PROMPTING HELPER / USER GUIDE

Prompting Helper: the whole app, chapter by chapter

From the first install to a finished storyboard: building prompts, characters, models and workflows, references and masks, tuning, repairs on the Canvas, and keeping your work safe.

13 CHAPTERS

PROMPTING HELPER / USER GUIDE

CONTENTS

In this book

01 Getting Started 3
02 Prompt Builder 19
03 Characters 38
04 Model Library 47
05 Workflows 63
06 Example Images 75
07 Regions, References & Masks 95
08 AI Analysis 109
09 Lineup & Tuning 121
10 The Phone 131
11 Canvas 143
12 Storyboard 156
13 Backup & Moving 173

Click a chapter to jump to it. Each chapter opens with its own list of sections. The PDF's bookmarks list every heading.

CHAPTER 01

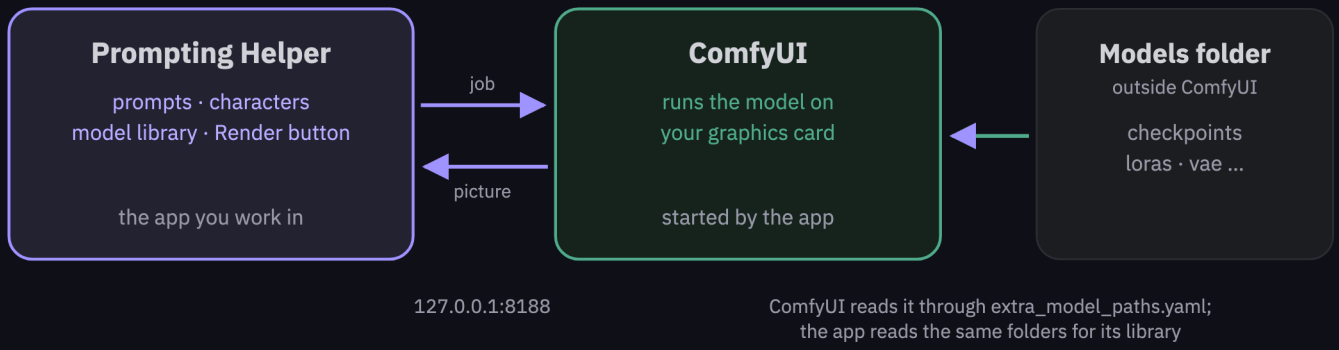
Getting Started: Prompting Helper + ComfyUI

From nothing installed to your first picture. You don't need any programming knowledge. Follow the steps in order, and don't skip any.

– How the pieces fit together 4
01 Install Prompting Helper 5
02 Set up ComfyUI with the assistant 5
03 The ComfyUI card in Settings 9
04 Where your models go 11
05 Show your models to the app 12
06 Workflows 13
07 Your first picture 14
08 Comfy Cloud (no graphics card needed) 16
09 Troubleshooting 16

How the pieces fit together

ComfyUI is the program that actually makes the pictures. It runs on your own computer and uses your graphics card. **Prompting Helper** is where you build prompts, manage characters and models, and press **Render**. It sends the job to ComfyUI and shows you the result. The app sets ComfyUI up for you, starts it, and keeps your models in a folder of their own, outside ComfyUI.



Graphics card
A Windows PC with an NVIDIA card (8 GB video memory or more), or a Mac with Apple Silicon (M1 or newer).

Disk space
About 20 GB free, more if you collect many models.

Internet
The setup downloads ComfyUI and several gigabytes of model files.

No graphics card? Skip to section 8: Comfy Cloud renders online.

CHAPTER 01 · SECTION 01

Install Prompting Helper

1. Go to `github.com/valerie-4659/promptinghelper-app/releases` (also available at `valerie-4659.itch.io/sd-prompt-manager`).
2. Download the file for your system from the newest release: **Windows** `promptinghelper-<version>.exe`, **Mac** `promptinghelper-<version>.dmg`, **Linux** `.AppImage` or `.deb`.
3. Install it.

> **A security warning on first start is expected**
>
> The app isn't signed with a paid certificate, so your system warns you once.
>
> - **Windows** ("Windows protected your PC"): click **More info**, then **Run anyway**.
> - **Mac** ("cannot be opened"): open **System Settings → Privacy & Security**, scroll down, and click **Open Anyway** next to the message about Prompting Helper.

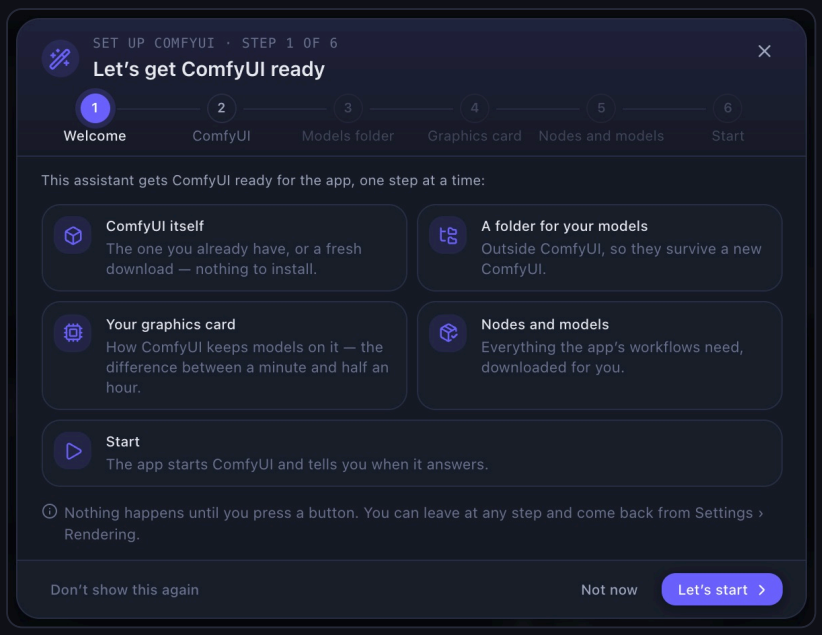
Updates. On Windows and with the Linux AppImage, the app offers to install a new version itself. On a Mac and with the `.deb`, the app tells you a new version is out and shows what to do: download it and install it over the old one.

CHAPTER 01 · SECTION 02

Set up ComfyUI with the assistant

The first time you start the app, and as long as it doesn't know any ComfyUI, the **setup assistant** opens by itself. It walks you through six steps. Nothing happens until you press a button, and you can leave at any step.

- Closed it? It is always in **Settings → 01 Rendering**: in **Output Integration** click **ComfyUI**, then **Setup assistant**.
- **Not now** closes it for this time. **Don't show this again** stops it opening on start; the button in Settings stays.
- The numbered steps at the top are buttons too. **Skip this step** goes on without doing a step.

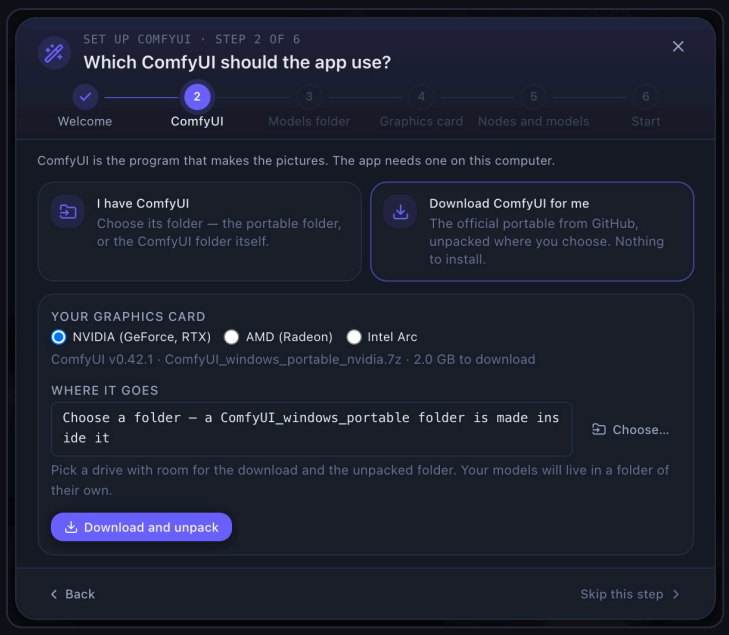


The welcome page names the five steps that follow. Press **Let's start**.

Step 2: ComfyUI

"Which ComfyUI should the app use?" You have three choices:

- **Keep this one**: only shown when the app already uses a ComfyUI.
- **I have ComfyUI**: choose its folder. The Windows portable folder (`ComfyUI_windows_portable`) and the `ComfyUI` folder inside it both work. The app reads everything else from it by itself: the start file, the port, its Python and its model folders.
- **Download ComfyUI for me** (Windows only): the app downloads the official portable from GitHub and unpacks it for you. You don't need 7-Zip.
  1. Under **Your graphics card**, check the card. The app picks it for you: **NVIDIA (GeForce, RTX)**, **AMD (Radeon)** or **Intel Arc**. The line below says which version it downloads and how big it is.
  2. Under **Where it goes**, click **Choose...** and pick a drive with enough room, for example `C:\AI\`. A `ComfyUI_windows_portable` folder is made inside it.
  3. Click **Download and unpack**. The download is about 2 GB. You can close the assistant meanwhile; the download goes on, and the progress is there when you open it again.



Step 2 with **Download ComfyUI for me** open. Already have ComfyUI? Click **I have ComfyUI** and choose its folder instead.

> **On a Mac**
>
> The download is a Windows build, so on a Mac the button is greyed out. Install ComfyUI yourself and choose its folder with **I have ComfyUI**. ComfyUI Desktop from `comfy.org/download` keeps its program inside the app, so the assistant may not recognise its folder. In that case start ComfyUI Desktop yourself and set the address by hand (section 3, "The manual way").

Step 3: Models folder

"Where your models live". Your models (checkpoints, LoRAs, everything the workflows load) go into a folder **outside** ComfyUI. If ComfyUI ever breaks, you put a fresh one in its place and it finds all your models again, without downloading anything.

1. The app suggests a folder, usually `ComfyUI-models` next to the portable folder. Keep it, or click **Choose...** and pick another.
2. Click **Use this folder**.

The app tells ComfyUI about this folder itself, by writing its own section into ComfyUI's `extra_model_paths.yaml` (in step 5). You never edit that file by hand.

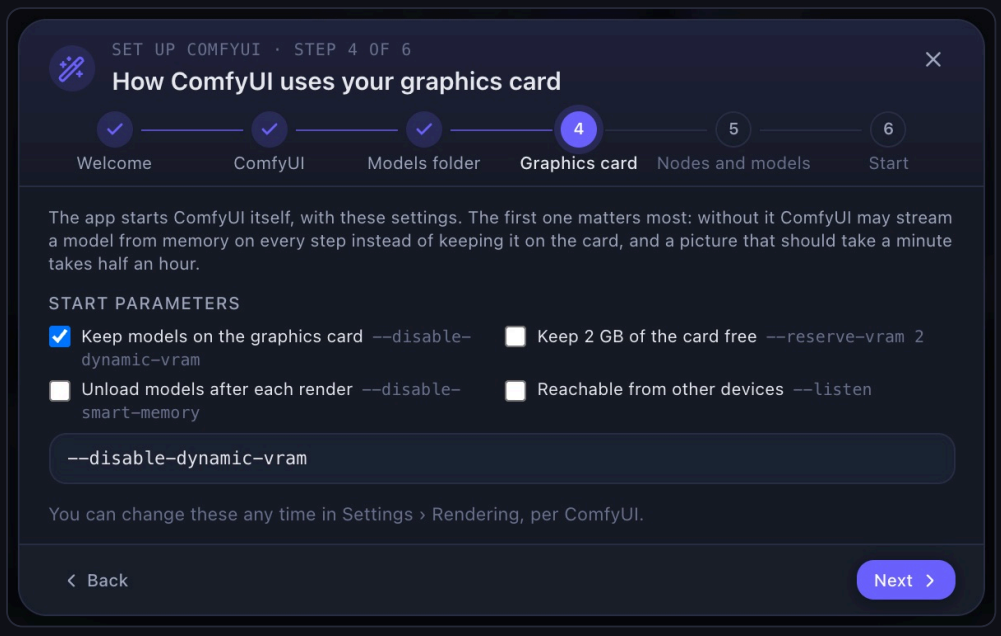
Step 4: Graphics card

"How ComfyUI uses your graphics card". The app starts ComfyUI itself, with the **start parameters** you tick here:

- **Keep models on the graphics card**: the important one, and ticked for you. Without it ComfyUI may stream a model from memory on every step instead of keeping it on the card. Krea 2 in particular then crawls at 30 seconds or more per step, and a picture that should take a minute takes half an hour.
- **Keep 2 GB of the card free**: leaves room for Windows and other programs. Helps when renders stop with "out of memory".

- **Unload models after each render**: frees the card for other programs, but rendering the same model again is slower.
- **Reachable from other devices**: ComfyUI answers on your network, not only on this PC. Anyone on your network can then use it.

The text field underneath takes any other ComfyUI parameter. For example, for live previews while rendering add `--preview-method auto`. Hover a switch to read what it does. Click **Next**.



Step 4. A switch only adds or removes its own parameter; the line below shows all of them and can be typed in.

Step 5: Nodes and models

The step lists what the app's workflows need besides your own checkpoints and LoRAs: telling ComfyUI where your models are, the Prompting Helper nodes, the node packs the Core workflows use, node packs you had in an earlier ComfyUI, and the model files (text encoders, VAEs, upscalers, detectors). Click **Set it up**.

Large files take a while. You can close the window; it goes on. When it is done the step says **Done.**, or how many could not be installed. Those are listed with the reason in **Settings → Rendering** (section 3).

Step 6: Start

Click **Start ComfyUI**. The first start takes a minute or two; the page tells you when ComfyUI answers: **ComfyUI is running — you can render.** Click **Done**.

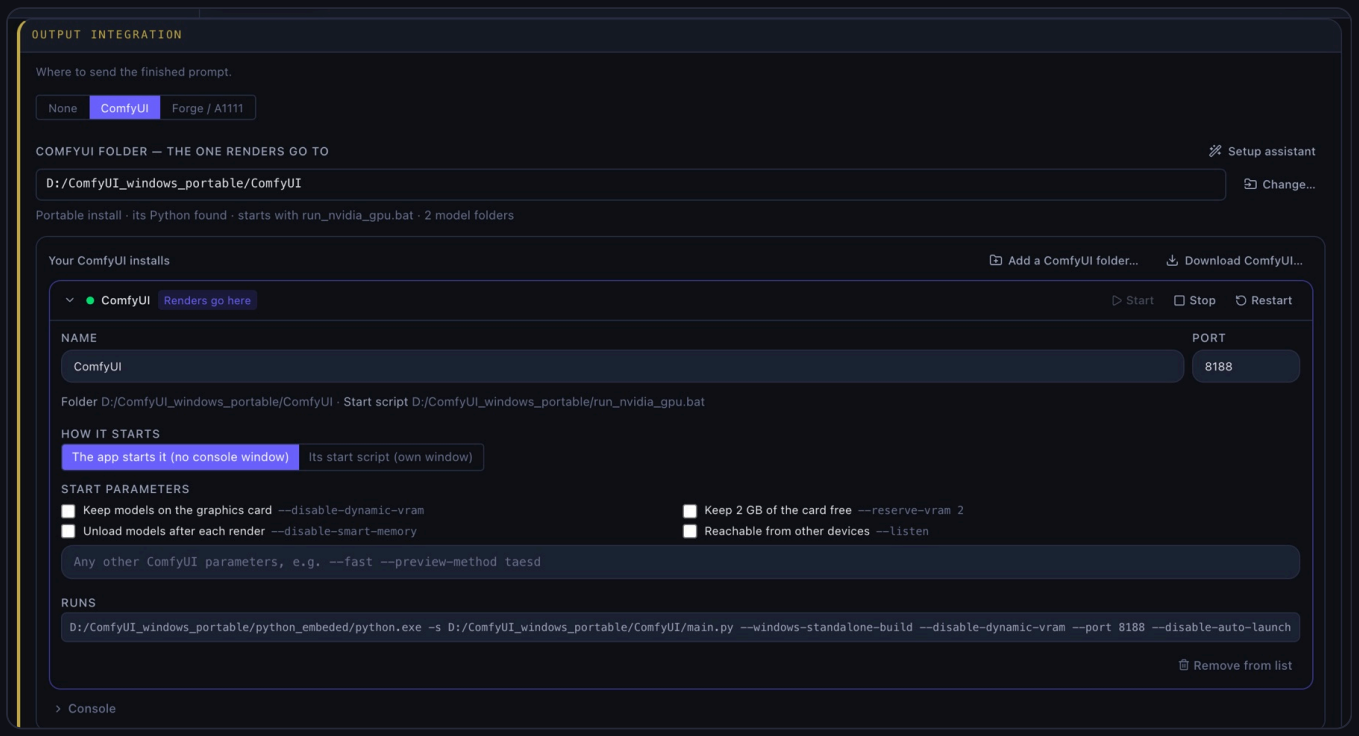
ComfyUI now runs without a console window and without opening a browser tab. You start, stop and restart it from the app (section 3).

> **Checkpoints and LoRAs are yours to pick**
> The assistant doesn't download checkpoints or LoRAs. You choose those yourself, for example in the Model Library's Civitai tab, and they go into your models folder (section 4).

CHAPTER 01 · SECTION 03

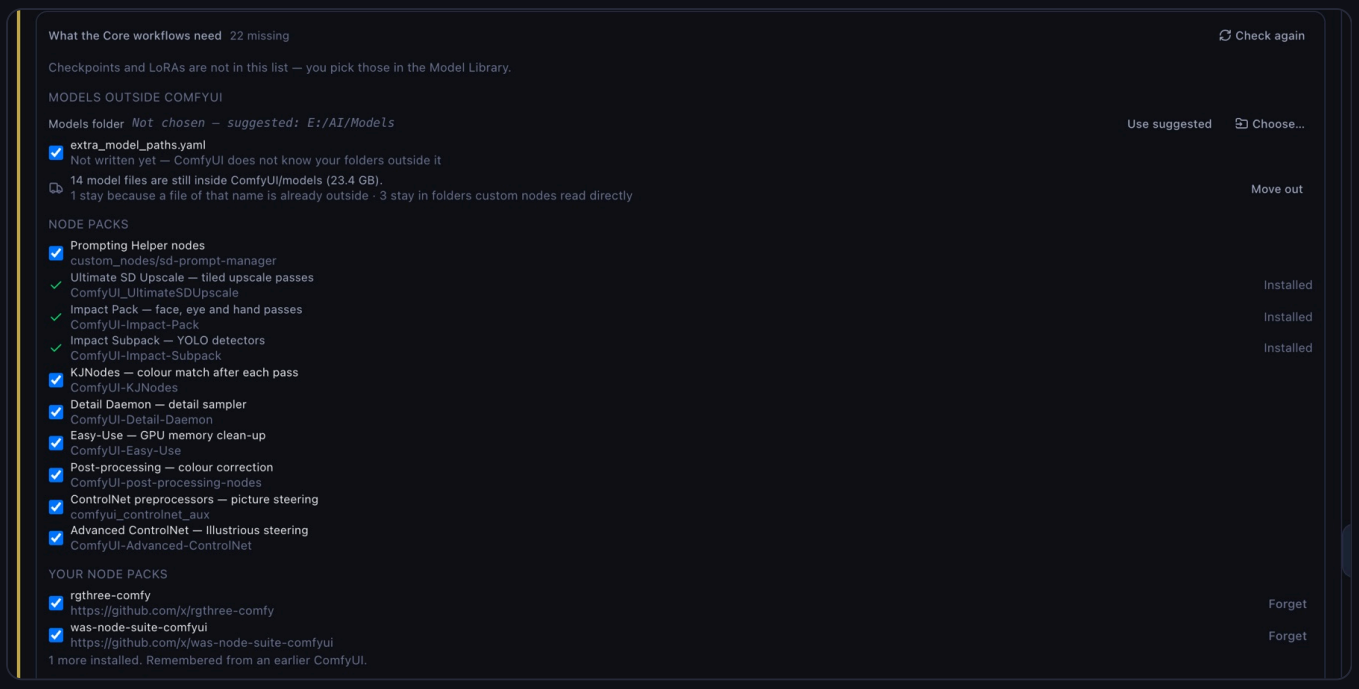
The ComfyUI card in Settings

Everything the assistant set up can be seen and changed in one card: **Settings → 01 Rendering → Output Integration → ComfyUI**.



The top of the card: the ComfyUI folder renders go to, and **Your ComfyUI installs** with the start parameters and the command the app runs.

- **ComfyUI folder — the one renders go to.** **Choose folder...** (or **Change...**) picks it. The line below says what the app found. A ComfyUI that is new to the app opens **Set up this ComfyUI**: it names the models folder and everything it will do, and **Set up** does it all in one go.
- **Your ComfyUI installs.** You can have several. One runs at a time; the one marked **Renders go here** gets your renders, and **Use this one** switches to another. Each has **Start**, **Stop** and **Restart** (restart after new nodes or new model folders). Open one to change its **Name**, **Port** and **Start parameters** (the same switches as in the assistant). **Runs** shows the exact command.
- **How it starts.** **The app starts it (no console window)** is the normal way. **Its start script (own window)** runs the `.bat` file as before, in its own window; the start parameters are not used then.
- **Console** under the list shows what ComfyUI prints. Look there when it doesn't start. **Add a ComfyUI folder...** and **Download ComfyUI...** add another install; **Remove from list** deletes nothing on disk.



What the Core workflows need: a tick for what is there, a checkbox for what is missing. At the bottom of the list one button gets every ticked piece and says what it will do.

- **What the Core workflows need** lists the models folder, `extra_model_paths.yaml`, the node packs and every model file. Missing pieces are ticked; untick what you don't want. The button at the bottom gets all ticked pieces at once (for example *Write the paths file, get 9 node packs and 12 models*). **Check again** looks at the folders again. Restart ComfyUI afterwards.
- **Models outside ComfyUI.** Model files still inside `ComfyUI\models`? **Move out** moves them into your models folder. Stop ComfyUI first. Nothing is overwritten: a file whose name is already outside, and folders custom nodes read directly, stay where they are, and the card says so.
- **Your node packs.** The app remembers the node packs of every ComfyUI it has seen. A new ComfyUI offers them again; **Forget** stops offering one.
- **Not downloadable from here.** A few files can only be downloaded after a login or from a web page. The list names each one and the folder it belongs in.
- **Advanced** holds the **API URL** (normally `http://127.0.0.1:8188`, filled in for you), the app's own **Port** for the optional Prompting Helper nodes, and the nodes as a ZIP. A change here needs **Save** at the top right.

Is it connected?

Open the **Dashboard**. Under **Connected services**, the tile **ComfyUI (Local)** should say **Running**. In the install list a green dot shows the same. If it says **Stopped**, start ComfyUI from the card (see Troubleshooting).

The manual way

You can do without the assistant. Install ComfyUI yourself: on Windows download `ComfyUI_windows_portable_nvidia.7z` from `github.com/comfyanonymous/ComfyUI/releases` and unpack it with 7-Zip; on a Mac install ComfyUI Desktop. Then choose its folder in the card, press the button under **What the Core workflows need**, and start it from the install list. If you start ComfyUI yourself

(ComfyUI Desktop, for example), only the address matters: open **Advanced**, set the **API URL** (ComfyUI Desktop usually uses port **8000**: `http://127.0.0.1:8000`; its **Settings → Server-Config** shows the port) and click **Save**.

CHAPTER 01 · SECTION 04

Where your models go

Models are the large files (usually `.safetensors`) that decide what your pictures look like. You download them from sites such as `civitai.com` or `huggingface.co`, or inside the app in the Model Library's Civitai tab. They go into **your models folder** from step 3 of the assistant, and each kind has its own subfolder, named like ComfyUI's own:

What you downloaded	Put it in	Examples
Checkpoint (a full image model, 2–7 GB)	`checkpoints\`	Illustrious, Pony, SDXL models
LoRA (a small add-on, 10–300 MB)	`loras\`	A style, a character, a pose
Anima / Krea 2 model	`checkpoints\` for the Core pipelines; `diffusion_models\` for the simple Character Preview starters	see the box below
Text encoder	`text_encoders\`	Needed by Anima and Krea 2
VAE	`vae\`	Needed by Anima and Krea 2; for others only if the model page says so
Upscaler	`upscale_models\`	4x-UltraSharp and similar

The setup fetches text encoders, VAEs and upscalers for the Core workflows itself. Checkpoints and LoRAs are what you add.

Anima and Krea 2: the workflow decides the folder

ComfyUI reads model files through a "loader" node, and each loader looks in one folder. The big **Core** pipelines that ship with the app (section 6) load Anima and Krea 2 the same way as any checkpoint, from `checkpoints`. The small **Character Preview** starters load them from `diffusion_models`. A file in the wrong folder fails with "Value not in list". Each workflow card in the app lists the files it needs.

- On Civitai the model page says what type a file is (Checkpoint, LoRA, and so on). Use that to pick the folder. Downloads from the Model Library's Civitai tab land in the right folder by themselves.

- You can make subfolders inside `checkpoints` or `loras` to keep things tidy. Both programs find files inside subfolders.
- Models already in `ComfyUI\models` keep working; ComfyUI reads both places. **Move out** in the card (section 3) moves them.
- If you add files while ComfyUI is running, click **Restart** in the card so ComfyUI sees them.

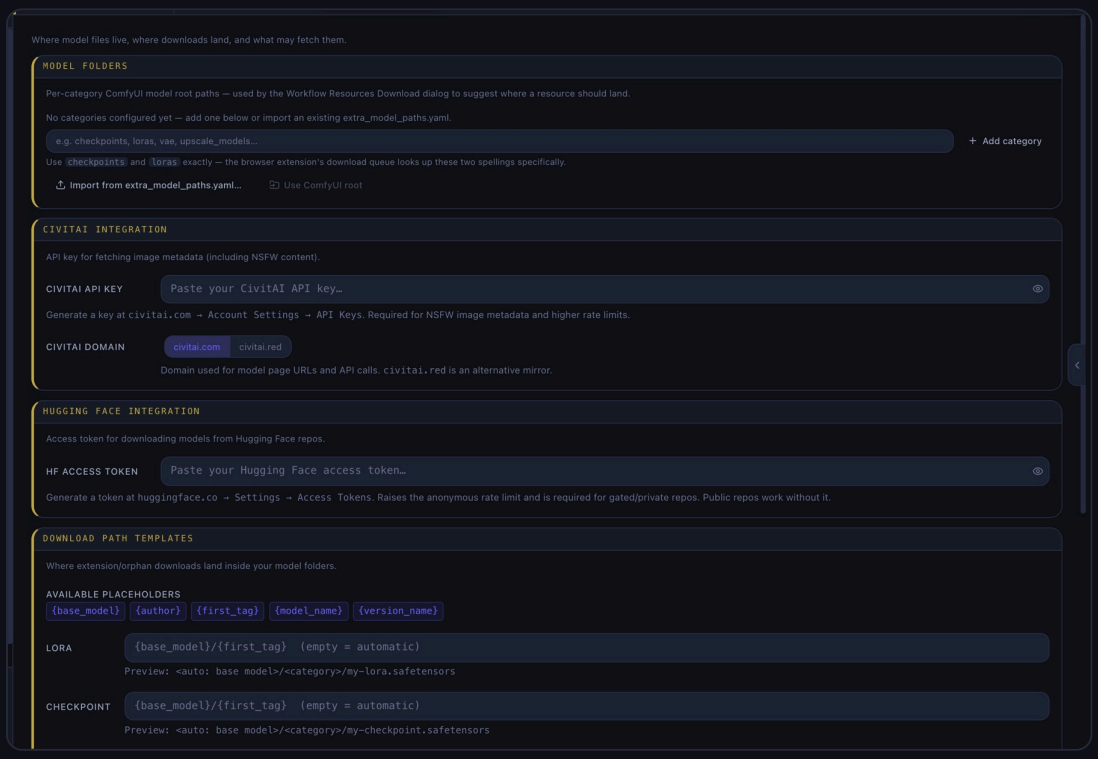
A good first model: any Illustrious-based checkpoint from Civitai. The app ships with a ready-made workflow for it.

CHAPTER 01 · SECTION 05

Show your models to the app

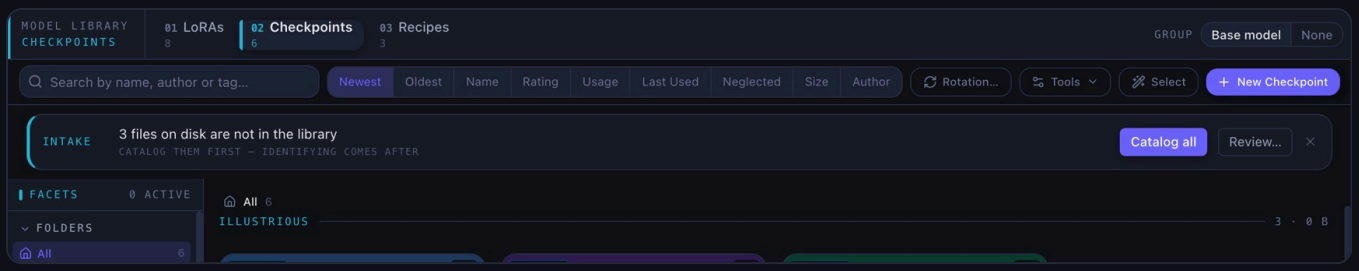
Prompting Helper keeps its own library of your models, with previews, notes and recommended settings. It fills it from the folders listed under **Model Folders**.

1. Open **Settings → 03 Models & sources** and find **Model Folders**. When you chose the ComfyUI folder and the models folder, the app filled in the `checkpoints` and `loras` folders itself, your models folder first. Check that they are there.
2. Missing? Click **Use ComfyUI root**, or add them by hand: type `checkpoints` into the field, click **Add category**, then add the folder. Do the same for `loras`. The names must be spelled exactly like that. Click **Save**.



Settings → Models & sources → Model Folders. **Use ComfyUI root** works once the ComfyUI folder is set. Already using an `extra_model_paths.yaml` of your own? **Import from extra_model_paths.yaml…** reads it.

3. Open **Model Library** in the sidebar and go to the **Checkpoints** tab. A bar says something like *"3 files on disk are not in the library"*. Click **Catalog all**. Do the same on the **LoRAs** tab.



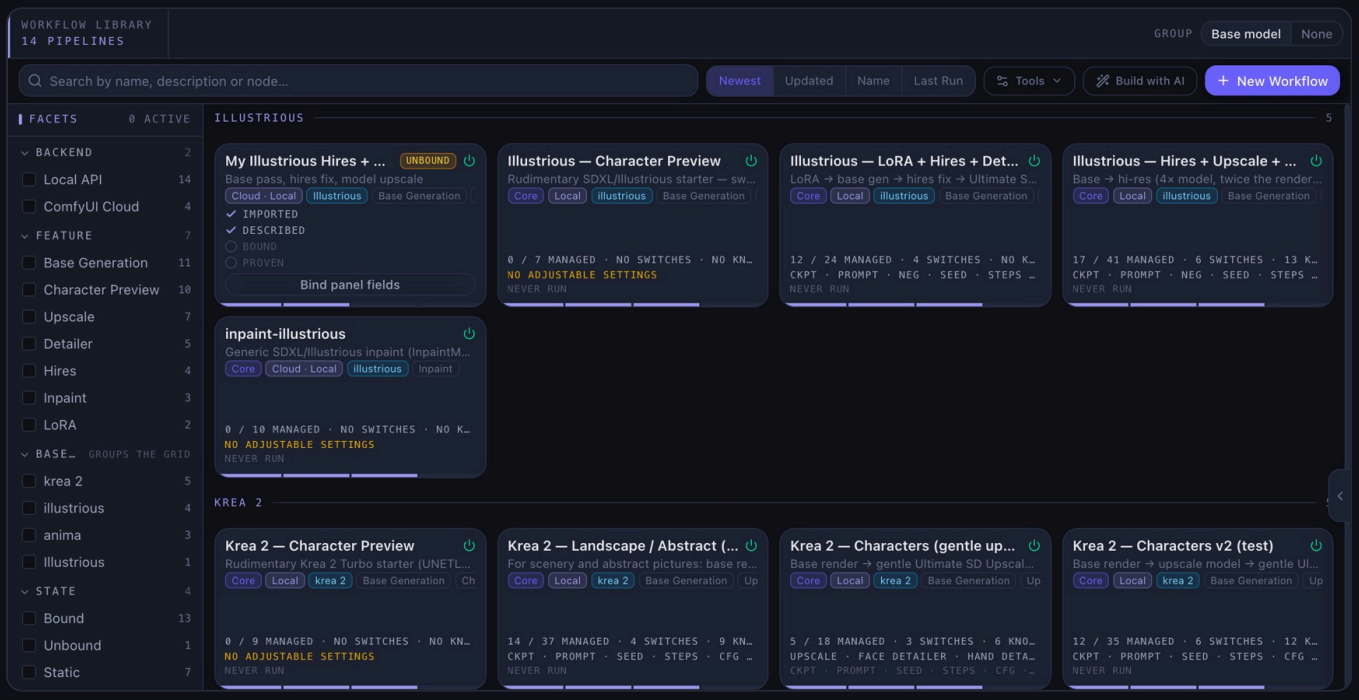
The intake bar. Catalog first; identifying (previews, base model, recommended settings) comes after.

Your models now appear in the checkpoint picker when you render. Added new model files later? Put them in your models folder, restart ComfyUI, then use **Tools → Rescan folders** in the Model Library (or restart the app) and catalog them again.

CHAPTER 01 · SECTION 06

Workflows

A **workflow** is the recipe ComfyUI follows to make a picture. The app ships with ready-made workflows, so for a first render you don't need to do anything here. Open **Workflows** in the sidebar to see them.



The Workflow Library, grouped by base model. **Core** marks the pipelines that ship with the app; **Local** and **Cloud** say where each one can run.

- **Character Preview** (Illustrious, Anima, Krea 2): small, plain workflows and the easiest start. Illustrious needs nothing extra; Anima and Krea 2 need their model, text encoder and VAE, which the card lists with download links.
- **Core pipelines**: Illustrious, Anima and Krea 2 with hires fix, upscale, face and hand detailers and a SeedVR2 clean-up, set up so Draft and Quality stay quick and the expensive stages run at Final. Krea 2 has two: **Landscape / Abstract** for scenery and **Characters** for figures. They need ComfyUI custom

nodes (Impact Pack, Impact Subpack, UltimateSDUpscale and others) and extra model files. The setup installs them (section 2, step 5, or the card in section 3); each workflow card lists them.
- **inpaint-...**: repair workflows used by the Canvas. They come set up; nothing to do.

Bringing in your own ComfyUI workflow

You only need this if you want to use a workflow you built in ComfyUI yourself.

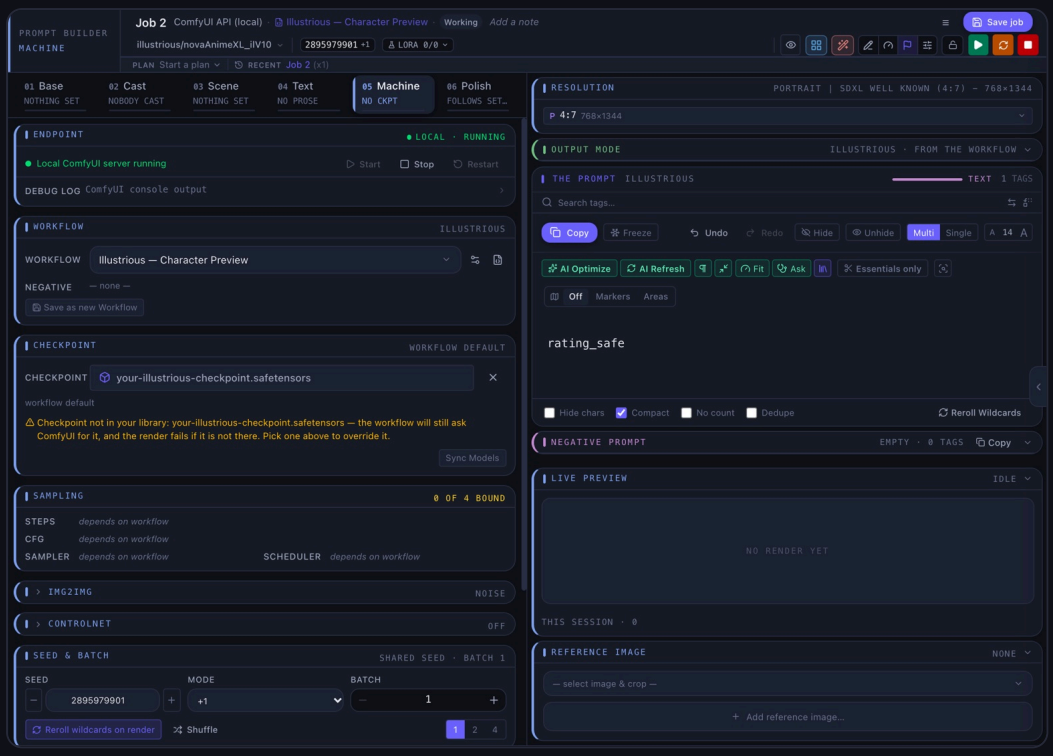
1. In ComfyUI, open your workflow and use **Workflow → Export (API)**.
2. In Prompting Helper, go to **Workflows → New Workflow**.
3. In the **Workflow** tab, next to **Workflow JSON**, click **Load file** and pick the file (or copy its text and click **Paste**). Used the normal Save by mistake? The app offers **Convert to API format**; click it while ComfyUI is running.
4. Give it a **Name** and fill in the **Base Model** (for example Illustrious).
5. Open the **Nodes & Bindings** tab and click **Guess**. The app finds the usual parts (checkpoint, prompts, seed, steps, size) so it can fill them in when you render.
6. Click **Save**.

> **Use New Workflow, not Import**
>
> The **Import** entry in the Workflows **Tools** menu only reads files that Prompting Helper exported itself and turns down a ComfyUI file with "Not a valid workflow export file". A file from ComfyUI always goes through **New Workflow → Load file**.

CHAPTER 01 · SECTION 07

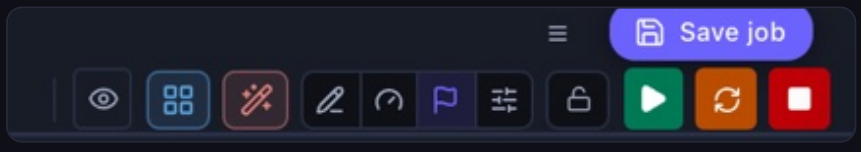
Your first picture

1. Go to **Jobs** in the sidebar and click **New setup** (on the Dashboard the button is called **New job**).
2. Choose **ComfyUI API (local)** as the service, then click **Create & open**. (Don't see it? Go back to section 3: it only appears after ComfyUI is selected in Settings → Output Integration. The assistant does that for you.)
3. The job opens in the Prompt Builder. Click the stage **05 Machine**:
   - **Endpoint** should say *Local ComfyUI server running*.
   - **Workflow**: pick **Illustrious — Character Preview** (or your own).
   - **Checkpoint**: click it and choose one of your models from the library. Until you do, the card warns that the workflow's placeholder file is not in your library.



05 Machine: Endpoint, Workflow and Checkpoint on the left; the prompt as it will be sent on the right.

4. Write your prompt in the other stages (Cast, Scene, Text), or add a character from your library. The finished prompt is always visible on the right under **The Prompt**.
5. Check the rung switch in the top-right corner, four icons: **Draft** (pencil), **Quality** (gauge), **Final** (flag) and **Custom** (sliders). Hover one to see its name. Picking a rung does not render.
   - **Draft** is a quick, cheap look, at a capped step count. It doesn't save a file.
   - **Quality** is a better look. It doesn't save a file either.
   - **Final** is the real render, with the whole workflow. **Only Final (and Custom) save the picture.** The app starts on Final.
   - **Custom** is Final with your own bypass switches deciding which stages run. You won't need it at first.
6. Click the green ▶ **Render** button, or press **Ctrl+Enter** (**Cmd+Enter** on a Mac). Ctrl/Cmd+1 to 4 pick the rungs; all of these keys can be changed in **Settings → Rendering → Keyboard Shortcuts**.



The top-right corner: view toggles, then the rung switch (Draft · Quality · Final · Custom), the seed lock, Render, Re-render and Stop.

The first render takes longer because ComfyUI loads the model into memory. Later renders are faster.

Where your pictures end up

- **ComfyUI's output folder**, for example `ComfyUI_windows_portable\ComfyUI\output\`.
- **Prompting Helper's own output folder**, if you set one: **Jobs → Output → Output folder → Choose…**. With **Auto-save** on (the default), every finished picture is saved there, sorted into a folder per day. This is the recommended setup.

CHAPTER 01 · SECTION 08

Comfy Cloud (no graphics card needed)

If your computer has no suitable graphics card, you can render online with Comfy Cloud. It's paid per use.

1. Create an account at `cloud.comfy.org` and copy an API key from **Account → API Keys**.
2. In Prompting Helper, open **Settings → 01 Rendering → ComfyUI Cloud**, paste the key into **Cloud API Key** and click **Test**. It should say **✓ Connected — API key is valid**. Click **Save**.
3. When you create a setup (section 7), choose **ComfyUI Cloud** as the service.

You don't need to install ComfyUI or download models for Cloud. Custom nodes are limited to what Comfy Cloud offers.

CHAPTER 01 · SECTION 09

Troubleshooting

The Dashboard says "Stopped".

ComfyUI isn't running, or the address is wrong. In Settings → Rendering → Output Integration → ComfyUI, click Start on your install and wait a minute. If it doesn't come up, open the Console under the install list: ComfyUI prints there why it stopped. Started ComfyUI yourself? Check the API URL under Advanced; ComfyUI Desktop usually uses port 8000, not 8188.

"The app cannot start this one itself".

The app found no Python for that ComfyUI. Start then runs its start script instead, in its own window, and the message says why. Your start parameters are not used that way.

"Download ComfyUI for me" is greyed out.

The portable is a Windows build. On a Mac install ComfyUI yourself and choose its folder (section 2).

Krea 2 (or another big model) is very slow: 30 seconds or more per step.

ComfyUI streams the model from memory on every step. Open your install in the card, tick **Keep models on the graphics card** and click Restart. It only works when the app starts ComfyUI (**The app starts it**); with the start script, add `--disable-dynamic-vram` to the script yourself.

"ComfyUI API (local)" is missing from the service list.

In Settings → Output Integration, select ComfyUI and click Save.

The checkpoint picker is empty.

The app lists its own library, not the folder directly. Do section 5 (Model Folders, then Catalog all in the Model Library).

The render fails with "Value not in list" or a missing model.

ComfyUI can't find the file the workflow asks for. Check that the model is in the folder that workflow reads (section 4; Anima and Krea 2 depend on the workflow), restart ComfyUI, and pick the checkpoint again under 05 Machine. In the card, **What the Core workflows need** → Check again shows which files are found; the `extra_model_paths.yaml` line must say **Written**, or ComfyUI doesn't look in your models folder. In the Model Library, Checkpoints → Tools → Auto-fill ComfyUI paths fixes names that don't match.

The render fails with a missing node type.

The workflow uses ComfyUI custom nodes you don't have. For the Core workflows, the button under **What the Core workflows need** installs them; restart ComfyUI afterwards. For a workflow of your own, install them in ComfyUI (the ComfyUI Manager is the easiest way).

Some files could not be installed.

The card shows the reason on each row. Click the button again to retry; files under **Not downloadable from here** you download yourself into the folder named there.

No live preview, only a progress bar.

Add `--preview-method auto` to the start parameters of your install and restart ComfyUI.

Pictures don't appear in my output folder.

Only Final (and Custom) save. Draft and Quality are for looking. Also check that an output folder is set under Jobs → Output.

Draft looks rough, or the upscale doesn't run.

That is on purpose: Draft caps the step count and the Core pipelines switch their expensive stages off below Final. Settings → Rendering → Render Steps sets how cheap Draft is.

Out of memory.

Your graphics card doesn't have enough memory for this model or size. Tick **Keep 2 GB of the card free** in the start parameters, or try a smaller resolution, a batch of 1, or a lighter model.

Where to go next

The other chapters walk through each part of the app: the Prompt Builder, Characters, Example Images, Workflows, Regions, References & Masks, Lineup & Tuning, the Canvas, the Model Library, the Storyboard, AI Analysis, and Backup & Moving.

CHAPTER 02

Working in the Prompt Builder

The Prompt Builder is where one picture is put together: who is in it, where they are, how it looks, and which model renders it. This chapter walks through the screen once, stage by stage, and ends with rendering.

– What it is for 20
01 Jobs and the Current Job 20
02 The screen at a glance 21
03 The six stages 22
04 The Description: one prompt, and its original 24
05 Placeholders: who plays whom 26
06 Mix examples: the Mix Board 27
07 The Prompt: what you see is what is sent 29
08 Output mode: writing for your model 30
09 Casting characters 31
10 The cast table: settings, order and weights 31
11 LoRAs 32
12 Rendering 32
13 The Recent trail: taking from another job 34
14 More in Polish 35
15 Saving and reusing a setup 36
16 Going further 36
17 Questions and problems 36

What it is for

Instead of typing a long prompt by hand, you choose things: a rating, a style, your characters, a background, a pose, a few lines of description. The Builder turns all of that into one prompt, written in the language of the model you render with, and keeps it on screen the whole time. When you press Render, exactly that prompt goes to ComfyUI (or another service) together with the model and settings you picked.

> **Before you start**
> Rendering needs a connected service, for example ComfyUI on your own computer. Setting that up is covered in the Getting Started chapter. You can build and copy prompts without any service connected.

CHAPTER 02 · SECTION 01

Jobs and the Current Job

Every piece of work in the app is a **job**. A job remembers its service (for example ComfyUI API (local)), its characters, tags and settings, so you can come back to it later exactly as you left it.

1. Open **Jobs** in the sidebar. Each card is one saved job.
2. Click **New setup** and choose a service. On ComfyUI API (local) and ComfyUI Cloud, also choose a **Base model**. Then click **Create & open**. The job opens in the Prompt Builder.
3. The Builder is always reachable as **Current Job** in the sidebar. It shows the job you opened last.

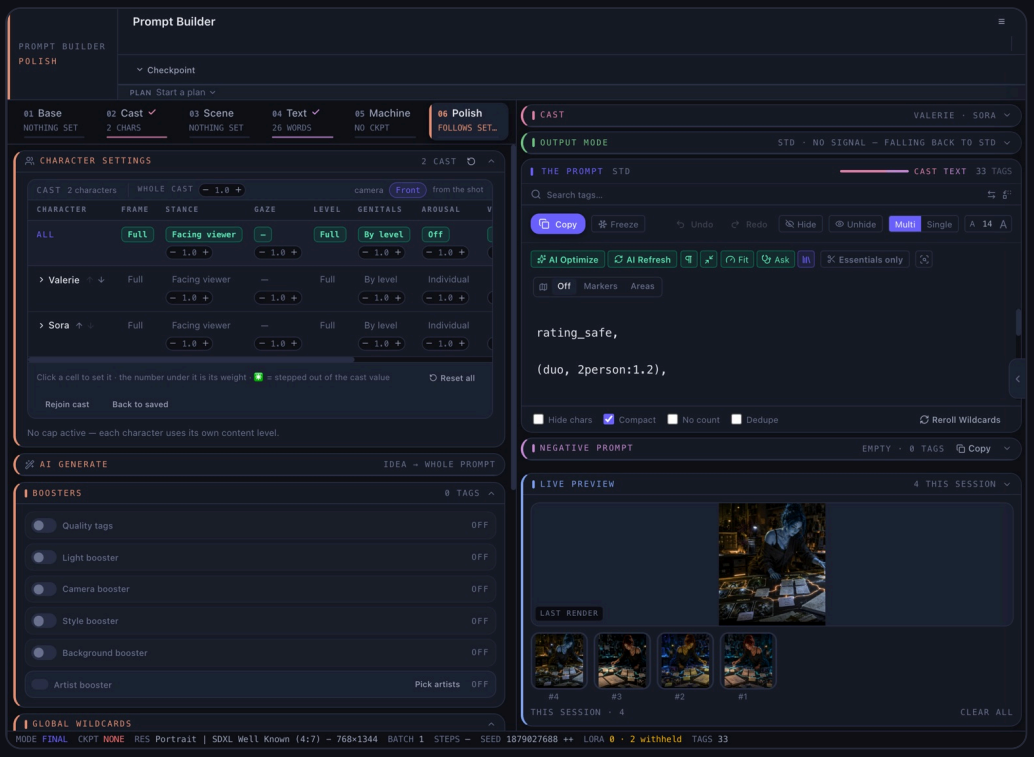
Saving. **Save job** at the top right stores the job's characters, tags and settings. Rendering also saves automatically.

Renaming. Click the job's name at the top left of the Builder, type, and press Enter (Escape cancels).

> **A job keeps its base model**
> A job on ComfyUI API (local) or ComfyUI Cloud is fixed to the base model you chose, for example Illustrious. The lock badge beside the job name says so. The Builder then offers only workflows and checkpoints of that family, and the output mode follows it. To work on another base model, start a new job (or use **New job from this**, section 13).

CHAPTER 02 · SECTION 02

The screen at a glance



The Builder on the Polish stage: the stage rail and its panels on the left, the cast, the output mode, the prompt and the previews on the right, the status bar along the bottom.

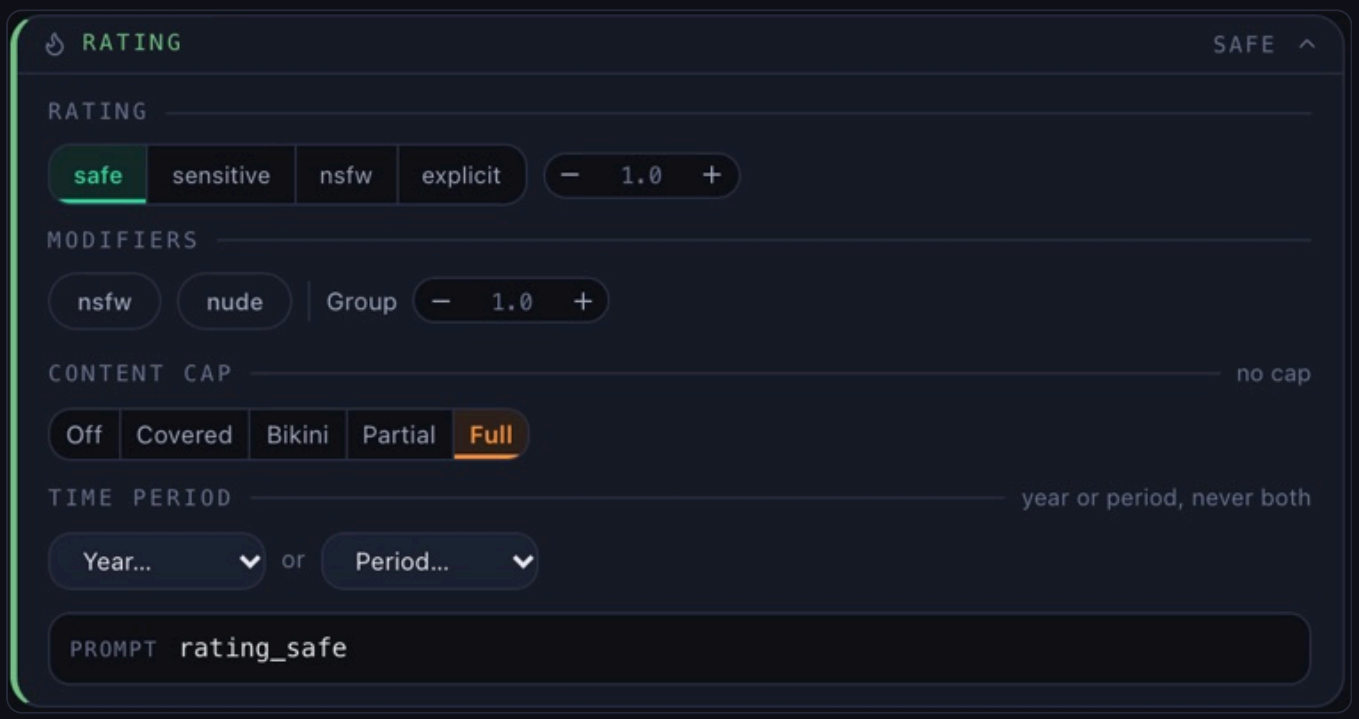
Area	What it does
Page head	Row 1: the job's name (click to rename), service, base model and workflow (a select, to switch the workflow). Row 2: the checkpoint, the seed, the LoRA pill, the STEPS/CFG pill and, when the workflow has some, its settings presets (**Settings…**). Row 3: Plan and the **Recent** jobs (section 13). On the right: Privacy, **Lineup**, **Repair**, the rung switch, the seed lock, and Render, Re-render, Stop (section 12).
Stage rail	The six stages, each with a short readout such as *2 chars* or *26 words*, so you see what is set without opening it.
Stage panels	The panels of the stage you clicked. The other stages keep their settings while hidden.
Cast, Output mode	Who is in the picture, and which model family the prompt is written for (section 8).
The Prompt	The finished prompt. This is what gets sent (section 7). The Negative Prompt sits right below it.
Live preview	The picture while it renders and afterwards, and a strip of this session's renders. With the window wide enough, the previews get a column of their own.
Status bar	A one-line summary of what the next render will use (section 12).

CHAPTER 02 · SECTION 03

The six stages

The stages follow the order in which a picture is usually made. You don't have to use all of them, and you can go back and forth at any time.

Stage	Panels	Use it for
01 Base	Rating, Global Style	How explicit the picture may be, and the overall look: rendering dimension, style engine, shading, line style, lighting, colour and more.
02 Cast	Characters & Areas, Regional prompting, Regional positioning	Who is in the picture. Regional prompting places each person in their own part of the frame (separate chapter).
03 Scene	Background, Pose, Shot & Lighting, Global Background	Where it happens, what people are doing, and the camera: view angle, shot size, lens, lighting, time of day.
04 Text	Description, Parked, AI Refinement	Your own words. The Description is the scene in plain text (section 4); Parked holds text that Replace and the AI cannot touch.
05 Machine	Endpoint, Workflow, Checkpoint, Sampling, img2img, ControlNet, Seed & batch, LoRA & triggers, Bypass & parameters, Detail pass	The technical side: which model and workflow render the picture. Only what the chosen service supports is shown.
06 Polish	Character Settings, AI Generate, then (folded) Boosters, Global Wildcards, Global Transformers, Style block, AI Suggest	Fine-tuning: how strongly and in which order the model reads what the other stages wrote.



01 Base → Rating: safe, sensitive, nsfw, explicit, an optional Content cap, and a Time period. The line at the bottom shows exactly what this panel adds to the prompt.

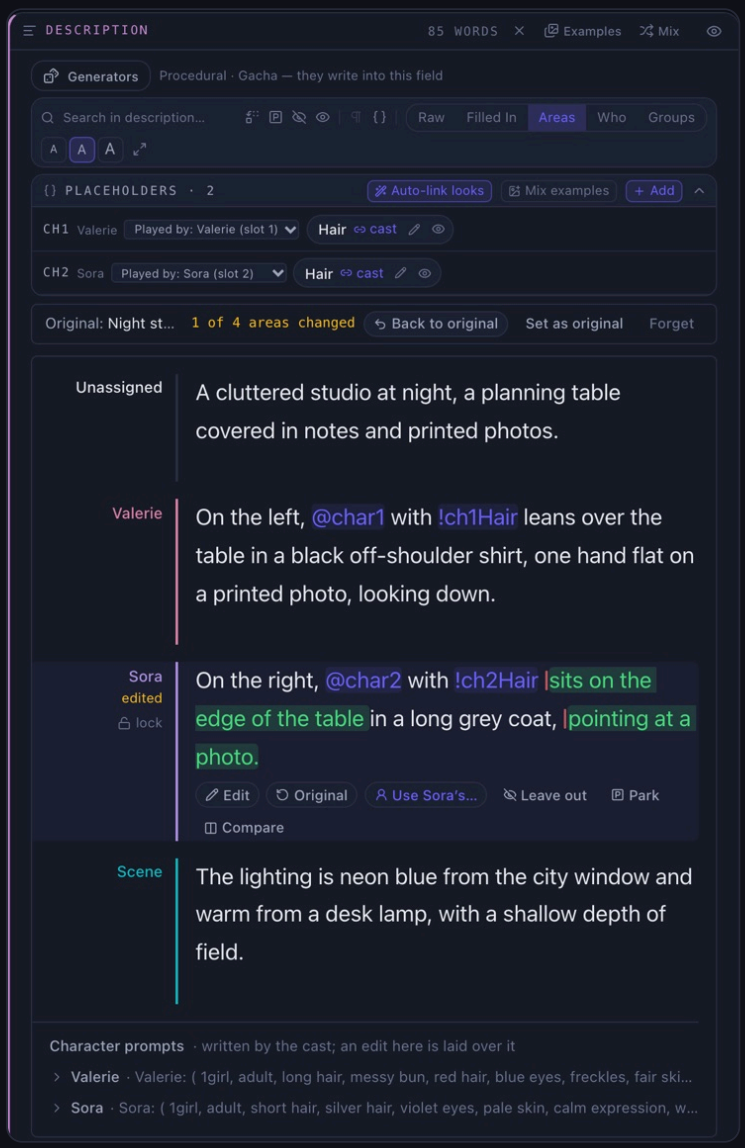
Every panel has an eye

The eye icon in a panel's head switches its contribution to the prompt off without losing what you set. Handy for comparing "with background" and "without".

CHAPTER 02 · SECTION 04

The Description: one prompt, and its original

The Description in **04 Text** is the scene in your own words. Everything you change about the text, you change here, where it is written. The app keeps the text it came from beside it, the **original**, so any part can go back to how it was.



The Description in the Areas view. At the top: the placeholder chips per person. Below: the original's line ("1 of 4 areas changed"), then the text cut into areas. Sora's area was edited: the new words are tinted, and under the pointer its actions show (Edit, Original, Use Sora's..., Leave out, Park, Compare). Under the text: the character prompts.

The head and the toolbar

- **Examples** opens your Example Images library without leaving the Builder, to replace or append the Description with a prompt you kept. **Mix** opens the Mix Board (section 6).
- The toolbar: search, **Dictionary replace**, **Park** (the selection goes onto the Parked shelf), Hide and Unhide, the ¶ button, and **{ }** to add a placeholder.
- **¶ Split into paragraphs** breaks a text that is one block before each person, what they do together, the light and the style. Only the spacing changes.
- Three **A** buttons set the type size; **Read large** opens the same field in a dialog.

AI Refinement, below the Description, rewrites it: **Edit** changes only the aspects you mark, **Enrich** builds it out, **Strip** takes aspects out, **Recast** puts your cast in place of the people the text describes, and one-click **Change** buttons redo the pose, outfit, expression, light & camera or background.

Five ways to read it

View	Shows
Raw	The text to type in. Placeholders are tinted; hover one to see what it sends now. The definition lines (`!ch1Hair: @cast`) are folded out of the editor into the placeholder strip; **Show** brings them back.
Filled in	The text with every placeholder replaced by its value. Click a value to edit it in Raw.
Areas	The text cut into areas (the scene, each person, the light...), each compared with the original.
Who, Groups	Which person each passage is about, and which kind of thing it describes.

The Areas view

The line at the top names the original and says how much changed, for example *1 of 4 areas changed*. Every Replace (from Examples, from the Mix Board, from the Crosspost Helper) makes the text it brings the new original. A text you wrote yourself has none: click **Set as original** to pin it. **Back to original** puts the whole text back in one undo step; **Forget** keeps the text and forgets where it came from.

The area's name stands in the margin, with *edited*, *new* or *removed* when it is no longer as it came in. Point at an area to see what you can do with it:

- **Edit** rewrites just this area. **Done** (Ctrl+Enter) writes it back.
- **Original** puts this area back as the original had it; the other areas stay. A removed area says **Bring back**, a new one **Remove**.
- **Leave out** takes the area out of the prompt. It stays listed as removed, one Bring back away.
- **Park** moves the words onto the Parked shelf (below).
- **Compare** shows the original and now side by side.
- **Use Valerie's...** (on a person's area) lets the character who plays her take her over: her whole look, her core, or single traits linked to the character.

- **lock** in the margin, and the trait chips in the action row, lock the area or one trait (see Locks below).

Character prompts, under the text

Under the areas, **Character prompts** lists each figure, folded. Open one to see her block twice: as the cast writes it, and *as the model gets it* (Krea 2's sentences, Anima's phrase…). An edit in either is laid over what the cast writes and is sent in its place; **Reset all** lets the cast write again. The magnifier in a row's head finds and replaces words in it, and says which area each hit is in.

Written by other stages, folded below, shows the words the pose, background, style and shot panels add. **Into the text** moves them into the Description as your own words and switches that panel's eye off; **Park** keeps a copy on the shelf.

Parked

The **Parked** panel (it replaces the old Custom field) holds pieces of text out of reach of Replace, the Mix Board and the AI analysis. Each piece is **Sent** (it stands in the prompt right after the Description) or **Kept** (here only). **+ Piece** adds one, **Into text** puts a piece back at the end of the Description.

Locks and Replace

Only a lock stops a Replace. Lock an area, or a single trait such as Outfit, and the next Replace from Examples, the Crosspost Helper or the Mix Board asks first: a sheet lists **Yours** and **New picture** side by side, with **Keep mine** or **Take new** per row. Locked rows start on Keep mine. The new prompt becomes the original; locks stay locked.

CHAPTER 02 · SECTION 05

Placeholders: who plays whom

A prompt taken from Example Images often speaks of people as `@char1`, `@char2`, and of their looks as placeholders like `!ch1Hair`. Their values are definition lines at the top of the Description (`!ch1Hair: @cast`), and the **Placeholders** strip shows them as chips, one lane per person.

- **A chip** names the trait and what it holds now: *cast* (linked to the character), its first words, or a state such as *undefined*, *unused*, *hidden*, *locked* or ← *Mix*. Click it to open the row; the pencil edits the value, the eye hides it from the prompt and keeps the value for later.
- **The hair is one placeholder**, `!ch1Hair`: colour and style together. An older text with separate hair colour and hairstyle placeholders is folded into one.
- **Played by**, at the head of each lane, chooses which cast slot plays this person. Picking another slot swaps the two numbers in the Description. **Played by: nobody** keeps the picture's own words for her.
- **Auto-link looks** (on by default) links the hair and eyes of every person the cast has a character for, by themselves. A line you unlinked or are typing in is left alone.
- **Who is who** appears when the main picture has two people or more: the picture with a numbered ring per person, and a list of who plays each. A click there is the same Played by. Where the text does not say

where someone stands, **Place them from the picture** asks the AI (one call).

CHAPTER 02 · SECTION 06

Mix examples: the Mix Board

The Mix Board takes parts of other pictures into this prompt: the palette of one, the outfit of another, the light of a third. Open it with **Mix** in the Description's head, or **Mix examples** in the placeholder strip.

The Mix Board. Left, the pictures: picture 1 is the main picture (the one the Description came from), 2 and 3 were added. Right, the prompt's inputs, each saying where it comes from. The palette comes from picture 3, the outfit from picture 2, and the lighting is a blend of 1 and 3 (60/40, Join). Every input nobody wired is drawn from the main picture, fainter.

Three lanes

- **Pictures**: the references on top (R1 is always the reference the Builder has selected now), then the main picture, then every picture you added with **Add picture**. Each picture lists its traits, read from the picture: art style, palette, lighting, background, framing, and per person outfit, makeup, hair, accessories, pose and expression. *not in the picture* means the reading found none. **View large** shows the picture whole; **S M L** in the head sets the size.
- **Blends**: one node for each input with two wires or more.
- **Prompt**: the inputs. Reference image and Description, then Scene (art style, palette, lighting, background, framing), then each person of the text.

Wiring

1. Drag a cable from a picture's trait onto an input of the prompt, or double-click the trait to wire it into the input of the same name. Or click an input and choose **Use** in its list.
2. A drop **replaces** the input's wire. Hold **Shift** while dropping, or choose **+ Blend**, to blend two pictures into one input.
3. Click a cable to remove that wire. **Clear wires** puts every input back to the main picture.
4. Click **Apply**. Every wired input is written into the Description in one undo step, and a wired reference becomes the selected one.

An input nobody wired keeps the main picture's text; nothing is written for it. A wire from another picture is what stays: when you later take a new main picture from Examples, the inputs wired from other pictures keep their values. (A Replace from Examples starts the board fresh otherwise; the pictures on it stay.)

Blends

A blend node has one slider per wire; the shares always add up to 100%.

Input	Modes
Text traits	**Join** puts both texts side by side, the larger share first (free). **Fuse with AI** writes one text out of them (one AI call, on Apply).
Description	Fuse only: the AI writes one Description from both, weighed by the shares.
Reference image	**Crossfade** mixes the pixels: light and colour come through, not two compositions. **Colours only** keeps the first picture's shapes with the others' colours and light. **Layers**: see below.

The reference as layers

In **Layers** mode the first picture is the base and the frame; every other one is a layer on it. **Edit layers** opens **Layer mix**: per layer a **Blend** mode (Multiply, Screen, Overlay, Additive, Subtract ...), **Opacity** and **Size**. With **Move** you drag a layer (the wheel scales it); **Hide** paints part of it away, **Show** paints it back. A mask moves with its layer: mask the part you want, then move it where it belongs. Apply saves the result into the Reference Library and selects it.

> **Readings cost one call each**
>
> A picture is read once when it is added (one AI call, a queue, one picture at a time). The main picture's reading also finds which words of the Description belong to which trait, so Apply never reads it twice. A failed reading says so on its node, with **Retry**.

CHAPTER 02 · SECTION 07

The Prompt: what you see is what is sent

The Prompt panel on the right is not a preview of something else: it is the prompt the render receives, word for word. If a tag isn't in there, the model won't see it. If it looks wrong here, the picture will be wrong too.

- **Copy** copies the prompt, for use in another program.
- **Freeze** locks the prompt so you can edit the text directly. While frozen, changes in the stages no longer reach it.
- **Undo / Redo** step through earlier versions of the prompt (Ctrl+Z / Ctrl+Shift+Z outside a text field).
- **Hide**: select some text, click Hide, and it is struck through and left out of Copy and the render. **Unhide** brings it back.
- **Multi / Single**: one line per block, or everything on one line. The two **A** buttons change the type size.
- **Off / Markers / Areas**: how strongly the prompt is colour-coded by where each part came from.
- Select a tag and press **Ctrl+↑ / Ctrl+↓** to make it stronger or weaker, for example `(red hair:1.2)`.
- Below the prompt: **Hide chars** (display only), **Compact**, **No count** (drops solo, 1girl, 2girls…), **Dedupe** (moves tags every character shares to the front) and **Reroll Wildcards**.

The AI row

Button	What it does
AI Optimize	Opens the options (service, ruleset, enhancements), then rewrites the prompt for the active model.
AI Refresh	Freezes the prompt as it stands and optimizes it again.
¶ Prose / **Simplify**	One-click rewrites: into flowing prose for the active model, or shorter and decluttered. Both freeze the prompt first.
Fit	Cuts the prompt to a token budget. Every character keeps her essential tags, and the answer is counted with the model's own tokenizer.
Ask	Ask the AI about this prompt: why something is in the picture, how to get something in on this model, why a render does not look like the prompt.
Prompt Library	Save the current prompt, or load one you saved.
Essentials only	Cuts every character block to the tags marked essential in the character editor: what makes them recognisable.
Tag check	Duplicates, contradictions, broken fragments, essentials that went missing, and sections past the model's token limit.

> **AI buttons need an AI key**
> Everything in the AI row except Essentials only and the Tag check needs an AI provider set up in Settings → AI.

CHAPTER 02 · SECTION 08

Output mode: writing for your model

Different model families understand prompts differently. Some want short tags with weights, others want full sentences. **Output mode** picks the rules the whole prompt is written in: tag weights, prose layout, booster vocabulary and the AI optimizer's target.

Output mode, opened. The readout says which mode is active and why: here "Illustrious · from the workflow". Below the modes: the switches that apply to the active one.

- **Tag-based:** STD (standard weights), Illustrious, NoobAI, Pony (with its own score_ tags), Anima.
- **Natural language:** FLUX, Krea2, Z (Z-Image Turbo), Qwen, WAN and Minimax (video).
- **GPT Image** writes a structured document with headings (Characters, Wardrobe, Pose, Setting...) instead of a tag list.

Normally the mode follows the checkpoint or workflow you picked in 05 Machine. Click a mode to set it yourself.

The switches under the modes

- **Cast in Description**: write `!ch1Look` into the Description where a figure stands ("on the left, !ch1Look, kneels ..."). On, the placeholder becomes the whole figure and her own block is dropped, so she is described once, in her own clause. Off, it is just her name and the block stays.
- **Prose layout** (Illustrious, Anima): rewrites the character blocks as sentences anchored in space ("in the foreground, Valerie is ...") instead of a tag list.
- **Natural-language modes** have their own: **Prose rewrite** (each character block as a flowing sentence, without AI), **Block labels**, and two guards against one figure's traits bleeding into another (**Anti-bleed**, **Face-bleed guard**).
- **Cast first** (Anima): the character blocks move ahead of the scene text. Anima reads the start of a prompt best.

- **Token count** (Anima, Krea 2): both read at most 512 tokens. The status bar shows the real count from the model's tokenizer, and the prompt tints what is past the limit.

CHAPTER 02 · SECTION 09

Casting characters

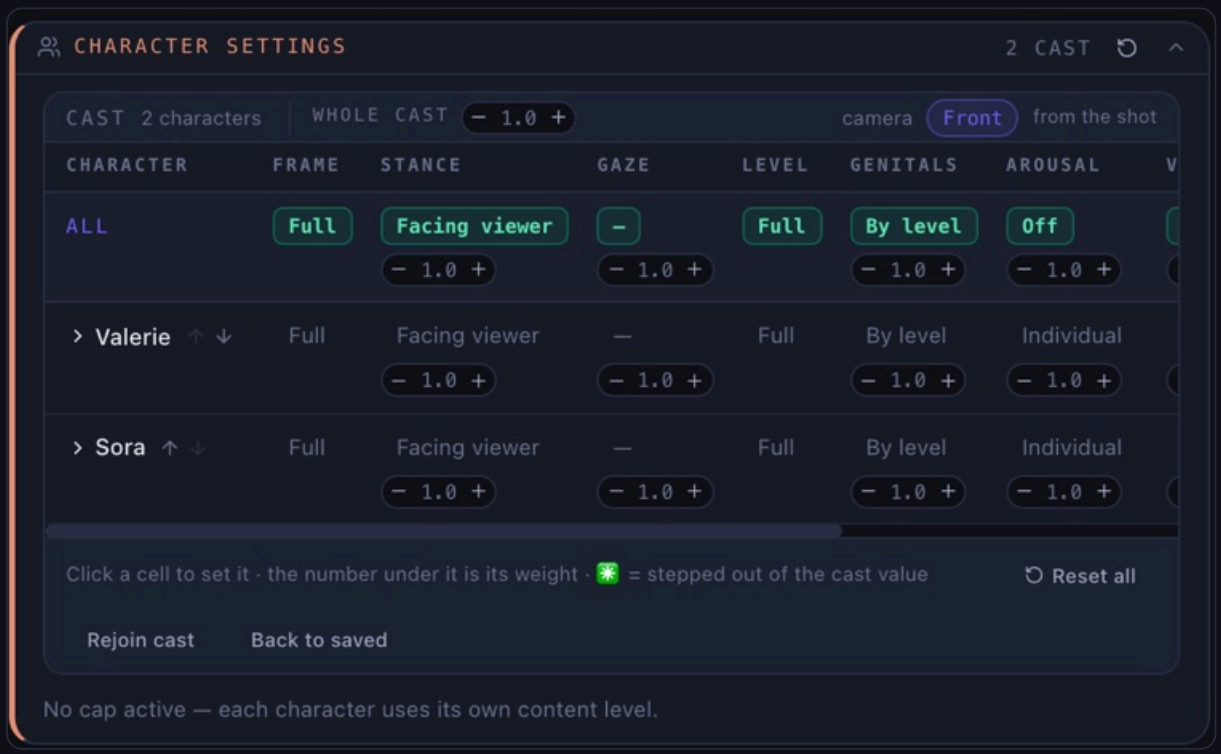
Characters come from your character library (see the Characters chapter). In **02 Cast → Characters & Areas**:

- **Add character** puts a character from your library into the picture. **Add temp** adds a one-off person just for this job. **Add area** adds an area card for regional prompting.
- Drag a row by its handle to reorder. The order is the order in which the characters reach the prompt.
- On each row, the eye hides that character for now, the arrow opens its settings for this job, and ✕ removes it.
- A yellow **Cast may read alike** note appears when two characters are described too similarly for the model to tell apart, and says which kinds of features would hold them apart.

CHAPTER 02 · SECTION 10

The cast table: settings, order and weights

06 Polish → Character Settings is one table for everything about the cast in this picture. It changes the job, never the character in your library.



The cast table. Columns: frame, stance, gaze, level, genitals, arousal, vibe, mess, role. Under each value sits its weight.

- **Click a cell** to change it. The **ALL** row changes a column for every visible figure; a figure's own row changes just that figure.
- **The number under a value is its weight**: how strongly that part of the figure is read. Under the ALL row it writes everyone; under a name, that figure.
- **Whole cast**, in the table's head, makes all figures stronger or weaker against the rest of the prompt, background and style included.
- **The arrows beside a name** set the order: earlier in the prompt is read first.
- **Camera** in the head says where the camera stands; "from the shot" follows the Shot & Lighting panel.
- **Rejoin cast** makes every figure follow the ALL values again; **Back to saved** drops this job's changes and uses the library values.

> **Where a weight does not arrive**
>
> Weights are numbers in tag prompts. Natural-language models (FLUX, Krea 2, Qwen ...) read emphasis as words instead; there, weights are translated into emphasis words (Krea 2) or left out, and the Whole cast control names the model it does not reach.

CHAPTER 02 · SECTION 11

LoRAs

LoRAs sit in **05 Machine → LoRA & triggers**. Each row is listed by the LoRA's name; the file path is in the tooltip, and pointing at the name shows its preview picture from the library.

- **Add LoRA** opens the LoRA picker: tiles with base model and stars, a **To add** column on the left, and a **Library | Civitai** switch to find and download one on Civitai. **Recipes** beside it opens your saved LoRA recipes.
- On a row: the switch turns it on or off, the slider or the number box sets the weight (type a value and press Enter), and ✕ takes the LoRA out of the chain. LoRAs that are part of the workflow itself are marked *from workflow* and cannot be removed here.
- A row keeps the range you gave the LoRA in the library; one without a range reaches −5 to +5. A stored weight outside the range turns amber.

CHAPTER 02 · SECTION 12

Rendering

1. **Pick a rung** in the page head: **Draft**, **Quality**, **Final** or **Custom**. The number on each is the step count that rung uses. Hover one to see what it does with your workflow: which stages go off, whether anything reaches the disk.
   - **Draft** and **Quality** are for looking: fast, and nothing is saved.
   - **Final** is the real render and saves the picture. The app starts on Final.

- **Custom** is Final with your own bypass switches (05 Machine → Bypass & parameters) deciding which stages run, instead of the workflow's plan.

Picking a rung doesn't render; it only decides what the next Render does.

2. Click the green ▶ (or press **Ctrl+Enter**, Cmd+Enter on a Mac). Orange re-renders (stops the current render and starts again), red stops.
3. The picture appears under Live preview.

The render queue. While something renders or waits, the status bar shows **QUEUE**, for example `1 RENDERING · 2 WAITING`. Click it for the queue: one block per service, sent first, then waiting in order. **Cancel** takes a waiting render out, **Stop** stops one that is rendering, **Cancel all waiting** clears the rest. A render already sent to ComfyUI Cloud cannot be stopped; the button says so.

Key	Action
Ctrl+Enter	Render
Ctrl+Shift+Enter	Re-render
Ctrl+.	Stop
Ctrl+L	Lock / unlock the seed
Ctrl+1 … 4	Draft, Quality, Final, Custom

Cmd instead of Ctrl on a Mac. These keys work even while you type in a field. If another program already uses one, change it in Settings → Rendering → Keyboard Shortcuts.

Seed. The seed pill in the head (for example `2895979901 +1`) decides the randomness. With +1 it moves on by one each time you press the same rung again, so each render is a new picture. The **lock** keeps the seed fixed, so you can repeat one picture and change only the prompt.

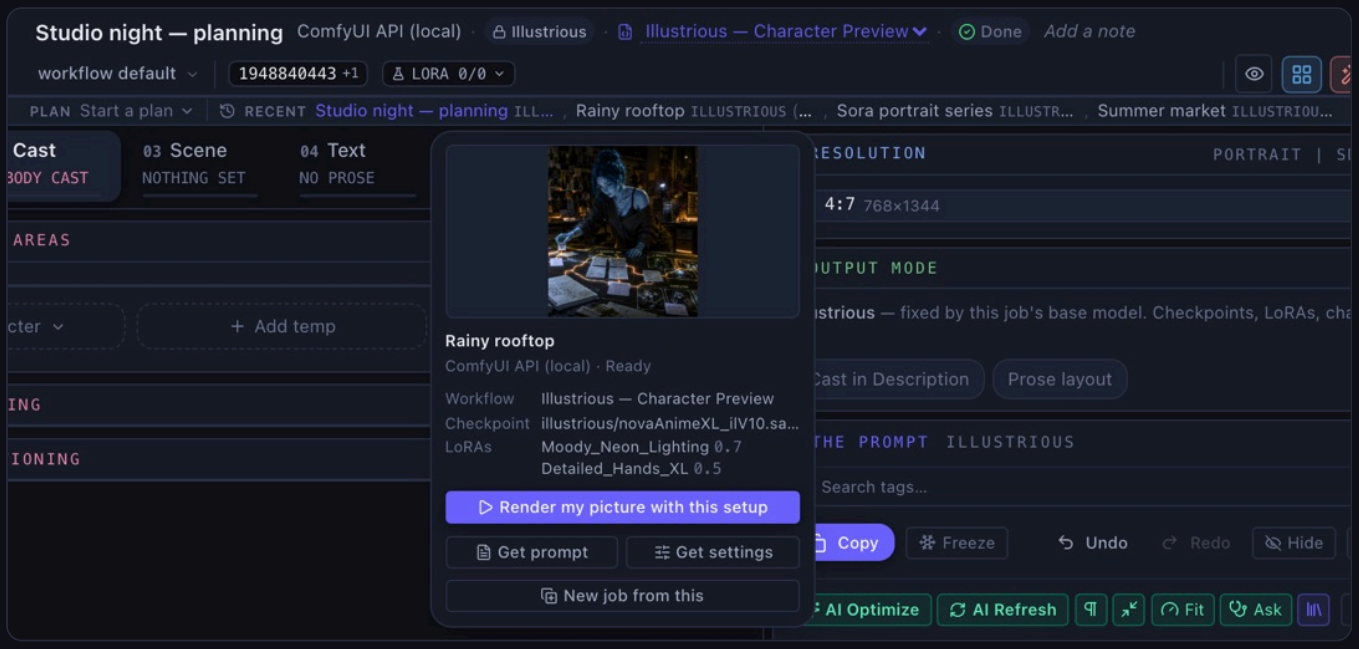
Status bar. The bottom line summarises the next render. Check it before rendering: **MODE** the rung · **CKPT** the model (red NONE = none picked) · **RES** the size · **BATCH** pictures per render · **STEPS** · **SEED** · **LORA** active LoRAs · **TAGS** how many tags the prompt has (and, for Anima and Krea 2, its token count) · **QUEUE** while anything renders or waits.

Steps and CFG. The STEPS/CFG pill in the head gets a dot when its values are off the checkpoint's sweet spot. Hover it, and **Use sweet spots** sets both in one click.

CHAPTER 02 · SECTION 13

The Recent trail: taking from another job

Row 3 of the page head lists your **Recent** jobs: the last six, in the order you first visited them, each with its base model. Click one to open it. Point at one to see its card: its newest picture, its workflow, checkpoint and LoRAs, and what you can take from it.



The card of "Rainy rooftop", shown from the open job "Studio night — planning". Its workflow, checkpoint and LoRAs are listed above the buttons.

Button	What it does
Render my picture with this setup	Renders the open job's prompt, resolution and reference through the other job's workflow, checkpoint and LoRAs, as a Final render. The result is filed under that job; neither job's settings change. With img2img on, the card has its own **Strength** for this one render. ComfyUI API and ComfyUI Cloud jobs only.
Get prompt	Puts the prompt that job last sent into this job, frozen, with its reference selection.
Get settings	After a warning, replaces this job's prompt, cast, description, style, output mode and boosters with that job's. The Machine stage (workflow, checkpoint, LoRAs, bypasses, parameters, seed) stays. **Undo settings from ...** appears in the Recent row to take it back.
New job from this	A new job with that job's prompt and settings and a fresh Machine stage, on a base model you choose in the card. It opens at once.

The other job is only read. On the open job's own card, only New job from this is offered.

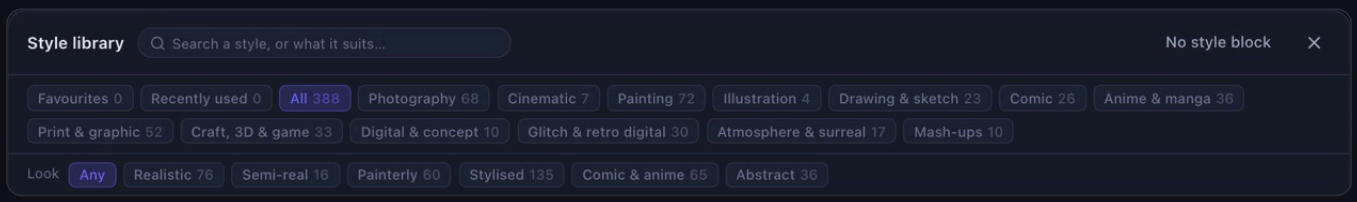
CHAPTER 02 · SECTION 14

More in Polish

- **AI Generate**: describe an idea and let the AI write it. In the Builder it writes a **scene** into a slot of its own, beside your Description, and your cast keeps coming from the characters; "Replace whole prompt" is the second button. A run keeps going if you close the window, and its door says when it is done.
- **Boosters**: ready-made quality, light, camera, style and background tags that fit the output mode, plus an artist booster.
- **Global Wildcards** and **Global Transformers** (theme, pose, expression, hair, makeup, cloth, body FX).
- **Style block**: one named style for the whole picture (see below).
- **AI Suggest**: ideas from the AI, from conservative to wild.

Pointing at 06 Polish in the stage rail shows its card with **Reset Polish**: boosters, transformers, wildcards, the cast weights and the character prompt edits go back to following the settings, in one click.

The Style library



The Style library's shelves: Favourites, Recently used, All, the families, and the Look row.

Click the Style block card to open the **Style library**, close to 400 styles. The tabs are Favourites, Recently used (the last ten), All, one per family (Photography, Painting, Comic, Anime & manga ...) and Hidden; the **Look** row filters by Realistic, Semi-real, Painterly, Stylised, Comic & anime or Abstract. Favourites, recent and hidden styles are kept across all jobs.

- On a tile: the star adds it to Favourites, **Hide** takes it off every shelf but Hidden, and the pencil edits it. Any style can be edited; your version takes the shipped one's place, and Reset to original brings the shipped text back.
- **Picture:** give a style a picture with **Paste** (or Ctrl+V over the tile), **Render** (one of this session's renders), **File**, or by dropping a file on the tile.
- A style reaches every model in the form it can read: a paragraph after the scene for Krea 2, FLUX, Qwen and Z; tags plus the paragraph for Anima; only tags for Illustrious, NoobAI and Pony. The panel says what it sends for the active model.

Wildcards

Type `{red dress|blue dress|green dress}` anywhere you can type tags, and one option is picked. Every render picks again; **Reroll Wildcards** under the prompt picks again right now. Word lists from files live on the Wildcards page.

CHAPTER 02 · SECTION 15

Saving and reusing a setup

A job belongs to one service. To reuse a picture idea across jobs, save it as a **prompt setup** from the ≡ menu in the page head: **Save** stores the characters, tags, description and settings; **Load…** brings a setup back. Once one is loaded you get **Update**, **History** and **Save as…**. **Copy from job…** copies another job's content into this one; **Reset** clears the Builder (it asks first).

CHAPTER 02 · SECTION 16

Going further

- **Lineup** (page head) compares candidates side by side: several seeds, checkpoints or workflows at once, at a cheap step count. Take the winner's setup into Tuning with one click. See the Lineup & Tuning chapter.
- **Repair** (page head) masks a part of the newest picture and has it redrawn. See the Canvas chapter.
- Other chapters cover Characters, Example Images, Workflows, Regional prompting and references, the Storyboard and AI Analysis.

CHAPTER 02 · SECTION 17

Questions and problems

I changed something in a stage, but the prompt doesn't change.

The prompt is probably frozen: click Freeze again. Also check the panel's eye: crossed out, the panel adds nothing. And under Character prompts, a figure you edited by hand stops following the settings until you reset it.

Some words are struck through in the prompt.

They were hidden with Hide and are not sent. Click Unhide, then the struck-through text.

The Render button does nothing, or CKPT says NONE.

Open 05 Machine, check that the Endpoint is running and pick a Checkpoint.

Every render looks the same.

The seed is locked. Unlock it next to the rungs.

My pictures aren't in the output folder.

Only Final and Custom save. Draft and Quality are for looking.

A pick in a character's settings is struck through and marked "held".

Something else writes that part at render time, for example the arousal stage replacing a genitals tag. The struck-through pick is kept, not lost; it applies again when the other setting lets go.

Two characters blend into one person.

Read the Cast may read alike note and give them more distinct features, raise their weights in the cast table, or use Regional prompting (Illustrious or Anima).

A Replace from Examples asked me what to keep.

Something in the Description is locked. Choose Keep mine or Take new per row; locks stay locked.

I edited the Description and want one part back.

Open the Areas view and click Original on that area. Back to original puts the whole text back.

On Anima or Krea 2, the end of my prompt seems ignored.

Both read at most 512 tokens. Switch on the token count in Output mode; use Fit, Essentials only, or Cast first (Anima) to bring the important part forward.

CHAPTER 03

Creating Characters

Build a character once, and use her in every picture. This chapter walks through the Characters page stage by stage, shows how to mark what makes her recognisable, how to check what she looks like, and how to put her into a picture.

— What a character is for 39
01 The Characters page at a glance 39
02 Create a character and set her identity 40
03 Body, face and clothes 41
04 Mark what makes her recognisable 42
05 Shortcut: fill a character from an existing prompt 43
06 LoRAs and model variants (optional) 44
07 Save 44
08 Check her with a test render 44
09 Put her into a picture 45
10 Tips and FAQ 45

What a character is for

A character is a saved description of one person: her face, hair, body, clothes and anything else that makes her recognisable. The app turns that description into a **character block**, a ready-made part of the prompt. When you add the character to a picture in the Prompt Builder, the block goes into the prompt for you. You don't retype her hair colour for every picture, and the same person looks the same across renders.

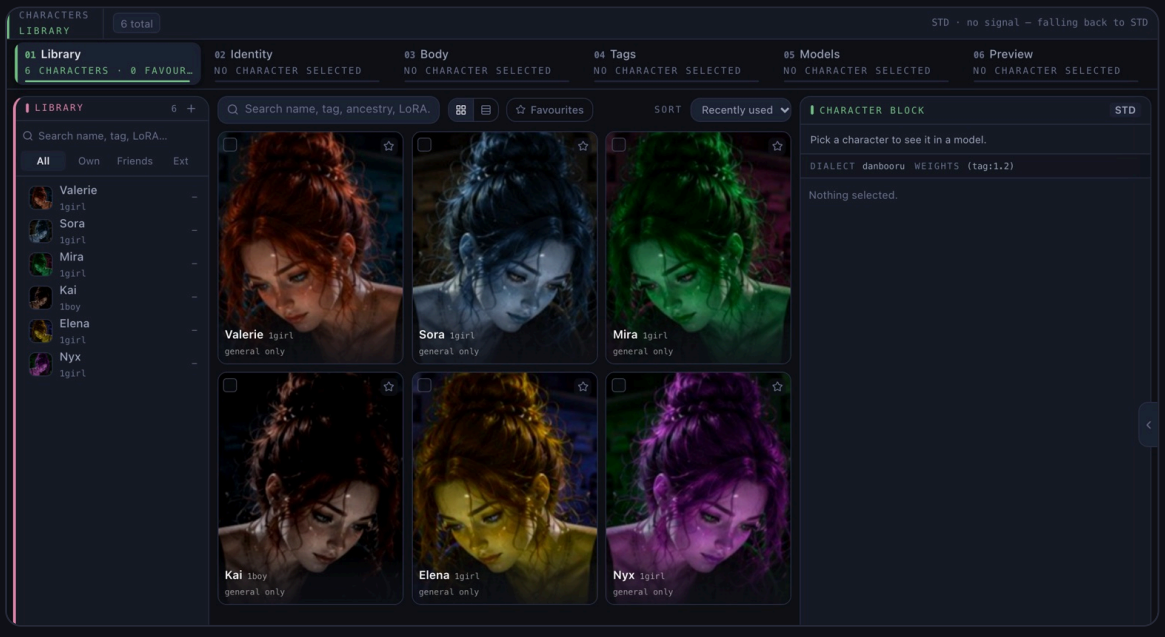
> **The stage order is the prompt order**
>
> The stages run in the same order the character block is written: Identity makes the opening line, Body the lines about the body and clothes, Tags the extra words. Reading the page top to bottom is reading the block top to bottom.

CHAPTER 03 · SECTION 01

The Characters page at a glance

Open **Characters** in the sidebar. The page has four parts that stay in place while you work:

- **Library** (left column): all your characters, with a search field and the filters All / Own / Friends / Ext. The **+** in its header creates a new character.
- **The stage rail** (top): 01 Library · 02 Identity · 03 Body · 04 Tags · 05 Models · 06 Preview. Each stage shows a short summary underneath.
- **The stage itself** (middle): the fields for the stage you picked.
- **Character Block** (right): the finished part of the prompt, exactly as the model will get it, in the dialect of the model named in its head. It updates while you type. Below it sits **Intake**, a quick way to fill a character from an existing prompt (section 5).

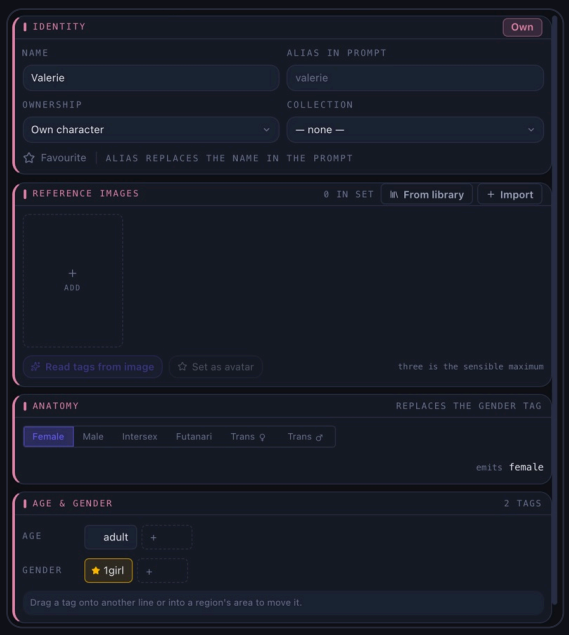


01 Library: every character as a card, with search, Favourites and sorting. Housekeeping at the bottom lists characters with unsaved changes or problems.

CHAPTER 03 · SECTION 02

Create a character and set her identity

1. Click the **+** in the header of the Library column. A character called *New Character* appears and is selected.
2. Go to **02 Identity**.



02 Identity: name and alias, ownership and collection, reference images, anatomy, age and gender, ancestry further down.

- **Name**: what you call her in the app. **Alias in prompt**: optional; if set, this word goes into the prompt instead of the name.
- **Ownership** (own, a friend's, third party) and **Collection** (main cast, side, retired) are for filtering. The star makes her a favourite.
- **Anatomy**: Female, Male, Intersex, Futanari, Trans ♀, Trans ♂. This writes the opening word of the block and replaces the gender tag; the rack shows what it emits.
- **Age & gender**: tags such as `adult` and `1girl`. While Anatomy is set, the gender line is marked suppressed and says why.
- **Ancestry** (further down): a region such as Japanese. It adds a small set of face tags; two switches turn the general or the specific half off.
- **Character type**: for non-human characters (ears, tail, wings). Leave it on Human for a normal person.

Mage.space character (optional)

If you render on Mage.space (with a Mage.space key under Settings → Rendering → Render Services), the **Mage.space character** rack links her to a character saved on Mage: **Create on Mage** makes one from her avatar or a reference picture, with a handle you choose; **Link existing** picks one already on your account. On Mage's Mango and Guava models she is then sent as `@handle`: hair, eyes, face and her other essentials come from that picture, clothing, pose and expression from here. **Unlink** forgets the link and deletes nothing on Mage. Unlinked, she is described from her tags there like everywhere else.

Reference images

The **Reference images** rack holds the pictures that stand for this character. Add them with **From library** or **Import**; three is the sensible maximum. **Set as avatar** uses the selected picture as her thumbnail. **Read tags from image** asks an AI to describe it as tags for you to read and copy; it writes nothing into the character.

CHAPTER 03 · SECTION 03

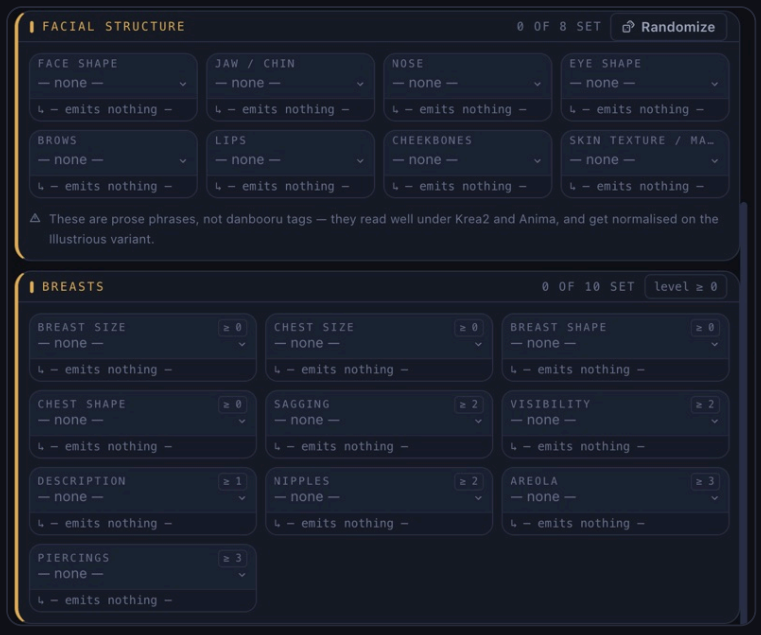
Body, face and clothes

Stage **03 Body** is where most of the description lives.

- **Visible area & content level** (top): a "what if" view. Pick a framing and a level (Clean, Suggestive, Revealing, Nude, Explicit), and every rack below shows whether its tags would reach the prompt. This only changes the view here; the real framing and level of a picture are set in the Prompt Builder.
- **Tag review**: **Check** looks through all of the character's tags for doubles, contradictions and tags in the wrong place; **Review with AI** does the same with an AI. Nothing changes until you press a fix.
- **Skin & build**: three lines of tags, Extras, Skin and General. Click the dashed **+** to add a tag.
- **Regions & areas**: the body is split into Head, Upper Body and Lower Body. Inside each, **Add area** and pick what it is (Hair, Eyes, Makeup, Clothing, Accessories, Feet ...). Each area has its own tags, an on/off switch and a weight. A Clothing area in Upper and one in Lower Body is her standard outfit.

> **Tags can be moved by dragging**
>
> Put a tag in the wrong place? Drag it onto another area, or onto a line of Skin & build. It keeps its weight and its on/off state. A tag you type is kept when you leave the field, Enter or not; Esc throws it away.

Facial structure, breasts and genitals: several tags per axis



Facial structure (eight axes) and Breasts. Each field says the content level it needs (≥ 2 and so on) and what it emits.

These racks are pickers over axes: face shape, jaw, nose, eye shape, brows, lips, cheekbones, skin texture; breast size, shape, nipples, areola and more; for genitals, the penis or vagina groups. **An axis can hold more than one entry.** The select adds a pick; every pick becomes its own row underneath, with its own × and, where the model reads weights, its own weight. An axis with nothing set shows *emits nothing*. **Randomize** fills the face at random.

Breasts and Genitals only reach the prompt when the picture's content level is high enough. Set nothing, and they add nothing.

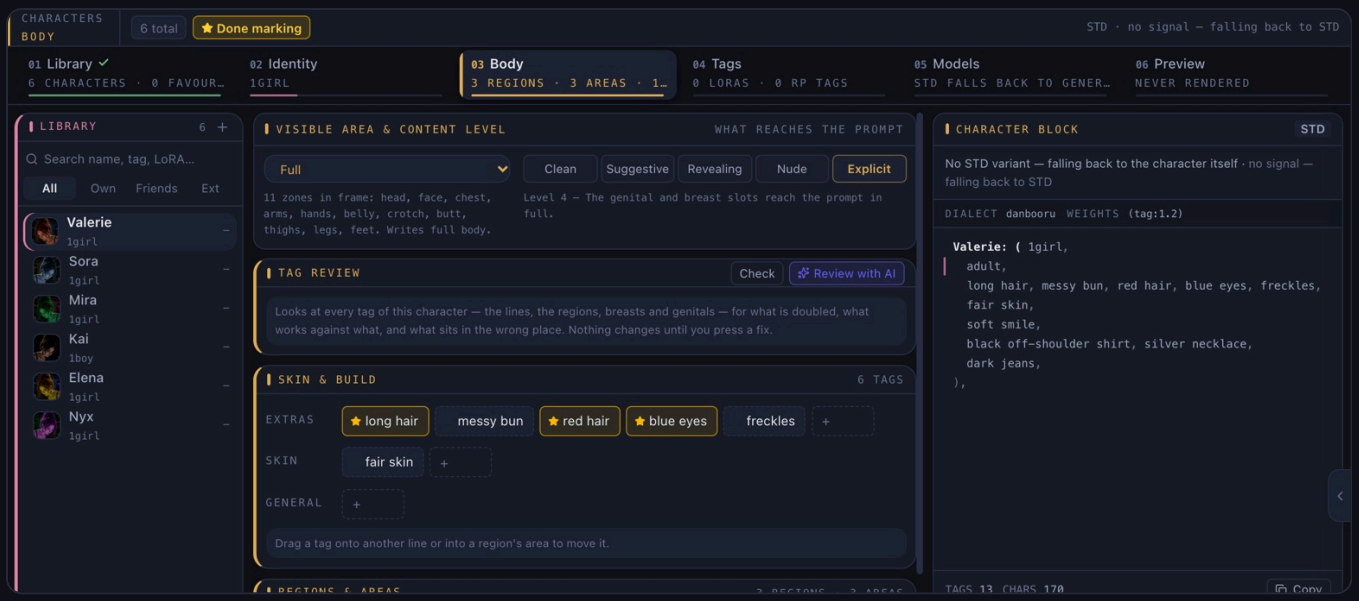
> **"held": a pick that something else writes**
>
> The arousal stage in the Builder can replace a few genital and nipple groups at render time. Where it does, the card shows your pick struck through with a **held** badge and names what writes it instead. Your pick is kept, not lost; it applies again when the stage lets go.

Free mixing is allowed: tags that seem to contradict each other can sit on one axis. Tag review only points things out. One exception the app makes on its own: when a figure's eyes are closed, the eye colour is left out of the prompt, because the lid hides it.

CHAPTER 03 · SECTION 04

Mark what makes her recognisable

Some tags carry a character: her hair colour and cut, her eye colour, her gender, her breasts and genitals, ears, tail, wings or horns. Those are her **essentials**, and they survive every cut the app makes: **Essentials only** in the Builder, the `!ch1Core` placeholder, and the AI's **Fit** to a token budget.



Mark essentials is on (the button now says Done marking). Starred chips are essential; the suggestion already starred long hair, red hair, blue eyes and 1girl.

1. Click **Mark essentials** at the top of the page. Every tag chip becomes a switch.
2. Click a tag to mark it essential, or to take the mark away. The app's suggestion is a starting point; it is never copied in, so changing it costs nothing.
3. Click **Done marking**, then save the character.

The star on a chip is also a switch on its own: click it to mark or unmark one tag without entering the mode. Marks belong to the character, so they hold in every workflow. Breasts and genitals being essential does not put them into every picture: the content level still decides whether they are written at all.

CHAPTER 03 · SECTION 05

Shortcut: fill a character from an existing prompt

Already have a prompt that describes her? The **Intake** rack under the Character Block sorts it for you.

1. Paste the prompt into the box. Weights, LoRA calls and BREAK are removed automatically.
2. Click **Parse**. The tags land in the **Pool**. Tags the app recognises get a destination straight away, for example "Head › Eyes". Quality tags like `masterpiece` are dropped.
3. Check each tag's destination and change it where it is wrong; *— nowhere —* leaves a tag out. **Classify with AI** asks an AI about the tags the app couldn't place.
4. Click **Assign**. The tags are written into the right racks; missing areas are created.

CHAPTER 03 · SECTION 06

LoRAs and model variants (optional)

You can skip this for your first character. It matters when you use a character LoRA, or render the same character with different base models.

- **04 Tags → Character LoRA, per base model**: a character LoRA is trained for one base model, so you pick it per model with **Choose LoRA…**. Doing so creates that model's variant. "No LoRA" is a valid answer: the tags then carry her identity. A LoRA set on the character as a whole shows as **General · flat lora**; pick the base model it was trained on in its row and click **Claim**, so only that model loads it and the others load none until they have their own answer.
- **Regional prompt tags**: extra tags used only while Regional Prompting is on in the Builder.
- **05 Models: one variant per base model.** Different base models understand different words. A variant changes a few fields for one model only, for example shorter tags for Illustrious and full sentences for Krea 2. Click **+ Create** on a model's card and change only what should differ; untouched fields follow the character. **Convert with AI** rewrites the fields for that model; **Only differences** hides the rest.

CHAPTER 03 · SECTION 07

Save

Changes are held in memory until you save. The page head then shows an amber **N unsaved** badge with a **Save** button; click it or press **Ctrl+S**. It saves every changed character at once. The status bar reads *all saved* when nothing is pending. Unsaved characters are also listed under Housekeeping on the Library stage.

CHAPTER 03 · SECTION 08

Check her with a test render

Stage **06 Preview** renders the character on her own, so you see whether the description works before you use her in a real picture. It needs ComfyUI connected and a workflow tagged **Character Preview**; the app ships them for Illustrious, Anima and Krea 2.

1. Pick a framing (Portrait / Full body / Sheet) and a background (Light / Dark), then click **Render**. The Preview render window opens.
2. On the left you keep editing the character while you watch the results; **Save character** saves those edits.
3. On the right: framing, background, an optional expression, quality tags, the workflow and backend, and the checkpoint (the workflow's own unless you pick one from your library). **Render**; **Stop** cancels. While rendering on your local ComfyUI API you watch the picture form.

4. Like a result? **Keep** files it under the character, sorted by the model that made it. **Use as Avatar** makes it her thumbnail.

> **Closing without saving undoes your edits**
> The ✕ closes the window and puts the character back the way she was when you opened it, unless you clicked Save character first. Saving does not close the window.

Back on the Preview stage, **Kept renders** shows everything you kept, and **Consistency check** lets an AI look for contradictions in the block.

CHAPTER 03 · SECTION 09

Put her into a picture

1. Open **Current Job** and click the stage **02 Cast**.
2. In **Characters & Areas**, click **+ Add character**. A searchable list of your library opens.
3. Click her name. She appears as a card, and her block is in The Prompt on the right.

Open the card (the arrow on its right) to change things for this picture only: clothing, hair, makeup, accessories, expressions and motions have pickers there, and **Vibe** sets her mood and bearing. These changes belong to the job; the character in your library stays as she is.

Several characters work the same way: add each one. Their order, what each one shows and how strongly each is read are set in one table, **06 Polish → Character Settings** (see the Prompt Builder chapter).

CHAPTER 03 · SECTION 10

Tips and FAQ

Where do I see what my character actually sends to the model?

In the Character Block on the right of the Characters page. Its head names the model's dialect; Copy copies it.

A tag I set doesn't appear in the block.

Check the area's on/off switch, the Visible area & content level dials on 03 Body (a framing can leave a region out), and whether the gender line is suppressed because Anatomy is set. Racks silenced by the dials say why.

"Essentials only" cut something I need.

Mark it: Mark essentials, click the tag (or its star), save.

The Render button on 06 Preview is greyed out.

ComfyUI is not selected in Settings → Output Integration, or no Character Preview workflow is available. See the Getting Started chapter.

I changed her outfit in the Builder; is the library version different now?

No. Changes on the Builder card stay with the job. To change her standard outfit, edit the Clothing areas on 03 Body and save.

What does "falling back to the character itself" mean?

The selected model has no variant, so it uses the character's normal fields. That's fine; variants are optional.

CHAPTER 04

Model Library: checkpoints, LoRAs and recipes

Every model file on your disk, turned into an entry with a name, a picture and the numbers that make it look good. The Prompt Builder reads those numbers, so you set them once instead of every day.

– What the library is for 48
01 Before you start: tell the app where your models are 48
02 Bring your files in 49
03 Find your way around the list 50
04 Set up a checkpoint 50
05 Set up a LoRA 52
06 Find and download on Civitai 53
07 Recipes: LoRA combinations in one click 54
08 How cheap is a cheap render: draft steps 56
09 Using the library in a job 57
10 Keeping the library tidy 59
11 Tips 60
12 Troubleshooting 61

What the library is for

A model file on its own is just a name like `waiIllustriousSDXL_v150.safetensors`. The **Model Library** turns that file into something you can work with: a title, the author, the base model it belongs to, a preview picture, and above all the numbers that make it look good: steps, CFG, sampler and scheduler for a checkpoint, a weight range and trigger words for a LoRA.

The rest of the app reads all of this. When you pick a checkpoint in a job, its steps and CFG, its base prompt and its negative prompt come along. When you switch on a LoRA, its trigger words come along and its weight slider stays inside its range. An hour spent here saves you retyping the same settings every day.

Every card shows these four rungs as a thin bar along its bottom edge.
An entry can be used at any rung; each rung makes it more useful.

The four tabs

Tab	What lives there
01 LoRAs	Small add-on files that add a style, a character, clothing, a pose or more detail. Each has trigger words and a weight range.
02 Checkpoints	The main image models. Each has a range and a sweet spot for steps and CFG, preferred samplers and schedulers, a draft step count, and optionally a base prompt and negative prompt.
03 Recipes	Saved combinations of LoRAs with their weights, switched on together with one click.
04 Civitai	Find LoRAs and checkpoints on Civitai by base model and download them straight into the library (section 6).

The first three tabs work the same way: a search field and sort buttons at the top, **Facets** on the left to narrow the list, and the cards grouped by base model (switch that with **Group: Base model / None** at the top right).

CHAPTER 04 · SECTION 01

Before you start: tell the app where your models are

The library is filled from folders on your disk. If you followed the *Getting Started* chapter, this is already done. Otherwise open **Settings** → **Models & sources** → **Model Folders** and add your `checkpoints` and `loras` folders (or click **Use ComfyUI root**, which fills them in from your ComfyUI install). The category

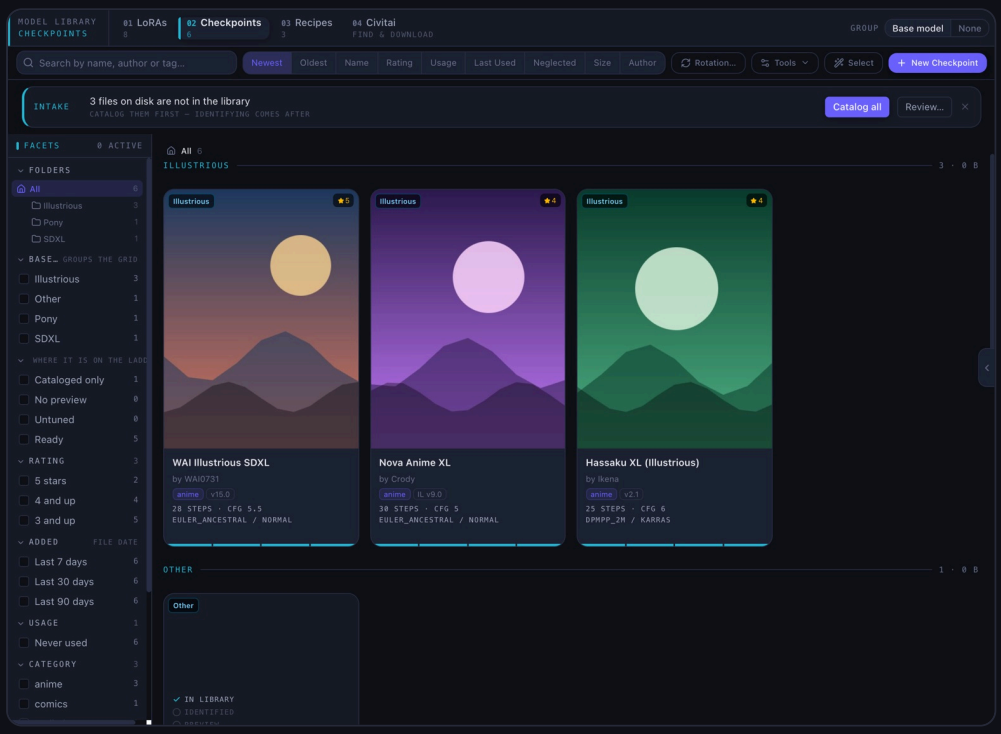
names must be exactly `checkpoints` and `loras`. Choosing your ComfyUI folder in **Settings** → **Rendering** fills these in by itself, including a models folder outside ComfyUI (see the Getting Started chapter).

The app indexes these folders a few seconds after it starts, and compares them with the library each time you open the LoRAs or Checkpoints tab.

CHAPTER 04 · SECTION 02

Bring your files in

When the folders and the library disagree, a bar appears above the cards. It always shows the most pressing of three stages:



The Checkpoints tab. The **Intake** bar says three files on disk are not in the library yet. The card under *Other* knows nothing but its filename; its checklist (bottom) shows which rungs are still missing.

1. **Intake**: "*N files on disk are not in the library*". Click **Catalog all**: every new file gets an entry, named after its file. Nothing is looked up online yet; this step only lists the files. Prefer to choose? **Review...** shows the files one by one.
2. **Setup**: "*N checkpoints know nothing but their filename*". Click **Set them up**. The grid narrows to exactly those entries and a queue walks you through them one after another.
3. **Repair**: "*N entries have no file on disk*". Click **Repair paths**. A file that moved is re-linked; one that is gone can't be.

The ✕ dismisses the bar until the numbers change.

> **The bar isn't there?**
> Then there's nothing new. You can always check by hand: **Tools → Rescan folders** or **Orphan files…** (see section 10).

CHAPTER 04 · SECTION 03

Find your way around the list

The **Facets** column on the left filters the cards; the count at its top says how many facets are active. The most useful group is **Setup — where it is on the ladder**:

- **Cataloged only**: entries that know nothing but their filename. Start here.
- **No preview**: entries without a picture.
- **Untuned**: entries whose numbers were never set (checkpoints still on placeholder values, LoRAs without a weight range).
- **Ready**: entries that have climbed all four rungs.

Further groups narrow by **Folders**, **Base model**, **Rating**, **Added** (by file date), **Usage** (*Never used*), **Category**, **Author**, **Tags** and **Scope**, which separates *Local* files from *Cloud only* entries. **Show only → Incomplete** collects every entry with a gap. The sort buttons at the top (*Newest, Oldest, Name, Rating, Usage, Last Used, Neglected, Size, Author*) change the order; *Neglected* weighs how rarely you use a model against how long you've had it.

Select switches the grid into multi-select for bulk edits and deletes. On the Checkpoints tab, **Rotation…** lets you pick several checkpoints and **Send to Rotation**, for the checkpoint rotation node of the ComfyUI (Local) extension path.

CHAPTER 04 · SECTION 04

Set up a checkpoint

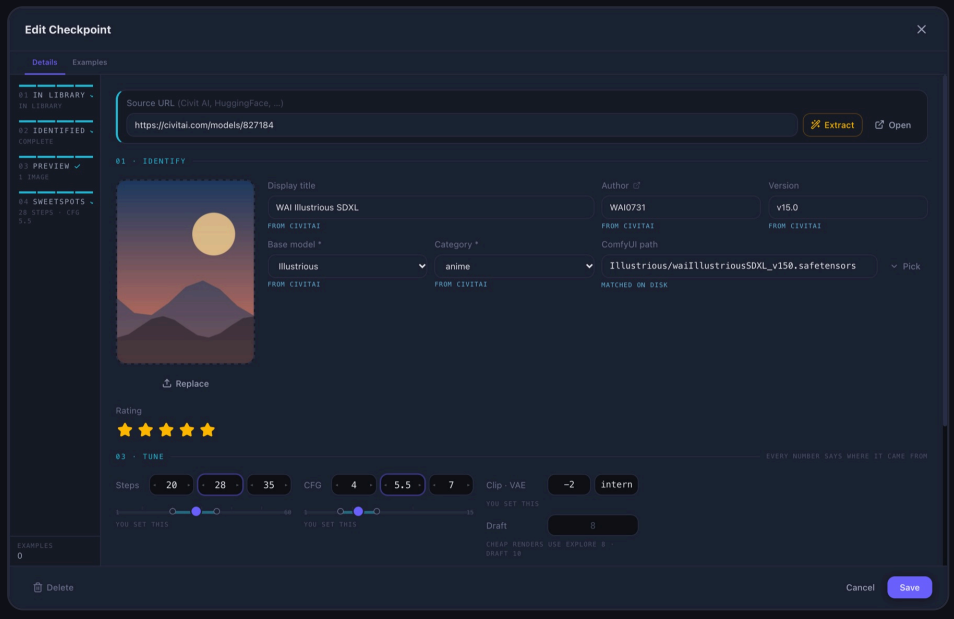
Click a card to open **Edit Checkpoint**. The rail on the left shows the four rungs and what is still missing; click a rung to jump to its part of the form. The form reads from top to bottom.

Fill it in automatically

1. Paste the model's Civitai page into **Source URL** and click **Extract**. The app reads title, author, version, base model, category, recommended settings and prompts, and lists them with pictures to choose from. Take rows one by one, or all at once with **Apply All**. Where the page leaves numbers out, **✨ AI Enhance** proposes them from the description (and from example prompts, once loaded). A `metadata.json` next to the file is read by Extract too, even without a URL.
   No URL yet? **DB Lookup** finds the model by the fingerprint (hash) of its file, and **Search Civitai**

searches by name. Both appear as long as there's no Civitai URL in the field. **Open** opens any URL, Hugging Face included, in your browser.

2. Check **Base model** and **Category**. They are required, and the base model decides which rules the Builder writes the prompt in.
3. **ComfyUI path** must match the file exactly as ComfyUI sees it, folders included. **Pick** offers the files your ComfyUI actually has. *MATCHED ON DISK* means it's right.
4. Add a picture (**Replace** once there is one) and give it a **Rating**.

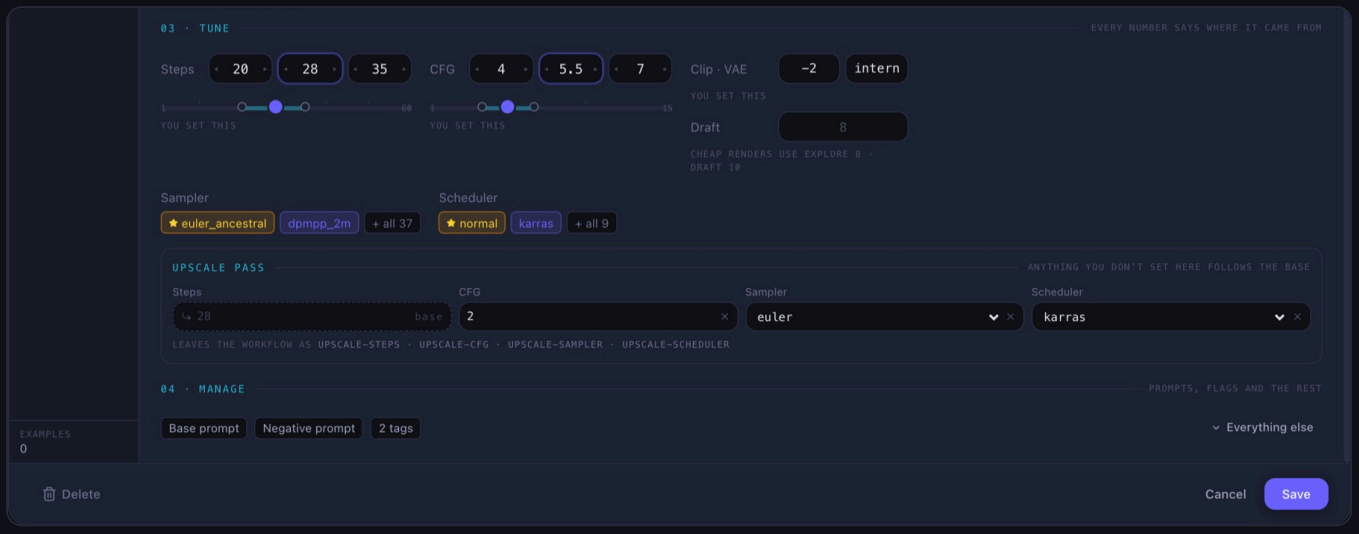


Edit Checkpoint. Under each field a small note says where the value came from: **FROM CIVITAI**, **YOU SET THIS**, **MATCHED ON DISK**.

The **Examples** tab at the top shows the model's example pictures from Civitai with the prompts they were made with (**Load Civitai examples**).

> **Extract talks to the internet**
> Extract, DB Lookup, Search Civitai and the Examples tab reach Civitai. For NSFW model pages and a higher rate limit, add a key in **Settings → Models & sources → CivitAI Integration**. AI Enhance needs an AI provider key.

03 · Tune: the numbers that make it look good



The Tune section. Each range is *min · sweet spot · max*; the highlighted middle value is the one a job starts with.

- **Steps** and **CFG**: three numbers each, the lowest, the best (the **sweet spot**) and the highest value that works. The Builder keeps you inside the range unless you switch **Override** on.
- **Clip · VAE**: clip skip and which VAE to use. Leave them if the model page doesn't say otherwise.
- **Draft**: how many steps cheap renders use with this model: Explore, the Lineup's cells and Tuning's Draft rung. Empty means it follows the counts in Settings (section 8); the note under it says what that is right now. Set it lower for turbo and lightning models.
- **Sampler** and **Scheduler**: the ones this model likes. The one with the ★ is the default; **+ all** opens the full list.
- **Upscale pass**: steps, CFG, sampler and scheduler for a workflow's upscale step. A dashed cell with ↳ follows the base value and shows it; set a value to take it over, ✕ gives it back. Each of the four follows the base on its own.
- **Your renders** appears once you have rendered with this model: it shows what you actually used, and **Take these** copies those values into the form. Nothing is written without that click.

04 · Manage

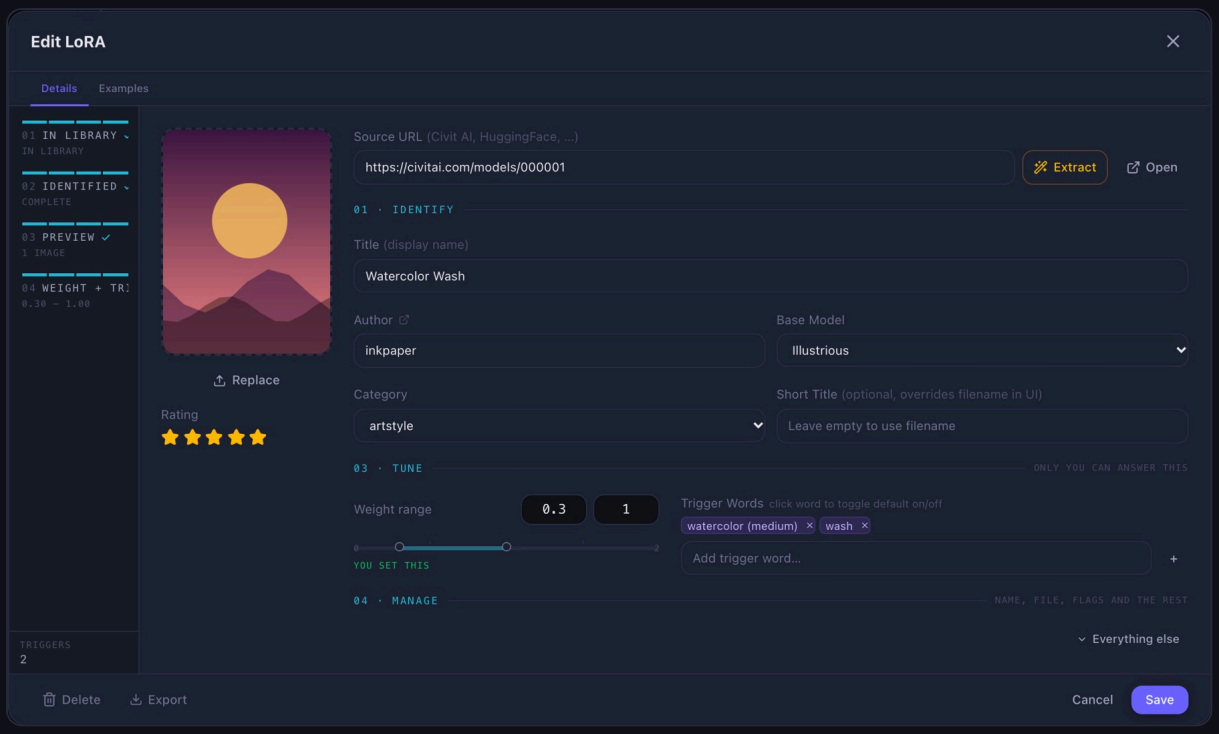
Base prompt and **Negative prompt** are words this checkpoint always needs, such as `masterpiece, best quality`. When you pick the checkpoint in a job, the base prompt goes in front of your prompt and the negative prompt becomes the job's negative. **Everything else** holds notes, tags, the Cloud path and flags such as *Metadata only* or *Flagged for deletion*.

Click **Save** (or Ctrl+S). Closing with unsaved changes asks *Discard your changes?* first.

CHAPTER 04 · SECTION 05

Set up a LoRA

A LoRA entry works like a checkpoint entry: the same rail, Source URL and Extract at the top, the identity fields, then its own **Tune** section.



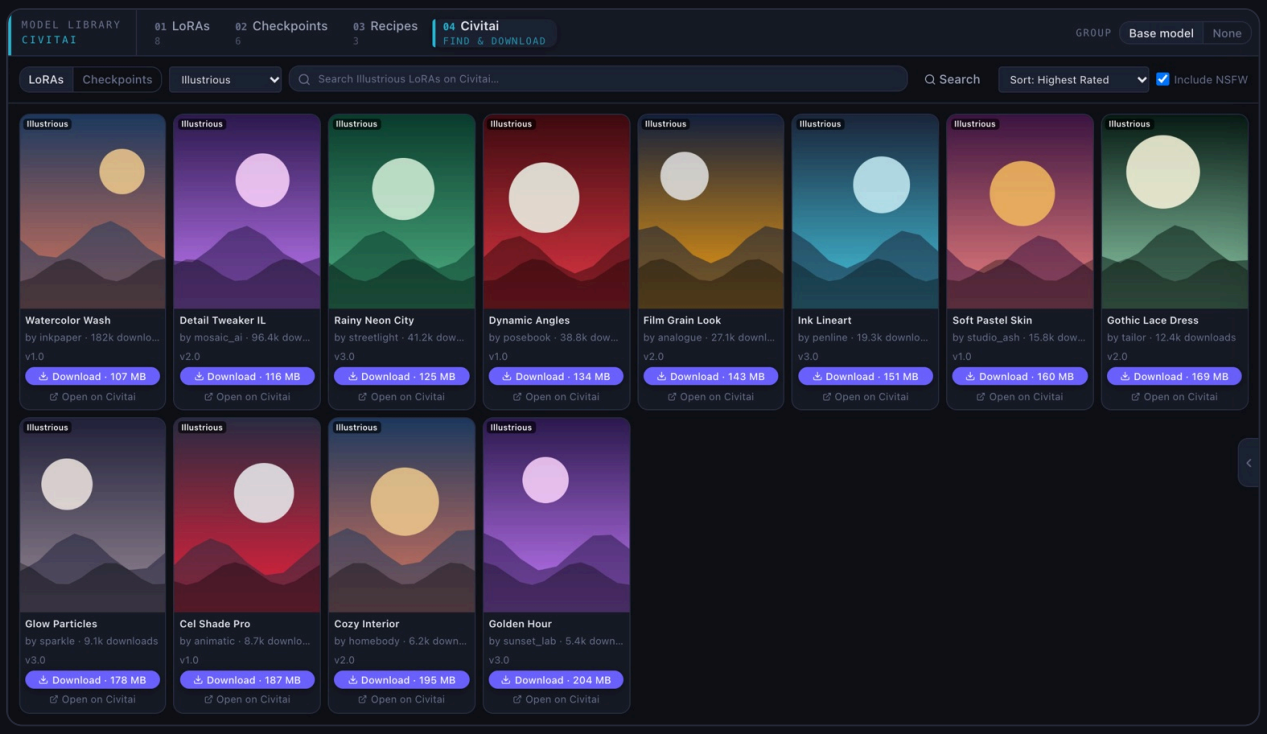
Edit LoRA. The fourth rung of a LoRA is **Weight + triggers**: the Tune section holds the weight range and the trigger words.

- **Weight range**: the lowest and highest weight that makes sense for this LoRA; negative values are allowed, for a LoRA that works backwards. It is the one thing no website can tell the app ("only you can answer this"), and every weight slider in the app (job, recipes, character LoRAs) keeps exactly this range. Without one, the job's sliders run from −5 to +5.
- **Trigger Words**: words the LoRA was trained on. When the LoRA is switched on, they are added to the prompt. *Click a word* to switch whether it is on by default; ✕ removes it, the field below adds one.
- **Short Title**: a shorter name. Everywhere a LoRA is listed (the job's racks, the session, the trigger words, the Recent strip) it shows by its short title, else its library title, else its file name; the file path is in the tooltip.
- Under **Everything else** (04 · Manage): the **ComfyUI Path** (relative to `models\loras\`), **External** for LoRAs that exist only on ComfyUI Cloud, **Hidden**, and the archive and deletion flags. **Export** at the bottom saves the entry on its own.

CHAPTER 04 · SECTION 06

Find and download on Civitai

The **04 Civitai** tab searches Civitai from inside the app. Pick **LoRAs** or **Checkpoints**, and the **base model** first: it starts on the one you are working with (a fixed job's family, else the loaded checkpoint's) and every search is filtered by it, down to the versions offered per model. Type a search and press **Search**, choose a **Sort** (Highest Rated, Most Downloaded, Newest) and whether to **Include NSFW**.



The Civitai tab, LoRAs for Illustrious. Each tile has its version, **Download** with the file size, and **Open on Civitai**. The pictures here are demo stand-ins.

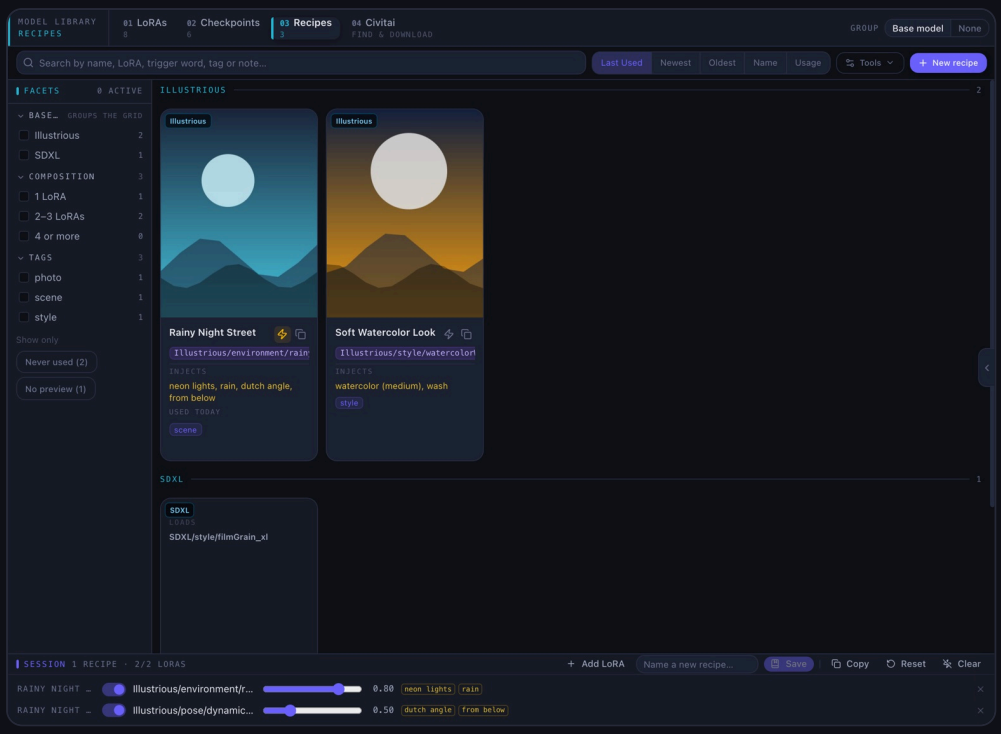
- A click on a tile opens the model: its pictures, the version to take, its **Trigger words** (with Copy), the description and tags.
- **Download** puts the file into your LoRA or checkpoint folder, under `<base model>\<category>`, and makes it a complete library entry, hashed like any other. A download keeps running when you leave the tab; a failed one says why, with **Try again**.
- A model you already have shows **In library** (matched by the file's fingerprint, else the Civitai version).
- The same browser is the **Civitai** side of the job's **Add LoRAs** window (section 9). There a finished download is picked for adding at once; a LoRA already in the library offers **Pick to add**.

For NSFW results and a higher rate limit, add your key in **Settings → Models & sources → CivitAI Integration**.

CHAPTER 04 · SECTION 07

Recipes: LoRA combinations in one click

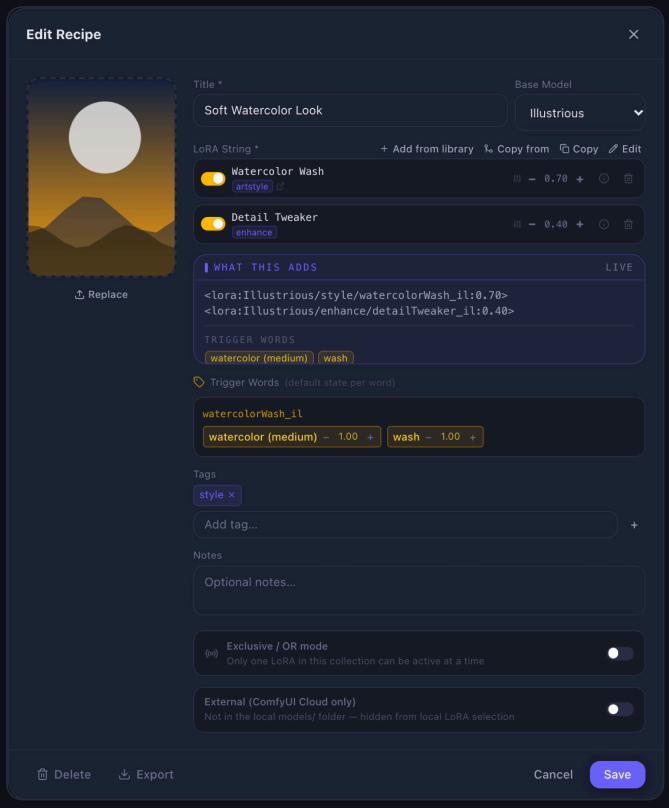
A recipe is a named set of LoRAs with their weights, for example a style LoRA at 0.7 plus a detail LoRA at 0.4. Instead of switching on and weighing every LoRA each time, you switch on the recipe.



The Recipes tab. Each card lists what it loads and which trigger words it **Injects**. The **Session** strip at the bottom shows what is switched on right now, and whether it fits the loaded checkpoint.

1. Click **New recipe**, give it a **Title** and **Base Model**, and add LoRAs with **Add from library**. Set each weight with – and +, switch a LoRA off without removing it. **What this adds** shows the resulting LoRA string and trigger words live.
2. Click the lightning button on a card (*Add to active session*) to switch the recipe on. Its LoRAs appear in the **Session** strip, each with an on/off switch, a weight slider and its trigger words.
3. Found a combination you like in the strip? Type a name into **Name a new recipe...** and click **Save**. **Add LoRA** adds a single LoRA, **Copy** copies the LoRA string, **Reset** puts every weight back to the recipe's own, **Clear** empties the session.

The Recipes facets add **Composition** (how many LoRAs) and, under **Show only**, *Not for <checkpoint>*: recipes whose base model doesn't match the loaded checkpoint. Such a recipe still loads, but it isn't what it promises.



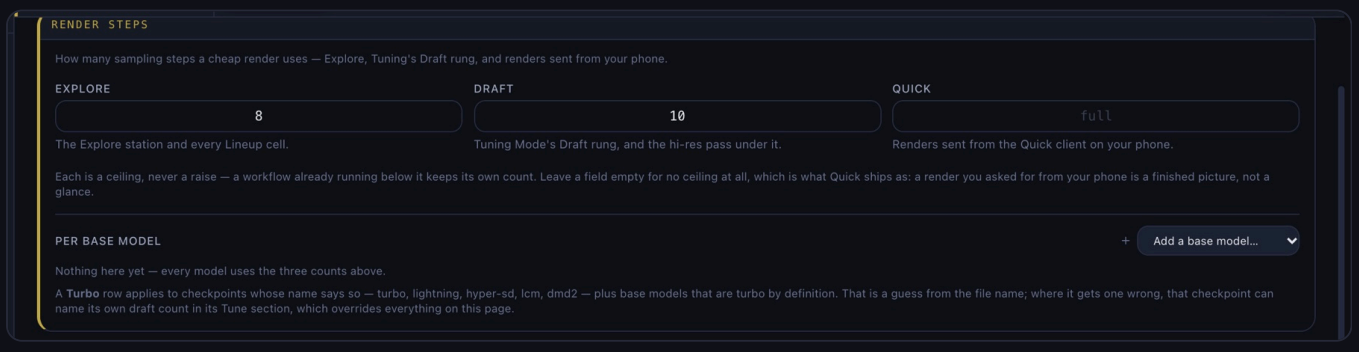
Edit Recipe, with the two switches at the bottom.

- **Exclusive / OR mode**: only one LoRA of this recipe can be active at a time. Useful for a set of alternative styles.
- **External (ComfyUI Cloud only)**: the LoRAs aren't in your local `models` folder, so the recipe is hidden from local LoRA selection.

CHAPTER 04 · SECTION 08

How cheap is a cheap render: draft steps

Exploring seeds and comparing candidates doesn't need full quality, so the app caps the step count for those renders. The counts live in **Settings** → **Rendering** → **Render Steps**:



Render Steps. Three counts, then exceptions per base model.

- **Explore** (default 8): the Explore station and every Lineup cell. **Draft** (default 10): Tuning's Draft rung and the hi-res pass under it. **Quick** (empty): renders sent from the Quick client on your phone.

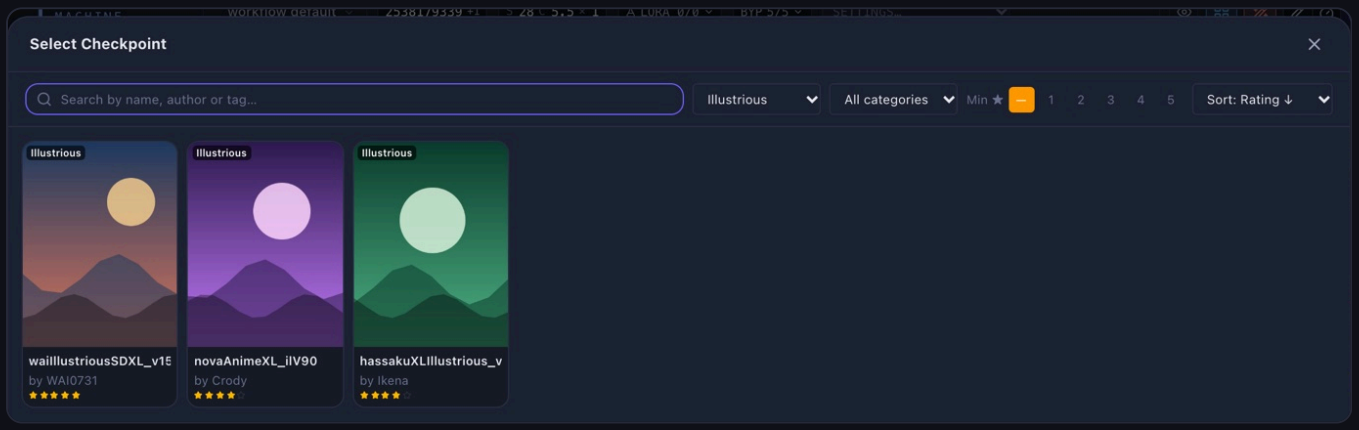
- Each count is a **ceiling, never a raise**: a workflow already sampling below it keeps its own count. Empty means no ceiling.
- **Per base model**: **Add a base model...** adds a row with its own counts; a blank cell inherits the count above. A **Turbo** row applies to checkpoints whose file name says so (turbo, lightning, hyper-sd, lcm, dmd2).

For a single checkpoint, the **Draft** field in its Tune section wins over all of this. The order is: the checkpoint's own count, then its base model's Turbo row (if the name looks like a fast model), then its base model's row, then the global count.

CHAPTER 04 · SECTION 09

Using the library in a job

A job on **ComfyUI API (local)** or **ComfyUI Cloud** is fixed to a base model when you create it (**New setup** asks for one). From then on the job only offers that family: its workflows, its checkpoints, its LoRAs. Go to stage **05 Machine** and click the field in the **Checkpoint** rack. The **Select Checkpoint** window lists your library.



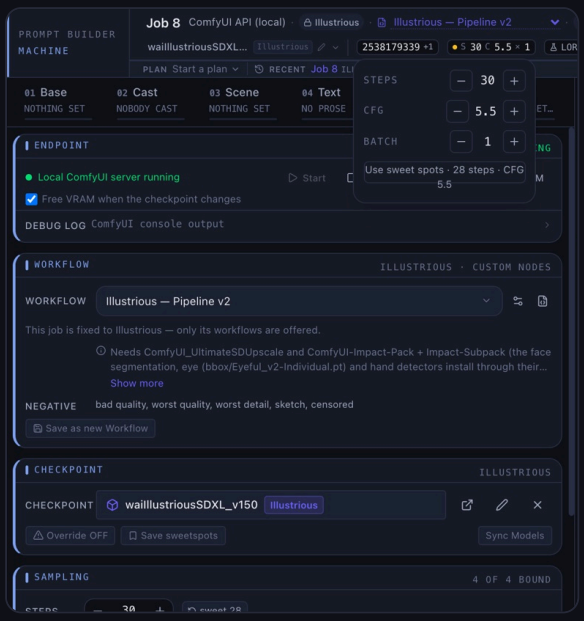
Select Checkpoint in a job fixed to Illustrious. The base-model filter is locked to that family; category, minimum rating and sort are free. The pencil on a tile opens its checkpoint form.

The same picker opens wherever a checkpoint is chosen (the Lineup, the Canvas, the character preview). Outside a fixed job, the base-model filter is preset but can be changed. Its filters are remembered while the app runs.

After you pick one, its library entry takes over:

- **Sweet spots.** In the **Sampling** rack, a green check means a value sits on the checkpoint's sweet spot; off it, a button like **sweet 28** sets it back with one click. In the head, the **S · C · ×** pill marks a value off the sweet spot with a dot; hover it and **Use sweet spots** sets steps and CFG at once.
- **Save sweetspots** saves the steps, CFG, sampler and scheduler you are using now as this checkpoint's own. The original values are kept, so you can go back to them in the checkpoint form.
- **Override** ignores the checkpoint's min/max range, for when you deliberately want to try values outside it.

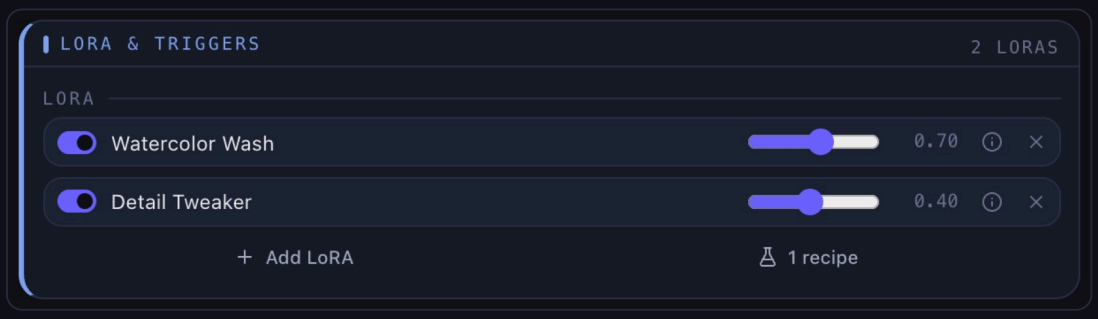
- **Sync Models** (local only) compares your checkpoint and LoRA libraries with what ComfyUI actually has.
- Beside the field: open the model's source page, the pencil to edit the checkpoint, and ✕ to go back to the workflow's own model. Once you have used two or more checkpoints of this base model, the last twelve sit under the field as small thumbnails, for switching back and forth.



After picking WAI Illustrious. The prompt starts with the checkpoint's base prompt, the workflow's negative is its negative prompt, and Steps and CFG start at its sweet spots. Steps was moved to 30 here: the pill in the head carries an amber dot, and its popover offers **Use sweet spots**.

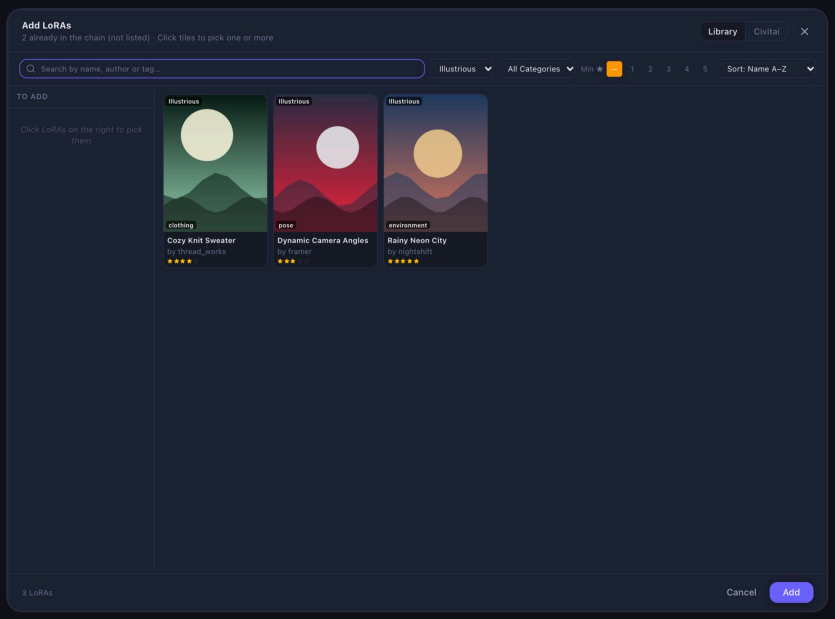
LoRAs in the job

The **LoRA** rack (in **LoRA & triggers**, and on ComfyUI API in the workflow's LoRA rack) lists every LoRA of the session as a row, by its name: a switch, a weight slider, the weight as a number you can type, an ⓘ that opens its library entry (name, trigger words, strength range), and ✕ to take it out. The slider keeps exactly the LoRA's own weight range; a stored weight outside that range shows amber. Hover the name to see the LoRA's preview picture. A LoRA the workflow's own graph loads cannot be removed here.



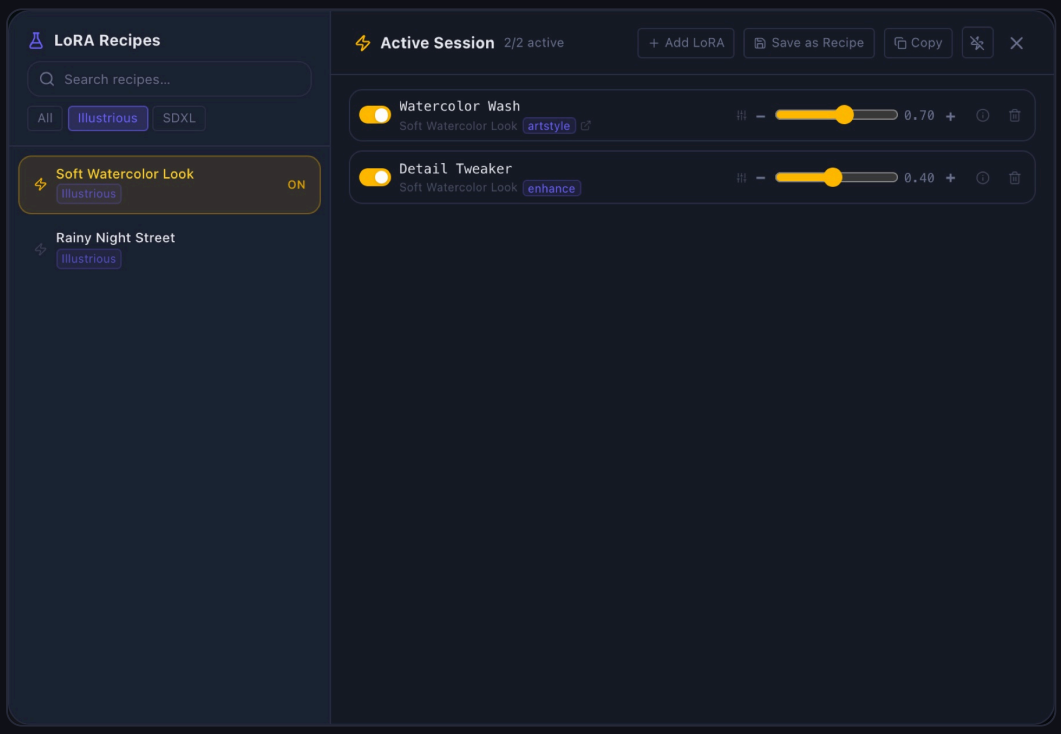
A recipe switched on: its two LoRAs as rows, each with its switch, a slider bounded by the LoRA's weight range, the weight, ⓘ and ✕. Below: **Add LoRA** and the recipes button.

- **Add LoRA** opens **Add LoRAs**, which works like the checkpoint picker: one row of filters, tiles with base model, stars and a pencil for the LoRA's form, and a **To add** column for what you picked. The **Library | Civitai** switch at the top finds a LoRA on Civitai instead (section 6). Not offered for LoRAs that live on ComfyUI Cloud only.



Add LoRAs in a job fixed to Illustrious. Click tiles to pick them into **To add**, then **Add**. **Library | Civitai** at the top right.

- **Recipes** (**Pick recipes...** while the rack is empty) opens **LoRA Recipes**: recipes on the left, filtered by base model, the **Active Session** on the right, with **Add LoRA** and **Save as Recipe**.

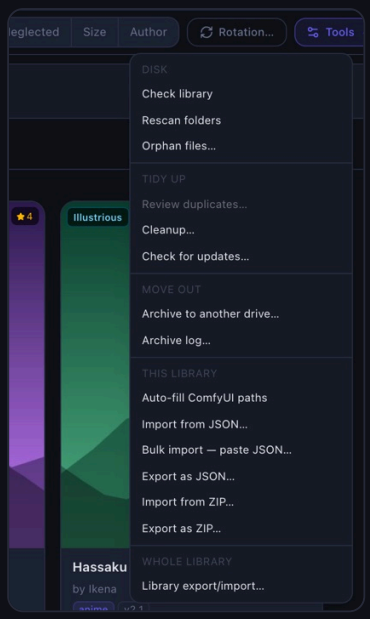


A recipe switched on: its two LoRAs, each with a switch and a weight slider bounded by the LoRA's weight range. A LoRA the workflow itself loads is listed on top, "from workflow".

CHAPTER 04 · SECTION 10

Keeping the library tidy

The **Tools** menu on each tab collects the maintenance jobs.



Tools on the Checkpoints tab.

Group	What it does
Disk	**Check library** compares every entry with your folders. **Rescan folders** indexes them again. **Orphan files...** lists files not yet in the library.
Tidy up	**Review duplicates...** and **Cleanup...** find older versions and duplicates so you can free disk space. **Check for updates...** asks Civitai whether a newer version exists.
Move out	**Archive to another drive...** moves files you rarely use to another disk; their entries leave the library. **Archive log...** tells you where they went.
This library	**Auto-fill ComfyUI paths** matches all checkpoints with ComfyUI's own list (needs ComfyUI running). **Import from JSON...**, **Bulk import — paste JSON...**, **Export as JSON...**, and **Import/Export as ZIP...** (ZIP includes the pictures).
Whole library	**Library export/import...**: checkpoints, LoRAs and recipes at once.

CHAPTER 04 · SECTION 11

Tips

- **Work down the ladder.** Catalog everything, click **Set them up**, and let Extract or DB Lookup do most of the typing. Then set weight ranges and sweet spots for the models you really use.
- **Let your renders teach the library.** Render a few pictures, then open the checkpoint and press **Take these**, or press **Save sweetspots** in the job when a setting works.
- **Set a Draft count for fast models.** A turbo or lightning checkpoint says what it has to say in far fewer steps than the shared count. If the file name gives it away, a Turbo row in Render Steps covers all of them at once.

- **Keep trigger words honest.** Switch off by default the words you don't want in every prompt; you can still switch them on per session.
- **Group by base model.** A LoRA made for Illustrious rarely works on Pony or SDXL. The grouping, the base-model badge and the *Not for* facet make that visible.

CHAPTER 04 · SECTION 12

Troubleshooting

The library is empty although my models are in ComfyUI.

No model folders are set, or they point somewhere else. Check **Settings → Models & sources → Model Folders**, then **Tools → Rescan folders**.

A render fails with "Value not in list".

The **ComfyUI path** doesn't match the file ComfyUI has. Use **Pick** in the checkpoint form, or **Tools → Auto-fill ComfyUI paths** for all checkpoints at once.

The picker in my job doesn't show a checkpoint I have.

The job is fixed to another base model, and the picker only offers that family. Check the checkpoint's **Base model** in its form, or start a new job for the other family.

Extract finds nothing.

Check that the Source URL is the model's page (`civitai.com/models/…`). Some pages need a Civitai key. Without a URL, try **DB Lookup** or **Search Civitai**.

A LoRA does nothing.

Check its switch and weight in **LoRA & triggers**, that its base model matches the checkpoint, and that its trigger words are in the prompt.

A card says a file is missing.

The file was moved or deleted. **Repair paths** in the bar re-links a moved file. A deleted one can be removed from the library.

The weight slider stops at an odd value.

It follows the LoRA's **Weight range** exactly. Widen the range in the LoRA's form (the ⓘ on its row opens it).

Cheap renders look unfinished with one model only.

Its draft count is too low for it. Raise the **Draft** field in its Tune section, or its base model's row in Render Steps.

CHAPTER 05

Bring your own ComfyUI workflows

How to get a workflow you built in ComfyUI into Prompting Helper, connect it to the app's fields, and decide which parts run for a quick look and which only for the final picture.

01 Prepare the workflow in ComfyUI 65
02 Bring it into the app 66
03 Describe it 66
04 Connect it: Nodes & Bindings 67
05 Make stages switchable 68
06 Decide what runs when: the Modes matrix 70
07 Setting presets: named sets of values 70
08 Every stage's picture 71
09 How many steps a quick look uses 72
10 Share what it needs: Resources 72
11 Let the AI help (optional) 72
12 Troubleshooting 73

What this is for

The app ships with ready-made workflows, and for most pictures they are enough. Once you want **your own** pipeline, for example a hires pass, a model upscale or a face fix, you build it in ComfyUI and bring it here. From then on the app fills in the prompt, the checkpoint, the seed and the size for you on every render, and the workflow shows up in the **Workflow** picker under **05 Machine** in the Prompt Builder.

> **Ready-made pipelines come set up**
> Besides the small **Character Preview** starters, the app ships **Core** pipelines for Illustrious, Anima and Krea 2: hires fix, model upscale, face and hand detailers and a SeedVR2 clean-up, with their stages already marked so Draft and Quality stay quick and the expensive passes run at Final, and with **setting presets** (section 7) that say when to take which. They are marked **Core** on their cards; you can use them as they are or copy one as the start of your own. Every node pack and model file they need besides your checkpoint and LoRAs comes with the one-click setup in **Settings → Rendering** (see the Getting Started chapter).

Which Core pipeline is which

A Core pipeline is named by model, purpose and version: *Illustrious — Pipeline v2*, *Anima — Pipeline v1*. A newer generation gets a higher number beside the old one, so the highest number is the current one; the stages it runs are in the description under the name.

Pipeline	Take it for
Krea 2 — Landscape / Abstract v1	Scenes without people: upscale, then a light detail pass over the whole picture. Its upscale prompt is always the scene text.
Krea 2 — Characters v2	Figures: a gentle upscale that keeps the expression, a face pass and an eye pass at Final. The hand pass is switched off until you have tried it.
Anima — Pipeline v1, **Illustrious — Pipeline v2**	Their models' full pipelines, with face and hand detailers; Illustrious also redraws the eyes at Final.
… · Comfy Cloud	The same pipeline for ComfyUI Cloud, which installs no custom nodes and has its own model list: what Cloud lacks is taken out or swapped. Use the plain one on your own ComfyUI or a rented one.
SeedVR2 — Enlarge v1	A free local way to 4K for the Enlarge page: the picture is enlarged, then SeedVR2 redraws the small structure sharp without inventing anything.

The **SeedVR2** clean-up stage in the Core pipelines is off at every rung until its models are installed; a missing model would fail the whole render. Switch it on in the Modes matrix once the setup has downloaded them.

Setting up a workflow of your own happens in four steps. The app tracks them on every workflow card:

The card's button always names the next step: "Fix the JSON", "Describe it", "Bind panel fields", "Run it once".

What you need

ComfyUI installed and connected (see the *Getting Started* chapter). Some steps talk to ComfyUI directly: converting a normal ComfyUI save and **Check workflow** only work while ComfyUI is running. The AI buttons (**Build with AI**, **Ask AI**, **Extend with AI**, **Describe**) only appear once an AI provider key is set in **Settings → AI**.

CHAPTER 05 · SECTION 01

Prepare the workflow in ComfyUI

This is the one part the app can't do for you, and it saves the most time later. Before you export, give every node you care about a title that starts with a short **stage marker** and a colon:

```
BASE: KSampler        HR: Upscale Latent        US: Upscale Image        FD: FaceDetailer
```

Everything before the colon is the stage. Nodes with the same marker become one section with its own colour, and the app offers to turn each stage into a switch (section 5). The exported file carries no ComfyUI groups or notes, so the titles are the only structure that survives.

A marker must…	Example
end at the first colon	`US2: Upscale` → stage `US2`
be at most 12 characters, with no spaces	`Second pass: …` is not a marker
be ALL CAPS / digits, or at most 4 characters	`HIRES`, `FD`, `Face`

A title that doesn't follow these rules still works; it just doesn't get grouped. Also worth doing before you export: set the values you want as defaults, and delete nodes you don't use.

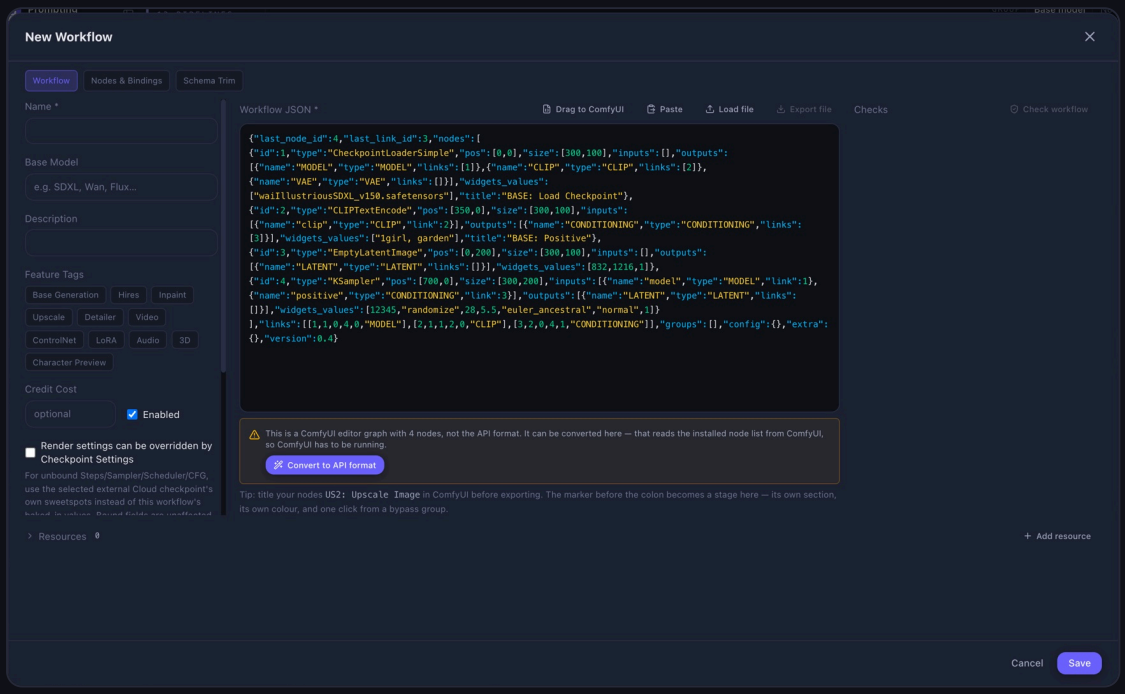
Export in API format

In ComfyUI use **Workflow → Export (API)**. The normal save is a different file (with node positions). The app can convert it, but only while ComfyUI is running.

CHAPTER 05 · SECTION 02

Bring it into the app

1. Open **Workflows** in the sidebar and click **New Workflow**.
2. In the **Workflow** tab, next to **Workflow JSON**, click **Load file** and pick your exported file. Or copy the file's text and click **Paste**.
3. If you picked a normal ComfyUI save by mistake, a yellow note appears under the JSON: *"This is a ComfyUI editor graph … not the API format."* Click **Convert to API format**. ComfyUI has to be running for this.



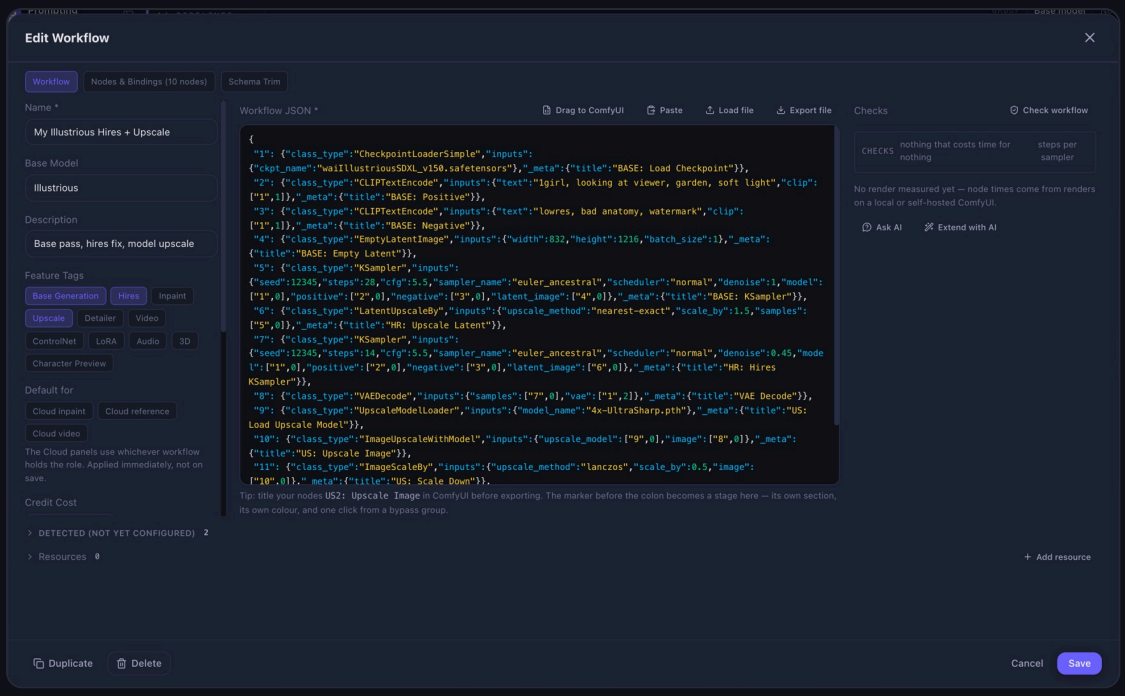
A normal ComfyUI save was loaded. The app recognises it and offers to convert it.

> **Don't use Tools → Import for ComfyUI files**
>
> The **Import** entries in the **Tools** menu of the Workflow Library ("Import from a JSON file…", "Paste JSON…") read files that *this app* exported, for moving workflows between computers. A file from ComfyUI is refused there with "Not a valid workflow export file". ComfyUI files always go through **New Workflow**.

CHAPTER 05 · SECTION 03

Describe it

The left column of the **Workflow** tab says what this workflow is. A workflow counts as described once it has a **Name**, a **Base Model** and at least one **Feature Tag**.



The **Workflow** tab: description on the left, the JSON in the middle, **Checks** and the AI buttons on the right, **Resources** at the bottom.

- **Name** and **Description**: what you'll see on the card and in the pickers.
- **Base Model**: the model family it's built for, for example *Illustrious*. The library groups its cards by this, and the checkpoint picker uses it as a starting filter.
- **Feature Tags**: Base Generation, Hires, Inpaint, Upscale, Detailer, Video, ControlNet, LoRA, Audio, 3D, Character Preview. They let you filter the library.
- **Enabled**: only enabled workflows appear in the Builder's Workflow picker.
- **Uses Custom Nodes**: tick it if the workflow needs extra ComfyUI node packs. ComfyUI Cloud can't install custom nodes, so the render panel warns when such a workflow is picked for Cloud.
- **Default for** (after the first save): makes this the workflow the ComfyUI Cloud panels use for *Cloud inpaint*, *Cloud reference* or *Cloud video*. Only one workflow can hold each role, and the change applies immediately.
- **Notes**: shown above the node boxes in the render panel and exported with the workflow. This is the place for "what to set" for whoever you share it with.

Click **Check workflow** in the right column to test the whole workflow against your running ComfyUI: installed nodes, inputs, links and model files. Every problem it lists links to the node in the JSON.

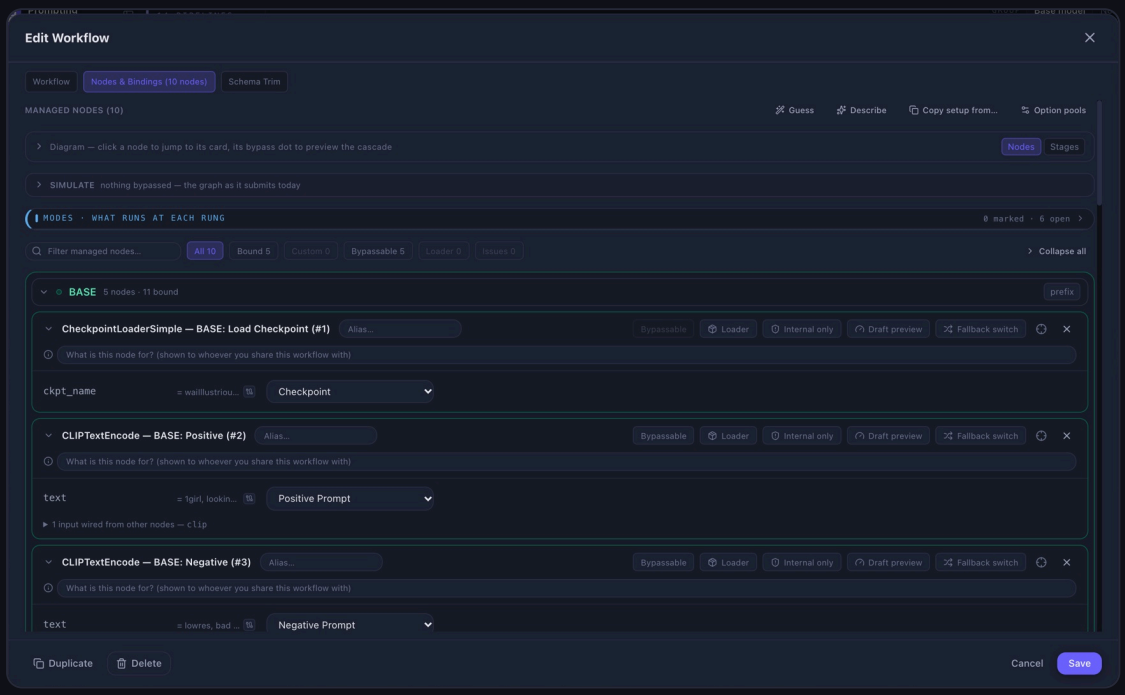
CHAPTER 05 · SECTION 04

Connect it: Nodes & Bindings

A **binding** tells the app which input of which node to fill. For example, the **Seed** field goes into the KSampler's `seed`. Without bindings the app can't put your prompt into the workflow; the card then says *"no render target"*.

1. Open the **Nodes & Bindings** tab.

2. Click **Guess**. The app finds the standard nodes and binds them. Those are the checkpoint loader, the positive and negative prompt, the sampler (seed, steps, CFG, sampler, scheduler), the empty latent (width, height, batch) and image loaders. A prompt node whose title contains "neg" becomes the negative prompt.
3. Look through the node cards. Each input has a menu on the right. **— not used —** keeps the value from the file. A **Panel field** (Checkpoint, Positive Prompt, Seed, Steps, CFG / Guidance, Width, Reference Image, Region masks ...) lets the app fill it in. **Custom field** puts your own control into the render panel.
4. To add a node that isn't managed yet, pick it under **Add nodes** at the bottom. **Suggested** shows the ones worth a look, and **Review** walks you through them one at a time.
5. Click **Save**.



After **Guess**: the base sampler's seed, steps, CFG, sampler and scheduler are bound to the app's fields. Below it is the **HR** stage, already a bypass group (section 5).

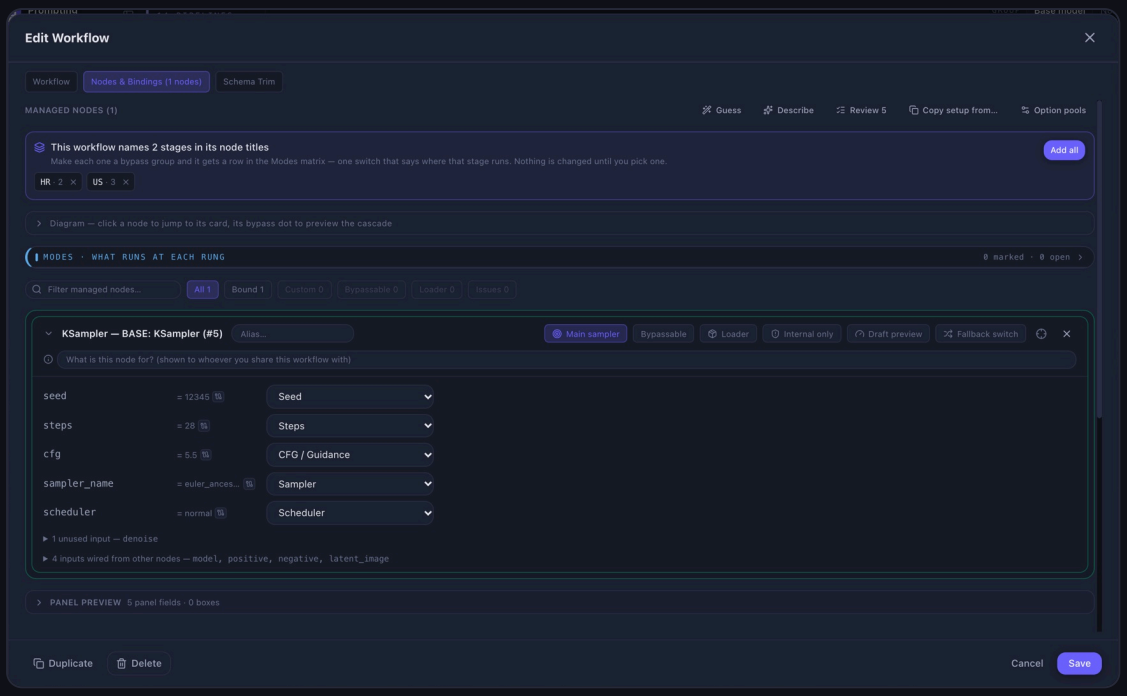
More buttons at the top of the tab: **Copy setup from...** copies the bindings of a similar workflow you already set up. **Describe** asks the AI what each node is for (needs an AI key). **Option pools** are shared choice lists, such as a set of samplers, that several workflows can use.

CHAPTER 05 · SECTION 05

Make stages switchable

A **bypass** switches a node off for one render. ComfyUI then routes around it. That is how one workflow can do both a quick look and a full render with hires and upscale.

If your node titles have stage markers, the tab greets you with *"This workflow names N stages in its node titles"*. Click **Add all** (or a single stage) and each stage becomes a **bypass group**: one switch for all its nodes. The app works out on its own what flows past a switched-off node. If it can't, a small **?** marks the nodes where you have to pick it yourself under **Wiring**.



The stage markers **BASE**, **HR** and **US** from the node titles. The app suggests turning HR and US into switches.

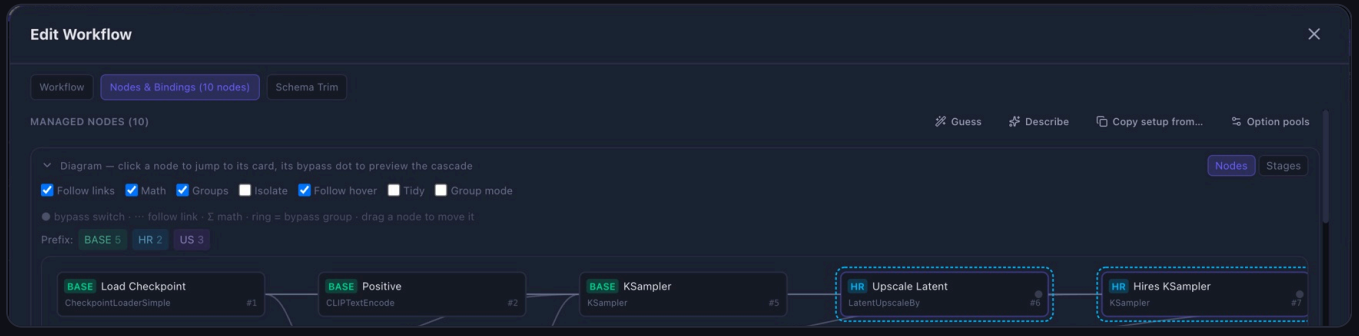
You can also do it by hand. On any node card, click **Bypassable**, then choose a **Group**. A group decides how its switches behave:

- **Linked — one switch for all**: the whole stage goes on or off together. This is the usual choice.
- **One always active** / **At most one active**: for alternatives, for example two different upscalers.
- **Master switch — members stay individual**: one switch turns everything off, but each member keeps its own switch while it's on.
- **Label only (no shared logic)**: just groups the nodes visually.

Bypassed by default starts the switch off. **Pin (shown collapsed and on the phone)** puts it in the compact row of the render panel and in the phone client.

The graph diagram

Above the node cards, the **Diagram** draws the workflow as a graph, coloured by stage. Click a node to jump to its card, or its bypass dot to preview what switching it off would do. You can **drag** nodes to where they read best; **Tidy** lays the graph out with fewer crossings (loaders beside what reads them), and **Reset layout** puts every node back and turns Tidy off. Both are remembered. Drag the diagram's lower edge to make it taller.



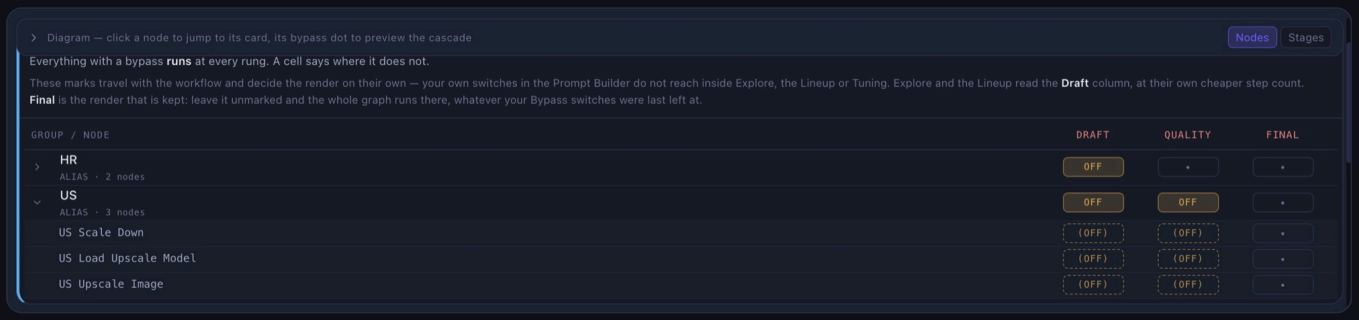
The diagram over the node cards, coloured by stage: BASE, HR, US.

CHAPTER 05 · SECTION 06

Decide what runs when: the Modes matrix

The Prompt Builder renders at three levels: **Draft** (a quick look), **Quality** (the model's sweet-spot step count, for comparing) and **Final** (the picture that gets saved). The **Modes · what runs at each rung** panel in the Nodes & Bindings tab decides, per stage, which of those levels it's skipped in.

> **The rule: everything runs unless you mark it**
>
> Each row is a bypass group, with its nodes folded underneath. An empty cell (·) means "runs here". Click a cell to mark it **OFF**. A member node follows its group; its cell shows **(OFF)**. Click a member's cell to decide it yourself.



A typical setup: the hires pass (**HR**) is skipped in Draft, and the upscale (**US**) in Draft and Quality. At **Final** everything runs.

- These marks travel with the workflow. Explore and the Lineup read the **Draft** column.
- The Builder's own bypass switches don't reach into Explore, the Lineup or Tuning; the marks decide there.
- Leave **Final** unmarked and the whole graph runs for the picture that gets saved. Mark a stage off at Final when it should never run yet, for example a pass you haven't tried on this model.
- The marks name every switchable node at every rung, so the Builder's bypass switches decide nothing at Draft, Quality or Final. When you want your own switches to decide, render on the fourth rung, **Custom**: Final, with the Builder's bypass switches instead of the matrix.
- A workflow nobody has marked runs its whole graph everywhere, just with fewer steps for a Draft. So a cheap look is only really cheap once you mark the expensive stages.

CHAPTER 05 · SECTION 07

Setting presets: named sets of values

A workflow can carry **setting presets**: named sets of values for its panel fields, such as "Landscape, more detail" or "Painted / illustration". In the Builder they sit at the head of **05 Machine → Bypass & parameters**. Pick one and the denoise values, models and factors it names are set for this job; the fields stay editable, and the bar says when you have changed something since.

- The buttons beside the select **update** the picked preset with the current values, **save as new**, or **delete** it. Presets are written into the workflow, so an export carries them.
- Presets hold values only: numbers, selects and switches. Never prompt text (a prompt is per picture) and never which stages run (that is the Modes matrix).
- A preset's description is the guide to it: say when to take which. The Core pipelines' presets are written that way.
- A preset sets **only what it names**, so presets can be layered. The Anima pipeline has two kinds: picture presets (models, upscales, everything) and **Shot** presets (*Shot — close-up / portrait*, *Shot — full body / small face*, *Shot — several figures*) that set only the face and hand passes. Pick a picture preset first, then a shot preset over it. Update writes only the fields the picked preset already names.
- The same presets can be switched in the Builder head, in the **Settings…** select beside the workflow.

Fields the app fills for you

- **The upscale prompt.** An upscale or detail pass gets its own short text, chosen by the cast: with a visible figure a text about skin and eyes, without one a text about edges and texture (a workflow made for scenes always sends the scene text). The field shows the text that is sent, why it was chosen and which nodes receive it. Edit it and it is yours (*Your text — sent as written*); **Let the app choose** gives the choice back.
- **Upscale model.** In the Core pipelines the model of each upscale stage is a select over one shared list, so you choose by purpose. **None — lanczos only, invents nothing** enlarges by the same factor without a model.

CHAPTER 05 · SECTION 08

Every stage's picture

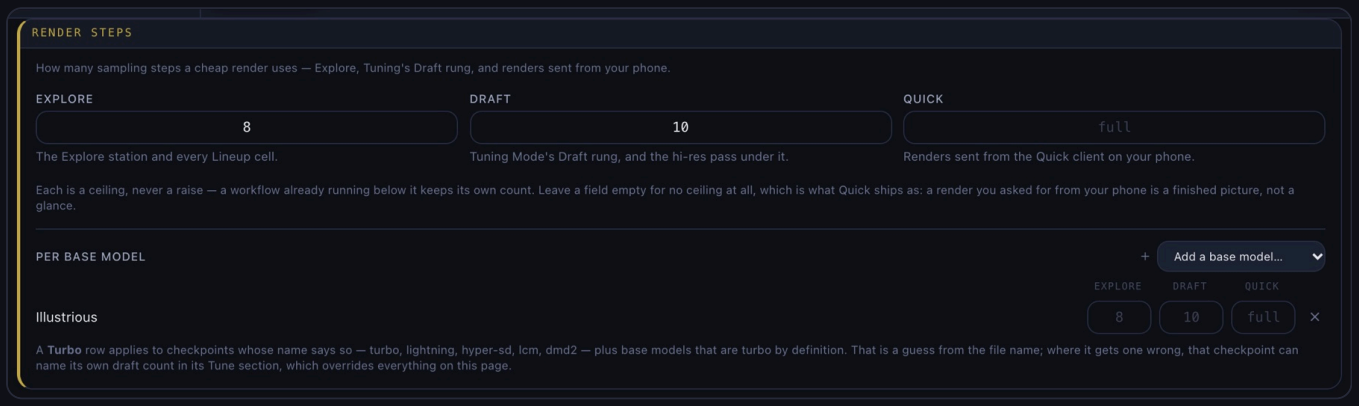
The Core pipelines show a picture after each stage that changes the whole picture: each upscale, the face pass, the picture handed to SeedVR2 and the one before the colour correction. They appear with the render in the Builder's previews; in the lightbox, the stage buttons under the picture walk through them. A stage that is switched off shows no picture under its name. To do the same in your own workflow, add a `PreviewImage` node after the stage; its title is the name the stage is shown and saved under. Put it in the stage's bypass group, so a stage that is off shows no picture.

- **Keep as final** (in the lightbox, on a stage picture of a saved render): writes that stage over the render's file, for example the picture before SeedVR2 when the clean-up went too far. The previous final is kept beside it as `…_final_….png`. An unsaved render has no file to replace, and the button says so.
- **Stage pictures** in the Output settings (**Jobs → Output** , off by default) saves every stage of a saved render next to it as `<render>_stage01-base_….png` and so on, so the Crosspost Helper stacks the whole pipeline.

CHAPTER 05 · SECTION 09

How many steps a quick look uses

The step count for cheap renders is set once for the whole app in **Settings** → **Rendering** → **Render Steps**. Each number is a **ceiling**, never a raise: a workflow already running with fewer steps keeps its own count. An empty field means no ceiling.



Explore covers the Explore station and every Lineup cell, **Draft** covers Tuning's Draft rung (and its hires pass), and **Quick** covers renders sent from your phone. Under **Per base model** you can add exceptions, here a row for Illustrious.

A **Turbo** row (for example "SDXL · Turbo") applies to checkpoints whose file name says so: turbo, lightning, hyper-sd, lcm, dmd2. If that guess is wrong for one checkpoint, give that checkpoint its own draft count in its Tune section in the Model Library. That count overrides this page.

CHAPTER 05 · SECTION 10

Share what it needs: Resources

At the bottom of the **Workflow** tab, **Resources** lists the model files and custom-node packs the workflow needs, each with a link. Click **Add resource** to add one. Whoever you share the workflow with can download each file straight into the right model folder from there. The library's **Has downloads** and **Uses custom nodes** filters find these workflows quickly.

To share a workflow, use **Tools → Export** in the Workflow Library. The file keeps the bindings, switches, marks and notes, and the other person brings it in with **Tools → Import**.

CHAPTER 05 · SECTION 11

Let the AI help (optional)

With an AI provider key set in **Settings → AI**, you get these extra buttons:

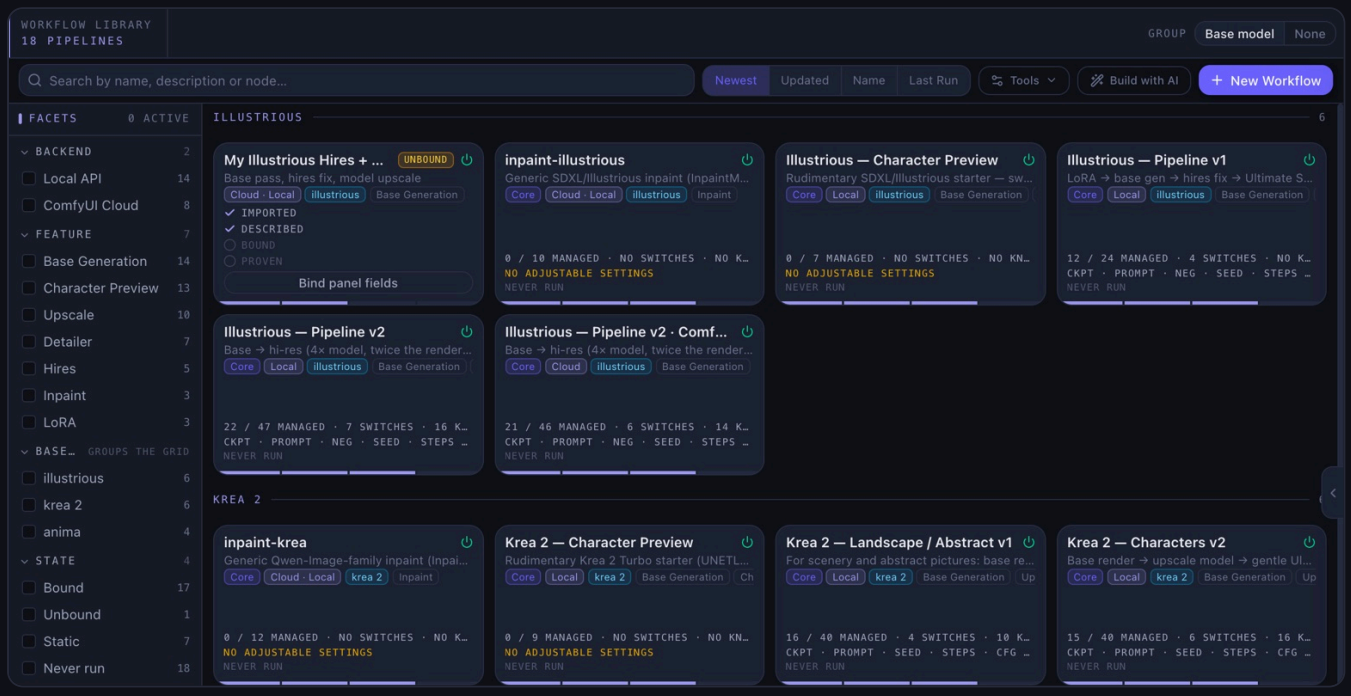
- **Build with AI** in the library head: describe a workflow in words and the AI builds one.
- **Ask AI** in the Checks column: ask why a workflow is slow or what a stage does.

- **Extend with AI**: describe a change and the AI proposes it. It's saved as a new workflow, so your original stays untouched.

Treat AI-built workflows like any import: run **Check workflow** and one test render before you rely on them.

CHAPTER 05 · SECTION 12

Troubleshooting



The Workflow Library. A card that isn't finished lists its setup steps and names the next one, here **Bind panel fields**. The Core pipelines sit beside it with their versions and their Comfy Cloud twins.

"Not a valid workflow export file".
You used **Tools → Import** with a ComfyUI file. Use **New Workflow → Load file** instead (Step 2).

My workflow doesn't show up in the Builder's Workflow picker.
Check that it's **Enabled** and that you clicked **Save**. The picker only lists enabled workflows that can run on the selected service.

The render ignores my prompt or my checkpoint.
The field isn't bound. Open **Nodes & Bindings**, click **Guess**, and check that the prompt and checkpoint inputs point at **Positive Prompt** and **Checkpoint**.

"Value not in list" or a missing node when rendering.
ComfyUI is missing a model file or a custom node. Run **Check workflow**; it names the node. Install what's missing, restart ComfyUI, and check again.

A Draft render takes as long as a Final.
Nothing is marked in the Modes matrix, so the whole graph runs. Mark the hires, upscale and detailer stages **OFF** in the Draft column (section 6).

Stage markers aren't being picked up.
Check the rules in Step 1: first colon, at most 12 characters, no spaces, and ALL CAPS or at most 4 characters. Then export again from ComfyUI.

CHAPTER 06

Example Images, readings and recasts

Keep the pictures you like, let the AI read a picture into a prompt for every base model you work with, decide once which of its people and places become placeholders, and let your own characters fill them in the Prompt Builder.

— What it is for 76
01 Collect pictures 76
02 Work on many at once 78
03 The picture window 79
04 Read a picture for your models 79
05 Where the detail goes 81
06 One tab per model 81
07 The page 83
08 Casting: decided once for every model 84
09 Scene only 86
10 Several people 87
11 Take from another picture 88
12 Rewrite: looser prompts 88
13 The picture's own prompt 89
14 Send it on 90
15 In the Prompt Builder 91
16 The Crosspost Helper 92
17 Tips 92
18 Troubleshooting 93

What it is for

You find a picture you love: the pose, the light, the room, the mood. Its prompt describes somebody else, though: that person's hair, eyes and clothes. Example Images is the library where you keep such pictures, together with their prompts. It does three things for you:

- **Keep and find.** Every picture with its prompt, model, LoRAs, a rating, a category, a purpose and tags.
- **Understand.** The prompt can be shown by who it describes or sorted into groups, so you see which words do what.
- **Reuse.** The AI reads the picture into a prompt for each base model you pick, every one in its own dialect, and the people and the place in it become placeholders. In the Prompt Builder your characters fill them.

The scene stays. Who stands in it is decided in the Builder.

What needs what

- Collecting, browsing, rating, the views, adding a model's prompt without reading, and sending to the Builder work without any setup.
- The AI tools (reading a picture, Scene only, Take from another picture, stripping the borrowed identity, Simplify pose & position, Outfit & style, rewriting as tags) need an AI provider key in **Settings → AI**. Without one, their buttons are not shown, and the start question says so. Reading a picture also needs a provider that can see images.
- Importing by CivitAI link works without a key for most pictures. When CivitAI hides a picture's generation data (NSFW pictures in particular), add a CivitAI API key in **Settings → CivitAI Integration**.
- The Crosspost Helper features need that app running and paired with this one.

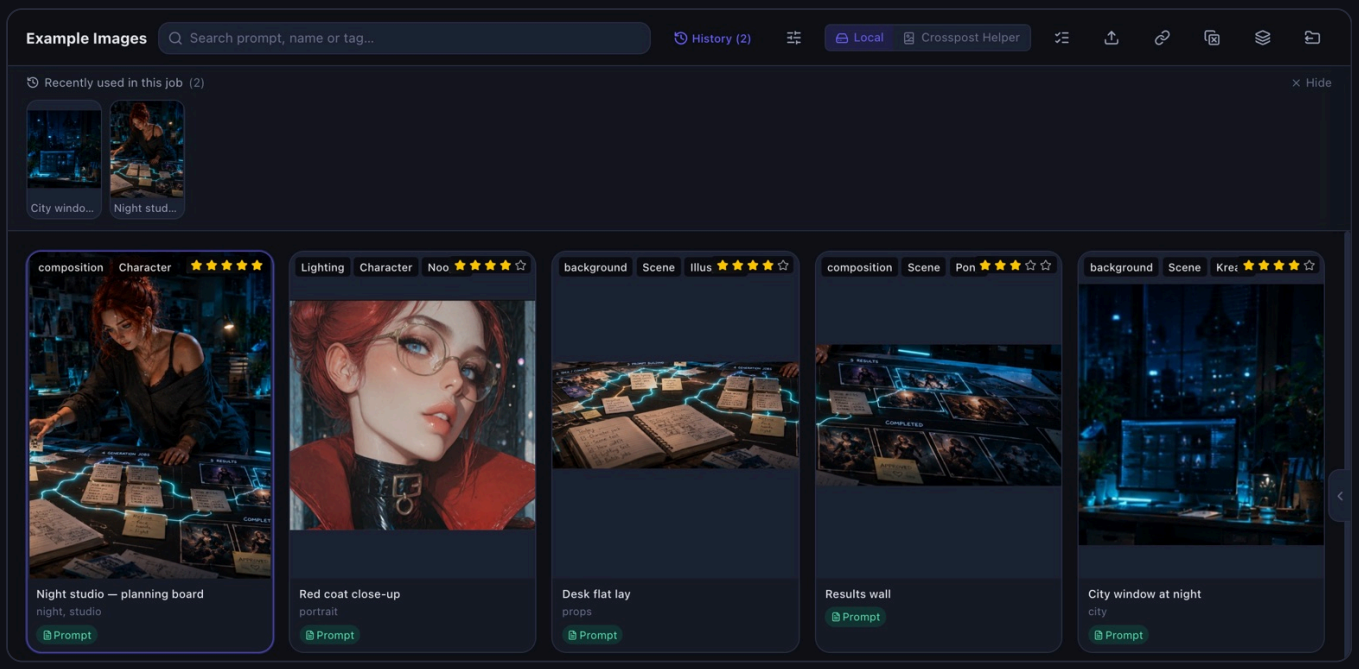
CHAPTER 06 · SECTION 01

Collect pictures

Open **Example Images** in the sidebar. There are four ways to add a picture:

- **Drag and drop** an image file onto the page.
- **Paste** an image with Ctrl+V (Cmd+V on a Mac) while you are not typing in a field.

- **Import from file** (the upload icon). The prompt is read from the file's metadata when it has any. A file without metadata is welcome too; its fields simply start empty.
- **Import from CivitAI URL** (the link icon). Paste a civitai.com/images/... link or just the image number. Prompt, model, sampler and LoRAs come along.



The library. At the top, *Recently used in this job*; each card shows purpose, category, base model and its stars, the whole picture, and whether it has a prompt.

- **Rate on the card.** Click a star on a card and the rating is saved at once; click the same star again to take it back.
- **Prompt** under the name says whether the picture has a prompt to take, and the model chips beside it name the base models it already has a prompt for.
- **To references** appears over the picture when you point at it: the picture goes into the Reference library and is selected, without opening it.
- **History** in the head opens *Recently used in this job*: the last 20 pictures this job took a prompt or a reference from. Click one to open it, **Hide** to fold the strip away. Each job keeps its own history, and its own page, search and filters.

The search field finds a prompt, a name or a tag. **Filters** narrows the grid by category, purpose, rating, the base model a picture was made on, and **People** (how many people the prompt describes). **Content** (Any, SFW, Suggestive, NSFW) works on the Crosspost Helper's library, whose tagger rated each picture; the local library has no such rating, and the control says so. **Clear filters** resets them all. The grid shows 60 pictures per page.

The switch **Local** / **Crosspost Helper** chooses which library you are looking at. Local is this app's own. The second is the Crosspost Helper's Reference Library, read over the bridge; see section 16.

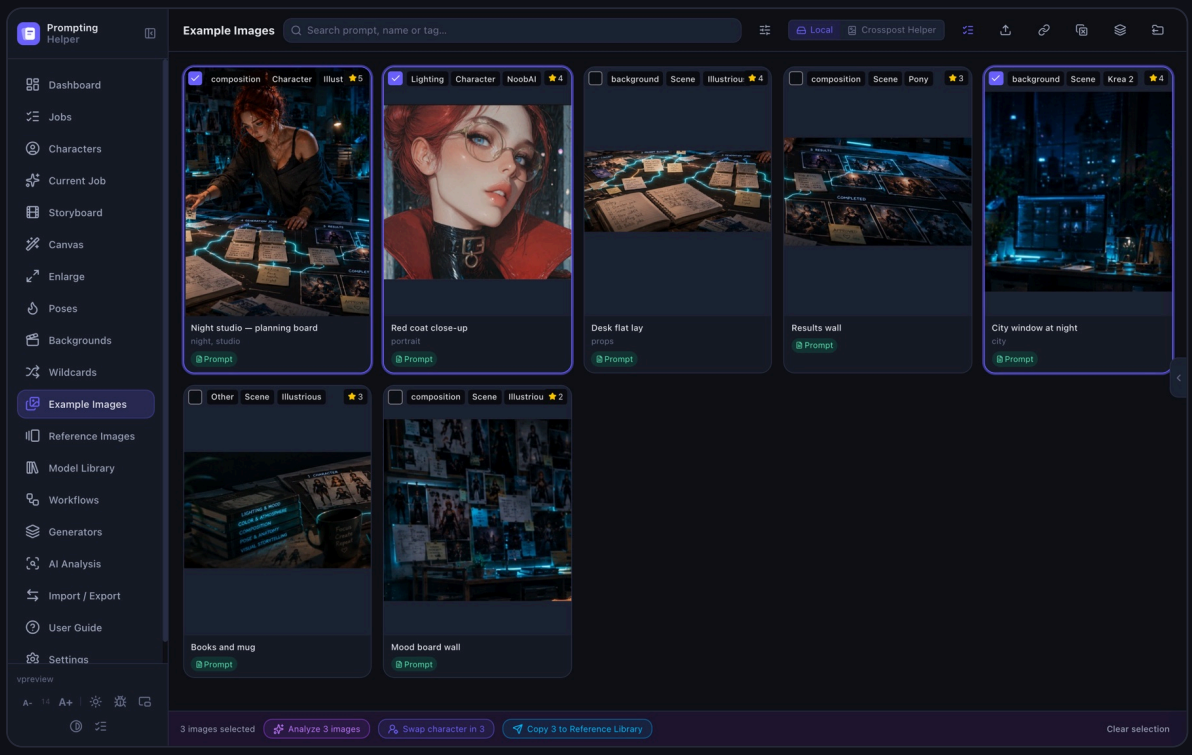
The toolbar, left to right

- **Select** (the ticked list): turns on selection mode, see section 2.
- **Import from file** and **Import from CivitAI URL**.
- **Look for duplicate images**: shows a count when the same picture arrived twice, and removes the extra copies.
- **Scratchpad**: collects single tags you click in pictures, to save them as a group later.
- **Export all as Prompting Helper bundle**: one file with every picture and its data.

CHAPTER 06 · SECTION 02

Work on many at once

Click **Select** in the toolbar and tick the pictures. A bar at the bottom offers three things:



Three pictures selected. The bar: Analyze, Swap character, Copy to Reference Library, and Clear selection on the right.

- **Analyze N images**: the AI looks at what the prompts have in common, which is how you find out why a set of pictures works.
- **Swap character in N**: replaces the character tags in every selected prompt and keeps the rest. No AI needed.
- **Copy N to Reference Library**: copies the pictures to the Crosspost Helper; see section 16.

CHAPTER 06 · SECTION 03

The picture window

Click a card. The window is a light table in three parts: **the picture** on the left, **the page** (the prompt for one base model) in the middle, and **the casting sheet** on the right once a picture has been read. One row of exits runs along the bottom.

- **The head.** The name, the stars, and on the right **Outfit & style**, **Copy**, **Swap character**, the **⋯** menu and Close.
- **The model row** under the name: **Original** (the picture's own prompt) and one tab per base model that has a prompt. Section 6 explains them.
- **The picture.** Click it to view it at full size. Once a reading has placed the people, a ring with a number shows where each one stands. Under it: Category, Purpose, Generated on, the stars, your tags, the source, and size, model, sampler, steps, CFG and seed. The arrow at the bottom collapses the picture column, so the page gets the width; the app remembers that for the next picture.
- **The ⋯ menu** holds **To Prompt Builder**, **Delete image**, and, when you opened the library from the Builder, **Append to Description**.

A picture with no model prompt yet opens on one question: which models should get one (section 4). Opened from the Builder, the window opens on the prompt for the base model the Builder is on; when the picture has none for that model, the question says so and has that model ticked.

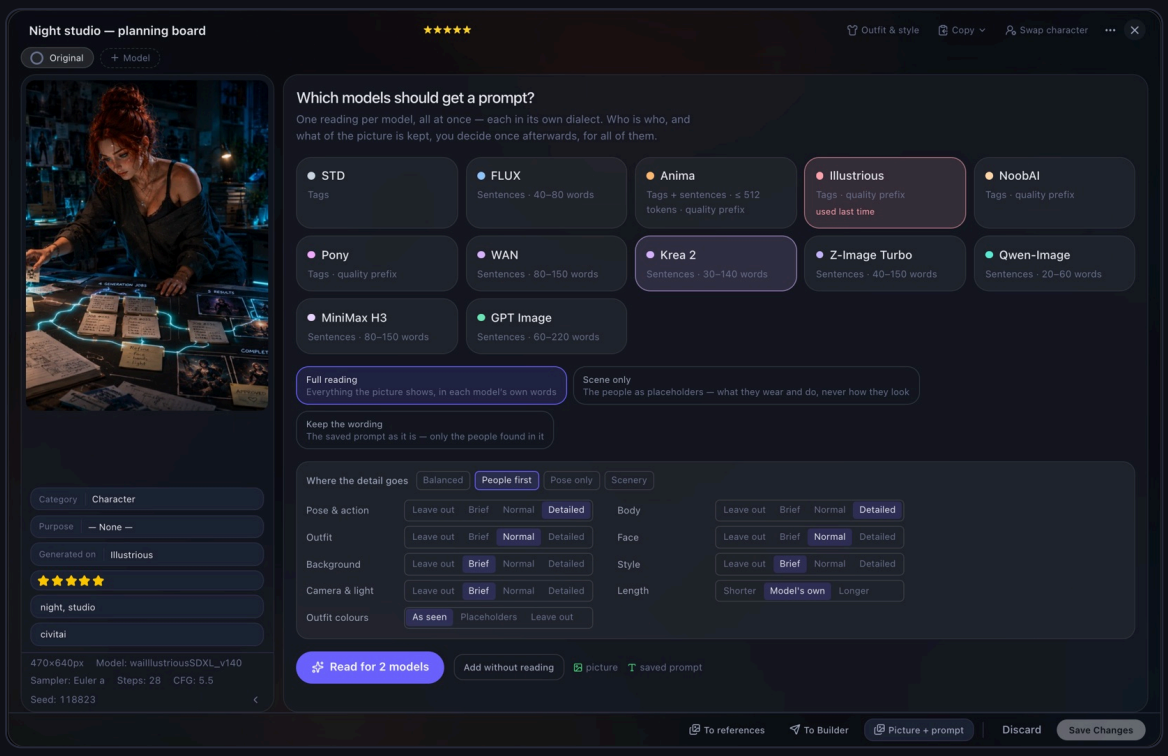
Outfit & style, and Copy

- **Outfit & style** reads only what the person wears and how the picture is made. **Read outfit** gives a name, the garments as tags and a description; **Save as outfit** keeps it as a Cloth Transformer template, which you apply from a character card's Cloth Transformer. **Read art style** gives a paragraph for the caption models and style tags for the tag models; **Save as Style Block** keeps it in the Style library, **Save & use in Builder** also puts it in use.
- **Copy** copies the outfit, the art style or the lighting on its own, to paste into another prompt.

CHAPTER 06 · SECTION 04

Read a picture for your models

The start question is **Which models should get a prompt?** Each card names a base model and what it reads: tags, or sentences with the word range it likes. Tick every model you work with; the models you picked last time are marked *used last time*.



Illustrious and Krea 2 ticked, Full reading chosen. Below: where the detail goes, the length and the outfit colours. **Read for 2 models** sends one call per model, at once.

One call per model, all at once. Every model gets its own reading in its own dialect, and one that fails leaves the others. A reading keeps running when you close the window; it lands in its tab anyway. Then choose how to read:

- **Full reading**: everything the picture shows, in each model's own words, with every person found.
- **Scene only**: the people as placeholders, what they wear and do but never how they look; see section 9.
- **Keep the wording**: the saved prompt as it is; only the people in it are found. Use it when the words are right and you only want the people sorted out.

Two sources. The AI sees the picture *and* the saved prompt. The picture may add to the prompt; it never drops what only the prompt knows. Where the two disagree you are told, not overruled. Click **saved prompt** to leave the prompt out and read the picture alone, which helps when the prompt is unusable (a metadata blob from another tool).

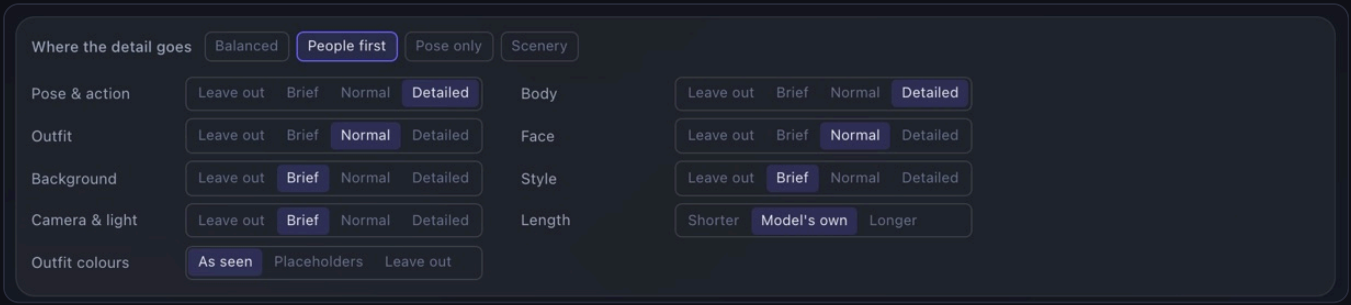
Add without reading adds the ticked models with the picture's own prompt as their text, for you to edit by hand. The model picker at the right end chooses which AI model reads; next to each provider's recommended vision model it offers that provider's **stronger** one, for a picture the usual one gets wrong.

A reading comes back **in paragraphs**: the setting, one paragraph per person, what they do together, the light, the style.

CHAPTER 06 · SECTION 05

Where the detail goes

Under the three ways to read sits **Where the detail goes**. It decides how much care each part of the picture gets, and it is remembered for every reading after.



People first: pose and body in detail, outfit and face normal, background, style and camera brief.

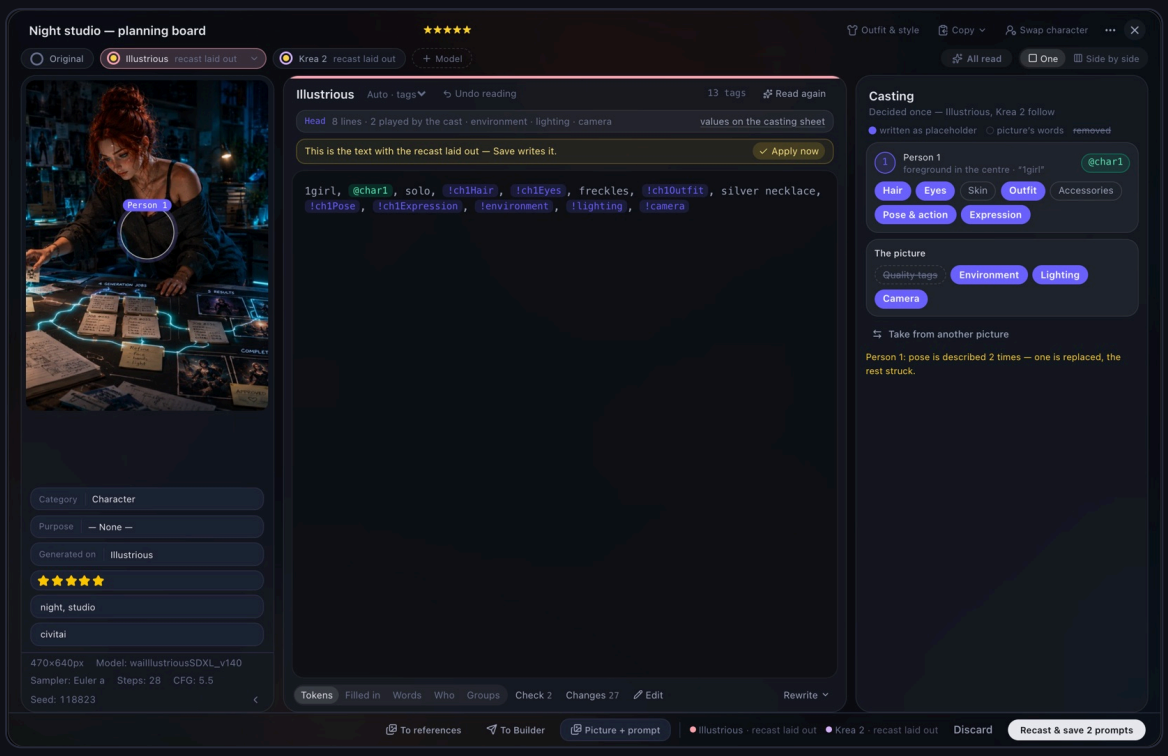
- **Seven areas**: Pose & action, Body, Outfit, Face, Background, Style, Camera & light. Each is **Leave out**, **Brief**, **Normal** or **Detailed**.
- **Presets**: **Balanced** (every part with the same care, how readings always were), **People first**, **Pose only** (the poses and the act in detail, the rest as short as it can be) and **Scenery** (setting, style and light in detail, the people brief).
- **Length**: **Shorter** (about 60% of what the model asks for), **Model's own**, **Longer** (about 140%).
- **Outfit colours**: **As seen** keeps every garment's colour. **Placeholders** writes each colour as a placeholder like `!ch1OutfitColour`, its value the colour seen, so you can change or empty it in the Builder. **Leave out** names no colour, and the image model picks them.

Poses are always read as geometry, per person: what they rest on, which way they face, the limbs, who is on top and who below, which part touches which. That is what a picture of several people gets wrong most often, so **People first** or **Pose only** are the presets to try when the poses come out muddled.

CHAPTER 06 · SECTION 06

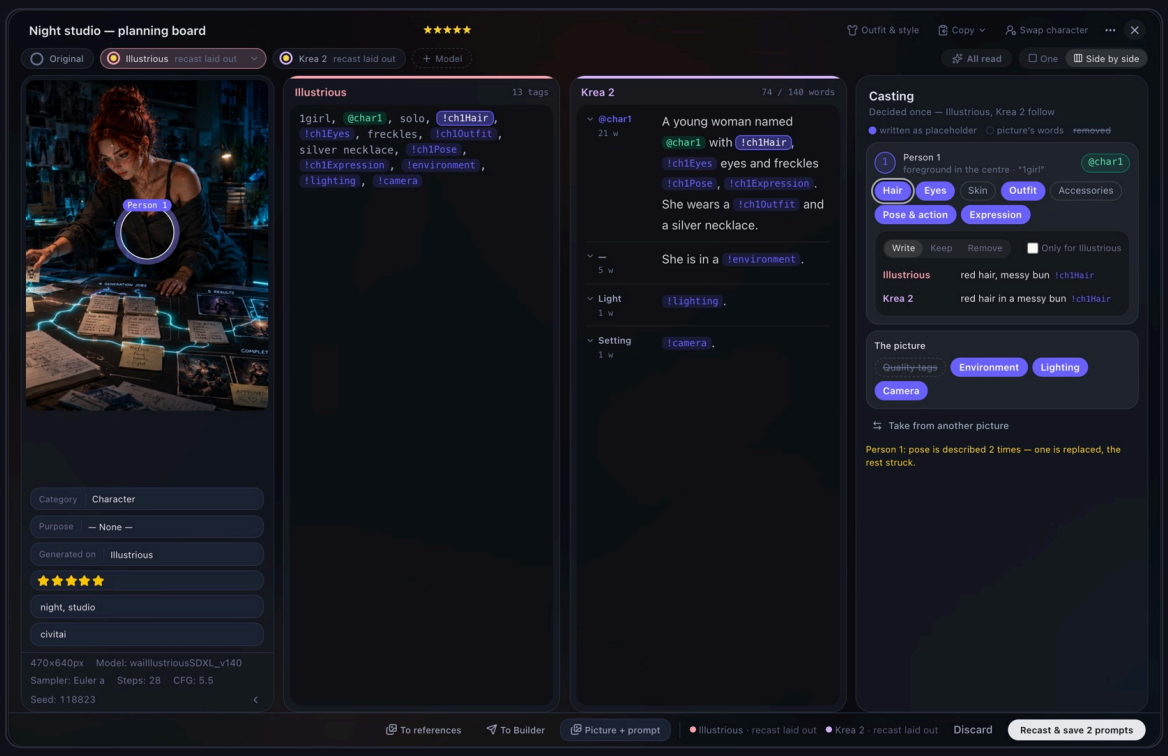
One tab per model

After the reading, the model row has a tab for each model, coloured like the model, with where it stands: *not read*, *reading…*, *failed*, *recast laid out*, *ready* or ✓ *rendered*.



Two tabs, both *recast laid out*. The page shows Illustrious; the casting sheet on the right is the same for both models. The Save button says **Recast & save 2 prompts**.

- **+ Model** adds a model that has no prompt here yet; **Read N** on the right reads every model added and not read yet.
- **The arrow on a tab** opens its menu: **Rename**, **Mark as rendered — it worked** (the guess becomes a fact, and the tab says ✓ *rendered*), **New version of this**, **Save a copy to the Prompt Library**, and Delete. With several versions the menu lists them; the first is the one the Builder takes.
- **One** / **Side by side** on the right: one model's page at a time, or every page next to each other. Side by side lines long texts up by paragraph, so the person in one column stands beside the same person in the next.



Side by side. Illustrious as tags, Krea 2 as labelled paragraphs, with one recast for both.

CHAPTER 06 · SECTION 07

The page

The page is one model's prompt. Its head names the model and the form (**Auto · tags** ; switch it to Sentences or Tags for the next reading), offers **Undo reading** , and counts the text in the model's own unit: tags, words against the range the model asks for, or tokens for Anima. Past the range the count turns amber and **Fit to N** shortens it, keeping every placeholder or naming the one it lost. **Read again** reads this model once more, as an unsaved change with Undo.

- **Head**: the placeholder lines are one line here (*8 lines · 2 played by the cast · environment · lighting · camera*); their values live on the casting sheet.
- **The recast**: while it is laid out but not written, an amber bar says *This is the text with the recast laid out — Save writes it.* **Apply now** writes it into this page at once (still saved only on Save). Once written, the line says so and offers **Recast again** .

At the bottom of the page:

Button	What it does
Tokens	The prompt as a template, placeholders as pills.
Filled in	The prompt as the model reads it, every placeholder replaced by its value.
Words	The reading's passages word by word: click a word to keep it, Shift-click for a run, assign a phrase to another person.
Who / Groups	Which person each tag belongs to; the prompt split into blocks.
Check	The Tag check: duplicates, contradictions, broken sentence fragments, placeholders filled, and where the picture disagrees with the prompt.
Changes	What this page changed against the picture's own prompt.
Edit	The raw text, head included, to type into.
Rewrite	The tools of section 12.

A prose page reads as labelled paragraphs (one per person, Light, Setting ...) that fold past the model's range; **Open all** / **Fold all** switches them all.

CHAPTER 06 · SECTION 08

Casting: decided once for every model

Recast turns what a prompt says about a person, and about the place, into placeholders. The picture decides which placeholders exist; the Prompt Builder later decides who fills them. The **Casting** sheet decides it **once**, for every model read: the sheet says *Decided once — Illustrious, Krea 2 follow*.

- `@char1`, `@char2` ... stand for whoever is in cast slot 1, 2 ... in the Builder.
- `!ch1Hair`, `!ch1Eyes`, `!ch1Outfit` ... are one line each for one trait of that person. `@cast` as the value means "take it from the character in that slot"; the picture's own words stay beside it as a kept line (`# !ch1Hair: red hair, messy bun`), used whenever the character has nothing for that trait.
- `!environment`, `!lighting`, `!camera` ... are the picture's own placeholders. Their value is the picture's own words (`!environment: night, cluttered studio`), so the prompt reads the same until you change the line in the Builder.

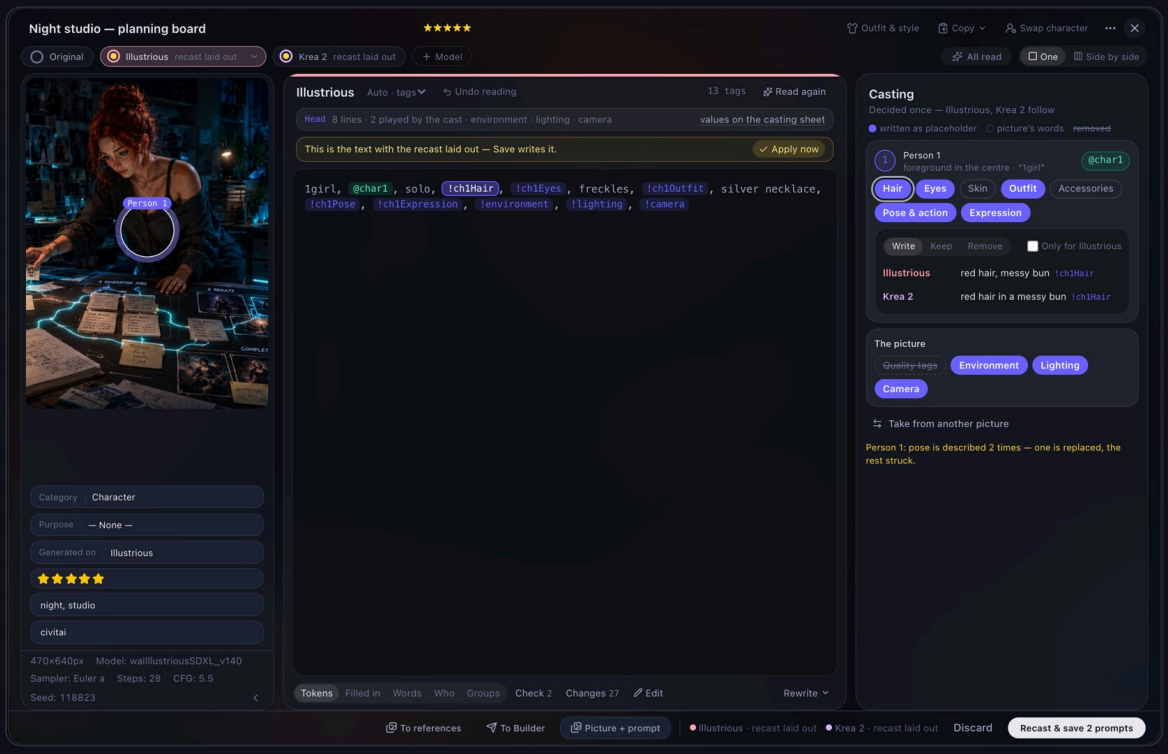
Hair is one placeholder. `!ch1Hair` holds colour and style together, so a cast character's hair arrives whole. Older texts with separate colour and style lines still work; in the Builder they are folded into one Hair line.

The pills

Each person has a card (number, where they stand, the name token) with a pill per trait the reading found. **The picture** has pills for Quality, Motif, Environment, Art style, Palette, Lighting and Camera, as far as the

picture has them; a picture without people is read in full, and its motif (an animal, a car, a building) becomes a line of its own.

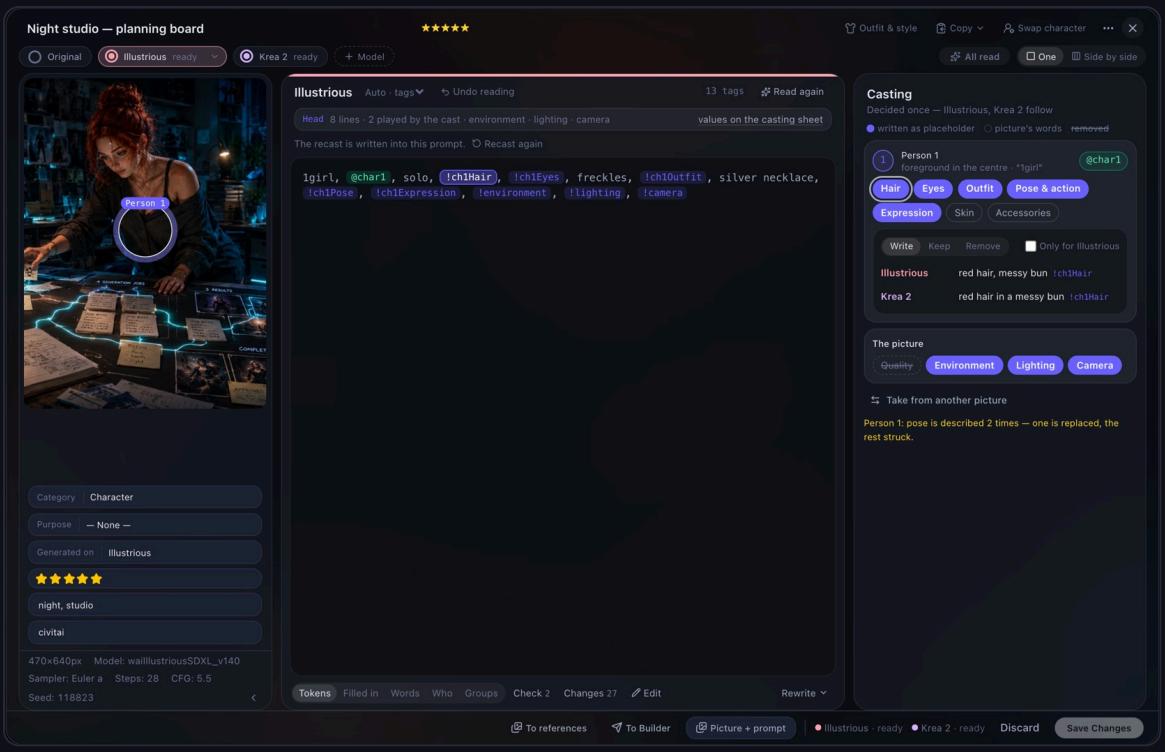
- **Filled**: written as a placeholder. **Outline**: the picture's own words stay. **Struck**: removed from the prompt.
- Click a pill to open it: **Write**, **Keep**, **Remove** (and on Quality **Per model**, which writes the target model's own quality prefix instead), with that trait's words in **every** model side by side. That is the one place to see whether the readings agree. A value can be edited there.
- **Only for <model>** decides that row for the open model alone; the others keep the shared decision.
- **Said twice**: when a trait appears in two places, pick the one that gets the placeholder; the other is struck, or click **keep** to keep it as written.



Hair opened: Write, Keep, Remove, Only for Illustrious, and the hair as each model's reading wrote it.

> **What an untouched row does**
>
> As little as it needs to. Of a person, hair, eyes, genitals, pose, expression, outfit and makeup are written as placeholders; skin, build and everything else keep the picture's words. Of the picture, every row is written except the quality tags, which are removed: those come from the checkpoint. Art style and palette are written too, so a reference can hand over its look alone.

Then **Recast & save N prompts** in the bottom row writes the recast into every model's page and saves; it says how many it writes. Once written, a page is *ready*, and nothing is pending until a new reading or Recast again.



The finished template. Hair, eyes, outfit, pose and expression are placeholders of person 1; environment, lighting and camera are the picture's; the quality tags are gone.

CHAPTER 06 · SECTION 09

Scene only

Scene only reads the picture with every person written as a placeholder and **never writes a look**: no hair, no eyes, no skin, no build. What stays is everything you borrow a picture for: pose, outfit, expression, props, framing, setting and light. Genitals are read as a placeholder of their own (`!ch1Genitals`), so the cast's anatomy takes their place.

Two controls appear beside it. **People as** decides what a person becomes:

Choice	In the prompt	In the Builder it becomes
`@char1`	the name	the character's name; her look comes from her own character block
`!ch1Look`	the whole figure	the full description, right where she stands in the scene
`!ch1Core`	the essentials	only what makes her recognisable (hair, eyes, identity)

Keep from the picture : tick **Outfit** or **Makeup** to keep the picture's own words for them instead of a placeholder.

Scene only: `@char1` stands for the person; outfit, pose, expression and the place are placeholders, and nothing describes how she looks.

> **When to use which**
>
> Use the full reading when the person in the picture should stay recognisable as a template you might fill with anybody. Use Scene only when you know you will cast your own character and do not want a borrowed look anywhere in the text.

CHAPTER 06 · SECTION 10

Several people

- Each person gets their own number (`@char1` , `@char2`), their own card on the sheet and their own lines.
- Everything about one person stays in that person's part of the prompt, and each is introduced with where they are ("on the left").
- A phrase pointing at another person ("sitting on the other woman's lap") becomes that person's token ("sitting on @char2's lap"), so the relation still holds with your cast.
- A person called by a look ("the blonde woman") is called by a name instead.
- People are matched across the models' readings by position and hair colour. When that cannot decide, the sheet says *Person order differs in <model>* and asks you to match them by hand; it never guesses.

Templates stay neutral on purpose

There is no character picker in Example Images. What the picture shows decides which placeholders exist, and every person's lines are bound to `@cast`. Who fills them is decided in the Builder, so one template works for every character.

CHAPTER 06 · SECTION 11

Take from another picture

Keep this prompt's people and take the pose and the place from another picture. Click **Take from another picture** under the casting sheet and pick the picture in the library. It is read as Scene only, and the ticked blocks put its words into the matching rows of this reading: **Pose** (and what they hold), **Environment**, **Light**, **Framing**, and on request **Expression** and **Style** (art style and palette).

- The rows of this picture's people are never filled; a row the other picture has nothing for keeps this prompt's words.
- Each taken row has its own Undo, back to this prompt's own words; the ✕ puts every row back.
- The recast then writes the taken words as usual (`!environment: neon bar`).

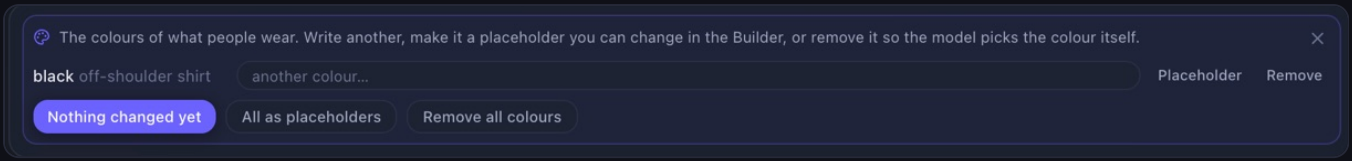
To combine several pictures in the Builder (one picture's light, another's outfit), use the Mix Board; see the Prompt Builder chapter.

CHAPTER 06 · SECTION 12

Rewrite: looser prompts

Rewrite at the bottom right of the page opens the tools that change the text. Every one shows its result before it replaces anything, and nothing is stored until you save.

- **Strip the borrowed identity**: takes the borrowed person's strongest features out, for every character in the prompt. Tick **Hair & eyes**, **Chest**, **Genitals** or **All three**, then **Find it in the prompt**.
- **Clean up** (only when there is something to remove): LoRA tags, weights, embeddings, BREAKs and repeated tags.
- **Outfit colours**: every garment colour in the prompt, head lines included. Write another colour, make it a **Placeholder** you can change in the Builder, or **Remove** it so the model picks the colour itself; **All as placeholders** and **Remove all colours** do it for every one. No AI needed.
- **Simplify pose & position**: one AI call, two levels. **Simpler** keeps the posture and the one main gesture; **Just the stance** keeps only standing, sitting, kneeling or lying, and which way they face. Left and right are kept when there are several people, and a result that lost a placeholder cannot replace the prompt.
- **Rewrite as tags**: rewrites prose as tags. Nothing is added or dropped, only the wording changes.
- **Split into paragraphs**: breaks a one-block text before each person, the light and the style. Only whitespace changes.



Outfit colours: one garment found. Type another colour, make it a placeholder, or remove it.

Strip the borrowed identity: three passages found and struck through. Click a word to keep it, or **Keep all** for a whole passage; *What the prompt becomes* shows the result.

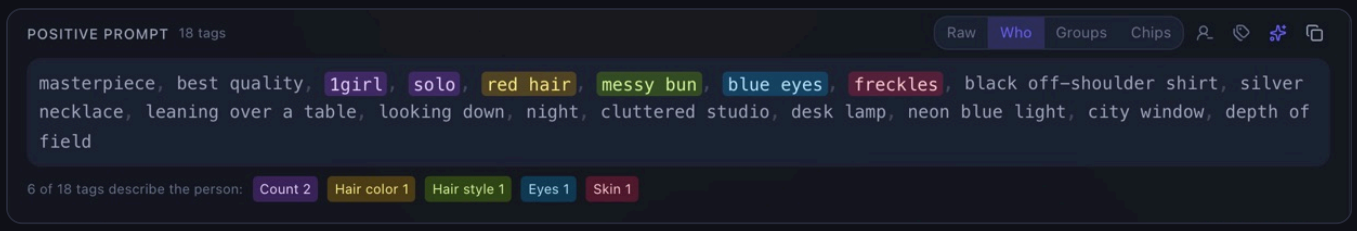
These make a prompt looser on purpose: a model that is not told the colour of a shirt, or every finger of a pose, has room to make something of its own.

CHAPTER 06 · SECTION 13

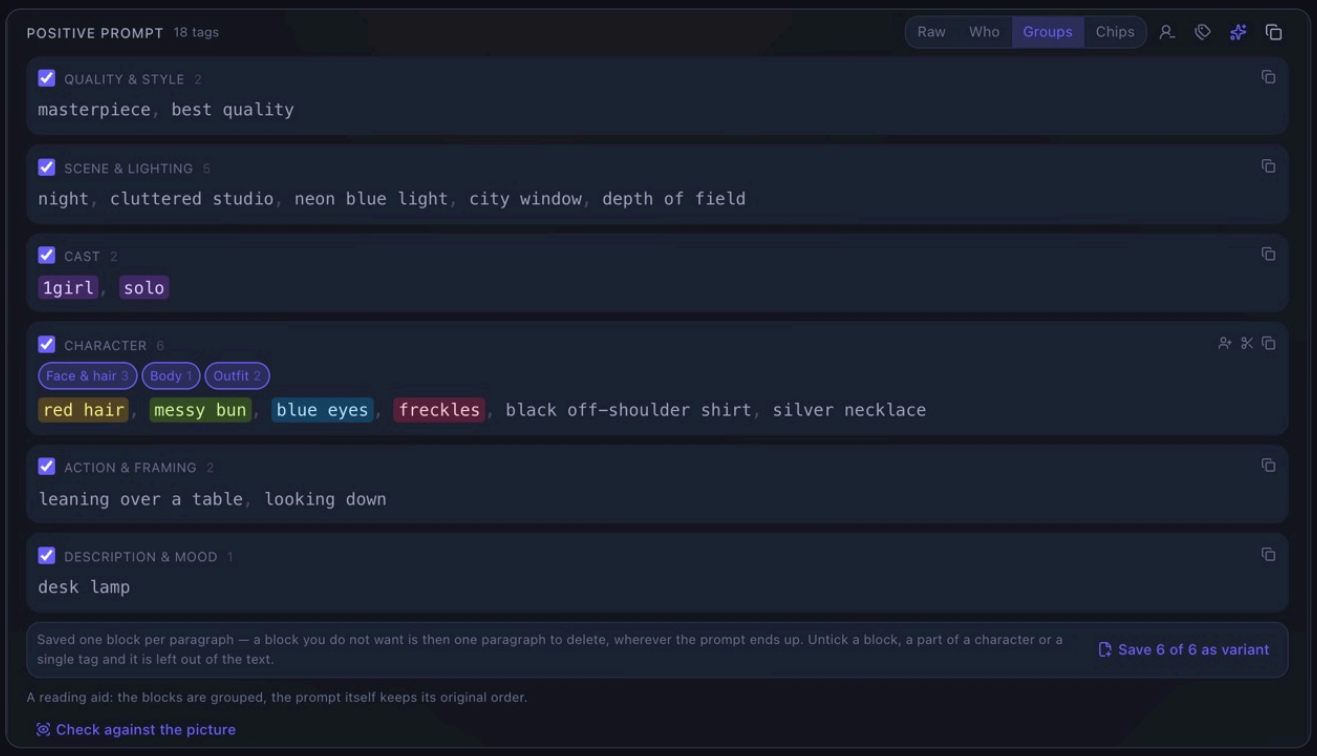
The picture's own prompt

The **Original** tab is the prompt the picture came with: the evidence that it came out this way. A reading never rewrites it. Above it are four views of the same text; they never change it, they only show it differently.

View	What it shows
Raw	The prompt exactly as it is.
Who	Colours the words that describe the person (count, hair colour, hair style, eyes, skin) and counts them.
Groups	Sorts the tags into blocks: quality & style, scene & lighting, cast, character, action & framing, description & mood.
Chips	Every tag as a button. Click one to put it in the scratchpad.



Who: 6 of 18 tags describe the person. Everything else is scene, light and quality, the part you usually want to keep.



Groups. Untick a block, a part of a character or a single tag, and **Save 6 of 6 as variant** keeps the rest as a new prompt, one block per paragraph. **Check against the picture** asks the AI whether the split matches what the picture shows.

The Original's row has its own tools: clean up, strip the borrowed person, rewrite as tags, copy, and the sparkle **Write prompts for base models**, which opens the start question. **Used LoRAs** lists the picture's LoRAs with their weights; **✦ Send to Recipe** starts a LoRA recipe with them and this picture. **Swap character** in the head (no AI) replaces the character tags directly, the whole person or slot by slot.

CHAPTER 06 · SECTION 14

Send it on

All exits are in the bottom row. Each one **saves your changes first** and closes the window, so nothing you send is lost. Beside them, every model's tab says where it stands.

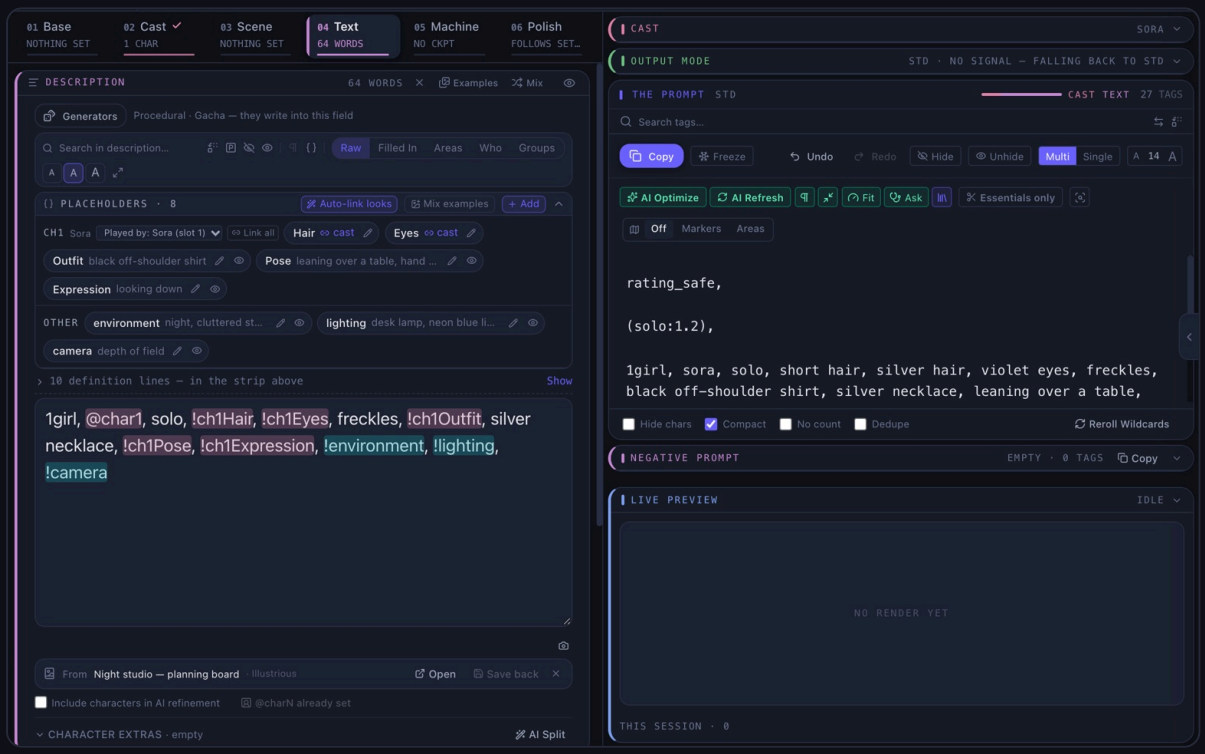
Button	What it does
To references	Copies the picture into Reference Images and selects it, ready for img2img.
To Builder	Sends the prompt on screen (Original or a model's page) into the Builder's Description. When you opened the library from the Builder, this button says **Replace**, and the ⋯ menu has **Append to Description**.
Picture + prompt	Both at once: the picture for img2img and the prompt for the Builder. The usual way to start from a picture.

The other direction works too: in the Builder's Description, click **Examples**. The same library opens without leaving the Builder, on the prompt for the Builder's model.

CHAPTER 06 · SECTION 15

In the Prompt Builder

The prompt lands in **04 Text**, in the Description. The placeholder lines are read into the **Placeholders** strip above the text, as chips per person:



Sora is cast in slot 1. Hair and Eyes are linked to the cast, Outfit, Pose and Expression hold the picture's words, and the picture's own environment, lighting and camera sit under OTHER. The finished prompt on the right says *sora, short hair, silver hair, violet eyes.*

- **A linked chip** (*cast*) reads from the cast slot and follows whoever is in it. **Auto-link looks** links hair and eyes of every cast person by themselves.
- **Played by** on a person puts any cast character into that person of the picture, or nobody.

- *These lines live in the Description. They never reach ComfyUI.* Only the finished prompt is sent.

Change the cast in 02 Cast and the same template dresses the new character. Nothing in the template needs editing. The Prompt Builder chapter covers the strip, Cast in Description (`!ch1Look` , `!ch1Core`), and the Mix Board, which combines several example pictures in one prompt.

The way back

A bar under the Description remembers which picture the text came from: **From <picture>** . **Open** opens that picture again; **Save back** (once you changed the text) writes your edit into the model's prompt it came from, or into a new prompt when it came from the Original, which is never overwritten.

CHAPTER 06 · SECTION 16

The Crosspost Helper

If you also use the Crosspost Helper, the two apps share pictures over a local bridge.

- **Look at its Reference Library**: switch the library to **Crosspost Helper** . Its pictures and their prompt versions show here read-only; the local tools are hidden while it is shown. Its tagger's rating makes the **Content** filter work there. A picture dropped onto the page in that view goes into the Crosspost Helper's library.
- **Copy local pictures over**: in the Local library, select pictures and click **Copy N to Reference Library** . It is a copy; the pictures stay here too. A picture that was copied before is skipped, so nothing arrives twice.

Goes over	Does not
the picture file; stars, category and purpose; every prompt as a version, the picture's own prompt labelled *Prompting Helper*	your manual tags, the generation data (seed, steps, sampler, model, LoRAs) and the base model. The message after the copy names what stayed behind.

A prompt taken from the Crosspost Helper into the Builder gets the same bar under the Description, with its own buttons: **Open here** shows the picture and every version of its prompt, **View in Crosspost Helper** jumps to that version over there, **Refresh** checks whether the prompt changed there since (and **Use newer** takes it), and **Save back** writes your edit there as a new version. The picture's own prompt is never written over.

CHAPTER 06 · SECTION 17

Tips

- **Read for every model you use, at once.** One reading per model costs one call each and gives every model its own words; the casting is still decided only once.

- **Read first, then recast.** A reading for your target model finds the people better than recasting an old prompt in another model's dialect.
- **Scene only for your own characters.** When you know who will stand in the picture, a template without any borrowed look is the cleanest start.
- **Poses wrong? Weight the reading.** People first or Pose only spends the words on who kneels where instead of the wallpaper.
- **Mark what you tested.** `Mark as rendered — it worked` on a tab after a good render, so you know later which prompts really work on which model.
- **Rate while it is fresh.** One click on a card's stars; purpose, category and stars make a large library searchable.

CHAPTER 06 · SECTION 18

Troubleshooting

I don't see the AI buttons, and the start question says no AI provider is set up.

Add a key in Settings → AI. Models can still be added without reading. Reading a picture needs a provider that can see images.

One model's tab says "failed".

Open that tab: it shows the model's own words. Read again, or pick the stronger vision model in the switcher on the start question. The other models are not affected.

"The reading found nobody in this picture."

The prompt may describe the person only indirectly. Assign a phrase to a person in the Words view, or read again.

"Person order differs in <model>."

The two readings numbered the people differently. Match them by hand on the sheet; the shared decisions reach only who is matched.

My changes are gone after closing the picture.

Changes are kept only after Save. Every button in the bottom row that sends something saves first, and Recast & save saves too. A reading that was still running lands in its tab anyway.

The Builder says "No character in that slot".

The template asks for a character in a slot nobody is cast in. Add a character in 02 Cast, set Played by, or type a value into the line.

CivitAI import fails for some pictures.

CivitAI hides some pictures' generation data unless you sign in. Add your CivitAI API key in Settings → CivitAI Integration.

Copy to Reference Library fails.

The Crosspost Helper is not running or not paired. Start it and pair the two apps, then try again; pictures already copied are skipped.

CHAPTER 07

Regions, References & Masks: who stands where, and what may change

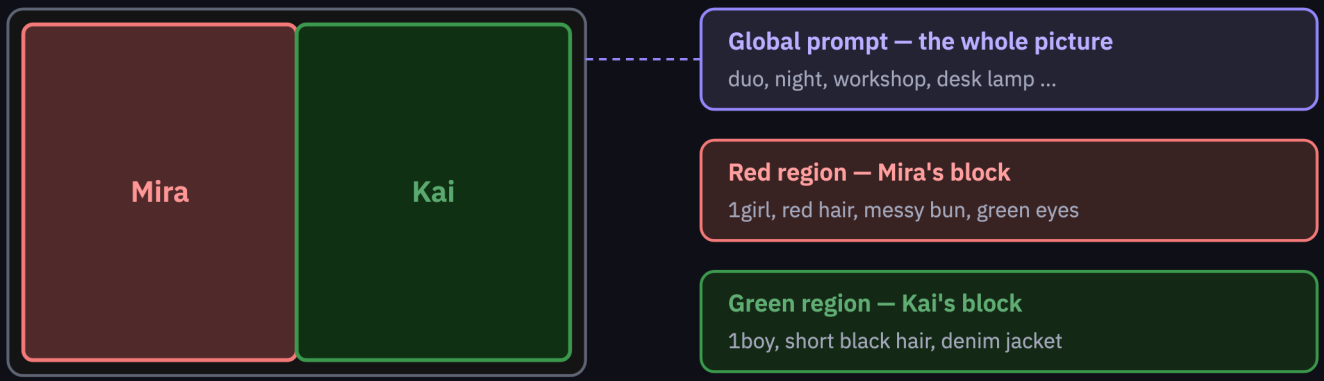
How to tell the model who stands where, how to start a render from a picture you already have, and how to mark the part of that picture the render may change.

– What these tools are for 96
01 Put your characters in the cast 97
02 Place them with Regional positioning 97
03 Switch on Regional prompting 98
04 Build a reference library 100
05 Colour maps and masks on a reference 100
06 The Reference image panel in the Builder 102
07 Start from the picture: img2img 104
08 Let only one area change: Free this area 105
09 Tips & troubleshooting 107

What these tools are for

A normal prompt is one long list of words for the whole picture. With two or more people in it, the model often mixes them up: the red hair ends up on the wrong person, or both get the same clothes. The tools in this chapter answer different questions:

Tool	What it answers	Where
Regional positioning	"Mira stands on the left, Kai on the right." A box per character on a grid.	Builder › **02 Cast**
Regional prompting	"Mira's words only apply inside her area." Each character's text is sent with its own mask, so traits stay on the right person.	Builder › **02 Cast**
Reference images	"Use this picture." A library of pictures you can crop, paint a colour map on, give masks, and send into a render.	Sidebar › **Reference Images**, and the **Reference image** panel in the Builder
img2img	"Start from this picture instead of from noise." The strength decides how much of it survives; the framing decides which part of it is used.	Builder › **05 Machine** › img2img
Free this area	"Only this part of the picture may change." A mask that limits where img2img redraws.	The Canvas, opened with **Modify image**



the region map — same shape as the render

The colours always mean the same thing

Regions are painted in five fixed colours, in this order: **Red, Green, Blue, Yellow, Pink** (short: **RGBYP**). Red is the first figure in your cast, Green the second, and so on. That is why there are at most five regions per picture.

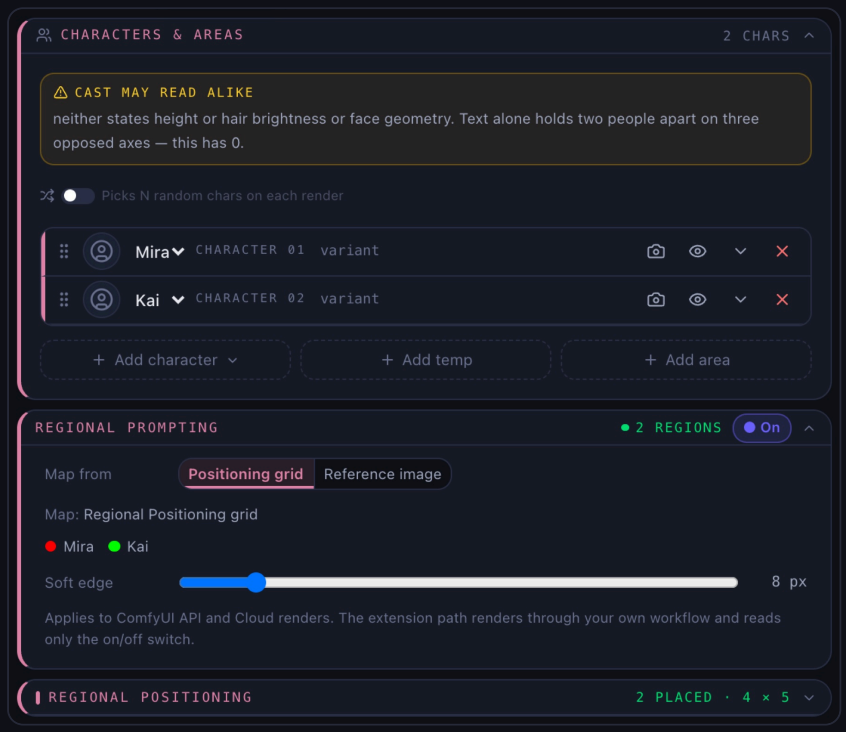
CHAPTER 07 · SECTION 01

Put your characters in the cast

Regions are drawn for *characters*, so start there. Open the Prompt Builder, click the stage **02 Cast**, and in **Characters & Areas** use:

- **Add character** — a character from your library.
- **Add temp** — a quick character that exists only in this session.
- **Add area** — a free-text part of the picture that is not a person ("a cat on the sofa", "the window"). It takes a region colour just like a character does.

The order of the rows is the order of the colours. Drag a row by its handle to change who is Red and who is Green.



Stage 02 Cast with Mira and Kai. Below them, Regional prompting is On and reads its map from the Positioning grid: Mira is red, Kai is green. The notice on top ("Cast may read alike") is the app pointing out that the two descriptions share no height, hair brightness or face geometry that would hold them apart.

CHAPTER 07 · SECTION 02

Place them with Regional positioning

Open **Regional positioning** (under Regional prompting) by clicking its title. It is a grid that stands for the picture.

1. Pick a **Preset** to start fast: **RG horizontal** puts the first character left and the second right, **RGB horizontal** does the same for three. The two vertical presets stack them top to bottom.

2. Or draw by hand: under **Mode** choose **Draw**, click a character's name, then click and drag across the grid. Boxes may overlap; the **×** beside a name removes that character's box.
3. To move a box, switch to **Position**, click it and drag. **Layer order (back → front)** appears below the grid: its ← and → arrows set who is in front where boxes overlap.
4. **Swap** mirrors the boxes, so the left figure goes right and the right one left. **Columns** and **Rows** make the grid finer (1 to 10 each). A preset splits the grid as evenly as the columns allow — with an odd number of columns one half gets the extra column.

Regional positioning with the RG horizontal preset on a 4 × 5 grid. With Output: Map it shows the colour legend and a preview of the exact map that is sent to ComfyUI, at the render size.

Text or Map?

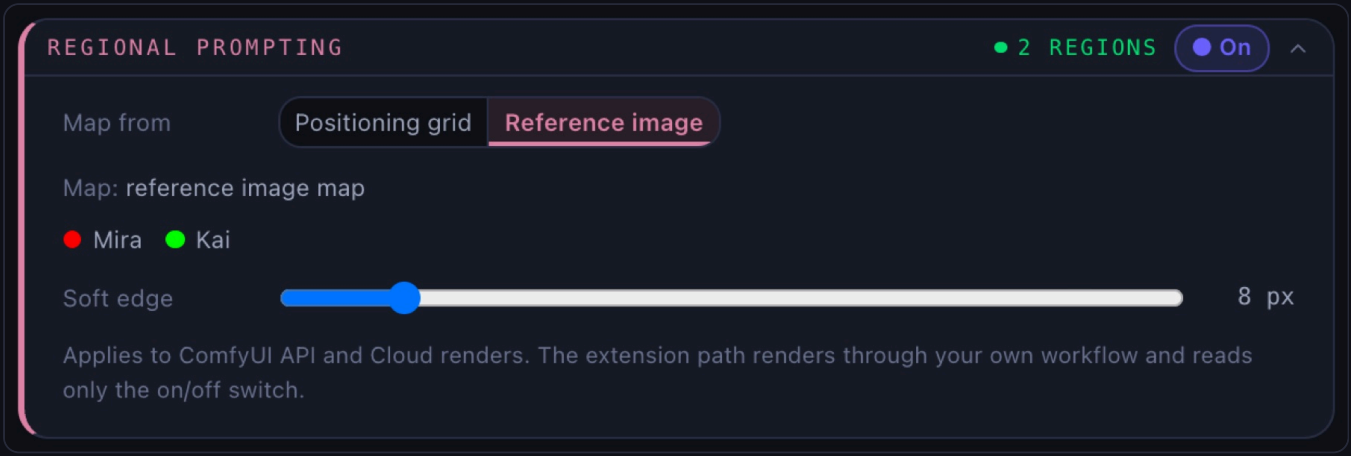
Text turns the boxes into words that are added to the prompt, like `[positions: Mira is on the left; Kai is to the right of Mira, on the far right]`. The line appears under the grid and you can edit it; it is written again whenever the grid changes. This works with every model, with or without Regional prompting.
Map turns the boxes into a colour map that Regional prompting can use (next step). The map is stretched to your render size.

CHAPTER 07 · SECTION 03

Switch on Regional prompting

In **Regional prompting**, click the switch in its header so it reads **On**. Now each character's text is sent only for its own colour, and the rest of the prompt is sent for the whole picture.

- **Map from**: **Positioning grid** uses the boxes from step 2. **Reference image** uses a colour map painted on the selected reference picture (see step 5). The choice is the only thing that decides — the source you did not pick is ignored.
- **Soft edge**: how far a region fades into its neighbour, in pixels of the render (0 = a hard edge, up to 64; default 8).
- The panel lists which colour carries which character. If someone has no colour, it says "No colour for … — goes into the global prompt", and that character's text is then part of the global prompt instead.



Map from: Reference image. The readout names the source ("reference image map") and the figures it found on it.

While it is on, the right column shows **Character & Area Prompts** under the prompt: one editable prompt per figure, exactly as it is sent for that region, with **Clean** (strips the name prefix), **Freeze** and **Reset**. Switching Regional prompting off throws those edits away and puts the cast back into the one shared prompt.

When it says "Unavailable"

Regional prompting only works with some model families. Out of the box these are **Illustrious** and **Anima**. The switch is locked (**Unavailable**) when no checkpoint is selected, when the checkpoint has no base model recorded, or when its base model is not on the list; the panel says which, and its button jumps to the Machine stage where the checkpoint is picked. Set a checkpoint's base model in the Model Library, or change the list in **Settings › Models & sources › Regional Prompting**.

It applies to renders through **ComfyUI API (local)** and **ComfyUI Cloud**. On the older extension connection your own workflow does the work and only reads the on/off switch.

Always say how many people there are

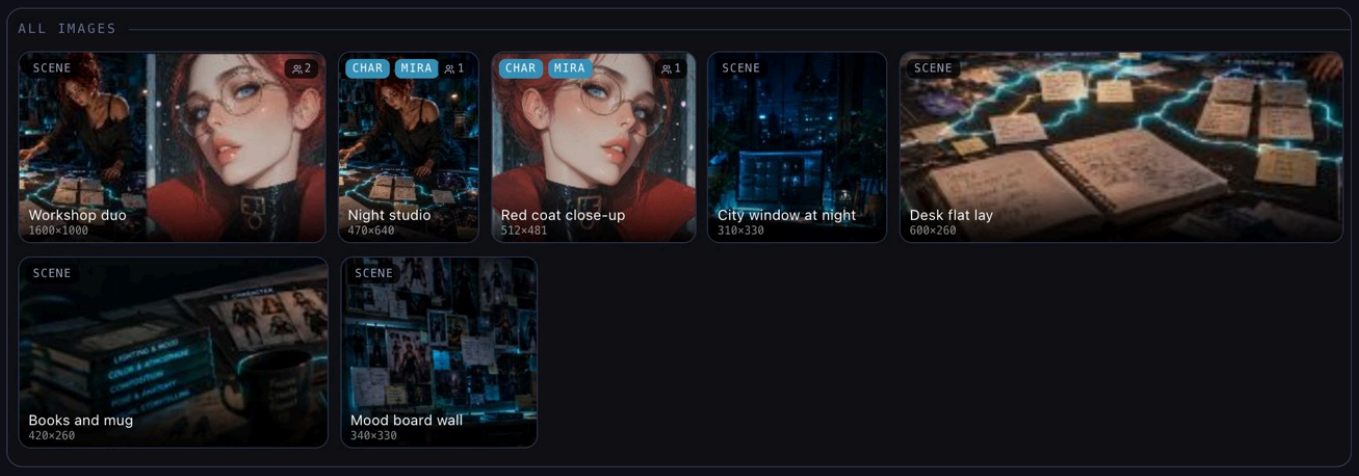
The global prompt needs a head count like `2girls`, `1boy 1girl` or `duo`. Without it the model tends to draw ONE figure and paint every region's traits onto it. The panel warns you when the count is missing.

CHAPTER 07 · SECTION 04

Build a reference library

Open **Reference Images** in the sidebar. This is your collection of pictures to work from: poses, faces, scenes, compositions.

- **Add pictures**: drag files onto the page, paste with Ctrl+V, or click **Import**.
- **Find them again**: the search looks through the picture's prompt, notes, name and crop labels; every word must appear. **Images** / **Crops** in the head switches between whole pictures and the cut-outs you made from them. Sort order and Filters sit beside it.
- **Look at one**: a click selects a picture, a double click (or **Inspect** on the card, or Enter) opens it large. ← and → walk through the set, Esc goes back to the grid. On the right you can name it, write **Notes** (searched like the prompt), mark it **Character** or **Scene**, type who is in it (**Person**) and set how many people are in it (**Persons**, 0–5). **Re-analyze** asks the vision model to classify it again.
- **Cut it to shape**: **+ Crop** under the picture (or **New crop** on a card) makes a crop in the exact shape of a render resolution, with **Best Fit** to pick the closest one. In a crop's details, **Use in Builder** selects it for your current job.



The library. Each card shows its kind (CHAR / SCENE), the person's name if you gave one, and how many people are in it.

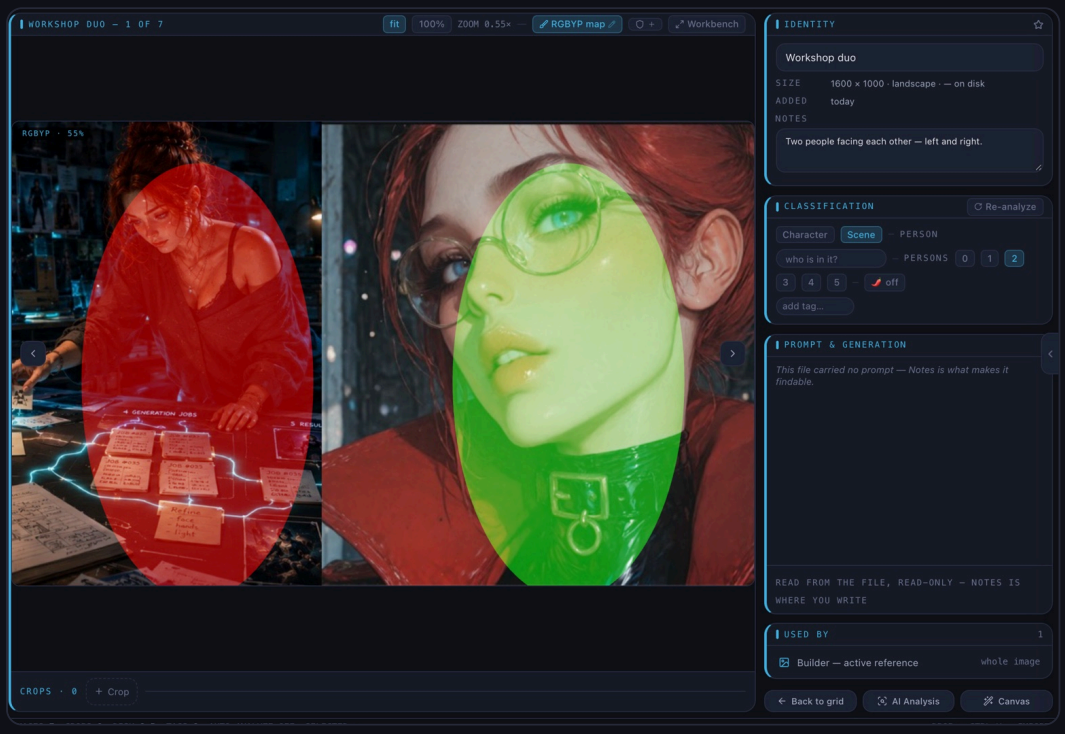
CHAPTER 07 · SECTION 05

Colour maps and masks on a reference

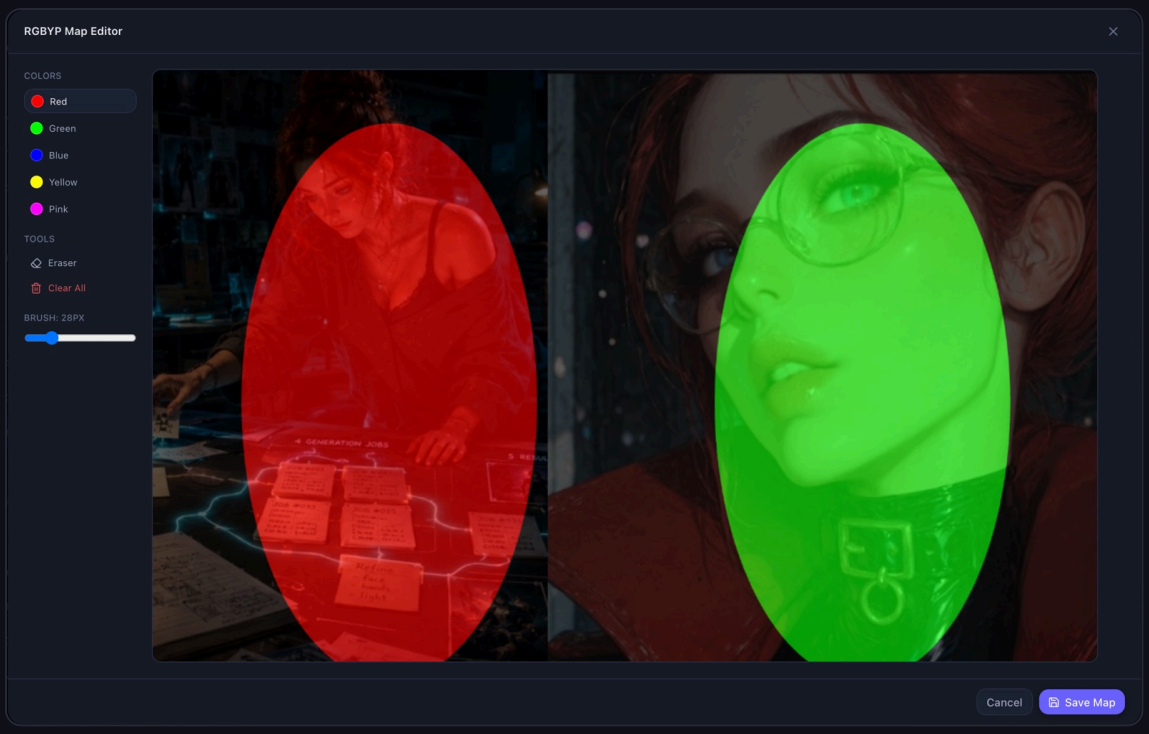
A reference picture can carry two kinds of painted overlay. Both are painted on top of the picture and saved with it; the picture itself never changes.

Overlay	What it is	Used by
RGBYP map	Coloured areas: red for the first character, green for the second, and so on. It says where each person is *in this picture*.	Regional prompting with **Map from: Reference image**
Mask	White = "this part", black = "not this part". You can keep several named masks per picture.	ControlNet's **Only inside the reference's mask**, and workflows that take a mask

In the large view, the buttons above the picture do this: **RGBYP map** shows the map over the picture (its pencil edits it; without a map the button reads **RGBYP map +** and paints a new one); each named mask has a shield button to show it and a pencil to edit it, and the shield with **+** paints a new mask. **Workbench** opens crops, map and masks side by side. **Canvas** at the bottom right opens the picture on the Canvas, whose **As reference** brings an edited result back into the library.



A picture of two people, opened large, with its RGBYP map shown over it.



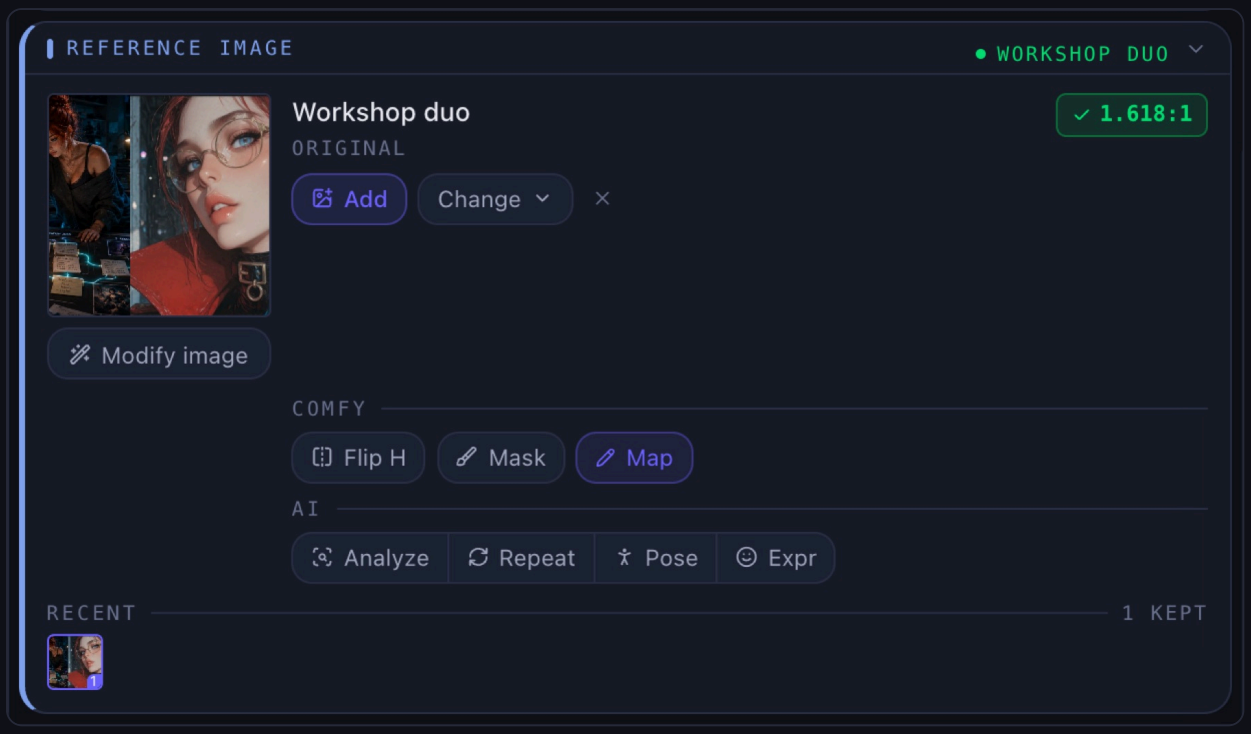
The **RGBYP Map Editor**. Pick a colour, paint over the person, use **Eraser** to fix edges (or **Clear All**), then **Save Map**. Rough shapes are enough.

> **Tip: a map from a real photo**
>
> Found a picture with exactly the composition you want? Paint Red over the left person and Green over the right one, select it in the Builder, and set Regional prompting to **Map from: Reference image**. Your characters now take those places.

CHAPTER 07 · SECTION 06

The Reference image panel in the Builder

In the Builder's right column, under the live preview, the **Reference image** panel holds the picture your current job uses. It is there when the output goes to ComfyUI.



The Reference image panel with a picture selected: its state line (ORIGINAL, a crop's name, or MODIFIED · NOT SAVED), the aspect badge, and the COMFY and AI rows.

- **Change** (or a click on the picture) picks another one from the library, including its crops. **Add** brings in a new one, and **×** clears the choice. **Recent** keeps the pictures you used lately one click away.
- The badge (here **1.618:1**) is green with a tick when your render resolution already has the picture's shape. When it doesn't, it is a button that sets the resolution to match.
- **COMFY** row — what travels into the render: **Flip H** mirrors the picture before it is sent (its map and masks are mirrored with it). **Mask** paints a *live* mask for this job; it overrides the library mask (a **Library** badge shows when a library mask is attached, and the **Mask** list under the panel picks one of the library's masks). **Map** / **Add map** edits the picture's RGBYP map.
- **AI** row — needs an AI provider in Settings: **Analyze** opens the full picture analysis, **Repeat** runs it again with the current settings, **Pose** and **Expr** read just the pose or the expression. Their text goes into **Ref notes**, never into your Description; the on/off chip there decides whether the notes go into the prompt.
- **Modify image** under the picture opens it on the Canvas (step 8).

Choosing a reference sets the resolution

Selecting a picture (or a crop) sets the render resolution to the closest one in its shape — that is what the green badge then confirms. If you want another shape, set the resolution *after* choosing the picture; img2img's **Builder resolution** then fits the picture to it (next step).

CHAPTER 07 · SECTION 07

Start from the picture: img2img

With a reference selected, open stage **05 Machine** and the **img2img** rack. Tick **Use reference**. The render now starts from your picture instead of from random noise. It works with **ComfyUI API (local)** and **ComfyUI Cloud**.

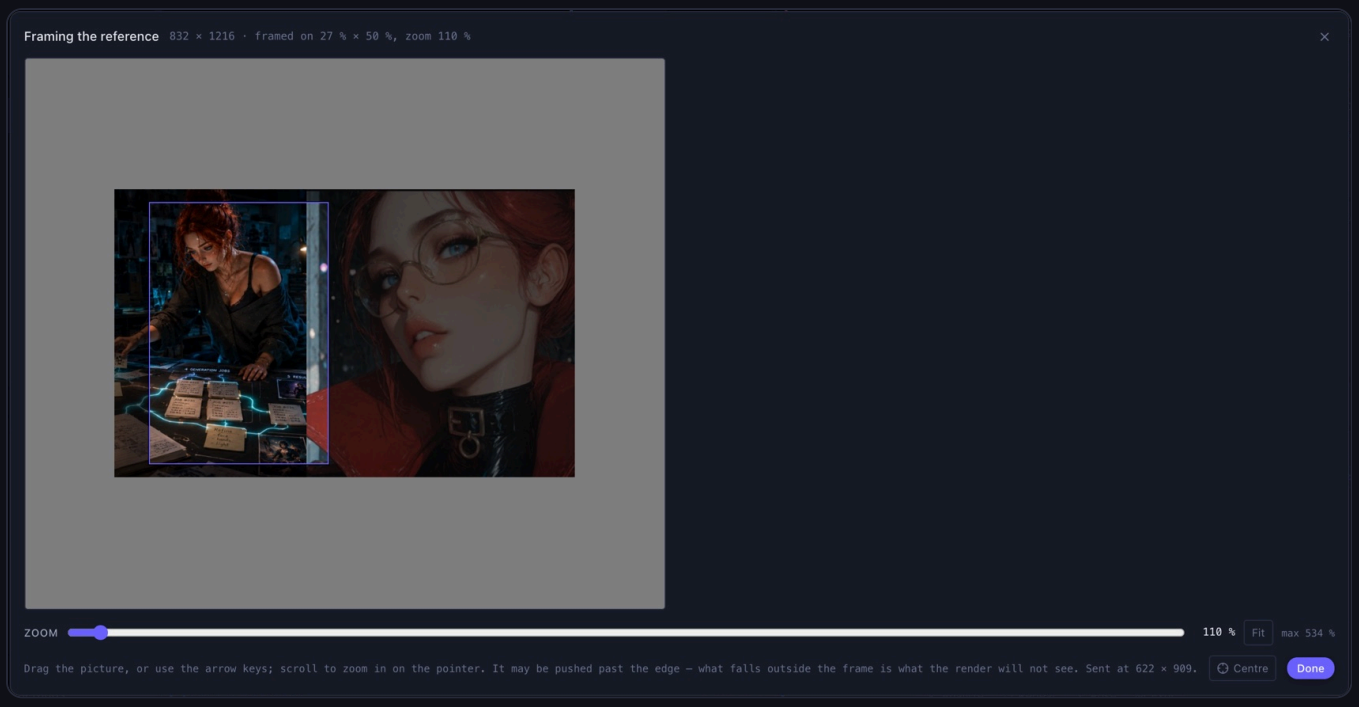
img2img on at strength 0.50, Builder resolution 832 × 1216, and the framing line with its small preview and **Adjust...**. The words next to the number say what it does.

- **Strength**: 0 keeps the picture as it is, 1 draws over it completely. Around 0.4–0.6 keeps the layout and changes the details. The line beside the slider says in words what the number does for this model. The same switch and number also sit in the Builder head (**img2img** with − / +) as soon as a reference is selected.
- **Depth map** (instead of **img2img**) takes only the shape of the scene — pose and composition — not its colours or faces. It is offered only on Anima checkpoints; on Illustrious the ControlNet rack below does depth.
- **Builder resolution** fits the picture to your render size. **Image shape** lets the picture keep its own shape and size; the resolution then becomes a readout.
- **Starts from**: **the reference**, or **a picture of its own** — paste, drop or pick one, for example a colour gradient — while the reference still serves as the scene for the regions.

Choosing which part of the picture is used

With **Builder resolution**, a small preview of the frame appears with **Adjust...**. It opens **Framing the reference**:

- Drag the picture (or use the arrow keys) until the outlined window shows what you want. The point you move is the part of the picture that lands in the middle of the render.
- Scroll to zoom in on the pointer, or use the **Zoom** slider; **Fit** goes back to no magnification. The slider stops at the most this picture allows and says so ("max 534 %").
- You may push the picture past the edge; the empty part of the frame is sent as flat grey. The line at the bottom says the size the cut is sent at.
- **Centre** centres it, **Done** keeps it. Escape, the **×** or a click outside puts it back as it was.



A wide picture framed for a tall render, at 110 % zoom. The outlined window is exactly what the render will see.

Fit description

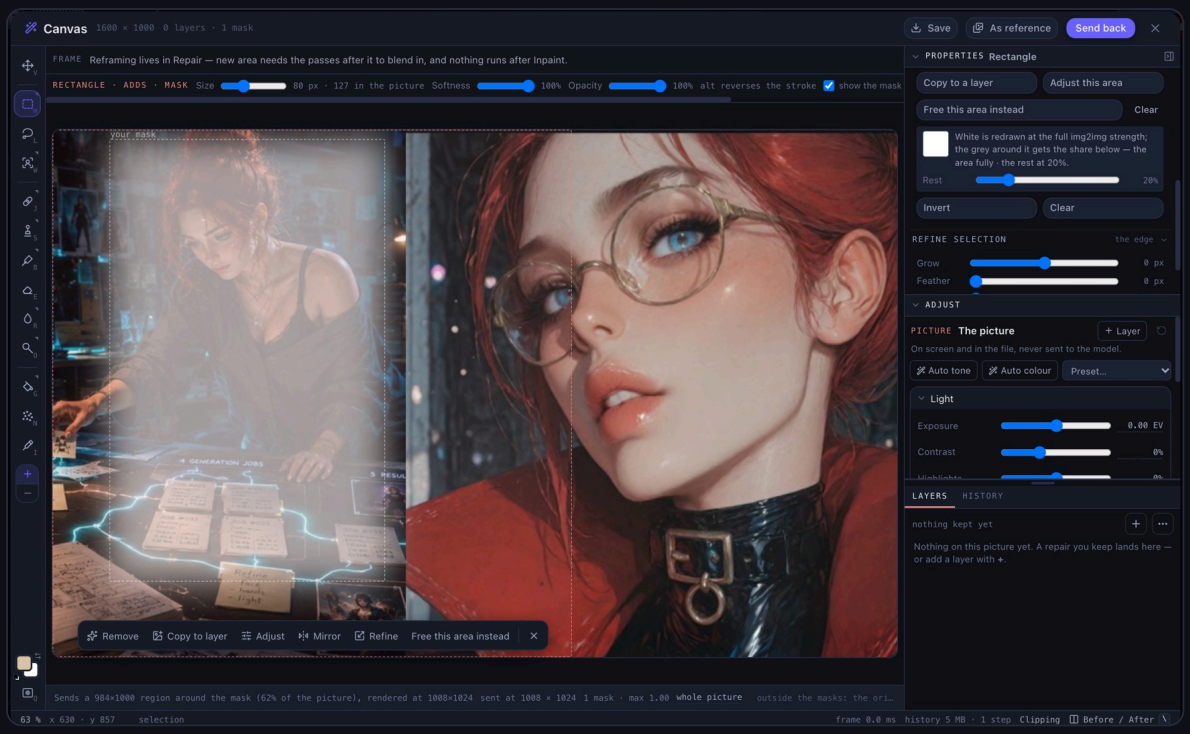
When the frame cuts part of the picture away and your Description is not empty, **Fit description** appears on the framing line. It asks the AI to cut the Description down to what this frame still shows — nothing is added, the rest stays word for word, and your Description itself is not changed. The fitted text is used only while frame and Description stay the same; otherwise the button reads **Fit again** and the full Description is sent.

CHAPTER 07 · SECTION 08

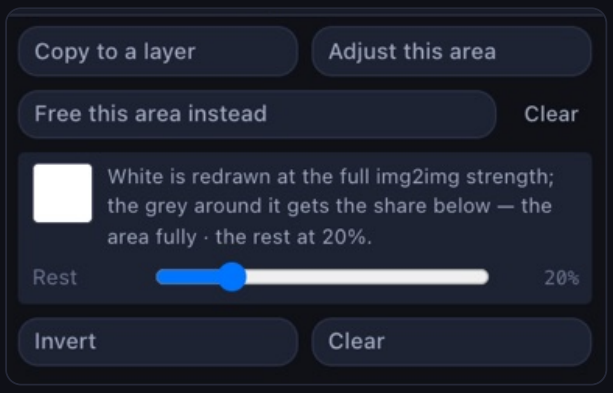
Let only one area change: Free this area

Normally img2img changes the whole picture by the same amount. If you want to redraw, say, the person but keep the room, free just that part:

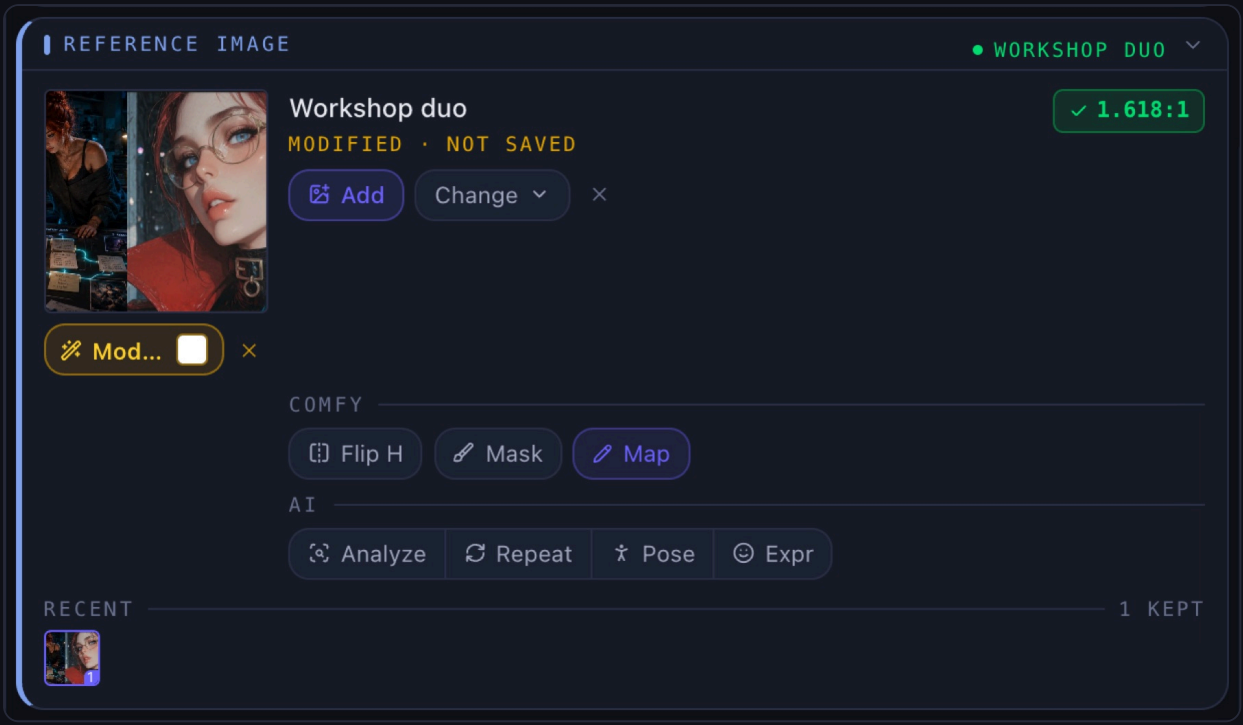
1. In the Reference image panel, click **Modify image**. The picture opens on the Canvas as a temporary copy — the library picture is never changed.
2. Select the area with a selection tool on the left rail: **M** Marquee (rectangle, ellipse), **L** Lasso (freehand or polygonal), **W** Select (Object, Select subject or Wand; see the Canvas chapter). **Q** paints the selection with a brush.
3. Click **Free this area** — in the bar that appears at the selection, or in the **Selection** card on the right.
4. Set **Rest**: how much the rest of the picture may change, as a share of the strength. 0% freezes it completely.
5. Click **Send back** in the head. The panel now shows **Modified** with a small mask thumbnail, and the img2img line says **in the freed area only**.



A rectangle around the left figure has been freed. White is redrawn at the full img2img strength; with Rest at 20 % everything else changes a little.



The Selection card after **Free this area**: the freed mask, what it means in words, the **Rest** slider, and **Clear** to go back to one strength for the whole picture.



Back in the Builder: **Modified** with the mask beside it, the state line MODIFIED · NOT SAVED, and the **×** that throws the painted copy away.

> **A selection alone does nothing outside the Canvas**
>
> Selecting an area only tells the Canvas's own tools (Fill, **Copy to a layer**, **Adjust this area**) where to work. Only **Free this area** turns it into something the render uses. The Selection card reminds you when you have a selection but nothing freed.

To undo it: click **Clear** next to **Free this area instead** on the Canvas, or click the **×** next to **Modified** in the Reference image panel to throw the painted copy away. The freed area comes back when you reopen the Canvas, so you can adjust it later — until the picture itself is replaced, reframed or punched into, which drops it.

The Canvas's other tools work on the copy too: paint, heal, grade, or **Punch in** (Properties) to take a window of the picture and scale it up without changing its shape. See the Canvas chapter.

CHAPTER 07 · SECTION 09

Tips & troubleshooting

> **Both people look the same, or the traits are swapped.**
>
> Check that Regional prompting says **On** and lists every character with a colour. Make sure the global prompt names the head count (`duo` , `1boy 1girl`). A little Soft edge (8–16 px) helps figures that touch, and the "Cast may read alike" notice tells you when the two descriptions give the model nothing to hold them apart.

Regional prompting says "no map".

With Map from: Positioning grid, draw at least one box in Regional positioning. With Map from: Reference image, the selected reference needs an RGBYP map — paint one with **Add map**.

A character is listed as "goes into the global prompt".

There are more figures than painted colours. Give that character a box (grid) or paint its colour on the reference map. A sixth figure never gets a colour.

The switch is locked and says "Unavailable".

See step 3: the selected checkpoint's base model has to be on the Regional Prompting list in Settings.

img2img says "No reference selected".

Pick a picture in the Reference image panel first (Change). The img2img switch only works with a picture selected.

My resolution changed by itself.

Choosing a reference, or sending a modified one back from the Canvas, sets the resolution to the picture's shape. Set the resolution you want afterwards.

My render ignored the area I marked.

You probably selected it but did not press Free this area, or the picture was replaced afterwards. Reopen Modify image: if the Selection card shows no mask thumbnail, nothing is freed.

img2img says it is off "while a picture is handed in".

A picture sent from Repair ("Send to Final") is the starting point of this render. Its row in the img2img rack has a button to start from noise again; after that the reference can be used — one source at a time.

Do I need Regional prompting to place people?

No. Regional positioning with Output: Text adds the positions as words and works with every model. Regional prompting is the stronger tool, for models that support it.

CHAPTER 08

AI Analysis: from a picture to words

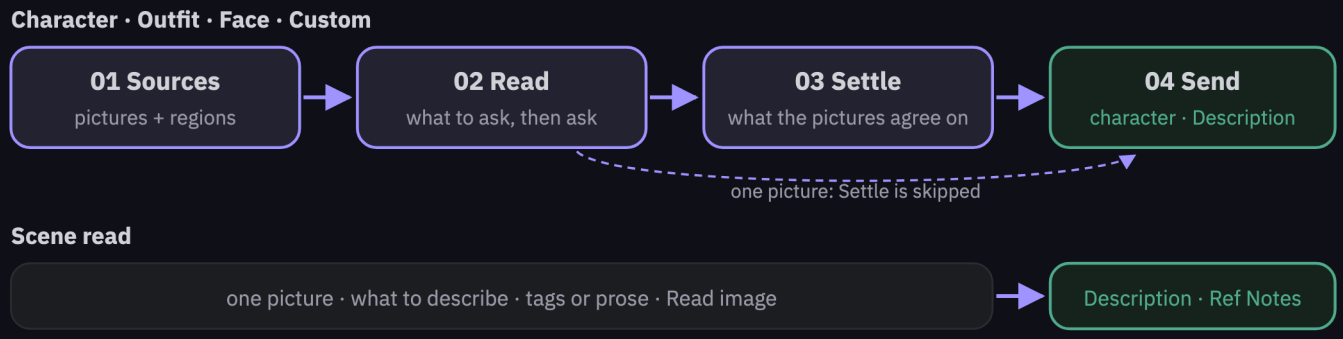
Let an AI look at a picture and write down what it sees: a character's looks as values you can save, checked against several pictures, or a whole scene as text for your prompt.

— What it is for 110
01 Add an AI key 110
02 The page at a glance 111
03 Sources: add pictures and mark areas 112
04 Read: choose what to ask, then ask 113
05 Settle: what the pictures agree on 114
06 Send: save it as a character 115
07 Scene read: text for your prompt 117
08 Other places the AI reads pictures 118
09 Tips 118
10 Troubleshooting 119

What it is for

You have a picture you like: a reference photo, an artwork, one of your own renders. You want the app to describe it in words the image model understands, so you don't have to guess the tags yourself. **AI Analysis** (in the left sidebar) sends the picture to an AI vision service and turns the answer into something you can use:

- **A character.** Identity, hair, eyes, face, skin, body, makeup, extras: read from one or several pictures of the same person, compared, and saved as a new character or merged into one you already have.
- **An outfit or a face** on its own, when only that part matters.
- **A whole scene** as tags or prose, sent to the Prompt Builder's **Description** or **Ref Notes**.



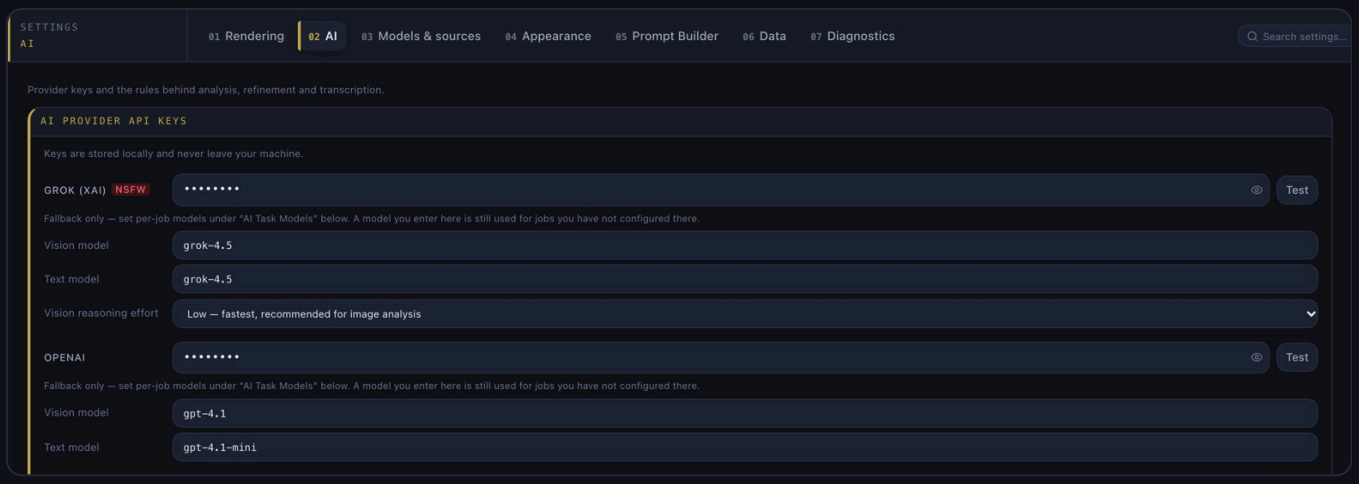
You need an AI service with a vision model

The app doesn't read pictures itself. It sends them to a service you choose: Grok (xAI), OpenAI, Google Gemini, Anthropic Claude, GLM (Z.AI), OpenRouter, or a local Ollama. All except Ollama need an API key from that service, and the service bills you for each read. ComfyUI is not needed for any of this, except for the optional test render in Send.

CHAPTER 08 · SECTION 01

Add an AI key

1. Open `Settings` → `AI`. The first section is **AI Provider API Keys**.
2. Paste your key into the field of your service and click `Test`.
3. Click `Save` at the top right.



Settings → AI → AI Provider API Keys. Keys are stored on your computer only. The **NSFW** badge marks the service that also reads explicit pictures (Grok).

For a local **Ollama** you enter its address (default `http://127.0.0.1:11434`) instead of a key. Each service also has a **Vision model** and a **Text model** field. They are a fallback: which model reads pictures is set per job under **AI Task Models** further down (the job is called *Image analysis*). The defaults work; change them only if you know you want another model.

> **Two things to know**
> **AI Analysis** only appears in the sidebar once a key (or an Ollama address) is saved. And the page's **Vision** picker only chooses the service while *Image analysis* has no models of its own under AI Task Models; once you set some there, those are used. The rules the Scene read sends with every picture are under **Image Analysis** in the same tab (Temperature, Max Tokens, Tags-mode and Prose-mode Rules).

CHAPTER 08 · SECTION 02

The page at a glance

Open **AI Analysis** in the sidebar. The row at the top decides what kind of read you make:

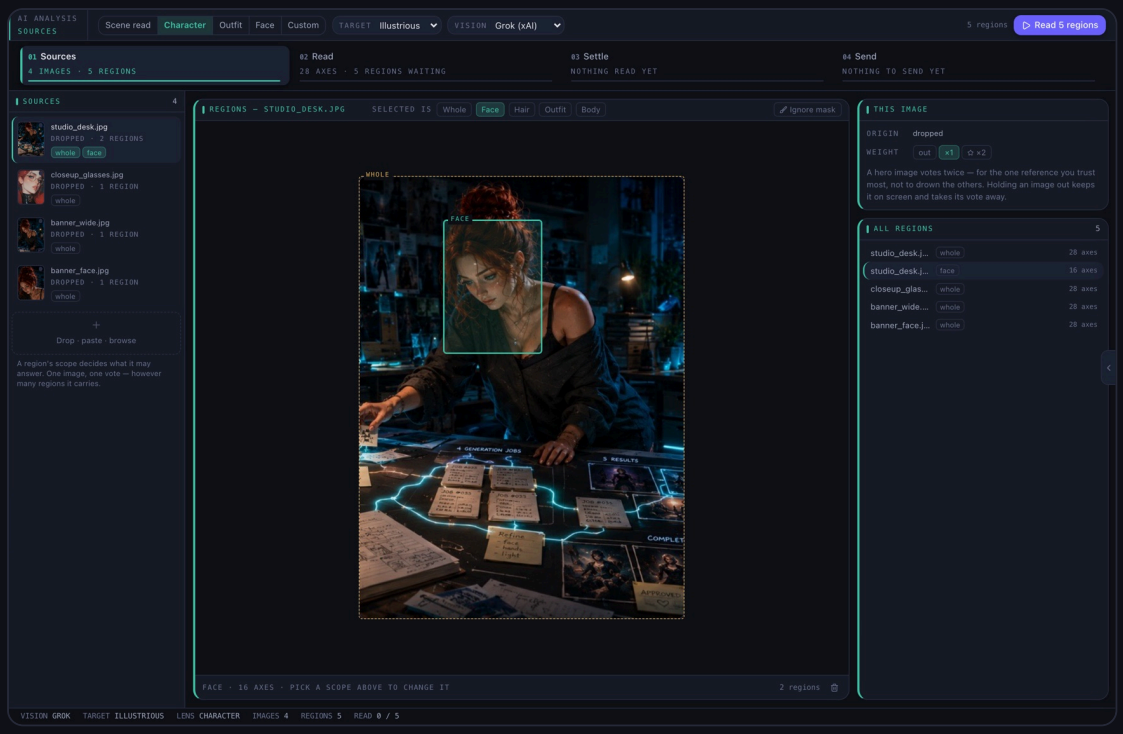
Control	What it does
Scene read	Describes one whole picture as tags or prose, for your prompt. Has its own page layout (section 7).
Character	Reads what makes a person: identity, hair, eyes, face, skin, body, chest, makeup, extras. Several pictures vote on the result.
Outfit	Reads only what is worn, not who wears it. A new picture gets an Outfit region.
Face	Reads only the head and face. A new picture gets a Face region.
Custom	Starts with every detail set to skip; you pick what to ask yourself.
Target	The image model the result is written for (Illustrious, Anima, Krea2 ...). It changes how the result is worded, not what is read. Not shown for the Scene read.
Vision	Which AI service does the reading. Lists only services with a key.
Read N regions	Starts the read. The number is how many areas will be read (**Read image** in the Scene read).

Below the head, the four stages **01 Sources · 02 Read · 03 Settle · 04 Send** each say where they stand ("4 images · 5 regions", "14 settled · 2 contested"). Every stage stays clickable; one that has nothing to work on yet says what it is waiting for. The **Sources** column on the left stays beside every stage, and the status bar at the bottom counts images, regions, reads, settled and contested values and outliers.

CHAPTER 08 · SECTION 03

Sources: add pictures and mark areas

1. Pick **Character** in the top row.
2. Drop pictures onto the box **Drop · paste · browse** on the left, paste one with Ctrl+V (Cmd+V on a Mac), or click the box to pick files. Use **several pictures of the same person** if you can; they vote on the result (section 5).
3. Click a picture to open it in the middle. Every picture starts with one region covering all of it (**whole** for Character).
4. To read only part of it, choose a scope above the picture (**Whole · Face · Hair · Outfit · Body**, "next drag is") and drag a rectangle over the area. The scope decides which details that area may answer: a face region is never asked about shoes or body build.



Four pictures of one character. The first carries a **whole** and a selected **face** region (16 axes). On the right, **This image** sets its weight, and **All regions** lists every area that will be read.

- **Select a region** by clicking it. The scope chips then change *its* scope ("selected is"); the bin at the bottom removes it.
- **Weight** (This image): **×1** is one vote. **★ ×2** makes it the **hero** image, the one you trust most, which counts twice (also the star on the thumbnail). **out** keeps the picture on screen but takes away its vote, and it is not read at all.
- **Ignore mask**, shown while a region is selected: paint over parts the AI should not look at, for example a second person in the frame. Those parts are blacked out before the region is sent.
- **One picture, one vote**, however many regions it carries. A face region on your clearest picture sharpens the face without giving that picture a second vote.

Pictures also arrive from other places: **Analyze** under a reference image in the Prompt Builder, and **AI Analysis** on a picture or crop in Reference Images. They land here as a new source, already selected.

CHAPTER 08 · SECTION 04

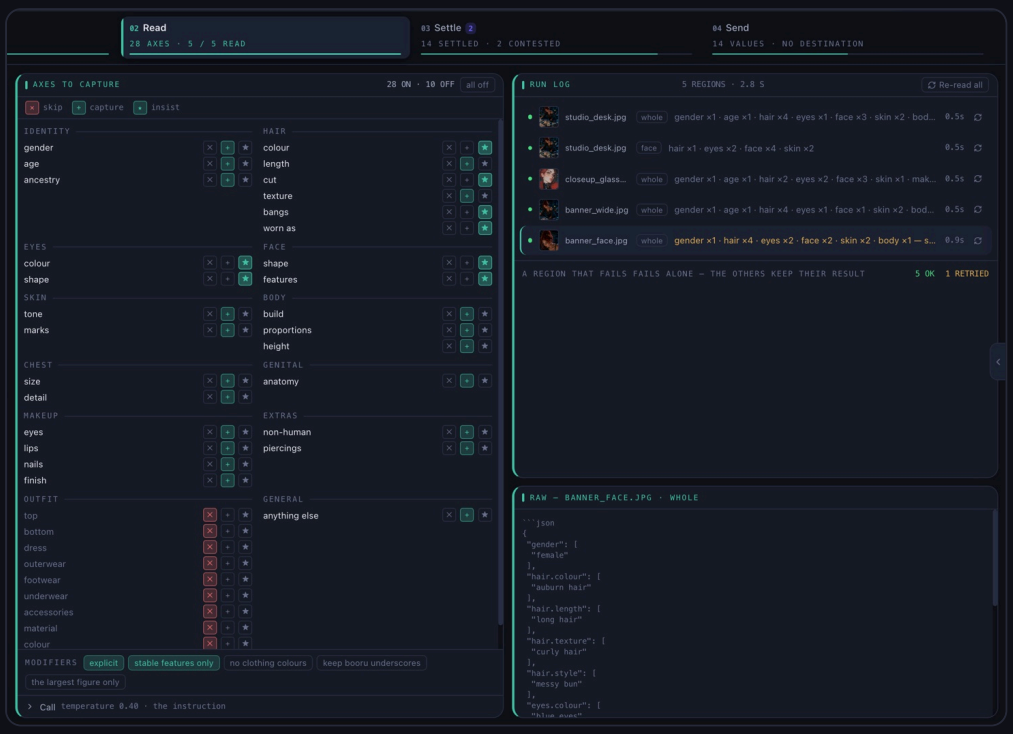
Read: choose what to ask, then ask

Open **02 Read**. On the left, **Axes to capture** lists every detail the AI can be asked about, grouped as Identity, Hair, Eyes, Face, Skin, Body, Chest, Genital, Makeup, Extras, Outfit and General. Each has three buttons:

- **✕ skip**: don't ask about it.
- **+ capture**: ask about it.
- **★ insist**: ask, and tell the AI this is what makes the person recognisable. The Character lens insists on hair colour, cut, bangs and style, eye colour and shape, and face shape and features by default: get

those wrong and the character looks like someone else.

Under the grid, **Modifiers** add rules: *explicit* and *stable features only* (both on by default), *no clothing colours*, *keep booru underscores*, *the largest figure only*. The folded **Call** row holds the temperature, a field for extra instructions, and the exact instruction one whole-image region is sent. Then click **Read N regions** at the top right.



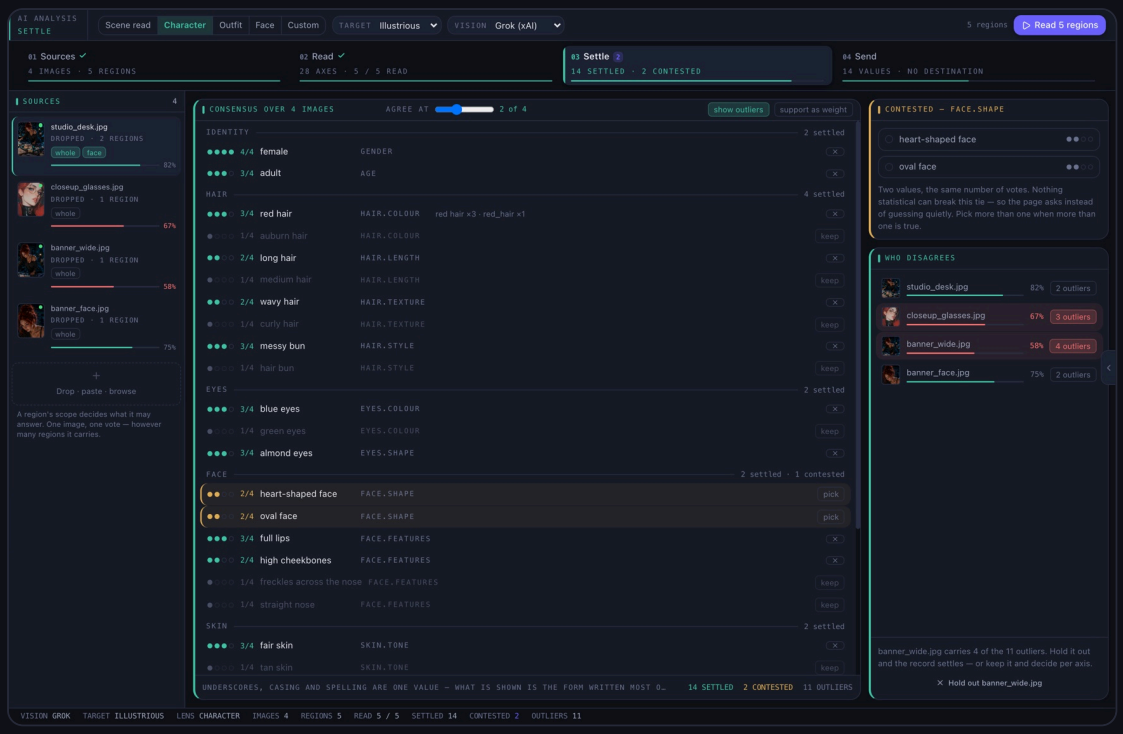
The axes on the left, the **Run log** on the right. One line per region, with what came back (hair ×4 · eyes ×2 …) and how long it took. The selected line shows its raw answer below; this one hit a rate limit and **succeeded on attempt 2**.

- Regions are read one after another. A busy service is asked once more by itself, and the log marks that line **retried**.
- **A region that fails fails alone**: the others keep their results. The circular arrow on a line reads just that region again; **Re-read all** starts over.
- A region whose scope has no axis left to answer is **skipped**, not sent ("no axis this region can answer").
- When the last region is back, the page moves on by itself: to **Settle** with two or more pictures, straight to **Send** with one.

CHAPTER 08 · SECTION 05

Settle: what the pictures agree on

With two or more pictures, the app compares their answers. Spellings are merged first (*red_hair*, *Red hair* and *red* on the hair colour are one value), then every picture casts one vote per value.



Consensus over four pictures. Each value shows how many pictures backed it (**3/4**, **2/4**); faded rows fell below the line. The face shape is a tie, so it is **contested** (top right). **Who disagrees** shows that *banner_wide.jpg* carries most of the outliers.

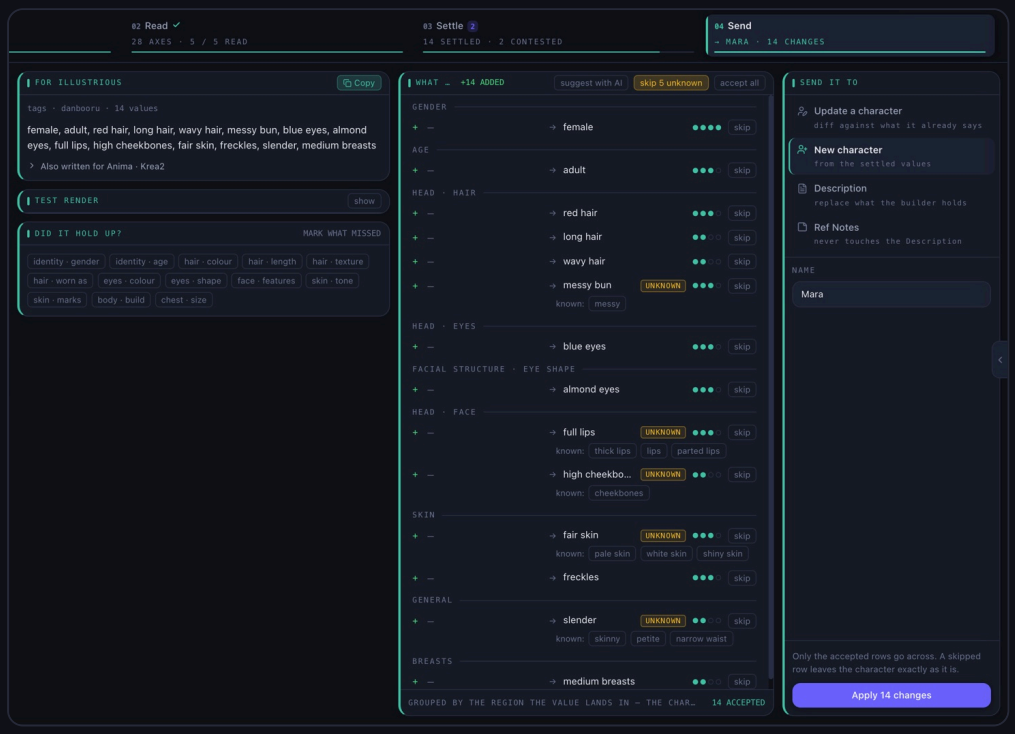
- **agree at**: the slider sets how much of the vote a value needs before it counts ("2 of 4"). It needs at least two pictures.
- A detail with one answer (hair colour, face shape) keeps the value with the most votes. An exact tie at the top is **contested**: the app asks instead of guessing. Click **pick** on the row or the value in the **Contested** panel; you may pick more than one when more than one is true.
- **keep** puts a faded value (an **outlier**) back in: one picture may be right. ✕ strikes a settled value; **struck · undo** brings it back.
- **show outliers** shows or hides the faded rows.
- **Hold out …** (bottom right) takes the picture that disagrees most out of the vote; **Let it vote again** undoes it.
- **support as weight** writes values that only some pictures backed with a lower weight, for target models that read weights.

CHAPTER 08 · SECTION 06

Send: save it as a character

Open **04 Send**. On the left is the finished result, written for your **Target** model, with **Copy**; **Also written for …** unfolds the same result for the other model families. In the middle, **What changes** lists every value grouped by where it lands in a character (Head · Hair, Facial structure · Eye shape, Skin, Breasts …). On the right, under **Send it to**, you pick where it goes:

Destination	What happens
Update a character	Pick one under **Which character**. The middle column then compares with what it already holds: **+** added, **~** replaced, **=** unchanged. Writes into the base character; per-model variants are untouched.
New character	Creates a character from the settled values. Type its **Name** first.
Description	Replaces the Prompt Builder's Description with the result for the Target.
Ref Notes	Writes the result into the reference notes. Never touches the Description.



The result for Illustrious on the left, the fourteen changes in the middle, **New character** named *Mara* on the right. **Apply 14 changes** writes them.

- **skip** leaves a single row out; only accepted rows are written. **accept all** brings every row back.
- **UNKNOWN** marks a value that isn't in the app's tag database (for tag models), so the image model may not know it. Keep it if you know it's right, click one of the **known:** alternatives to write that instead (**undo** puts the read's value back), let the AI find some with **suggest with AI**, or skip them all with **skip N unknown**.
- **Test render** (**show**) renders the result once, beside two of your pictures, to check whether it reproduces the person. It goes out as its own job, so nothing open in the Builder is touched. It needs an enabled workflow tagged *character-preview* in the Workflows library and a place to render it (ComfyUI API or ComfyUI Cloud).
- **Did it hold up?** lists every detail of the result. Click one the render missed: it is marked and the page jumps back to Settle to decide it again.

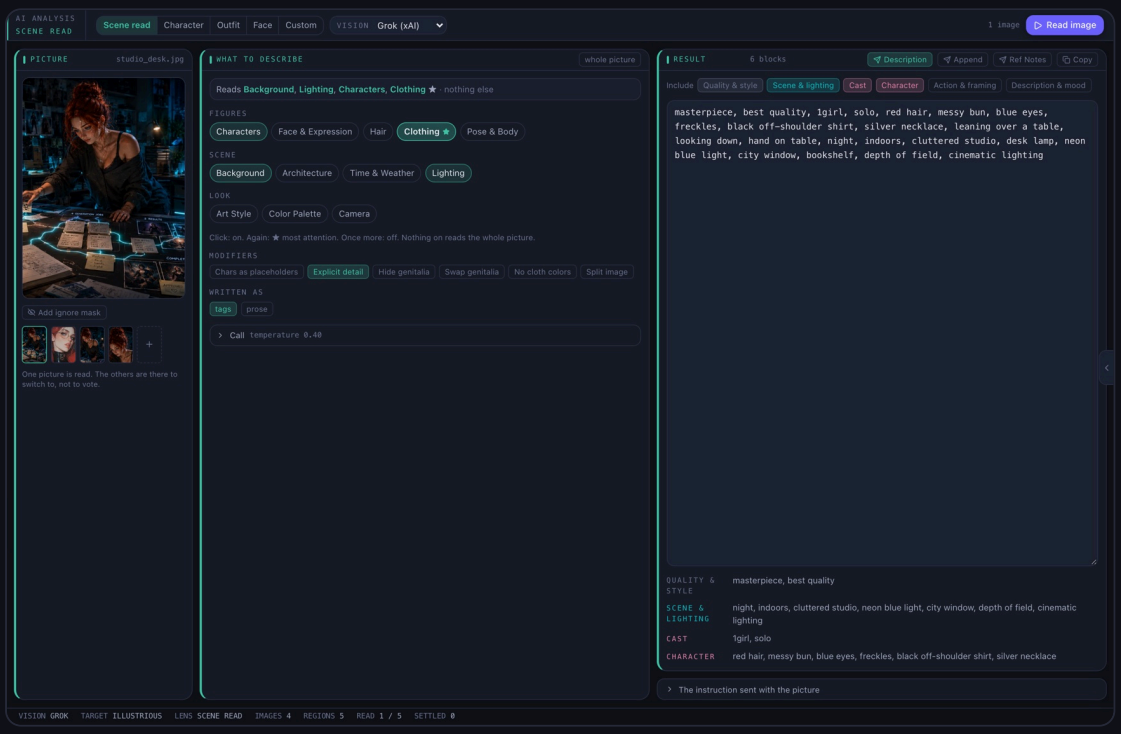
Finally click **Apply N changes**. The button briefly says **Done**.

CHAPTER 08 · SECTION 07

Scene read: text for your prompt

When you want a description of the whole picture rather than a character, click **Scene read** in the top row. The stages disappear and the page shows three columns:

1. **Picture**: the one that is read. The thumbnails below are your other pictures, there to switch to, not to vote. **Add ignore mask** hides parts of it.
2. **What to describe**: topics in three groups, **Figures** (Characters, Face & Expression, Hair, Clothing, Pose & Body), **Scene** (Background, Architecture, Time & Weather, Lighting) and **Look** (Art Style, Color Palette, Camera). Click once to turn a topic on, again for ★ most attention, once more to turn it off. The line on top says what will be read ("Reads Background, Lighting ... nothing else"); with nothing on, the whole picture is read, and **whole picture** clears every pick. Below: **Modifiers** (Chars as placeholders, Explicit detail, Hide genitalia, Swap genitalia, No cloth colors, Split image), **Written as** tags or prose (prose with short, medium or detailed), and the folded **Call** row.
3. Click **Read image**. The answer appears under **Result**, where you can still edit it.



A tag result, split into blocks (Quality & style, Scene & lighting, Cast, Character ...). Under **Include** you choose which blocks the buttons send.

The buttons above the result send the **included blocks**:

- **Description** replaces the Prompt Builder's Description.
- **Append** adds them to the end of the Description.
- **Ref Notes** writes them into the reference notes and never touches the Description.
- **Copy** copies them to the clipboard.

> **Check Include before you send**
> After a read only the **first** block is included. Turn on the others you want, or the Description gets only "masterpiece, best quality". A prose answer is one block and goes as a whole.

The instruction sent with the picture, under the result, shows exactly what the AI was asked.

CHAPTER 08 · SECTION 08

Other places the AI reads pictures

The same reading is available elsewhere without opening this page:

- **Prompt Builder → reference image**, the **AI** row: **Analyze** opens this page as a window, on the Scene read, with the reference already loaded. **Repeat** reads again with the Scene read's current settings, and **Pose** and **Expr** do quick pose or expression reads. All three write prose into Ref Notes (replacing what was there) and never touch the Description.
- **Reference Images**: **AI Analysis** on a picture or a crop hands it to this page.
- **Example Images** read a picture together with its prompt; that has its own chapter.
- **AI Generate** (Prompt Builder, stage 06 Polish, under Character Settings) writes a scene into its own slot and can read a reference picture, too. **AI Optimize** and **AI Refresh** in the prompt toolbar rewrite the prompt text itself; they don't look at pictures.

CHAPTER 08 · SECTION 09

Tips

- **Three to five pictures** of the same character give a much steadier result than one. Mix a close-up for the face with a full-body shot.
- **Draw a face region** on your clearest picture. A face read on a small area is more precise than the same face as a detail of a full-body shot, and it adds no extra vote.
- Mark your best picture as **hero (★ ×2)** instead of removing the others.
- **Switching Target after the read** rewrites the result for the other model. You don't need to read again.
- **Look at the raw answer** in the Run log when a value surprises you, before you decide in Settle.
- For explicit pictures choose a service that allows them (the NSFW badge in Settings). Others may refuse or leave details out.

CHAPTER 08 · SECTION 10

Troubleshooting

There is no AI Analysis in the sidebar.

No key is saved. Add one under Settings → AI → AI Provider API Keys (or an Ollama address) and click Save.

The Vision picker only shows "Ollama (local)".

Only Ollama is set up; every other service needs its key saved first.

The Read button can't be clicked.

There is no picture or region yet, or every detail is set to skip (the Call row then says *Every axis is off — nothing would be asked*). Set some to + or pick a lens again.

A line says "no axis this region can answer".

The region's scope and the lens don't overlap: an Outfit read on a *whole* region, for example, since Whole covers the person, not the clothes. Select the region and give it the Outfit scope.

One region failed.

Click the arrow on that line in the Run log to read it again. The other regions keep their results. Repeated failures usually mean the service refused the picture or the key has run out of credit; the raw panel shows the error.

Settle says there is nothing to compare, or the slider is greyed out.

It needs at least two pictures that vote. With one picture, go straight to Send.

Apply does nothing.

Pick a destination under Send it to first, and for Update a character, the character.

The result describes the wrong person.

Draw a region around the right person, paint an Ignore mask over the other one, or turn on the modifier *the largest figure only*.

Test render isn't available.

It needs an enabled workflow tagged *character-preview* in the Workflows library, plus a connected ComfyUI API or ComfyUI Cloud (see the Getting Started chapter). Until then the Render button says *No character-preview workflow is enabled in the workflow library*.

CHAPTER 09

Lineup & Tuning

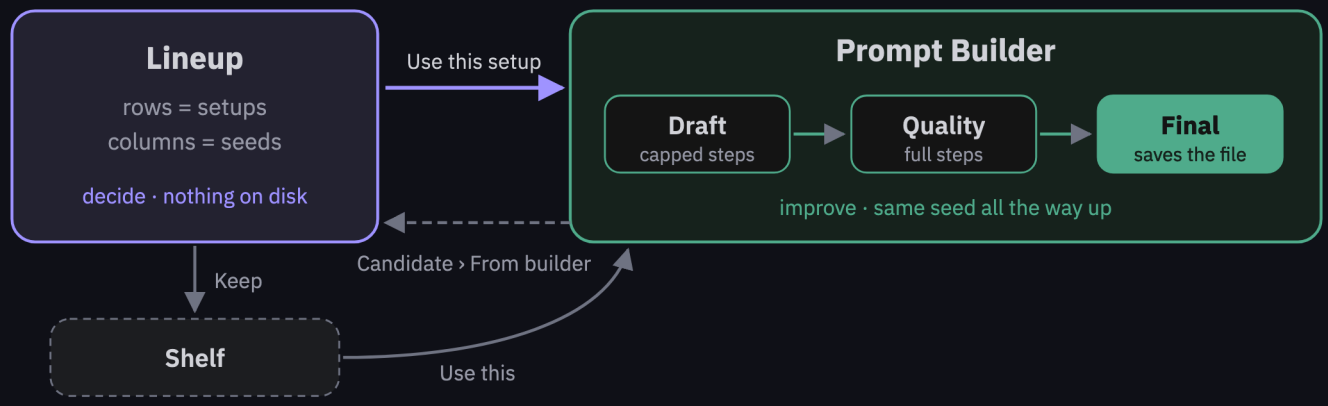
Decide first, then improve. The Lineup puts several setups and seeds side by side so you can pick a winner; the Builder's Draft, Quality and Final buttons then take that one picture from a quick look to a saved file.

01 Open the Lineup 122
02 Read the board 123
03 Add candidates 124
04 Seeds and the cost of a glance 125
05 Judge 126
06 Keep an idea on the Shelf 126
07 Take the winner into the Builder 127
08 Tune: Draft, Quality, Final 128
09 Tips 129
10 Troubleshooting 129

What this is for

A single render tells you very little. Was the checkpoint wrong, the LoRA too strong, or was it just an unlucky seed? The **Lineup** answers that by rendering a grid: every **row** is a candidate setup (workflow, checkpoint, LoRAs, sampling and prompt), every **column** is a seed. You look, give thumbs, and take the winner into the Builder with one click.

Back in the Builder, the render buttons **Draft · Quality · Final** are the **tuning** part: cheap looks while you adjust the wording, then a proper render that goes to disk. The two halves form one loop:



An API target
The Lineup renders through **ComfyUI API (local)** or **ComfyUI Cloud**, not the older plugin connection.

A workflow
Pick one in the Builder first (05 Machine › Workflow). Without it the Lineup won't open.

The basics
This chapter assumes you know the Prompt Builder. See the *Prompt Builder* chapter for that.

Nothing in the Lineup is saved to disk
Every cell is a quick, cheap render. Pictures only reach your output folder when you render on **Final** in the Builder. If you want to keep an idea for later without finishing it now, put it on the **Shelf** (Step 6).

CHAPTER 09 · SECTION 01

Open the Lineup

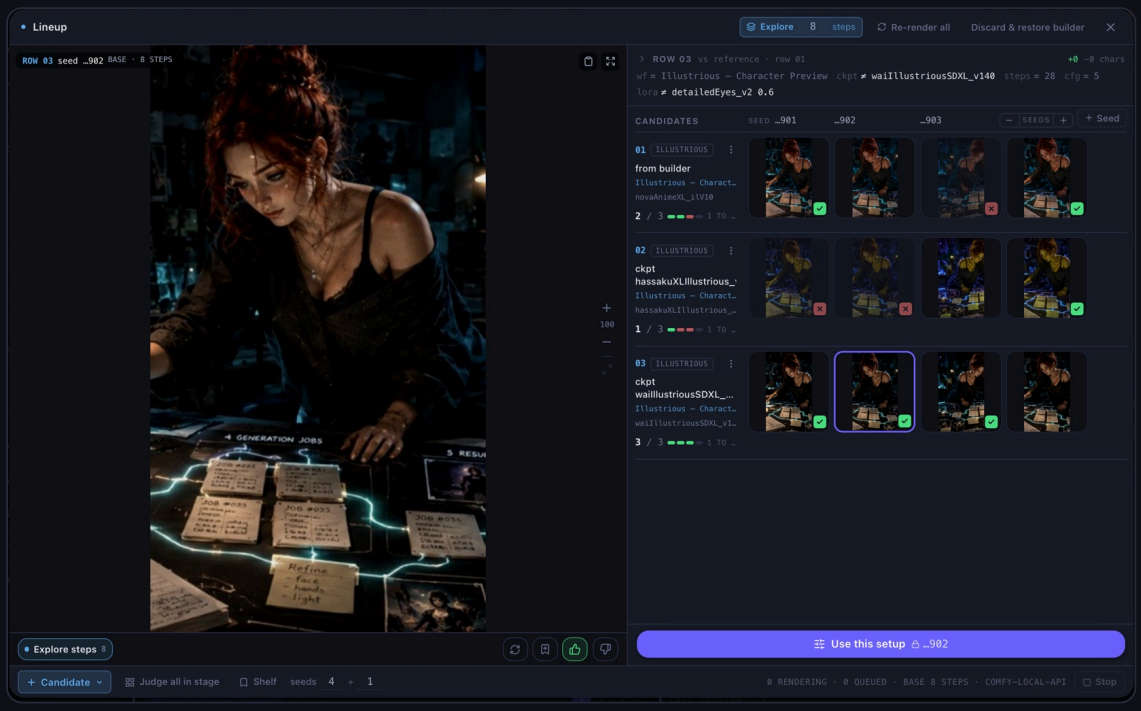
1. Set up your prompt in the Builder as usual and make sure a workflow is picked.
2. Click the **Lineup** door in the Builder head, next to the render buttons. On a narrow window it shows only its grid icon.

The board opens with one row, **01 · from builder**: your Builder's setup and prompt, rendered on four seeds starting at your current seed. If no workflow is picked you get *"Pick a workflow first — this renders through the API path."*

Closing is not throwing away. The ✕ (or Escape) only hides the board: rows, pictures and running jobs all stay, and the Lineup door brings you back to exactly where you were. Only the two **Discard** buttons in the head clear the board (see Step 8).

CHAPTER 09 · SECTION 02

Read the board



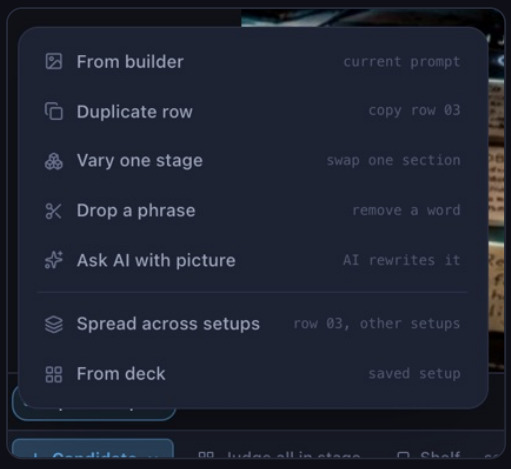
Three candidates (three checkpoints, one with a LoRA) on four seeds. Left: the **stage** with the picked cell. Right: the **board**, with thumbs on the cells. The pictures here are demo stand-ins, not real renders.

Part	What it tells you
Row head	The row number, its base model (*ILLUSTRIOUS*), a short label saying how it differs (*ckpt hassakuXL…*, *+ LoRA …*), its workflow and checkpoint, and its score: thumbs up out of judged cells, plus how many are still *to judge*.
Cells	One picture per row and seed. A green tick is a thumb up, a red cross a thumb down. A cell still rendering shows its progress; a failed one shows the error.
Stage (left)	The cell you clicked, large. Its badge names the row, the seed and *BASE · 8 STEPS*: every cell is a quick base render.
Diff line (top right)	What the picked row changes against the reference row: workflow, checkpoint (*ckpt ≠ …*), steps, CFG, LoRAs and the prompt (*+0 −0 chars*). Click it to unfold the prompt.
Footer	**Candidate**, **Judge all in stage**, **Shelf**, the seed settings, and on the right how many cells are rendering or queued, with **Stop**.

CHAPTER 09 · SECTION 03

Add candidates

Click **+ Candidate** in the footer. Every new row renders on the same seeds as the others, so the comparison stays fair.

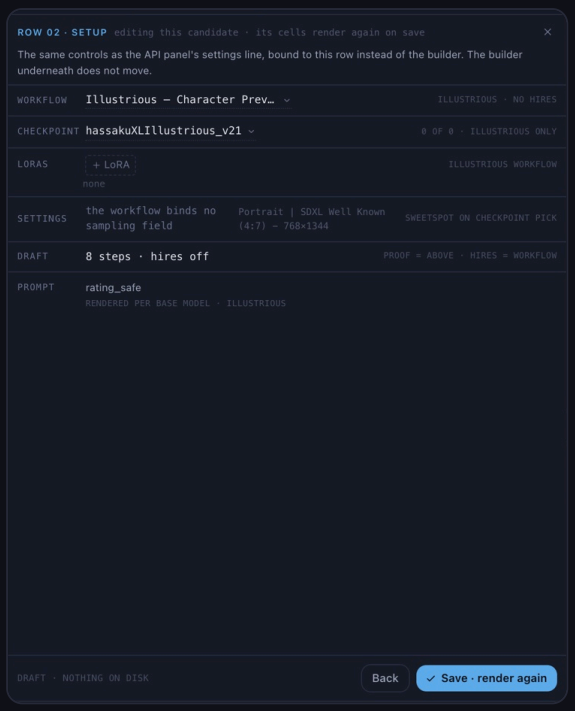


The Candidate menu.

- **From builder**: the Builder's current prompt and setup as a new row. This is also the way **back** from tuning: wording you improved in the Builder becomes a new candidate.
- **Duplicate row**: a copy of the picked row, opened for editing. Use it for "same, but another CFG".
- **Vary one stage**: replace one section of the prompt (for example the scene) and keep the rest.
- **Drop a phrase**: the same row without one phrase, to find out which word causes a problem.
- **Ask AI with picture**: an AI looks at the picked cell and rewrites the prompt from your instruction. Needs an AI provider with vision set up in **Settings › AI**.
- **Spread across setups**: the picked row on several saved checkpoint sets at once, one row per workflow + checkpoint.
- **From deck**: start from a setup saved in the Job Deck.

Change a row

Each row head has a ⋮ menu: **Edit setup**, **Duplicate · edit setup**, **Checkpoint settings** (opens that checkpoint in the library) and **Remove**. Edit setup opens the row's settings on the right:



Workflow, checkpoint, LoRAs and settings for this row only. The Builder underneath does not change. **Save · render again** writes the change and re-renders the row.

> **Editing a row throws away its pictures**
>
> A cell is a picture of one setup. When you save a changed setup, that row's old pictures and thumbs go and every seed renders again. If you want to keep the original to compare against, use **Duplicate · edit setup** instead.

CHAPTER 09 · SECTION 04

Seeds and the cost of a glance

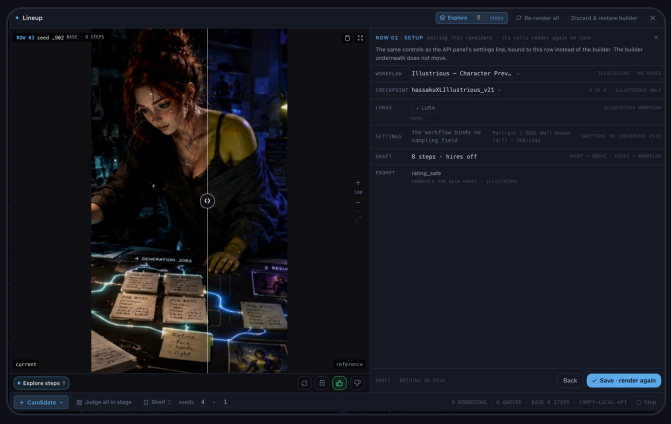
- **How many seeds:** the first number next to **seeds** in the footer (2–6). The second number is the **distance** between seeds. Neighbouring seeds look alike on some models, so raise it to spread them out.
- **Move all columns:** `–` and `+` beside **SEEDS** in the board header shift every column one step back or on. The rows stay, all cells render again.
- **One more column:** `+ Seed` adds the next seed for every row.
- **How cheap a cell is:** the `Explore 8 steps` field in the head sets the step count for all cells. The default lives in **Settings › Render Steps**. A checkpoint that names its own draft count uses that instead, which is what you want for turbo models.

Your seed count and distance are remembered. The board itself is not: an app restart always starts with an empty Lineup.

CHAPTER 09 · SECTION 05

Judge

1. **Click a cell** to put it on the stage.
2. Under the stage, use **thumb up** ("this seed works for this candidate") or **thumb down**. The row's score updates.
3. For many cells, click **Judge all in stage**: it pins the first cell you haven't judged yet. Then use the keyboard: ↑ thumb up, ↓ thumb down, ← → walk the seeds.



Wipe: the picked cell on the left (*current*), the reference on the right. Drag the handle to compare. The buttons below: re-render this cell, Keep, thumb up, thumb down.

Compare two cells directly

Shift-click a second cell to make it the **reference**. It gets a small *REF* mark, and the stage turns into a wipe between the two. The diff line at the top then compares against that row too.

A re-rendered cell also keeps its previous picture: **vs previous** under the stage wipes against it instead.

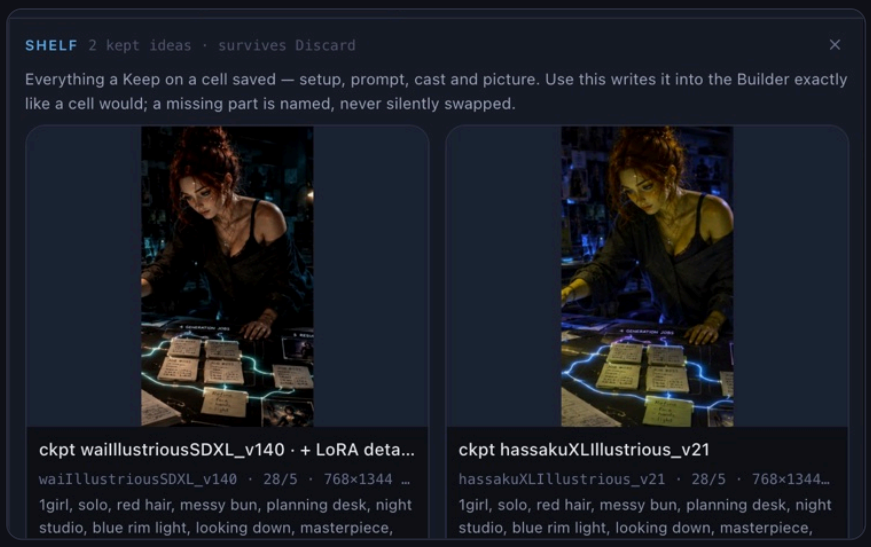
Render again

- The circular arrow under the stage: this one cell, same row, same seed.
- **Re-render all** in the head (or **⌘/Ctrl + Enter**): every cell. Rows that follow the Builder pick up its current prompt and image format.
- **Stop** in the footer cancels everything still rendering or waiting.

CHAPTER 09 · SECTION 06

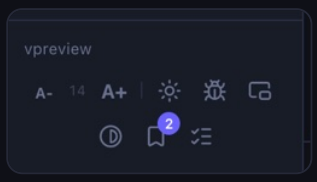
Keep an idea on the Shelf

Found something good but not ready to finish it now? Click the **bookmark** button under the stage (*"Keep — save this setup, prompt, cast and picture on the Shelf to finish later"*). It becomes a **kept idea**: a copy of everything needed to carry on later, plus its picture. Unlike the board, the Shelf **is saved** and survives Discard and restarts.



The Shelf inside the Lineup. **Use this** takes an idea straight into the Builder. A part that no longer exists in your library (here a LoRA) is named, never silently swapped.

Open it with **Shelf** in the Lineup footer, or from **anywhere in the app** with the bookmark badge at the bottom of the sidebar. The badge only appears once something is kept.



The Shelf badge in the sidebar.

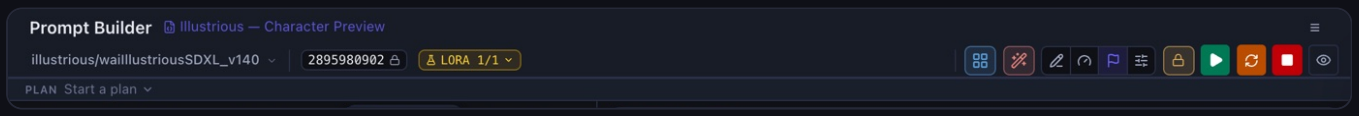
The trash icon deletes an idea and its picture. That can't be undone.

CHAPTER 09 · SECTION 07

Take the winner into the Builder

1. Click the cell you want, so it is on the stage.
2. Click **Use this setup** at the bottom right. The button shows the seed it will take (🔒 *…759*).

The board closes and the Builder now holds that row's **workflow, checkpoint, LoRAs and sampling**, the cell's **seed** and **image size**, and the row's wording as a **frozen prompt**. The cell's picture is in the Builder's preview strip, so you can compare what you render next against it. The first time you do this, the app remembers how your Builder looked before, so you can get it back later.



The Builder head after **Use this setup**: the row's checkpoint and LoRA, the cell's seed, the rung switch (Draft · Quality · Final · Custom), and the seed **padlock** (lit) left of the green Render button.

> **About the frozen prompt**
>
> **The Prompt** now says **FROZEN**: it is exactly the wording the cell used, and you can edit it directly. Click **Frozen** to unfreeze it and go back to the live prompt built from your stages. The cast is not copied: if the row was made with other characters than the Builder has now, a note says so (*"Cast differs — …"*).

CHAPTER 09 · SECTION 08

Tune: Draft, Quality, Final

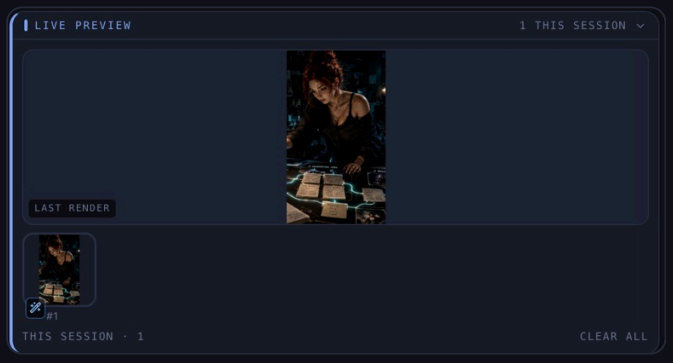
The four segments next to Render, Draft · Quality · Final · Custom (⌘/Ctrl + 1 to 4), choose **which rung** the next render goes out on. The number on each is its step count; hover it to see what that rung does with your workflow. Picking a rung does not render; the green **Render** button (and ⌘/Ctrl + Enter) fires whatever is lit. The status bar also names it (**MODE DRAFT**).

Rung	Steps	Saves a file?	Use it for
Draft (pencil)	capped, cheap	no	Trying wording. Change a word, render, look.
Quality (gauge)	the checkpoint's full count	no	Checking the picture properly before committing.
Final (flag)	full	**yes**	The keeper. Final writes to your output folder.
Custom (sliders)	full	**yes**	Final, with your own bypass switches deciding which stages run instead of the workflow's Modes matrix.

On a Core pipeline, a Final render shows a picture after each stage (upscale, face pass, before SeedVR2 …) in the previews. If an earlier stage is the better picture, open it in the lightbox and press **Keep as final**: it replaces the saved file, and the previous final is kept beside it. See the *Workflows* chapter.

The seed padlock

Unlocked, rendering twice on the same rung gives you two *different* pictures: the seed moves on. **Lock it** (the padlock left of Render) and Draft repeats the exact same picture, so the only difference on screen is the word you changed. Climbing from Draft to Quality or Final always keeps the seed that is standing, with or without the lock. Click the padlock again (or press Ctrl+L) to let the seed move.



The cell's picture waits in the preview strip as *LAST RENDER*, so the next Draft can be compared with it.

Going back and forth

- **Back to the board:** click the Lineup door again. Everything is still there.
- **Send tuned wording back:** in the Lineup, `+ Candidate` › **From builder** makes the improved prompt a new row, rendered on the same seeds as the others.
- **Finish a kept idea:** Shelf › **Use this** does the same as Use this setup.

- **Clear the board:** **Discard & restore builder** stops every job, drops every cell and gives the Builder back what it looked like before your first Use this setup. **Discard & keep setup** does the same but leaves the Builder as the last taken setup left it.

CHAPTER 09 · SECTION 09

Tips

- **Change one thing per row.** Two rows that differ in checkpoint *and* LoRA can't tell you which one made the difference. The row labels and the diff line help you check.
- **Judge by row, not by cell.** A checkpoint with 3 of 4 thumbs up is a better bet than one lucky cell on a row with 1 of 4.
- **Mark your workflows.** A cell is only really cheap if the workflow says what to skip in quick renders (upscalers, detailers). That is set per workflow in its Modes matrix; see the *Workflows* chapter. Unmarked, every cell runs the whole graph, just with fewer steps.
- **Turbo models:** give the checkpoint its own draft step count in the Model Library, so it doesn't get the global count.
- **Use the Shelf generously.** Keeping costs nothing and survives everything; the board is gone after a restart.

CHAPTER 09 · SECTION 10

Troubleshooting

"Pick a workflow first — this renders through the API path."
The Lineup needs a workflow on the **ComfyUI API (local)** or **ComfyUI Cloud** target. Choose one in the Builder under **05 Machine › Workflow**, then open the Lineup again. You also get this message when the job uses the older **ComfyUI (Local)** connection, which the Lineup can't render through: create a setup on **ComfyUI API (local)** instead.

I don't see the Lineup door.
It sits in the Builder head beside the render buttons, only while the Lineup is closed. On a narrow window it shows just its grid icon (the Lineup tooltip tells you which one).

A cell shows an error like "No workflow selected, or it was deleted."
That row points at a workflow that is missing or not enabled. Use the row's ⋮ **› Edit setup** to pick another, then **Save · render again**.

Use this setup is greyed out.
No cell is on the stage. Click one first; the setup comes from the cell you picked.

My Lineup pictures are not in the output folder.
That's by design: cells are never saved. Take the winner into the Builder and render on **Final**, or Keep it on the Shelf.

Final looks different from the Draft I picked.
Check that it is the same seed (the SEED readout in the head). More steps legitimately add detail, and samplers of the *ancestral* and *SDE* family draw fresh noise at every step, so with those a higher rung can move more than the details. Which sampler a checkpoint runs on is set by the checkpoint in the Model Library.

The board was empty after restarting the app.
The Lineup is session-only. Only what you put on the Shelf is saved.

CHAPTER 10

The Phone

The whole Prompt Builder on your phone, over your home network. You open a job, change it with your fingers, and press Render; the desktop does the rendering and keeps the files. This chapter shows how to switch it on, how to connect, and what happens when the phone and the desktop both changed the same job.

---

01 Switch it on (on the desktop) 132
02 Open it on the phone 134
03 Find your way around 135
04 Open a job 136
05 Save the job, and who wins 136
06 Render and stop 138
07 Results, and what you can do with a picture 139
08 The machine: workflow, checkpoint, ComfyUI 140
09 Phone or desktop? 140
10 Troubleshooting 141

What this is

The phone app is not a smaller copy of the app. It is the **same Builder** (the same stages, cards and fields as on the desktop) laid out for a narrow screen. Everything it does goes through the desktop app on your computer:

- The phone works on **its own copy of a job**. Nothing on the desktop changes until you press **Save** on the phone.
- A render you start on the phone runs **on the desktop**, through the same render paths as a render started there. It is filed under the phone's job, and a Final render lands in the desktop's output folder, as always.
- Your libraries (characters, checkpoints, LoRAs, workflows, styles) come **from the desktop**. A change you make to them on the phone is saved there too.

The desktop app running
The phone talks to the app on your computer. When the app is closed, the phone can do nothing.

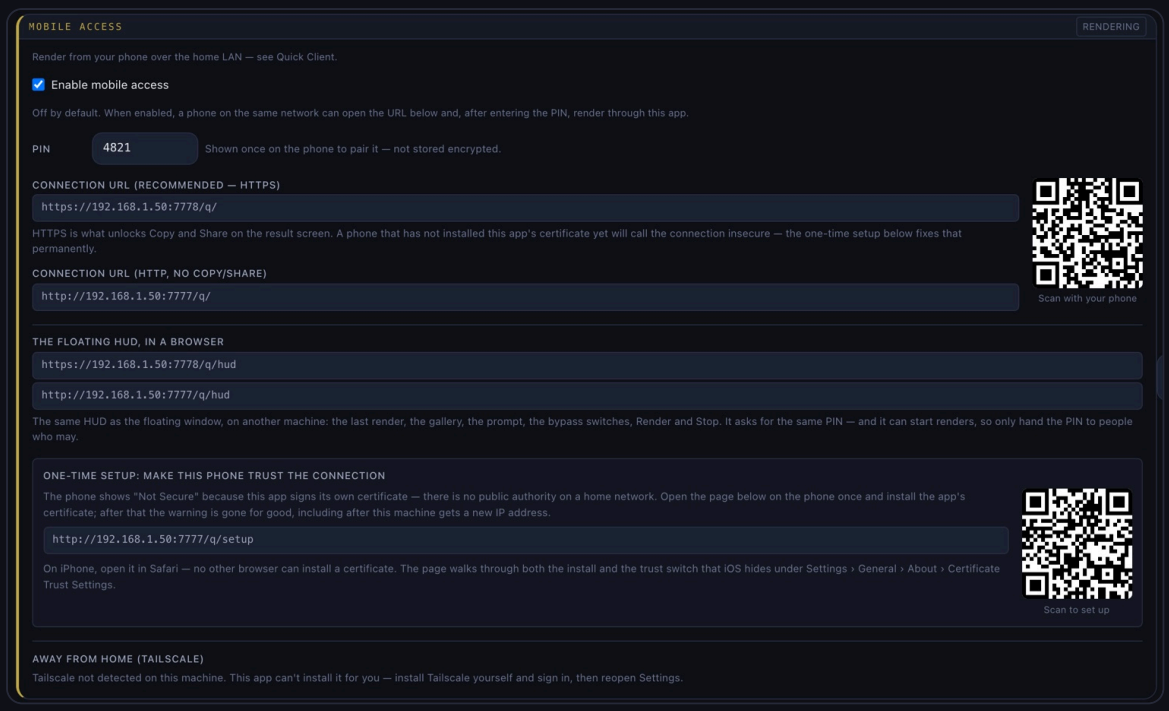
The same network
Phone and computer on the same Wi-Fi or home network (or both in the same Tailscale account, see Step 2).

The Builder basics
This chapter assumes you know the Prompt Builder. See the *Prompt Builder* chapter for that.

CHAPTER 10 · SECTION 01

Switch it on (on the desktop)

1. Open **Settings › Rendering** and scroll to **Mobile Access** (or type "Mobile" into **Search settings...**).
2. Tick **Enable mobile access**. It is off by default.
3. Type a **PIN**, for example four digits. The phone asks for it once to pair.
4. Press **Save** at the top right of the Settings page.



Mobile Access. The app finds your computer's network address itself and shows the address to open on the phone, with a QR code beside it. The addresses in this picture are examples; yours will differ.

The section then shows two addresses:

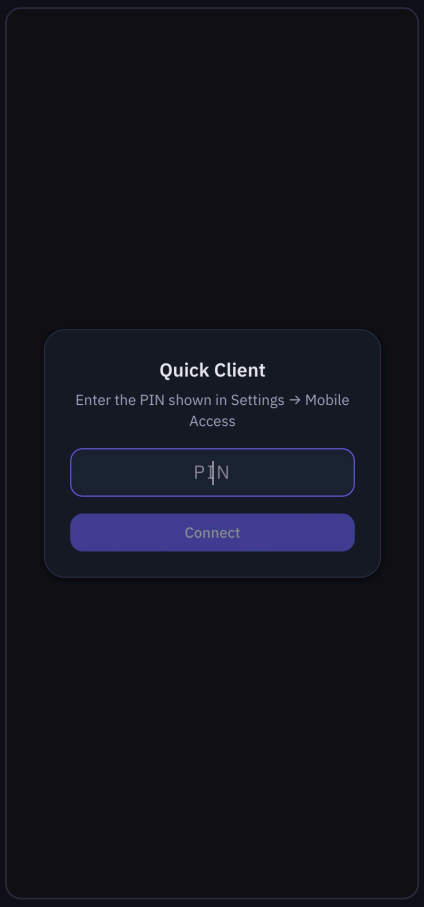
Address	What it is for
Connection URL (recommended — HTTPS)	The secure address. Only over this one can the phone **copy** text and pictures, **share** a result and **paste** a picture. Until the phone trusts the app's certificate (see the next step), the phone's browser calls this connection insecure.
Connection URL (HTTP, no Copy/Share)	The plain address. Everything else works, but Copy says *"Copying needs HTTPS"* and pasting a picture says *"Pasting a picture needs HTTPS"*.

Both end in **/q/**. That address opens the whole Builder; you do not need to type anything after it. If the section says *"No LAN network detected"*, the computer is not connected to a network yet.

> **The PIN is the only lock**
>
> Anyone on your network who knows the PIN can open your jobs and start renders. It is stored on your computer unencrypted. Give it only to people who may use the app. Several wrong PINs in a row make the desktop wait longer and longer before it accepts another try.

CHAPTER 10 · SECTION 02

Open it on the phone



The first time: the PIN screen.

1. Scan the QR code under **Scan with your phone** with the phone's camera, or type the address into the phone's browser.
2. Enter the PIN and tap **Connect**. The phone remembers it; you are asked again only when the desktop app was restarted or the pairing ran out.
3. The app loads your libraries from the desktop and opens the **Jobs** list (Step 4).

Make the connection secure, once

Below the addresses, **One-time setup: make this phone trust the connection** has its own address and QR code (**Scan to set up**). Open it on the phone and follow the page: it installs the app's certificate. After that the HTTPS address works without a warning, even after your computer gets a new network address. On an iPhone, open that page in **Safari**; no other browser there can install a certificate.

Full screen

The **Full screen** button in the phone app's head (the two arrows) hides the browser's address bar and toolbars. Tap it once after each opening; the browser only allows it from a tap. iPhone Safari has no such mode, so the button is not shown there.

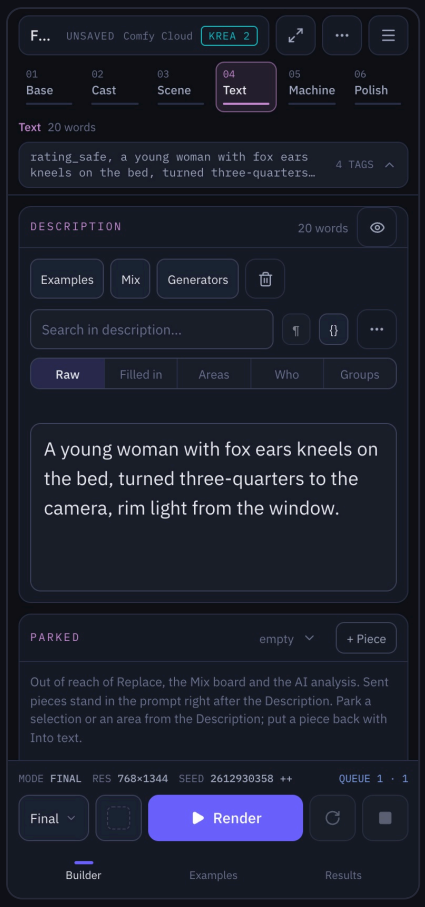
You can also add the page to your **home screen** (in the browser's menu). A home-screen icon opens the Builder without the browser around it, but it remembers the computer's network address: if that address changes, the icon stops working and you need a new one. The Full screen button works on any address.

Away from home

If **Tailscale** is installed on your computer and signed in, Mobile Access shows an extra address under **Away from home (Tailscale)**. It works from anywhere, as long as the phone has Tailscale too, signed into the same account. The app does not install Tailscale for you.

CHAPTER 10 · SECTION 03

Find your way around



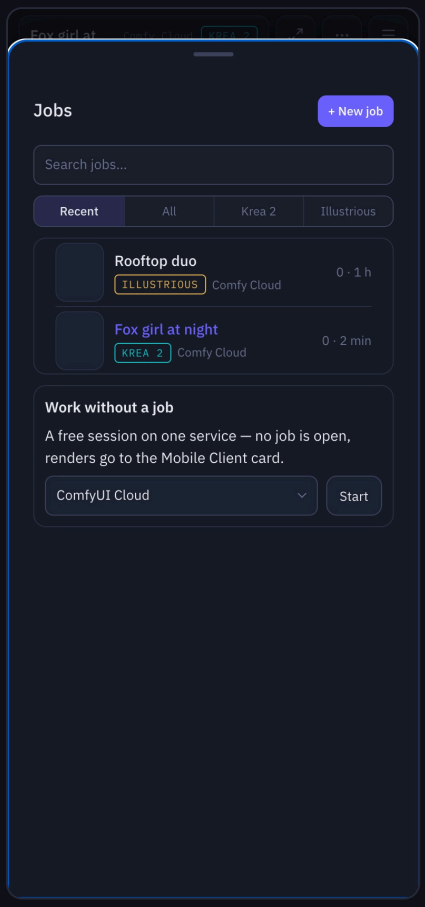
A job open on stage **04 Text**: the Description, as on the desktop.

Part	What it does
Head	The job's name, the service it renders on and its base model. A tap on the name opens the same menu as ⋯. Then **Full screen**, ⋯ (*Setups and more*) and ≡ (*Jobs*). A mark like *UNSAVED* or *OPEN ON DESKTOP* appears beside the name when there is something to know (Step 5).
Stage rail	The Builder's six stages: **01 Base · 02 Cast · 03 Scene · 04 Text · 05 Machine · 06 Polish**. A tick marks a stage you have filled in. A long press opens a stage's details.
Prompt strip	The prompt as it will be sent, with its tag count. Tap it to open it; drag its edge to make it taller.
Status line	**MODE · RES · STEPS · SEED · QUEUE**, like the desktop's status bar. Tap it for the details and the queue.
Render row	The rung (**Final** and so on, Step 6), a small thumbnail of the last render (it opens the render previews), **Render**, **Re-render** (the same prompt again) and **Stop**.
Tabs	**Builder**, **Examples** (your Example Images) and **Results** (Step 7).

The cards inside each stage are the desktop's cards: the cast, regional prompting, references with their masks and maps, the Description with its views, the Mix board, generators, boosters, LoRAs, workflow settings. They work the same way; where a desktop window would be too large, it opens full screen.

CHAPTER 10 · SECTION 04

Open a job



The Jobs list (≡ in the head).

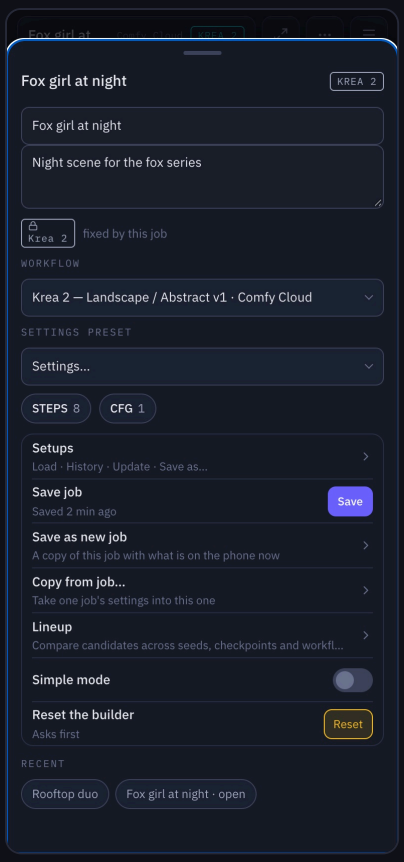
- **Recent** shows the jobs you opened on the phone, **All** every job on the desktop. If your jobs use different base models, there is a filter button per base model too. **Search jobs...** finds a job by name.
- Each row shows the base model, the service, how many renders the job has and when it was last used. *· open on desktop* means the same job is open on the computer right now. You can still open it on the phone.
- Tap a job to open it. The phone gets a copy of the job as the desktop last saved it.
- **+ New job** creates a job, the same way **New setup** does on the desktop: pick the service and, for ComfyUI API (local) and ComfyUI Cloud, the base model. As on the desktop, the base model cannot be changed later.
- **Work without a job** starts a free session on one service. No job is open; the renders go to the **Mobile Client** card on the desktop.

> **The phone keeps a draft**
>
> A phone browser may close a tab in the background without asking. So while a job is open, the phone keeps your changes in a draft of its own. When you open that job again, **Unsaved phone changes** offers **Restore them** or **Discard them**. Saving the job removes the draft.

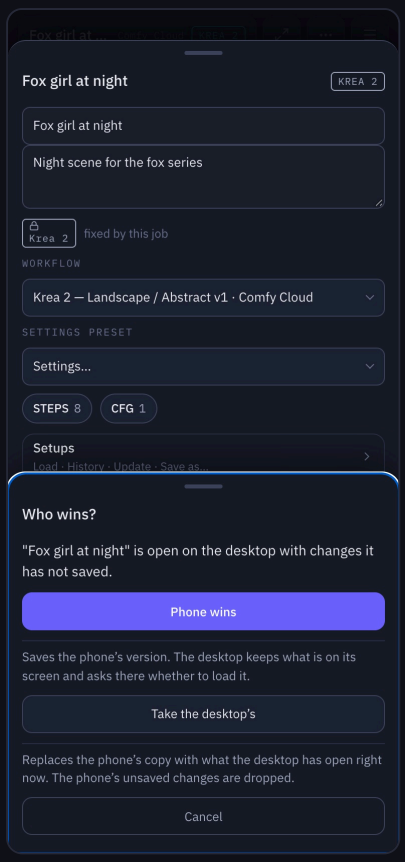
CHAPTER 10 · SECTION 05

Save the job, and who wins

The phone does not save by itself. When you are happy with your changes, open ⋯ and tap **Save** in the **Save job** row. The row says when the job was last saved, or *Unsaved changes on the phone*.



`•••`: name, note, workflow, settings preset, and the job's actions.



The job changed on the desktop too: **Who wins?**

The same menu also has **Setups** (load and save Prompt Setups), **Save as new job** (a copy with what is on the phone now), **Copy from job...** (take another job's settings into this one), **Lineup**, **Simple mode** and **Reset the builder**. Under **Recent** are your last jobs, for a quick switch.

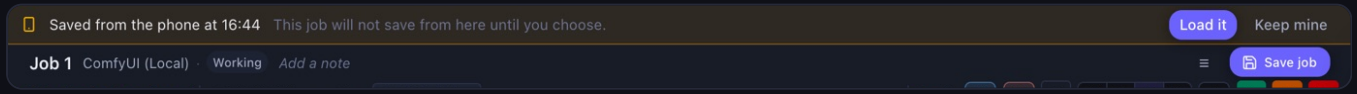
When both sides changed the job

If the job was saved on the desktop after the phone opened it, or it is open on the desktop with changes nobody saved, the phone's Save stops and asks **Who wins?**:

- **Phone wins** saves the phone's version anyway.
- **Take the desktop's** replaces the phone's copy with the desktop's version. Your unsaved phone changes are dropped.
- **Cancel** leaves both as they are. The phone keeps its changes.

What the desktop shows

When the phone saves a job that is **open on the desktop**, the desktop does not change the picture under your hands. Its Builder head shows a line instead:



The desktop after a save from the phone.

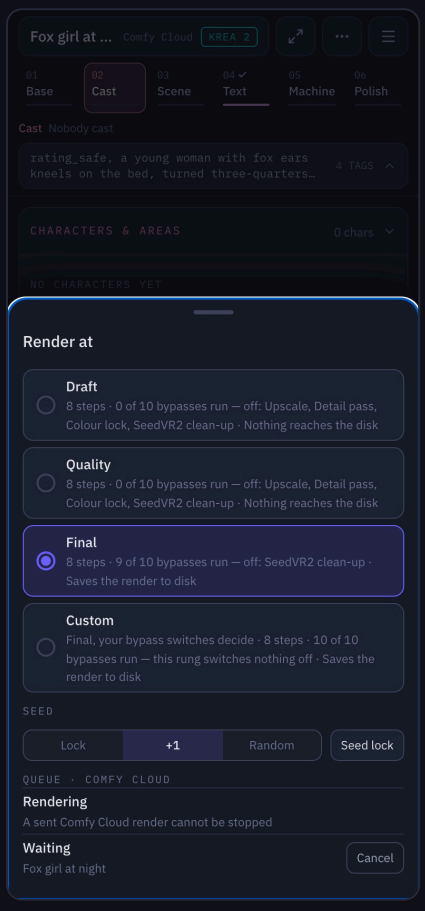
- **Load it** loads the job as the phone saved it. What the desktop had on screen is replaced.
- **Keep mine** saves the desktop's version over the phone's.

Until you choose, that job is **not saved from the desktop**, not even automatically, so nothing gets lost by accident. On the phone, the mark beside the job's name tells you where things stand:

Mark	Meaning
UNSAVED	You changed something on the phone and have not saved it yet.
SAVED	The phone's version is saved.
OPEN ON DESKTOP	The job is open on the computer too.
CHANGED ON DESKTOP	The job changed on the desktop since you opened it. A save will ask who wins.
DESKTOP ASKED	You saved; the desktop now shows **Load it** · **Keep mine** and waits for an answer there.

CHAPTER 10 · SECTION 06

Render and stop



Tap the rung button to choose where a render goes.

The button at the left of the render row says which **rung** a render runs at, as the Draft · Quality · Final switch does on the desktop. Tap it to open **Render at**:

- **Draft** and **Quality** are cheap looks. Nothing reaches the disk.
- **Final** is the real render and saves it to the desktop's output folder.
- **Custom** is Final, but your own bypass switches decide what runs.

Each line says what that rung does with this job's workflow: steps, which parts are switched off, whether it saves. The same sheet sets the **Seed** (`Lock` , `+1` , `Random`) and shows the **queue** of the service: what is rendering and what is waiting, with `Cancel` for a waiting render.

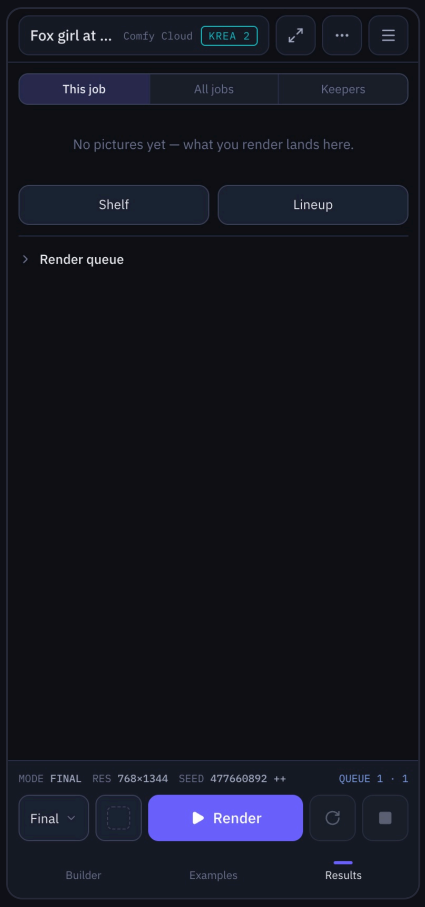
Then tap `Render` . The status line shows the progress; while ComfyUI sends pictures during a render, you see them come in. **Stop** (the square) stops the render. A render already sent to **ComfyUI Cloud** cannot be stopped; the phone says so.

A render keeps going when the phone's screen goes off. When you look again, the phone reconnects and catches up on what it missed.

The Render button is greyed out until a job is open (*"Open a job first"*). On a target that renders differently, it carries that target's own name, for example **Send to Forge**.

CHAPTER 10 · SECTION 07

Results, and what you can do with a picture



The Results tab, here before the first render.

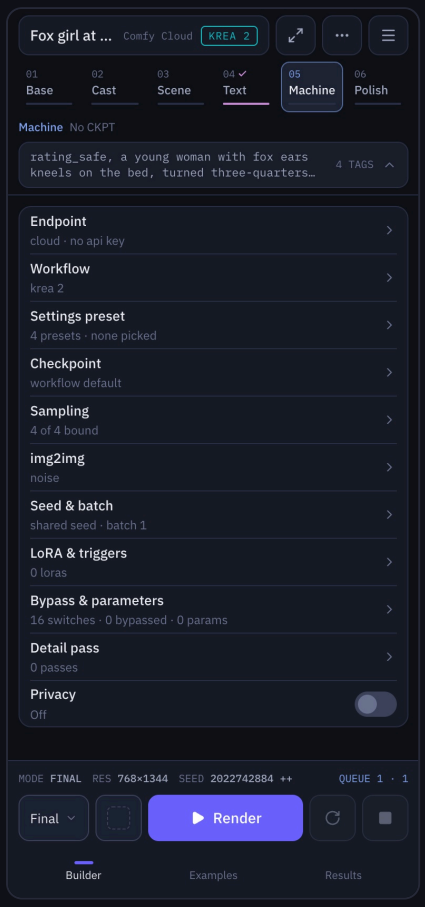
Results shows **This job**, **All jobs** and **Keepers**, plus doors to the **Shelf** and the **Lineup** and the **Render queue**. Tap a picture to open it large; the arrows go to the next one. Under the picture:

Keep as final	Writes a stage picture over the render's file, as on the desktop.
Reuse seed · **Render again**	Take that picture's seed into the Builder, or render it again with it.
Enlarge · **Topaz...**	Enlarge the picture, or upscale it with Topaz (needs a Topaz key on the desktop).
Repair	Opens the repair bench in finger form: selections, brushes, the grade, inpaint suggestions. **Send back** writes the repaired picture over its render, and after a Keep, **Send to Final** hands it to the Final rung, as on the desktop (see the *Canvas* chapter).
To reference	Adds the picture to the Reference library.

The keeper flag and the CivitAI button work on the phone too.

CHAPTER 10 · SECTION 08

The machine: workflow, checkpoint, ComfyUI



Stage **05 Machine**, one row per part of the render setup.

Stage **05 Machine** lists the same parts as on the desktop: **Endpoint**, **Workflow**, **Settings preset**, **Checkpoint**, **Sampling**, **img2img**, **Seed & batch**, **LoRA & triggers**, **Bypass & parameters**, **Detail pass**. Tap a row to open it.

For a local ComfyUI, **Endpoint** has the server's controls: whether it is running, **Start**, **Stop**, **Restart**, **Free VRAM** and the **Debug Log** (ComfyUI's console). They act on the computer, not on the phone. If the desktop knows several ComfyUI installs, **ComfyUI install** picks the one to start; when a Legacy install is set up, Start and Restart aim at it unless you pick another.

Setting up ComfyUI itself (choosing the folder, downloading models and nodes) is done on the desktop, in **Settings › Rendering**. Until then the phone says *"Set up ComfyUI in the desktop's Settings › Rendering first"*.

CHAPTER 10 · SECTION 09

Phone or desktop?

Almost everything in the Builder works on the phone. What needs the computer itself stays there; pressing it on the phone says *"That only works on the desktop"*.

Works on the phone	Only on the desktop
Every Builder stage and card, the cast, references, masks and maps, regional prompting, the Description, Mix board, generators, boosters	Choosing files and folders on the computer, **Show in folder**, opening Finder or Explorer
Opening, creating, saving and copying jobs; Prompt Setups; the Lineup	Export and import (ZIP backups, character files, storyboard exports)
Rendering on every target, Stop, the queue	Setting up ComfyUI (Settings › Rendering)
Results, Keep as final, Repair, Enlarge, Topaz, To reference, CivitAI	Settings with keys and passwords: the phone sees only that a key is *set on the desktop*
Starting, stopping and restarting ComfyUI	Importing storyboard audio and exporting storyboard bundles

The phone never moves the desktop's Builder
Rendering from the phone does not change the checkpoint, the resolution or the bypass switches of the Builder open on your computer. The phone's render carries its own settings.

CHAPTER 10 · SECTION 10

Troubleshooting

"Cannot reach the desktop app. Is it running, and is this phone on the same network?"
Check that the app is open on the computer and that both are on the same network. Guest Wi-Fi often keeps devices apart, and a phone on mobile data is not on your network. A firewall that blocks the app gives this message too: on Windows, allow the app for private networks when Windows asks, or in the firewall settings.

"Mobile access is switched off on the desktop." or "No PIN is set on the desktop yet"
Tick **Enable mobile access**, type a PIN and press **Save** in Settings (Step 1).

"Wrong PIN." or "This device is no longer signed in — enter the PIN again."
Type the PIN shown in **Settings › Mobile Access**. The pairing ends when the desktop app is restarted; then the phone asks for the PIN again. That is normal.

The address worked yesterday and today it doesn't.
Your computer probably got a new network address. Open Mobile Access again and scan the new QR code. A home-screen icon has the old address in it: make a new one.

The phone says the connection is not secure.
Do the one-time setup in Step 2, or use the HTTP address. Over HTTP only Copy, Share and pasting a picture are missing.

My changes from the phone are not on the desktop.
The phone saves only when you press **Save** under ⋯. If the job is open on the desktop, the desktop waits for **Load it** or **Keep mine** in its Builder head.

"That only works on the desktop"
That button needs the computer itself (a file dialog, a folder, an export). Do it on the desktop; see the table in Step 9.

CHAPTER 11

Canvas: retouching, grading and repairs

A picture editor built for AI renders. Heal a spot, push a chin with Liquify, grade with adjustment layers, upscale with Topaz, or send any part of the picture back to ComfyUI to be drawn again. If you know Photoshop, most keys are where you expect them.

– What the Canvas is for 144
01 Open a picture 144
02 Find your way around 145
03 The tools 146
04 Retouching without losing the original 147
05 Grading: Adjust, presets and looks 148
06 Select an area 149
07 Repair a spot with ComfyUI 150
08 Layers 152
09 Depth of field, tilt-shift and lens blur 152
10 Upscale with Topaz 153
11 Save your picture 153
12 Tips 154
13 Troubleshooting 154

What the Canvas is for

A render is often almost right: one hand has six fingers, the skin is blotchy, a colour bleeds from the hair into the forehead, an eye sits three pixels too low. Rendering the whole picture again would change everything else too. The Canvas gives you two ways to fix it:

- **By hand**, with retouching tools: healing, clone, remove, dodge and burn, smooth, colour, blur and sharpen, Liquify. Small faults (under about one percent of the picture) are faster and more faithful this way. No AI, no ComfyUI.
- **With a repair**: mark the part that is wrong, say what should be there, and ComfyUI redraws just that area with an inpaint workflow. Everything outside the mask stays exactly as it was.

Retouching, layers and grading are **non-destructive**: strokes land on their own layers, adjustments are layers, and the original stays underneath.

> **What you need for repairs**
> A connection to **ComfyUI API (local)** or **Comfy Cloud** (see the Getting Started chapter), and an inpaint workflow. Three come with the app (*inpaint-illustrious*, *inpaint-anima*, *inpaint-krea*), tagged and ready. Topaz needs a Topaz API key in Settings. Everything else works without either.

CHAPTER 11 · SECTION 01

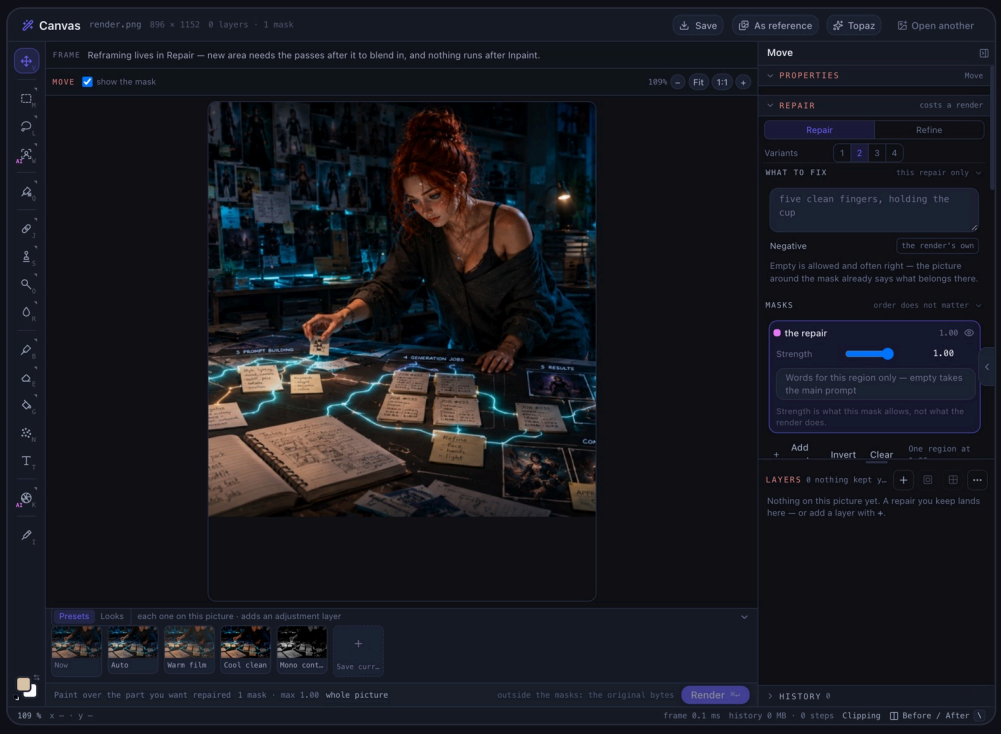
Open a picture

Click **Canvas** in the sidebar. With nothing open, the page offers: drop a picture, paste one with Ctrl+V (Cmd+V), **From the Gallery**, **From a file**, or **New canvas** (width, height, a background colour; mid grey is a good start if you plan to render over it). Below them, **Unfinished repairs**: the Canvas saves your work as you go.

Other ways onto the Canvas: **Repair** in the Prompt Builder head (opens your newest render), the small wand on every render thumbnail in the previews, **To Canvas** in a render's lightbox, **To Canvas** on a Gallery tile (shown on hover) and in the Gallery's full view, and the Canvas action in Reference Images. From a saved render, the Canvas knows that render's file, so your Save lands beside it (section 11).

CHAPTER 11 · SECTION 02

Find your way around



The Canvas. Head: Save, As reference, Topaz, Bake, Open another. Left: the tool rail. Above the picture: the options of the tool in hand. Under it: the strip of **Presets** and **Looks**. Right: the dock with Properties, Repair, Adjust and Lens, then Layers and History. Bottom: the repair strip with Render, and the status bar.

- **The rail** holds one button per tool group, with the group's key in its corner, in six sections: Move, the selections and the frame, the Mask, the four retouching groups, the makers (brush, eraser, fill, noise, text), and the Lens and the eyedropper at the end. The button shows the tool you used last in that group; **Shift plus the key** walks through the group, and the small tick in the corner says there is more. A pink **AI** marks a tool that runs a model; a pink dot says you have already used that group on this picture.
- **The options bar** shows the settings of the tool in hand, and on the right **where the stroke goes** ("→ new Retouch layer", "→ into *portrait*"), so a stroke never lands somewhere you didn't expect. On a narrow window it wraps onto a second line.
- **The dock** is one column: **Properties** (the tool in hand), **Repair**, **Adjust** and **Lens**, each a section you fold with its head, then **Layers** and **History** below. The folds are remembered. The fold button at the top gives the picture the width.
- **The status bar**: zoom, the pixel under the pointer and its colour, the active layer, history memory, **Clipping** (shows burnt-out lights and crushed shadows) and **Before / After**. Hold **** to see the original.
- **Navigate**: Ctrl/Cmd + wheel zooms at the cursor, Space + drag pans, **⌘0** fits, **⌘1** shows 100 % (one picture pixel per screen pixel). Hold **⌘Space** and click to zoom in, **⌥⌘Space** to zoom out, or drag a box to zoom onto it. The zoom reaches 3200 %; from 200 % pixels show as sharp squares, and from 800 % **⌘'** lays a pixel grid over them. Zoomed in past Fit, a **navigator** at the bottom right shows where you are; click or drag in it to move.
- **Spring-loaded keys**: hold **I** for the eyedropper or **⌘** for Move; let go and the tool you had comes back.

CHAPTER 11 · SECTION 03

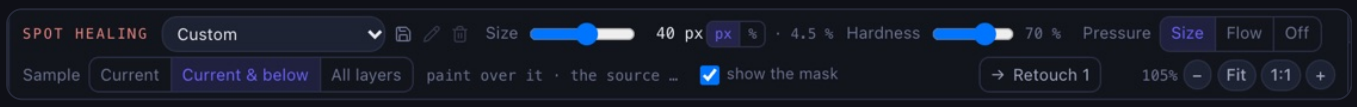
The tools

Key	Group	Tools
V	Move / Transform	Pick a free layer up and move it; ⌘T for free transform.
M	Marquee	Rectangle, Ellipse.
L	Lasso	**Lasso** (draw a loop); **Polygonal lasso** (click the corners; Enter, a double-click or the first corner closes it).
W	Select	**Object** (click a thing or drag a box round it, and a model on your computer finds its edge; Alt = "not that"), **Select subject** (the picture's main subject in one press, ⇧⌘U), **Wand**. See section 6.
C	Frame	Crop and outpaint. Reframing a render lives in Repair from the Builder.
Q	Mask	**Mask brush** (paint the repair mask; Alt takes it back), **Mask eraser**, **Gradient mask** (drag a ramp: linear, radial or a band), **Depth mask** (click a depth: everything behind it, in front of it or round it).
J	Heal	**Spot healing** (paint over it, the source is found), **Healing** (Alt-click a source: texture from there, light from here), **Remove** (filled from the picture around it), **Patch** (draw around it, drag onto clean picture).
S	Clone	**Clone stamp** (Alt sets the source), **Restore** (paint the original back).
B	Brush	**Brush**; **Colour** (hue and chroma only, the light stays: the tool against colour bleeding); **Smooth** (calms blotchy skin, keeps the edges of eyes and lips).
E	Eraser	Eraser; Layer mask eraser (hide parts of a placed picture, Alt brings them back).
R	Focus	Blur, Sharpen, Smudge.
O	Tone	**Dodge**, **Burn**, **Sponge**, into a tone layer (see below).
G	Fill	Fill from a point; Gradient (drag from start to end).
N · T	Noise · Text	Scatter grain; a text layer.
K	Lens	**Depth of field**, **Tilt-shift**, **Lens blur**, each made as a new layer (section 9).
I	Eyedropper	Takes a colour off the picture. **Sample size** (Point, 3×3, 5×5, 11×11) and **Sample** (Current layer, All layers) sit in the options bar. After one pick it hands back the tool you came from; ⇧-click keeps the eyedropper.

Brush keys: **[** / **]** size, ⇧**[** / ⇧**]** hardness, digits set strength or opacity in 10 % steps, **X** swaps the colours, **D** resets them. **?** shows the whole key sheet.

Brushes and brush presets

Every tool keeps its own brush: size, hardness, opacity, **Flow** (how much each dab lays down) and what pen pressure drives (**Size**, **Flow** or **Off**). The size is measured in picture pixels, or with the **px | %** switch beside it in percent of the picture, so a stroke is the same at every zoom. The **Preset** select in the options bar and in Properties offers *Hard round*, *Soft round*, *Airbrush low flow* and *Detail small hard*; the buttons beside it save the current brush as a preset of your own, rename it or delete it. It reads *Custom* when the brush matches no preset.

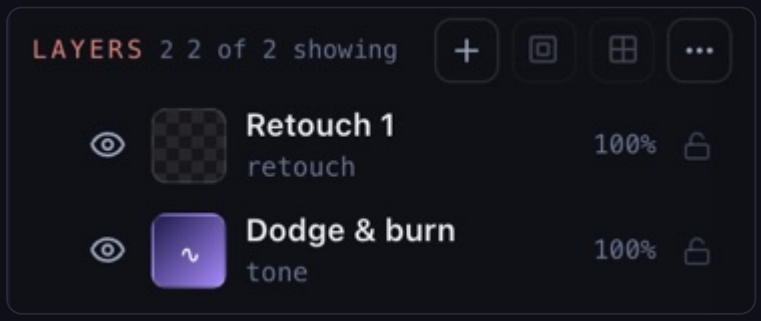


Spot healing in hand: the brush preset, Size in px or %, Hardness, Pressure, Sample (Current · Current & below · All layers), and where the stroke goes (→ Retouch 1).

CHAPTER 11 · SECTION 04

Retouching without losing the original

- **Sample: Current · Current & below · All layers.** With "Current & below" (the default for a rendered picture), a healing or effect tool works on a **Retouch layer** above the picture. The original stays untouched; the Eraser or Restore brings it back, and the layer has an opacity.
- **Picking the tool makes its layer.** When you choose a retouching tool, the layer it writes into exists at once (Retouch 1, or the tone layer), so the options bar can name it.
- **Dodge, Burn and Sponge** never write into pixels. They paint into a **Dodge & burn** tone layer that brightens, darkens, saturates or desaturates what lies under it. Options: **Range** (Shadows / Midtones / Highlights), **Exposure** (stroke strength) and **Protect tones** (luminance changes, hue stays). Erase on it, fade it, switch it off: it stays editable.
- **Frequency separation** (in Properties for the healing tools) splits the picture into **Low** (colour and tone) and **High** (texture). Smooth the Low layer to even out skin without touching the pores on High; **Join** merges the pair again.



Layers after picking Dodge and Spot healing: a **Dodge & burn** tone layer and **Retouch 1**, both over the untouched picture.

Liquify

⇧⌘X opens Liquify in the same window: Forward Warp, Bloat, Pucker, Push Left, Smooth, Restore and a Freeze mask that keeps parts from moving. It is made for what neither a heal nor a render fixes well: a chin a touch too long, a shoulder too high, one eye lower than the other. **Apply** puts the result on a layer that remembers the warp, so opening Liquify on it again continues where you left off.

Transform

⌘T on a layer (or on a selection, which lifts it out as a layer): eight handles and a rotation stem. Corners keep the layer's proportions; **Shift** frees them, **Alt** scales from the centre, Shift while rotating snaps to 15°. The pivot can be moved, edges snap to the frame, its centre and thirds and to other layers (hold ⌘/Ctrl to stop snapping), arrows nudge by 1 px (Shift 10 px), and the options bar takes exact numbers. **Enter** applies, **Esc** cancels. A transform always resamples from the original pixels, so turning twice costs nothing.

CHAPTER 11 · SECTION 05

Grading: Adjust, presets and looks

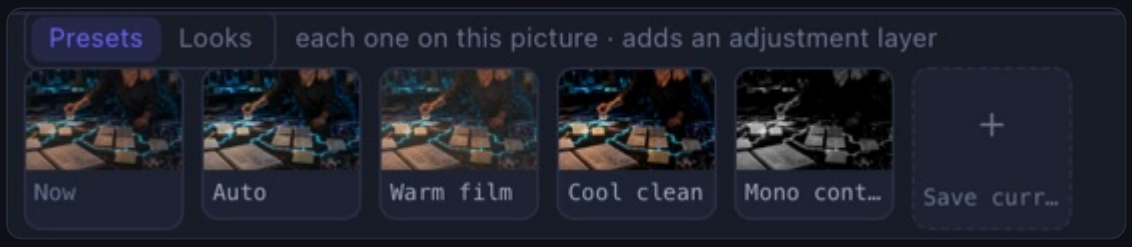
The **Adjust** section of the dock edits one grade at a time, and its head says which: **The picture** (a grade over everything, on screen and in the file, never sent to the model) or the adjustment layer you picked in Layers. Under the head sit **Auto tone** and **Auto colour**, then seven drawers: **Light**, **Tone curve** (Curves and Levels), **Colour**, **Colour mixer**, **Detail**, **Effects** and **Look**. A drawer that changes something carries a dot and its own reset.

Adjust on the picture: Auto tone, Auto colour, the Light drawer open, Tone curve below it.

+ Layer in the head adds an **adjustment layer** instead. Each has its own mask, opacity and blend mode; **⌥-click** an entry to add it clipped to the selected layer, so only that layer is graded (for example only the figure).

Group	Adjustment layers
Light	Exposure, Levels, Curves, Shadows / Highlights, Brightness / Contrast
Colour	White Balance, Vibrance, HSL, Colour Balance, Black & White, Selective Colour, Recolour
Look	LUT, Vignette, Grain
Detail	Clarity, Dehaze, Sharpen, Denoise

The strip under the picture: Presets and Looks



Presets: every tile is this picture through that preset. **Now** is the picture as it stands; **Save current** keeps the grade you are editing.

- **Presets**: a click adds the preset as a new adjustment layer on top, named after it, so its opacity is how strongly it applies and one undo takes it away. **Save current** keeps the grade you are editing as a preset of your own; the shipped ones (Auto, Warm film, Cool clean, Mono contrast) cannot be deleted.
- **Looks**: the picture's LUT, also as tiles. **None** takes it off; **Import** reads `.cube` files, or drop them on the strip.
- The arrow on the right folds the strip to one line.

Generated skies and studio backdrops often band. The grade is computed in floating point and dithered on the way back to 8 bit, so grading doesn't make the bands worse.

CHAPTER 11 · SECTION 06

Select an area

A selection says where the next thing happens: a fill, a copy, an adjustment, or a repair. Make one with a Marquee (**M**), the Lasso or Polygonal lasso (**L**), or the **W** group:

- **Object**: click a thing, or switch **Prompt** to Box and drag a box round it, and a model on your computer finds its edge. With **Highlight on hover** on, the object under the pointer lights up before you click. Alt-click says "not that".
- **Select subject** (⇧⌘U): the picture's main subject in one press.
- **Wand**: everything that looks like the spot you click.

Object and Select subject need a model, downloaded once

Both run on your computer, not on ComfyUI. Their models are downloaded in `Settings → Models & sources → Local Models`: **Object selection** (MobileSAM) and **Select subject** (BiRefNet lite), each with **Download** and a size. The Canvas never starts a download itself; where a model is missing, it shows a dashed **Model in Settings** chip instead.

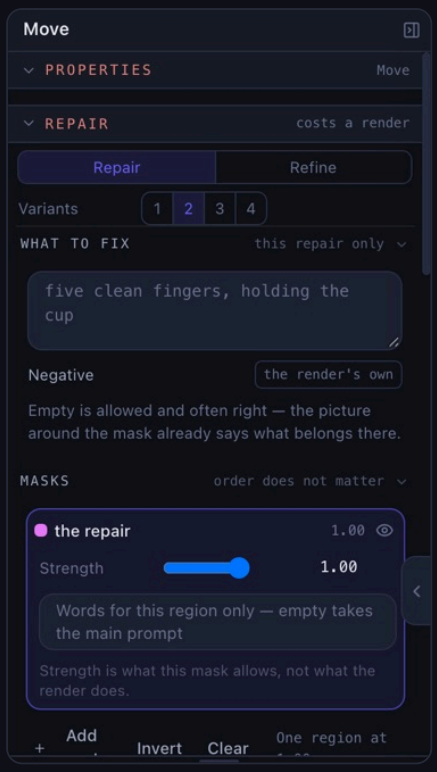
Into, in the options bar, decides what the selection tools make: the **repair mask** (or, on a reference picture, a **Selection**), or the **layer mask** of the picked layer. Properties then shows the selection's own card: how much of the picture it covers, Select all (⌘A), Deselect (⌘D), Invert (⇧⌘I), Reselect (⇧⌘D) and its edge.

Under an active selection a small bar offers what it can become: **Repair this**, **Remove** (filled from the picture around it, no model), **To mask** (the picked layer's mask), **Layer via copy** (⌘J), **Adjust area** (an adjustment layer that reaches only this area), **Mirror** (a mirrored copy on a layer, for the other eye or earring) and **Refine** (grow, shrink, feather or smooth the edge).

CHAPTER 11 · SECTION 07

Repair a spot with ComfyUI

1. **Mark the area.** Paint it with the Mask brush (**Q**), or use a selection with **Repair this**. Mark a little beyond the edges: the model needs room to blend. A dashed box shows your mask, a larger one what is actually sent.
2. **Say what to fix.** After the stroke a **card** opens beside the mask with the words for this region; press Enter to type in it, Esc hands the keyboard back to the brush. Describe what should be there, for example *five clean fingers, holding the cup*: short, and about that spot only. Empty is allowed and often right; the picture around the mask already says what belongs there. The same field is in `Repair → What to fix`.
3. **Check the setup** (bottom of the Repair section; usually already right).
4. **Render**: in the card, the button at the bottom right of the picture, or ⌘/Ctrl+Enter.



Repair: Repair or Refine, how many Variants a send makes, What to fix, and the masks with their own strength and words. Send and Setup follow below.

- **Variants** (1 to 4): how many suggestions one send makes, each with its own seed.
- **Strength**, per mask: at 1.00 the area is drawn again from nothing; lower values keep more of what is there. For a small fix (a blurry eye) try 0.5 to 0.7; to replace something, stay at 1.00.
- **Add mask** creates a second area with its own strength and words, so one render can fix two things. **Invert** swaps mask and rest; **Clear** empties it.
- **Send**: the coherence edge (the soft ring where old and new blend) and the size the area is rendered at. The word in the strip under the picture switches between sending the **region** around the mask (more detail for a small fix) and the **whole picture**.
- **Setup**: **Target** (your ComfyUI or Comfy Cloud), **Workflow** (the inpaint workflow for your model family; it can be a different family than the render's), and **Checkpoint**, the workflow's own unless you pick one in the app's model picker. The picker offers only models this workflow can load.

Choose a suggestion

The suggestions arrive as tiles in the card, each saying whether it is waiting, rendering or done. You compare them **in the picture**: hover a tile, or press ← / →, and that suggestion is shown at the spot. Click a tile, or **Keep** (Enter), to lay it on the picture as a layer named after the region; a second Keep replaces it. **N more** (⌘Enter) adds new seeds and keeps the ones you have; **Stop** stops those still on their way; **Done** (⌘D) clears the mask. Esc closes the card and leaves the mask; the suggestions are still there when you open it again, and they keep rendering if you leave the Canvas.

At the bottom of the card: **Remove · 3** fills the masked area from the picture around it three ways (no model, no render) for you to choose from, **Mirror** copies the area mirrored onto a layer, and **Edge** grows, shrinks or softens the mask.

Refine detail

Switch the Repair section (or the card) from **Repair** to **Refine** when a spot is right but soft: eyes, lashes, a hand. Refine renders the spot larger than the model's working size at a low **Creativity**, so it adds detail without drawing something new; **Detail** sets how much larger. Its words are optional (*sharper lashes*); empty uses the main prompt. When the area is too small to gain anything, the card first offers **Enlarge N×** (the whole picture, on this computer, as a new canvas), **Topaz…** or **Refine anyway**.

CHAPTER 11 · SECTION 08

Layers

Every kept repair, retouch, tone and adjustment is a layer. Add your own with **+**: a paint layer, text, another picture, a watermark. The top of the list is on top of the picture; drag rows to reorder. Switching a layer off, fading it or reordering costs no render.

- Each layer is **in render** (the model sees it in the next repair) or **on top** (never sent; for a caption or watermark).
- **Pick several**: ⇧-click a range, ⌘/Ctrl-click to add one; the eye, Delete and moving act on all of them.
- Every command is in the row's ⋯ menu and on a right-click, with its key: **Duplicate layer** (⌘J; with a selection, **Layer via copy**), **Layer via cut** (⇧⌘J), **Merge selected** (⌘E), **Merge down** (⇧⌘E), **Stamp visible** (⌥⇧⌘E: everything you see as a new layer), **Flatten into the picture**, **Rename** (F2). Clip to the layer below with ⌥⌘G. The blend-mode list previews a mode while you hover it.
- **Layer masks**: **Add layer mask** gives a layer its own mask, white shows, black hides. With that layer picked, the Mask brush paints the mask and the Mask eraser hides. On the mask thumbnail, ⌘-click makes a selection from it, ⌥-click shows the mask alone, ⇧-click switches it off.
- **Lock transparency** (/): the brush, the stamp and the eraser stay inside the layer's own pixels, for recolouring a piece without painting past its edge.
- **History**, below Layers, steps back through what you did. ⌘Z / ⇧⌘Z undo and redo while the Canvas has focus.

CHAPTER 11 · SECTION 09

Depth of field, tilt-shift and lens blur

The **Lens** group (**K**) blurs the picture the way a camera would, and every result is a **new layer** with its own mask, so the picture underneath stays as it was.

- **Depth of field** first needs the picture's depth. **Read depth** in the dock's Lens section runs a depth preprocessor once on your ComfyUI; every control after that is instant. If the picture changes, it says so and offers **Read again**. Then click the part that should stay sharp. The **focus strip** shows the sharp band and the falloff (drag its handles), with **Focus**, **Range**, **Falloff**, **Edge**, **Amount** and **Bloom** beside it; **Show**

on the picture tints the zones (sharp clear, falloff amber, blurred blue). **Make** (Enter) writes the layer; **Use as a mask** turns the same depth into a repair mask, a selection or a layer mask.

- **Tilt-shift**: drag a band across the picture; its handles move, widen and turn it. **Blur** and **Falloff** set the look, **+ saturation, + contrast** makes a town read as a model. **Make**.
- **Lens blur**: blurs the selection, the picked layer or the whole picture as a lens does, with **Radius** and **Bloom**. Apply writes it; Esc puts the preview away.
- The **Depth mask** in the Q group uses the same depth: click a depth and everything behind it, in front of it or round it goes into the mask.

The live preview needs WebGL; without it, Make still works.

CHAPTER 11 · SECTION 10

Upscale with Topaz

Topaz in the head opens **Upscale with Topaz**: every Topaz model, each with the settings it takes (creativity, faces, description, variations), a scale of 1–8×, PNG or JPEG. A size Topaz cannot take is caught before it is sent, with the size that fits.

Every run is kept, with the picture it was sent, so you can compare any two: a run against its input, or one run against another. Runs take minutes; you can close the window meanwhile. **Use as canvas** continues on the chosen run as a new picture; the old one keeps its layers.

CHAPTER 11 · SECTION 11

Save your picture

- **Save** writes the picture as you see it, every layer and the grade, into your output folder (with the watermark every finished render gets). When the picture came from a saved render, it lands **beside that render** as `<render>_canvas_<time>.png`, carrying the render's prompt data along, so the Crosspost Helper can stack the two. It also appears in the **Gallery**.
- **As reference** saves it into Reference Images, for example as the start of an img2img render. Layers set to "on top" are left off.
- **Bake** flattens everything into one picture and saves it the same way. Afterwards the separate layers are gone from that picture.
- **Open another** puts this picture away; your work stays under Unfinished repairs.

When you came from a render, the repair replaces it

Opened with **Repair** from the Prompt Builder, the Canvas is a window over the Builder, and the repaired picture **replaces the render it was made from**: when you leave (Send back, ✕, or Close anyway), the picture as it looks now is written over that render's file, in its format. Nothing is written if you changed nothing. In Tuning there is also **Send to Final**, which hands the repaired picture to the Final pass instead.

CHAPTER 11 · SECTION 12

Tips

- **Small fault, hand tool.** A blemish, a stray hair, a speck: Spot healing or Remove is faster and more faithful than a render.
- **Colour bleeding** (red hair tinting the forehead): the Colour brush with the right colour picked next to it.
- **Plastic skin**: frequency separation, then Smooth on Low only.
- **Mask generously for a repair**, and describe only the spot. One problem per mask.
- **Run again** a few times; keep the best one.
- **Paint first, then repair**: a rough stroke in the right colour, kept in render, repaired at medium strength, steers the model.

CHAPTER 11 · SECTION 13

Troubleshooting

"Repairs cannot run yet — comfy-local has no inpaint path".

Your active connection is the older plugin one. Switch to ComfyUI API (local) or Comfy Cloud, or set the Target in Setup.

The repair fails with "Value not in list" or a missing file.

ComfyUI doesn't have the model the workflow asks for. Pick a checkpoint under Setup → Checkpoint; if the Canvas lists files this machine hasn't got, put them into the models folder your ComfyUI reads first (see the Getting Started chapter).

Object or Select subject shows "Model in Settings".

Their model is not downloaded yet. Open Settings → Models & sources → Local Models and click Download on that row. It is downloaded once and kept.

Read depth fails, or Depth of field says "needs the depth".

The depth is read on your ComfyUI with `DepthAnythingV2Preprocessor` from the ControlNet auxiliary nodes. Check that ComfyUI is running and has that node pack; the one-click setup in Settings → Rendering installs it with the core workflows' node packs.

My stroke did nothing.

Read "→ where" in the options bar: the stroke may have gone into a Retouch or tone layer that is switched off or faded, or into a mask. Check the Layers list.

The repaired area looks pasted on.

Make the mask larger or raise the coherence edge, and render again. A suggestion you kept is a layer: paint its layer mask with the Mask eraser to soften the seam, or lower its opacity.

A key does something else than I expected.

The Canvas uses Photoshop's letters. Press ? for the sheet.

I closed the app in the middle of a repair.

Open the Canvas again: your work is under Unfinished repairs. A render that was still running is lost.

CHAPTER 12

Storyboard: a story told in pictures

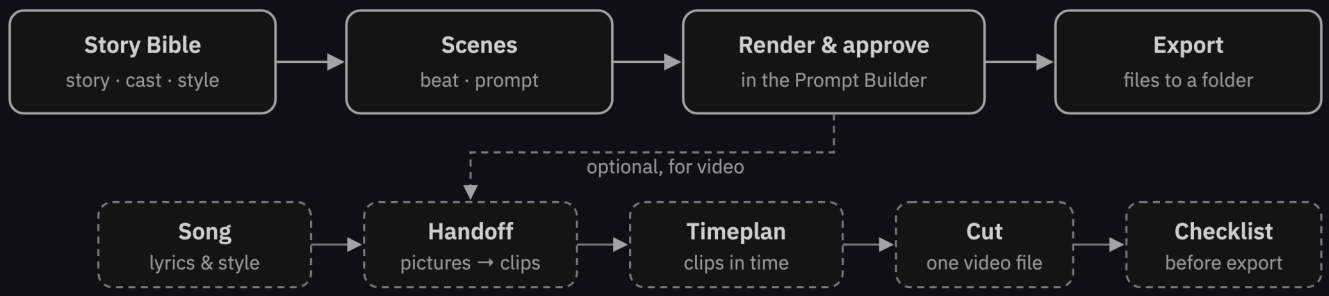
Turn a story into a series of consistent pictures, and if you like into video clips and one finished edit. The same page also makes card decks and illustrated series. This chapter walks you through a whole storyboard, from the first line of text to the exported files.

– What the Storyboard is for 157
01 Find your way around 157
02 Story Bible: what every scene shares 158
03 Break the story into scenes 159
04 Work on one scene 160
05 Render and approve 162
06 Export the pictures 164
07 Optional: Song 165
08 Optional: video clips with Handoff 166
09 Optional: Timeplan 167
10 Optional: Cut, one finished video 168
11 Checklist: before you export 169
12 Card decks 170
13 Series: pictures for an article 171
14 Tips and questions 172

What the Storyboard is for

For one picture, the Prompt Builder is all you need. For twenty pictures that belong together (the same characters, the same look, one scene after another) you need one place that holds the story, the cast and the style. That place is **Storyboard** in the sidebar.

A storyboard holds your story text, the characters in it, the places it happens and one locked style. You break the story into scenes, write a prompt for each, render them in the Prompt Builder and approve the picture you want per scene. Approved pictures are saved as files in a folder you choose. From there you can export them, or go on to video clips, a timeline and a finished video.



The picture path along the top is all you need. The video stages below it are optional and appear only in a storyboard (a deck or a series has no running time).

A render setup
A working connection to ComfyUI or another service (see the Getting Started chapter), and the characters of your story in the character library.

An AI provider
For Break into scenes, Generate, the rewrite buttons and the song writer: a key under Settings → AI → AI Provider API Keys. Everything else works without one.

Video, if you want it
A Wavespeed key under Settings → Rendering → Render Services to make clips in the app. ffmpeg for the Cut ships with the app.

CHAPTER 12 · SECTION 01

Find your way around

Open **Storyboard** in the sidebar. Everything happens on this one page.

Top to bottom: the header, the stage rail with its numbered steps, and the scene strip.

- **The header.** On the left it says what is open (*Storyboard · Story*, *Deck* or *Series*). The name with the small arrow switches between your storyboards and deletes one. On the right: **Placed** (how much

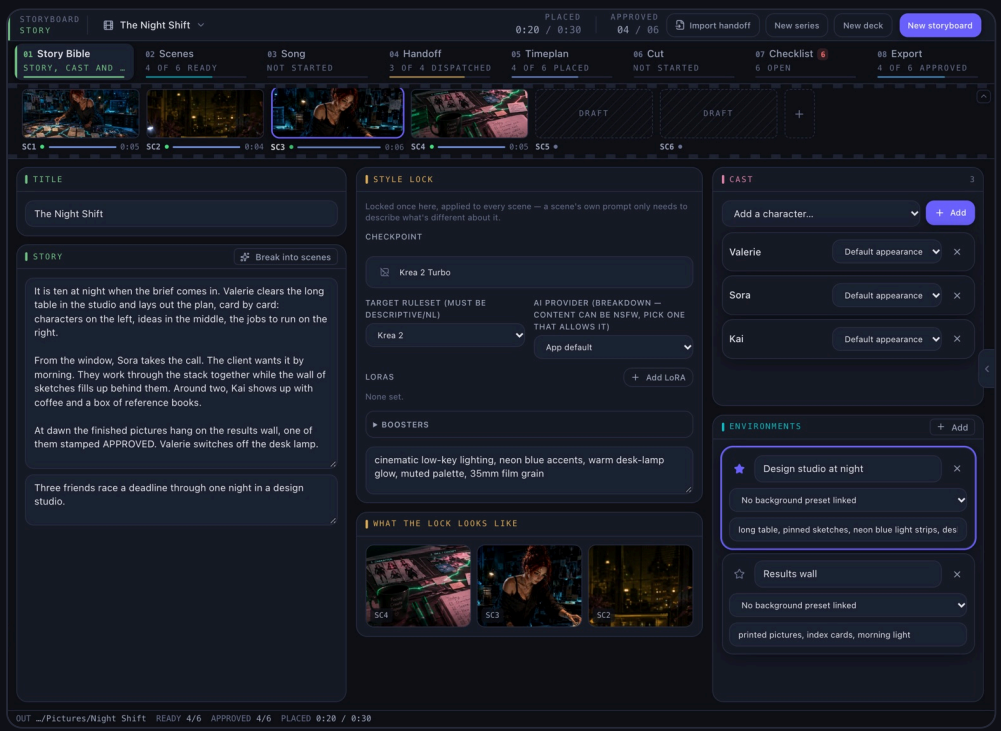
running time is on the timeline, against the target) and **Approved** (scenes with an approved picture). Then **Import handoff**, **New series**, **New deck** and **New storyboard**. On a narrow window the first three move into a **···** menu.

- **The stage rail**: **01 Story Bible** · **02 Scenes** · **03 Song** · **04 Handoff** · **05 Timeplan** · **06 Cut** · **07 Checklist** · **08 Export**. Each stage shows where it stands (*4 of 6 ready*, *plan drafted*) and fills its bar as you go; a tick means done, a red number means items are open. You don't have to go in order, and you can skip stages you don't need.
- **The scene strip**: one tile per scene (SC1, SC2 ...). A picture means the scene has an approved image; *DRAFT* means a draft scene without one, *NO RENDER* a scene marked Ready that has not been rendered yet. The dot under a tile shows its state, the line beside it how long its clip runs. Click a tile to open the scene, drag it to reorder, hover it to **duplicate** or **delete** it. The **+** tile adds a scene. The arrow at the right hides the strip when you need room.
- **The status bar** at the bottom: **OUT** (the output folder, red *NOT SET* until you pick one), **READY**, **APPROVED** and **PLACED**. It is a readout only.

CHAPTER 12 · SECTION 02

Story Bible: what every scene shares

Click **01 Story Bible**. Fill it in once; every scene reads from it.



The Story Bible: title and story on the left, the style lock in the middle, cast and environments on the right.

- **Title** and **Story**: paste or write the whole story. The small field below it is an optional one-line synopsis.
- **Style Lock**: the look that applies to every scene, so a scene's own prompt only has to describe what is different about it.
  - **Checkpoint**: click *Not set* to pick the model from the checkpoint library.

- **Target ruleset**: how prompts are written for your model. Only natural-language rulesets are offered (FLUX, FLUX.2, GPT Image, Krea 2, Anima (Prose), Qwen-Image), because a scene prompt is prose.
- **AI provider** for the scene breakdown. Stories can be NSFW; pick a provider that allows your content. *App default* follows your app-wide choice.
- **LoRAs** (`Add LoRA`) and **Boosters**: a reminder of what this storyboard uses. They are not applied automatically; set them in the scene's prompt panels.
- A free **style notes** field, for example *cinematic low-key lighting, muted palette*.

- **What the lock looks like**: the scenes approved so far, side by side, so you can see whether the style really holds.
- **Cast**: choose a character in *Add a character...* and click `Add` . The select beside each name picks which appearance (variant) that character has by default. Only characters added here can appear in scenes.
- **Environments**: the places the story happens. `Add` one, name it, optionally link a background preset from your library and add notes. The star marks the global default place.

CHAPTER 12 · SECTION 03

Break the story into scenes

Add scenes one by one with the `+` tile, or let the AI do the first pass: click `Break into scenes` above the story. Optionally give a target scene count, pick the AI provider and click `Generate` . You get a numbered preview with chapter labels first, then decide: `Add N scenes` or `Discard` .

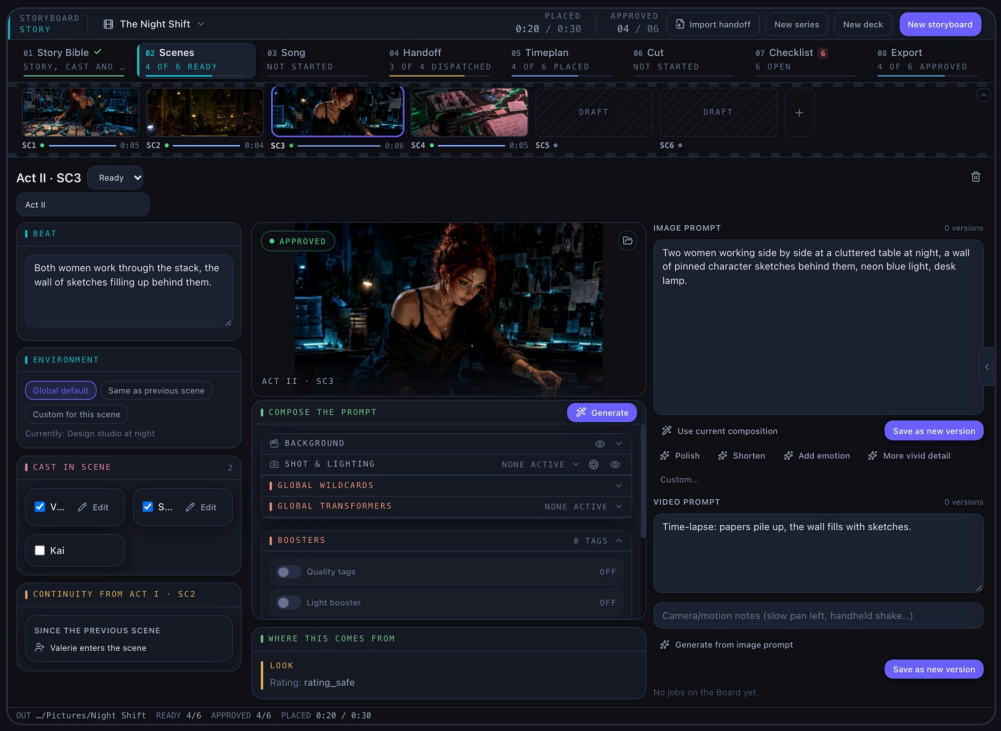
The preview of a breakdown, before anything is added.

> **It never overwrites anything**
> New scenes are always added as drafts at the end. Existing scenes stay exactly as they are, so you can run it again on a longer story without losing work.

CHAPTER 12 · SECTION 04

Work on one scene

Click a tile in the strip. The scene opens in three columns: its facts on the left, its picture in the middle, its prompt on the right.



An open scene. The middle shows the approved picture; a scene without one says *No render yet*.

The facts (left)

- At the top: the scene's name (*Act II · SC3*), a status select (**Draft** or **Ready**), an optional chapter label such as *Act I*, and the bin to delete the scene.
- **Beat**: what happens in this scene, in one or two sentences.
- **Environment**: **Global default** (the starred place), **Same as previous scene**, or **Custom for this scene**. The line below says where the scene currently takes place.
- **Cast in scene**: tick who appears. **Edit** next to a name opens that character's card for this scene: outfit, expression and pose here belong to this scene only, not to the library.
- **Continuity**: what changed since the previous scene (who came and went, a new place), so jumps are easy to spot.

The picture (middle)

The approved image with its label, and a folder button that opens the file. Renders made for this scene that are not approved yet line up below it as **candidates**, each with an **Approve** button.

The prompt (middle and right)

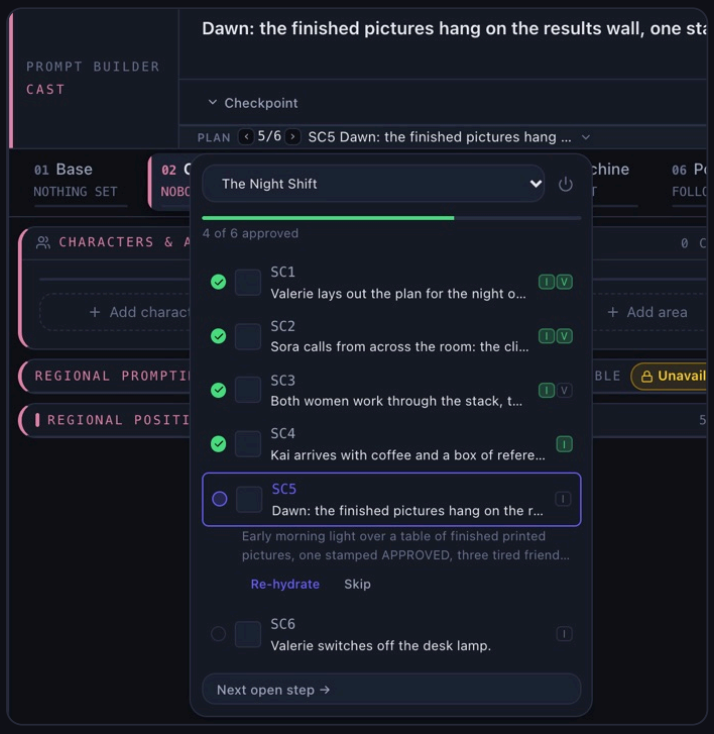
- **Compose the prompt** holds the real Prompt Builder panels for this scene: Background, Shot & Lighting, Global Wildcards, Global Transformers and Boosters. What you set here belongs to this scene only.
- **Generate** opens AI Generate with the beat as the idea and everything set above, and writes a finished prompt into the image prompt.
- **Where this comes from** lists which parts of the composition came from where (the look, the cast, the place).
- **Image prompt**: this text is what gets rendered. A new scene is filled from the composition once; **Use current composition** pulls it in again. You can also just type. Nothing is kept until you click **Save as new version**; earlier versions stay under **Image prompt history** and can be restored.
- The AI buttons **Polish**, **Shorten**, **Add emotion**, **More vivid detail** and **Custom...** rewrite the text in the box.
- **Video prompt** and the camera/motion field are only needed for video clips. **Generate from image prompt** drafts one for you. Save it with its own **Save as new version**.
- **Send to Job** at the bottom sends this scene's prompt to a setup on the Jobs page and starts it (**Send this scene's prompt**). Without setups it says *No jobs on the Board yet.*

CHAPTER 12 · SECTION 05

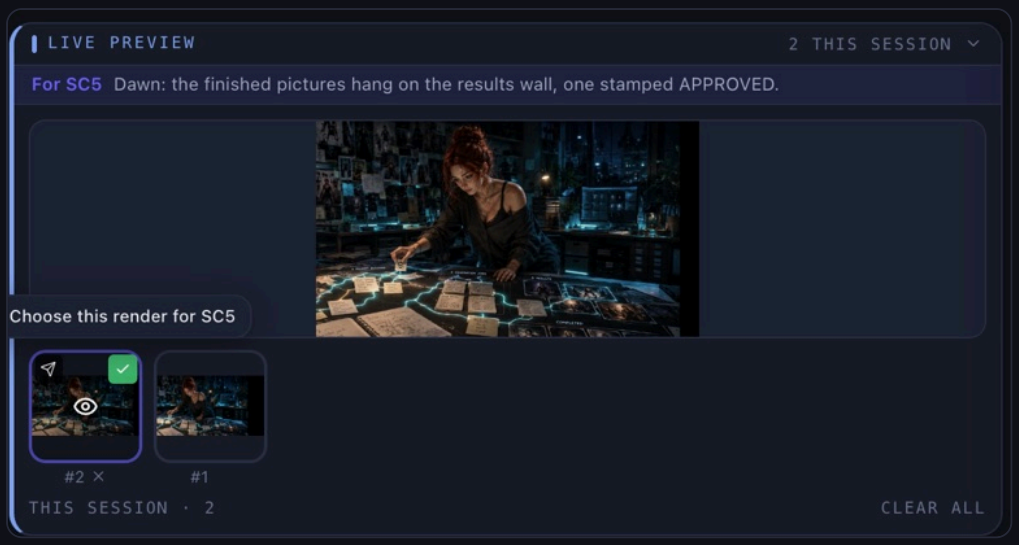
Render and approve

A scene is rendered in the Prompt Builder, the same one you use for single pictures. The Builder's **Plan** chip walks you through the storyboard scene by scene.

1. Go to **Current Job** in the sidebar. In the row under the Builder's head, click **PLAN · Start a plan** and pick your storyboard in *Switch plan…*. The Builder loads the first scene that has no approved picture: its prompt, cast and settings.
2. The scene's image prompt stands in **The Prompt**, marked *Frozen*, so the Builder renders exactly that text. The head names the step (*Step 5 of 6 — The Night Shift*).
3. Render as usual. The previews get a bar saying *For SC5* and the beat. If the scene has a video prompt, an **Image / Video** switch on that bar says which slot you are approving for.
4. Hover the render you like and click the tick in its top-right corner (*Choose this render for SC5*).
5. The first time, the app asks for an output folder for this storyboard. The picture is saved there, the scene is marked approved, and a message offers **Next: SC6 →** to load the next open scene.



The plan: every scene with its state (approved, waiting for review, current), **Re-hydrate** and **Skip** on the current one, **Next open step** at the bottom. The arrows beside *5/6* step without opening the list; the power button hands the Builder back.



The previews with a scene loaded: *For SC5* on top, the tick on each finished render.

Opening a scene on the Storyboard page

Clicking a scene in the strip also loads it into the Builder's panels, so you can compose it there. Leaving the Storyboard page hands the Builder back. To render, use the Plan chip as above.

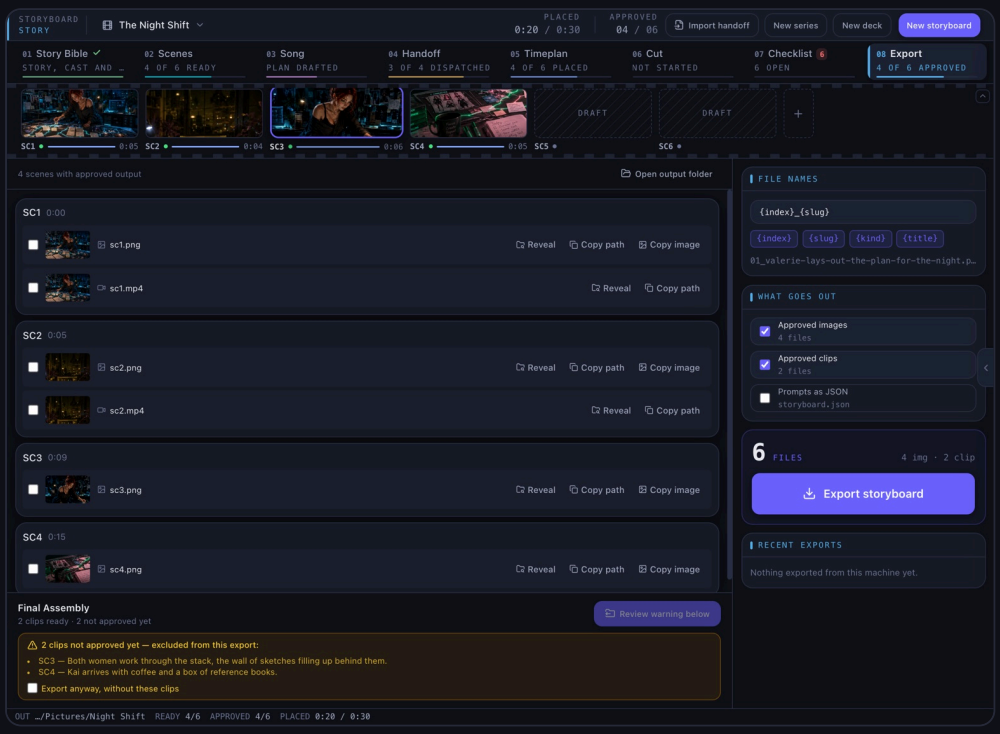
Approved means saved

Approving writes the file into the output folder right away, named after the scene number and beat (*sc03-…*). Approving another render for the same scene replaces the file; the previous one is archived, not deleted. Renders made while a scene is loaded also land on that scene's own job card, so the Jobs page can say what each render was for.

CHAPTER 12 · SECTION 06

Export the pictures

If you only want the pictures, skip straight to **08 Export**.



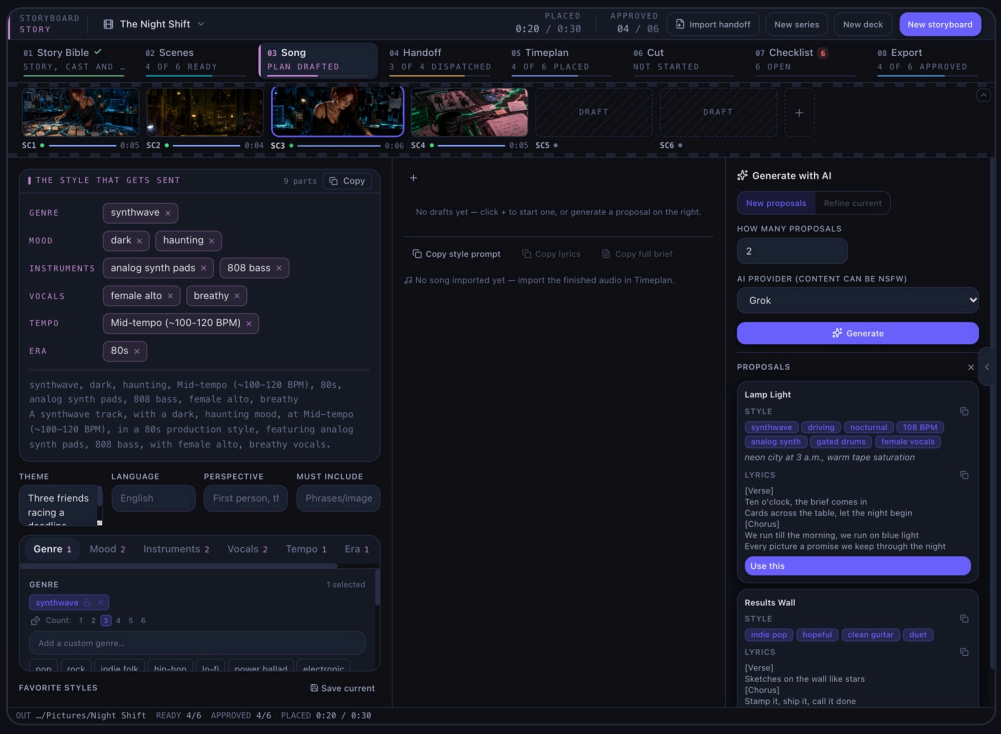
Left: every scene with approved output. Right: file names, what goes out, and the export button.

- The list on the left: **Reveal** shows a file in your file manager, **Copy path** and **Copy image** copy it, and you can drag files straight onto another program. **Open output folder** opens the folder.
- **File names**: a pattern for the exported names. Click the chips `{index}`, `{slug}`, `{kind}` and `{title}` to build it; the line below shows an example.
- **What goes out**: tick **Approved images**, **Approved clips** and, if you want the text too, **Prompts as JSON**.
- Click **Export storyboard** and choose a folder. **Recent exports** remembers where the last ones went.
- **Final Assembly** below the list writes the approved clips in timeline order into an *assembly* folder, numbered, with a manifest, an ffmpeg script and an EDL for a video editor. It names clips that are not approved yet; tick *Export anyway, without these clips* to go on.

CHAPTER 12 · SECTION 07

Optional: Song

03 Song helps you write a song for your video, for a music tool such as Suno. The app does not make the audio itself.



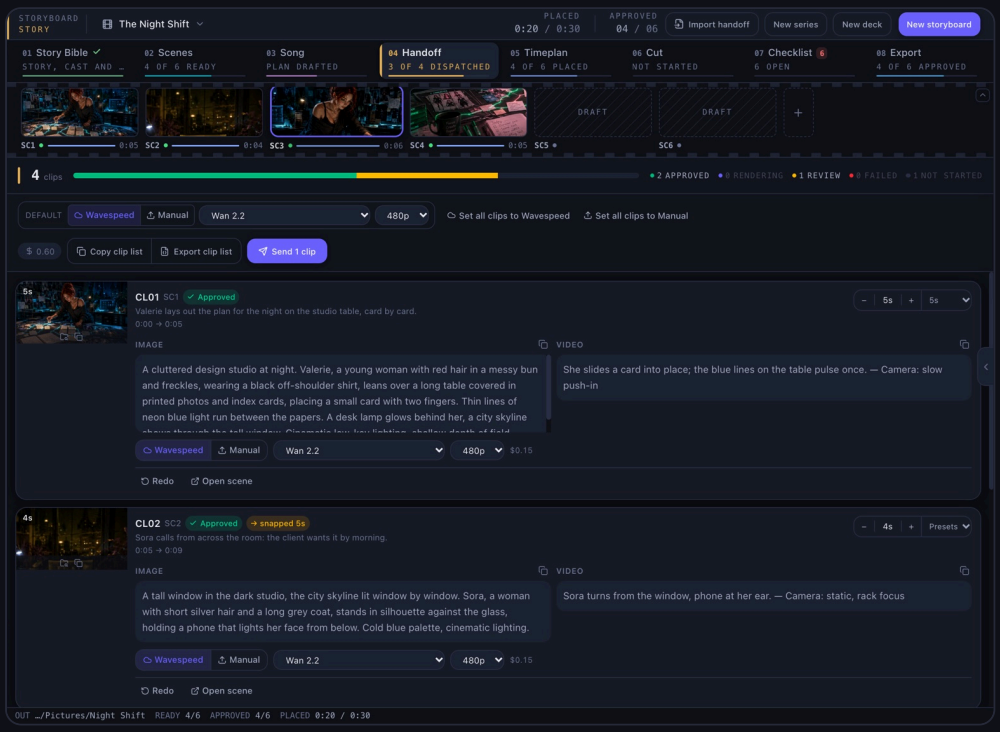
The style that gets sent on the left, the drafts in the middle, Generate with AI on the right.

- Start by writing what the song is about, or click one of the sounds under **Or start from a sound** (*Dark Synthwave*, *Epic Orchestral* …): each fills the style in for you.
- **Theme**, **Language**, **Perspective** (whose voice it is) and **Must include** steer the lyrics. The tabs Genre, Mood, Instruments, Vocals, Tempo, Era and Avoid build the style; **The style that gets sent** shows the result. **Favorite styles** keeps a style for later.
- On the right, **Generate with AI**: **New proposals** (how many, which provider) or **Refine current**. Each proposal brings lyrics and a style; **Use this** takes it into a draft. The **+** above the drafts starts one by hand; mark sections with `[Verse]`, `[Chorus]` and so on.
- **Copy style prompt**, **Copy lyrics** and **Copy full brief** take it to your music tool. The finished audio file is then imported on the Timeplan.

CHAPTER 12 · SECTION 08

Optional: video clips with Handoff

04 Handoff turns approved pictures into video clips. Every scene placed on the Timeplan becomes a clip card (CL01, CL02 ...) with its picture, its image and video prompts and its length. The bar at the top counts clips by state: approved, rendering, review, failed, not started.



The defaults bar and two clip cards. A clip whose length the video model cannot do is snapped to the nearest one it can, and says so.

Every clip is made one of two ways:

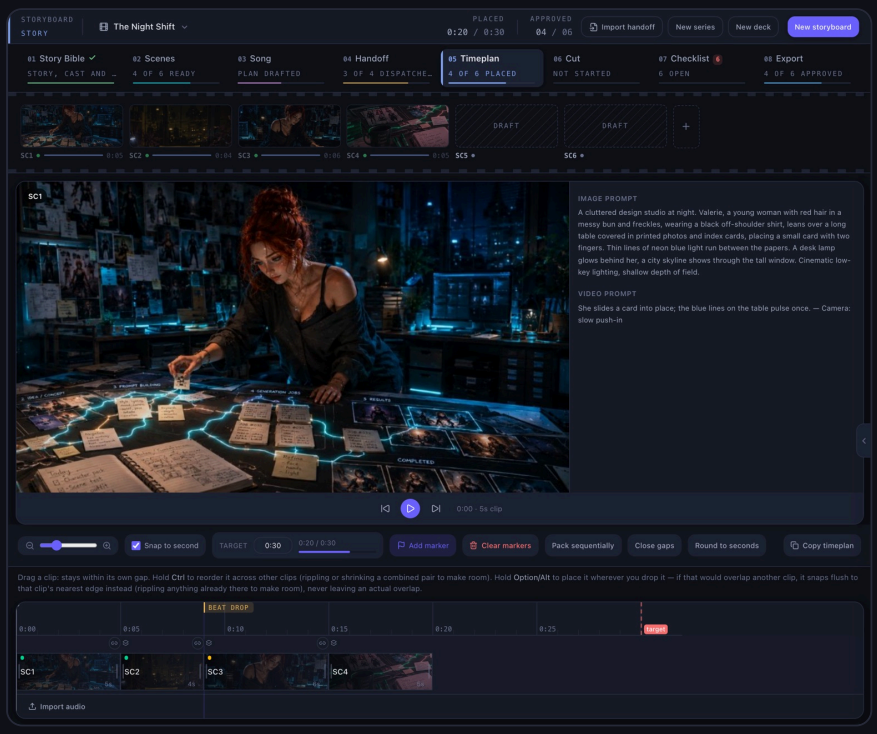
- **Wavespeed**: the app sends the clip to the Wavespeed video service (needs a key, costs money). The default model and resolution are in the bar, each card can override them, and the bar shows the estimated cost. **Send N clips** sends every Wavespeed clip that has not been made yet or failed. A finished clip appears on its card; click **Approve**, or **Redo**.
- **Manual**: you make the clip in another tool. **Copy clip list** and **Export clip list** give you all prompts at once. When a clip is done, drop the file onto **Import external video** or paste a link and click **Use link**; it is approved right away.

Set all clips to Wavespeed and **Set all clips to Manual** switch every card at once. The length control (−, +, presets 3/5/8/10 s) sets how long the clip runs; **Open scene** jumps to the scene. Scenes with a video prompt that are not on the timeline yet are listed separately.

CHAPTER 12 · SECTION 09

Optional: Timeplan

05 Timeplan decides when each scene plays.



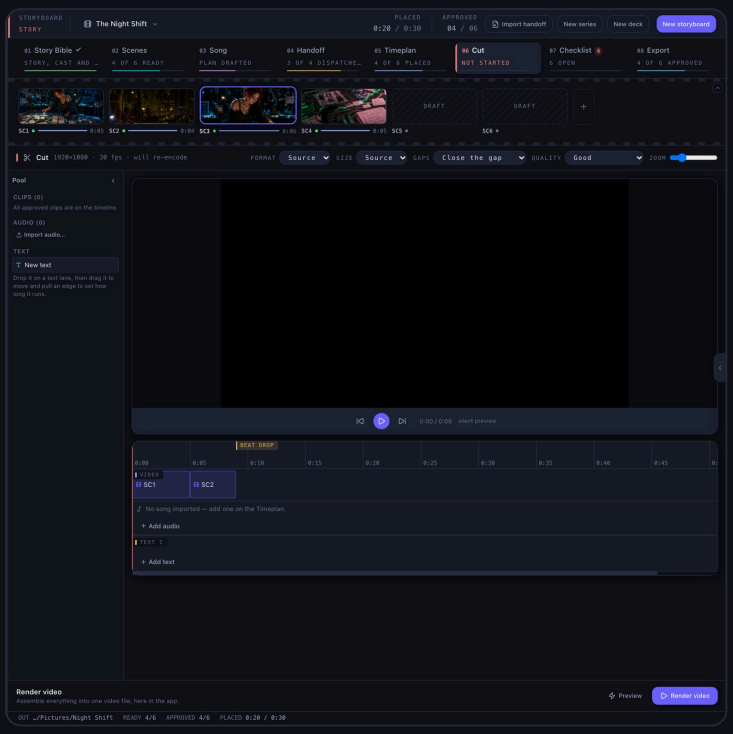
The preview with the scene's prompts on top, the toolbar, and the timeline with a marker (*Beat drop*) and the target line.

- Drag a clip along the timeline; pull its edge to change its length. A plain drag stays within its own gap; hold **Ctrl** to reorder across other clips, **Option/Alt** to drop it anywhere (it snaps flush instead of overlapping).
- Between two touching clips sit two small buttons: the link **combines** them (they move together and their shared edge trims one against the other), the layers icon cycles a **crossfade** of 0.25, 0.5 or 1 s (none is a hard cut).
- **Target** is how long the video should be; the bar beside it shows how much is placed. **Snap to second** keeps edits on whole seconds; the slider zooms.
- **Add marker** sets a named point (for example *Chorus starts*), **Clear markers** removes them. **Pack sequentially**, **Close gaps** and **Round to seconds** tidy the timeline; **Copy timeplan** copies it as text.
- **Import audio** under the clips adds your song as a waveform, so you can place scenes on the music. **Analyze lyrics** finds when each line is sung and shows the lines under the waveform, with a search box to jump to a word. It needs an OpenAI or ElevenLabs key; pick which under Settings → AI → Song Transcription.

CHAPTER 12 · SECTION 10

Optional: Cut, one finished video

06 Cut assembles the approved clips, your music and text into one video file, inside the app.



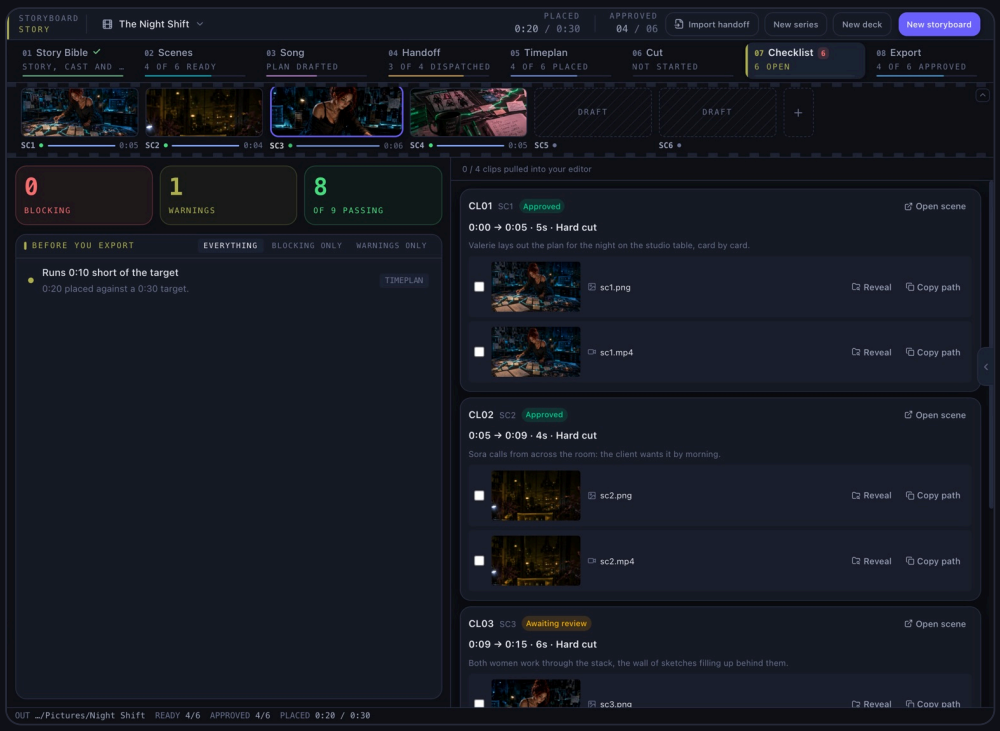
The Pool on the left (clips, audio, text), the preview, the video, audio and text lanes, and Render video at the bottom.

- Only **approved clips** appear here; the video lane says so when there are none. The song from the Timeplan is the music; **Import audio...** in the Pool or **Add audio** adds more.
- **New text**: drop it on a text lane, drag it to place it and pull an edge to set how long it runs. **Add text** adds a lane.
- At the top: **Format** (Source, 16:9, 9:16, 1:1, 4:3), **Size** (Source up to 2160p), **Gaps** (Hold last frame, Fill with black, Close the gap) and **Quality** (Draft (fast), Good, Best (slow)). The line beside *Cut* says what will come out and whether it has to be re-encoded.
- **Preview** renders a quick look into its own file; **Render video** writes the finished one. Joining clips without re-encoding is quick; a long edit that needs re-encoding can take several minutes. ffmpeg comes with the app; Settings → Rendering → Video Rendering only matters if you want another build.

CHAPTER 12 · SECTION 11

Checklist: before you export

07 Checklist collects everything still open in one list, for example *No output folder is set* or *Runs 0:10 short of the target*. Each item names the stage it belongs to.



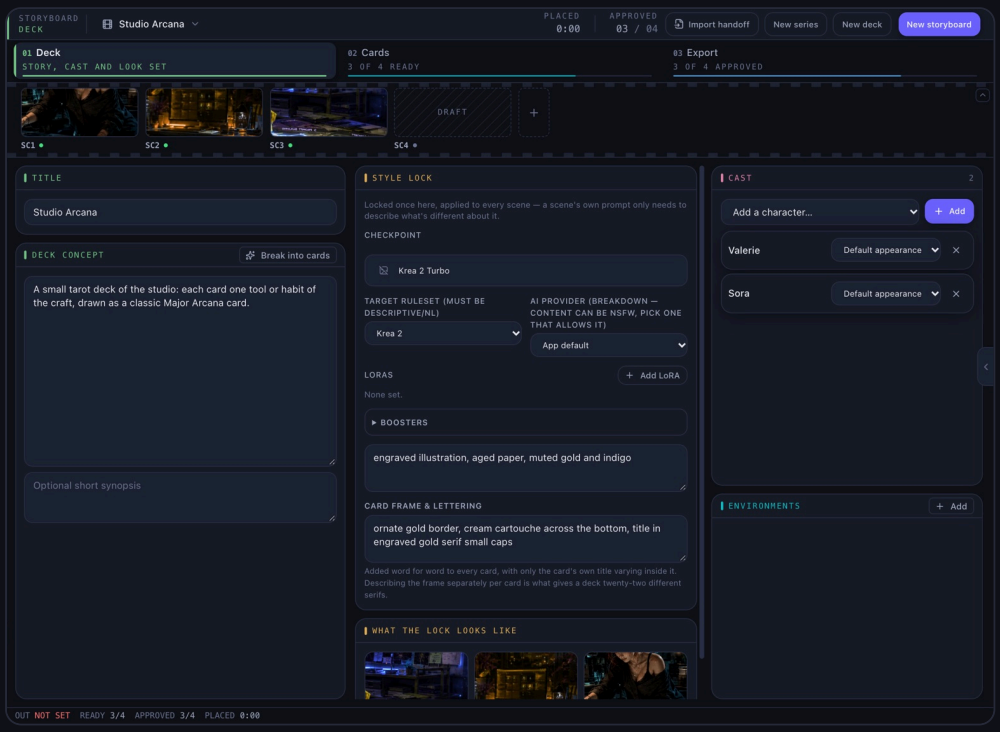
Blocking items stop an export, warnings are worth a look. The right side lists every clip in timeline order with its files.

Filter with **Everything**, **Blocking only** and **Warnings only**. On the right, every clip with its timing and transition, **Open scene**, and its picture and video files to reveal, copy or drag into a video editor. The ticks beside the files count how many you have pulled into your editor.

CHAPTER 12 · SECTION 12

Card decks

New deck starts a set of cards: tarot, playing cards, a character set. One locked look, many cards, no running time. A deck has three stages: **Deck**, **Cards** and **Export**. Song, Handoff, Timeplan, Cut and Checklist are hidden, because they all need clips placed in time.



The Deck stage: a deck concept instead of a story, and the extra field Card frame & lettering.

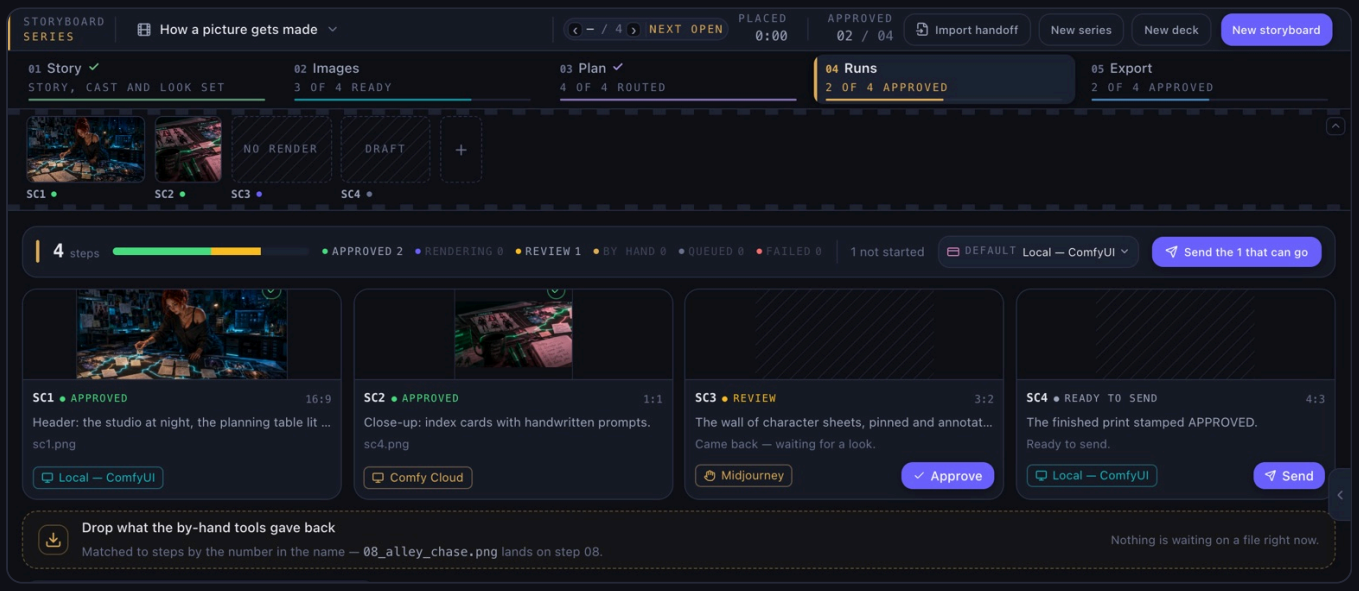
- **Deck concept** replaces the story; **Break into cards** turns it into draft cards the same way Break into scenes does.
- **Card frame & lettering** in the style lock is added word for word to every card, with only the card's own title varying inside it. That is what keeps the frame and the lettering the same on every card.
- Each card has a **Card title** field (for example *THE STAR*) next to its chapter label. The model draws the title onto the card, so check the spelling before you approve.
- Everything else (cast, prompt, render and approve) works as for scenes.

CHAPTER 12 · SECTION 13

Series: pictures for an article

New series starts pictures that illustrate a story or an article, where every picture may use its own tool and its own shape. The stages are **Story**, **Images**, **Plan**, **Runs** and **Export**. In the header, *‹ 05 / 12 ›* *Next open* steps through the open images.

- An image has a **Caption** instead of a beat, and a **How this one is made** box: **Made by** (a service the app sends to, a tool you use by hand, or an existing job card to borrow) and the **Ratio**. A value not set there shows the series' own, marked *From the series*. A picture can be marked *Explicit — keep this one on this machine*.
- **Plan** lists every step with its ratio and tool: how many are sent from here, made by hand, still need a tool, and how many ratios are in use. **Copy the by-hand prompts** copies all prompts for the by-hand tools at once.
- **Runs** shows what is out being made and what has come back. **Send** on a step, or **Send the N that can go**, starts the jobs; **Approve** and **Redo** decide on what came back. Pictures from by-hand tools are dropped onto *Drop what the by-hand tools gave back*; they are matched to steps by the number in the file name (`08_alley_chase.png` lands on step 08).



Runs: one tile per step with its state, ratio and tool, and below them the drop zone for by-hand results.

Import handoff reads a handoff file from ArticleHelper: every place in the article that expects a picture becomes a step, with the author's description as caption and prompt. Importing the same article again into its series adds new places and updates changed descriptions, but keeps every text you changed here. Characters the file names but your library doesn't have are listed in a message.

CHAPTER 12 · SECTION 14

Tips and questions

My scenes don't look alike.

Set a checkpoint and your style notes in the Story Bible's Style Lock, and keep scene prompts about what is different in that scene. *What the lock looks like* shows your approved scenes side by side, which makes drift easy to see.

A character is missing from "Cast in scene".

Only characters in the Story Bible's Cast can be ticked. Add them there first.

I changed the image prompt, but the render didn't change.

Click **Save as new version**. The text box is a draft until you do; the Builder renders the saved text.

There is no tick on my renders in the Builder.

No scene is loaded. Pick the storyboard under **PLAN** in the row under the Builder's head; the tick and the *For SC...* bar appear once a scene is active.

Where are my approved pictures?

In the storyboard's output folder, the one you picked the first time you approved. The status bar's OUT shows whether one is set, and **Open output folder** on the Export stage opens it.

Break into scenes or Generate does nothing or shows an error.

These need an AI provider under Settings → AI, and one that accepts your content. Pick another provider in the dialog's AI provider select.

The Cut stage is empty.

It only shows approved video clips. Approve clips on the Handoff stage first.

My deck has no Timeplan or Song.

Decks and series have no running time, so the video stages are hidden. Make a storyboard if you want clips.

CHAPTER 13

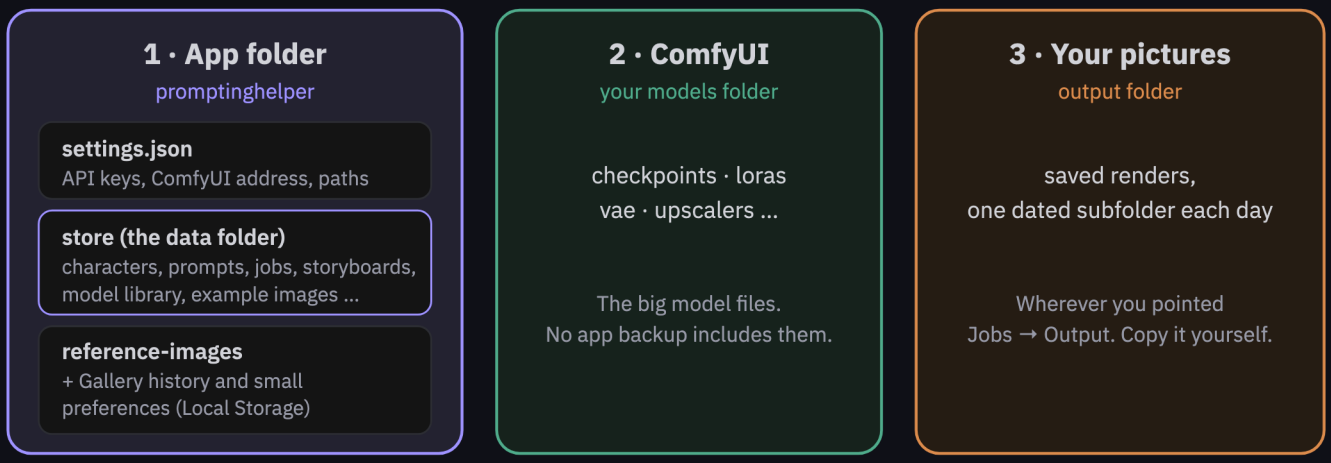
Backup & Moving: keeping your work safe

Where Prompting Helper keeps your work, which backup covers what, and how to take everything with you to another computer without losing anything on the way.

– Your work lives in three places 174
01 Find the app folder 174
02 Make a backup 175
03 What each backup covers 178
04 Move to a new computer 179
05 Keep the data folder somewhere else 180
06 Update the app safely 181
07 Questions 181

Your work lives in three places

Before you back anything up, it helps to know that your work is spread over three separate places. The app only takes care of the first one. The other two belong to ComfyUI and to your own folders.



Back up often
Copy the whole app folder (section 2, D). It is the only backup that keeps everything.

Before a big change
Make a Full Backup ZIP and a Library export as a second safety net.

Your own copy
Model files and finished pictures are never in an app backup. Copy them yourself.

CHAPTER 13 · SECTION 01

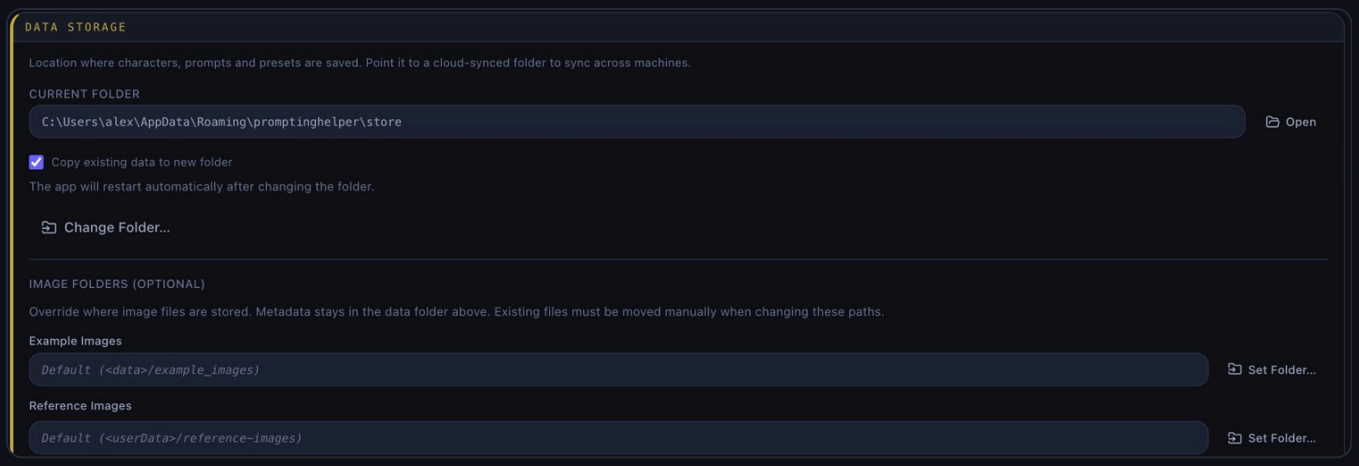
Find the app folder

Everything the app stores for you sits in one folder on your computer, called `promptinghelper`. Where it is depends on your system:

System	App folder
Windows	`C:\Users\<your name>\AppData\Roaming\promptinghelper` Quick way: press **Win+R**, type `%APPDATA%\promptinghelper` and press Enter.
Mac	`~/Library/Application Support/promptinghelper` In Finder: **Go → Go to Folder...** and paste the path.
Linux	`~/.config/promptinghelper`

Inside it, the subfolder `store` is the **data folder**: characters, saved prompts, jobs, storyboards, Lineup snapshots, Canvas projects, example images, the model library (`model_library.sqlite` plus preview pictures) and more. The app shows you exactly where that folder is:

1. Open **Settings** and go to the **06 Data** tab.
2. Under **Data Storage**, **Current folder** shows the path. Click **Open** to see it in Explorer or Finder.



Settings → 06 Data → Data Storage. **Open** shows the data folder. Under **Image Folders (optional)** you can keep example and reference images somewhere else; empty means the default place.

Next to the data folder, not inside it

The app folder also holds `settings.json` (API keys, your ComfyUI installs with their start parameters, model and output paths), `comfy-node-packs.json` (the ComfyUI node packs the app remembers), `datadir.json` (only if you moved the data folder), `reference-images`, `logs`, and the app's own browser storage (`Local Storage`) with your render history for the Gallery and many small preferences. If you copy only `store`, all of these stay behind.

Older versions: promptcharmanager

Before the app was called Prompting Helper, its folder was named `promptcharmanager`. On the first start under the new name the app moves that folder over by itself, so you don't need to do anything. If you restore an old backup copy of the folder, give it the new name.

CHAPTER 13 · SECTION 02

Make a backup

There are several kinds of backup. They cover different things, so read the table in section 3 before you rely on one of them.

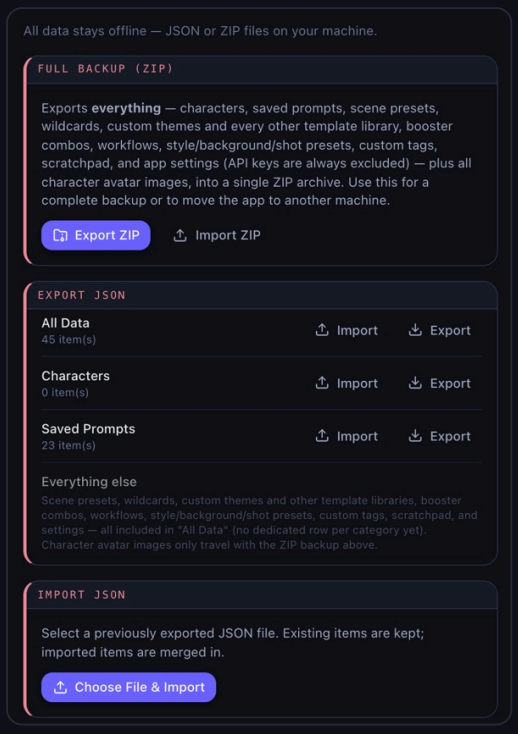
A · Full Backup (ZIP): characters and saved prompts in one file

1. Click **Import / Export** in the sidebar.
2. In **Full Backup (ZIP)**, click **Export ZIP** and choose where to save it. The suggested name is `sdpromptmanager-backup.zip` in your Documents folder; adding the date to the name helps.

What the ZIP actually contains: your **characters** with their avatar pictures, **saved prompts**, the **pose and background presets** from the Poses and Backgrounds pages with their preview pictures, your **LoRA recipes**, custom shot tags and your custom autocomplete tags.

The ZIP holds less than its description says

The text on the page lists wildcards, template libraries, booster combos, workflows, style/background/shot presets, custom tags, the scratchpad and app settings as well. In the current version these are **not written into the file** (the same goes for the JSON export). Back them up with the exports below, or with a copy of the whole app folder (D). Settings, and with them all API keys, never travel in a ZIP or JSON backup.



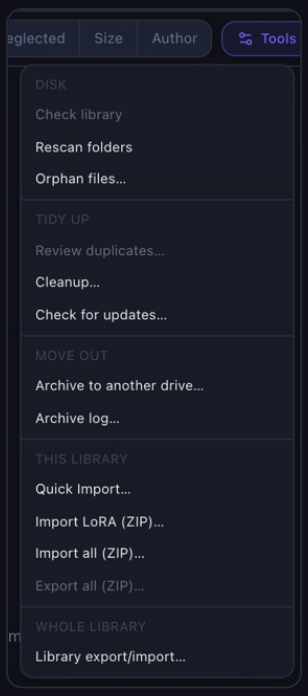
The Import / Export page. **Export JSON** saves the same content as a plain file, or only the characters or only the saved prompts, without any pictures.

B · Model Library: your LoRAs and checkpoints

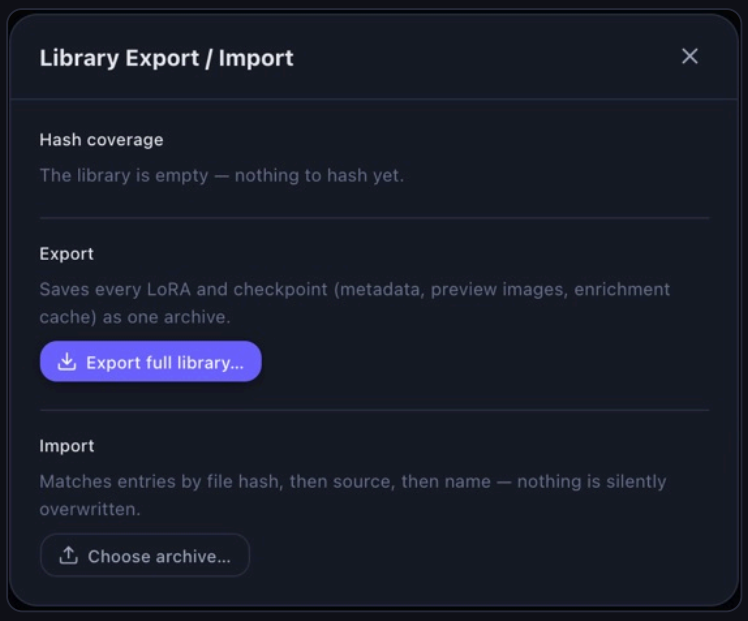
The Model Library has its own export:

1. Open **Model Library** and click **Tools**.
2. Under **Whole library**, choose **Library export/import...**.
3. Click **Export full library...**. This saves every LoRA and checkpoint entry with its notes, settings, preview images and the enrichment cache as one archive (suggested name `sdpm-library-export.zip`).

This is the library's information about your models, not the model files themselves. The **This library** group in the same menu has **Export all (ZIP)...** for just the LoRAs or just the checkpoints of the tab you are on; for a backup, the whole-library export is the better choice because it merges cleanly on import.



Model Library → Tools. **Library export/import...** is at the bottom.



The **Library Export / Import** dialog. An import matches entries by file hash, then source, then name, and shows what it will do before **Confirm import**.

When you import, you choose what happens when both sides have the same model: **Newest wins** (whichever side was updated more recently), **Prefer import** (the file always overwrites) or **Keep local** (never overwrite, only add what is new).

C · Wildcards and workflows

These two are not in the ZIP, so they have exports of their own:

- **Wildcards**: on the Wildcards page, **Export as ZIP** saves all your wildcard files.
- **Workflows**: on the Workflows page, **Tools** → **Export all N workflows** saves your workflow library as one JSON file. **Import from a JSON file...** in the same menu brings it back.

D · Copy the whole app folder: everything, exactly as it is

1. **Close Prompting Helper completely.** Copying while it runs can catch a file half-written; the model library database in particular keeps changes in a side file until the app closes.
2. Copy the whole `promptinghelper` folder from section 1 to an external drive or cloud storage.
3. If you set a custom data folder or custom image folders in **Settings → 06 Data**, copy those folders too.

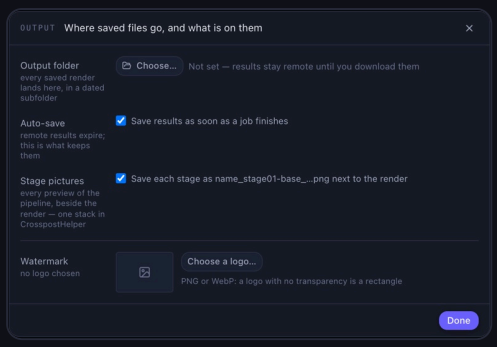
This is the only method that also keeps your settings and API keys, wildcards, templates and presets, reference and example images, jobs and their run history, storyboards, Lineup snapshots, Canvas projects, the Gallery's render history and small preferences such as which panels were open.

CHAPTER 13 · SECTION 03

What each backup covers

What	ZIP (A)	Library (B)	Own export (C)	Folder copy (D)
Characters (with pictures), saved prompts	✓	—	—	✓
Pose and background presets, LoRA recipes	✓	—	—	✓
Wildcards, workflows	—	—	✓	✓
Templates, booster combos, style presets, scratchpad	—	—	—	✓
App settings and API keys	—	—	—	✓
Model Library (LoRA and checkpoint entries, previews)	—	✓	—	✓
Reference images, example images	—	—	—	✓
Jobs, storyboards, Lineup snapshots, Canvas projects, Gallery history	—	—	—	✓
Model files (`.safetensors`)	—	—	—	— copy your models folder
Finished pictures	—	—	—	— copy your output folder
ComfyUI itself and its custom nodes	—	—	—	— reinstall; the app offers your node packs again

Your output folder is set under **Jobs → Output**. Saved renders land there in a dated subfolder, together with the stage pictures of a pipeline if **Stage pictures** is on.



Jobs → **Output**: the output folder, auto-save, stage pictures and the watermark. This is the folder to copy for your pictures.

CHAPTER 13 · SECTION 04

Move to a new computer

The most reliable way is to take the whole app folder along (method D). Plan about half an hour, plus the time to copy your models.

On the old computer

1. Make a **Full Backup (ZIP)** and a **Library export** (section 2, A and B). You may not need them, but they are your safety net.
2. Close Prompting Helper and ComfyUI.
3. Copy to an external drive: the `promptinghelper` app folder (and a custom data folder, if you set one), your models folder (the one the app set up outside ComfyUI, or ComfyUI's own `models` folder), and your output folder with your pictures.

On the new computer

1. Put your models folder back on the new computer.
2. Install Prompting Helper. If you open it once, close it again before the next step.
3. Paste your copied `promptinghelper` folder into the same place (section 1's table), replacing the one there.
4. Start Prompting Helper. Your characters, prompts, jobs, library and settings are back. If ComfyUI is not on the new computer yet, the **Setup assistant** (Settings → Rendering) downloads a fresh one (on Windows) or takes the one you have, points it at your models folder, and offers **Your node packs** from the old computer for installing (see the Getting Started chapter).
5. Check the paths, because drive letters and folder names often differ on a new machine:
   - **Settings → 01 Rendering → Output Integration**: the ComfyUI folder. Choose it again if it moved; its start file, port and model folders are read from it.
   - **Settings → 03 Models & sources → Model Folders**: where your checkpoints and LoRAs are. **Import from extra_model_paths.yaml...** fills them in from ComfyUI's own file.
   - **Settings → 06 Data**: the data folder and any custom image folders.
   - **Jobs → Output**: your output folder.

Click **Save** (top right in Settings) after changing anything.

6. The Model Library rescans your model folders on every start. To check right away, use **Tools → Rescan folders** or **Tools → Check library** in the Model Library.

A custom data folder that isn't there

If you had moved the data folder (section 5) and that path doesn't exist on the new computer, the app quietly starts with the default data folder, which is empty. Copy the folder over, then in **Settings → 06 Data** untick **Copy existing data to new folder**, click **Change Folder...** and pick it.

Different user name or drive on the new computer?

Character pictures are remembered with their full path. If that path changed (for example a new Windows user name), pictures can go missing even though the files were copied. Run **Import / Export → Import ZIP** with the backup from step 1: it restores the character pictures.

Only have the backup files, not the folder?

That works too, with less coming back: install the app, then use **Import ZIP**, **Library export/import... → Choose archive...**, and the wildcard and workflow imports. Imports are added to what's already there; nothing is deleted. Afterwards enter your API keys and settings again, set your paths, and copy your reference images over by hand if you still have them.

CHAPTER 13 · SECTION 05

Keep the data folder somewhere else

You can move the data folder, for example to a bigger drive or into a cloud-synced folder so a second computer sees the same characters and prompts.

1. Open **Settings → 06 Data → Data Storage**.
2. Leave **Copy existing data to new folder** ticked, so your current data comes along. Untick it only to point the app at a folder that already holds your data.
3. Click **Change Folder...** and pick the new folder. The app restarts by itself. Current folder then shows the new path and names the default below it.

Good to know

- Only the data folder moves. `settings.json`, reference images and the Gallery history stay in the app folder, so API keys, paths and the render history are not synced to a second computer.
- The copy overwrites files of the same name in the chosen folder. Pick an empty folder, unless it already holds your data and you have unticked the copy.
- The old folder is left as it was. Delete it yourself once everything works.
- If the chosen folder can't be found at start (an unplugged drive, a sync app not running yet), the app quietly uses the default folder instead. Your data isn't lost; plug the drive in and restart.
- With a cloud-synced folder, don't have the app open on two computers at once. Both write to the same files.

CHAPTER 13 · SECTION 06

Update the app safely

Updates never touch the app folder, so your data stays where it is. When a new version is out, a banner at the top of the window says **Version … is available**, with **What's new** for the changes, and a small **New** badge appears at the version number in the sidebar (which opens the About page).

- **Windows** and Linux **AppImage**: click **Download and install**, then **Restart and install**. If you don't restart, the update is installed when you next quit the app.
- **Mac**: click **Download** (or **Download …** on the About page), then replace the app in your Applications folder with the new one. macOS doesn't install unsigned updates in place.
- **Linux .deb**: update through your package manager.

The ✕ on the banner hides it until the next version. A quick copy of the app folder before a big update costs a minute and is worth it.

CHAPTER 13 · SECTION 07

Questions

I imported a backup and now have some things twice.
Imports merge: existing items are kept and imported items are added. Items with the same internal ID are updated rather than duplicated, but an item you re-created by hand is a different item. Delete the extra copies.

My settings and API keys are gone after importing the ZIP.
That's expected: a ZIP or JSON backup carries no settings at all. Enter your keys again under **Settings → 02 AI** (AI providers), **03 Models & sources** (CivitAI, Hugging Face) and **01 Rendering** (ComfyUI Cloud, render services), and set your paths.

My wildcards or workflows are missing after a ZIP restore.
They are not in the ZIP. Restore them from their own exports (section 2, C) or from a copy of the app folder.

The Model Library shows my models as missing.
The entries point to files in a place that no longer exists. Make sure the model files are in your models folder, check **Settings → Model Folders**, then use **Tools → Rescan folders**.

Can I move from Windows to Mac, or the other way?
Yes. Your characters, prompts and library travel fine. All paths (the ComfyUI folder, model folders, output folder) must be set again, because they look completely different on the other system.

Is Reset Data on the Import / Export page a way to start fresh?
No. It permanently deletes all characters, all saved prompts, or both (**Everything** means just these two). There is no undo. Make a backup first.

Is anything stored online?
No. Backups are files on your own computer. Only if you put the data folder into a cloud-synced folder, or render with a cloud service, does anything leave your machine.